environmental-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `environmental_engineering_ai999x.about()` for the same answer.

Every capability environmental-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import environmental_engineering_ai999x

result = environmental_engineering_ai999x.solve('enveng_stability_class', u='2.5 m/s',
condition='strong')
result.show()
# stability = A-B
```

By topic

topic	capabilities
Air pollution / atmospheric stability	Pasquill-Turner atmospheric stability class
Air pollution / box model	Box model of an air basin (well-mixed, time-dependent)
Air pollution / concentration units	Gas concentration: ppm (by volume) to mass concentration, Gas concentration: mass concentration to ppm (by volume)
Air pollution / dispersion coefficients	Briggs dispersion coefficients sigma y and sigma z
Air pollution / emissions	Sulfur dioxide emission from fuel sulfur
Air pollution / Gaussian plume	Gaussian plume concentration at a receptor (ground reflection), Ground-level centreline concentration of a plume, Gaussian plume concentration from the stability class, Gaussian plume trapped below an inversion lid (multiple reflections), Stack plume: effective height, dispersion and ground-level concentration
Air pollution / maximum concentration	Maximum ground-level concentration (constant sigma y/sigma z), Maximum ground-level concentration and its distance (Briggs coefficients)
Air pollution / particulate control	Gravity settling chamber collection efficiency, Electrostatic precipitator efficiency (Deutsch-Anderson), Electrostatic precipitator plate area for a target efficiency, Cyclone cut diameter and fractional efficiency (Lapple)

topic	capabilities
Air pollution / plume rise	Briggs plume rise and effective stack height (neutral or unstable air), Briggs plume rise in stable air, Holland plume rise
Air pollution / wind profile	Wind speed at height (power law)
Gas transfer / air stripping	Packed-tower air stripping: stripping factor, transfer units and height
Gas transfer / Henry's law	Henry's law constant conversion between forms, Dissolved concentration in equilibrium with a gas (Henry's law), Temperature dependence of Henry's constant (van 't Hoff)
Gas transfer / two-film theory	Overall liquid-phase transfer coefficient and flux (two-film theory)
Noise / community noise	Day-night (Ldn) and day-evening-night (Lden) sound levels
Noise / decibel arithmetic	Adding sound levels in decibels, Source level after removing background noise
Noise / exposure	Equivalent continuous sound level Leq
Noise / occupational exposure	Occupational noise dose and 8-hour TWA (OSHA, 5 dB exchange)
Noise / propagation	Sound level at another distance (point or line source), Sound pressure level from sound power level
Solid waste / energy recovery	Heating value of waste from its elemental composition (modified Dulong)
Solid waste / generation	Municipal solid waste generation and collection trucks
Solid waste / landfill gas	Landfill methane generation (first-order decay, constant acceptance)
Solid waste / landfills	Landfill life with daily cover and waste growth
Solid waste / properties	Moisture content of mixed solid waste
Wastewater / activated sludge	Food-to-microorganism ratio (F/M), Solids retention time (mean cell residence time), Hydraulic retention time and volumetric organic loading, Activated sludge (complete mix) effluent substrate and biomass (Lawrence-McCarty), Minimum (washout) solids retention time, Waste biomass production (observed yield), Oxygen requirement of an activated-sludge process, Return activated sludge ratio, Sludge volume index (SVI)
Wastewater / aeration	Field (actual) oxygen transfer rate of an aeration system, Overall oxygen transfer coefficient KLa from two DO readings, KLa fitted to a reaeration test, Dissolved oxygen during aeration and the transfer rate
Wastewater / anaerobic treatment	Methane production from COD removed (anaerobic digestion)
Wastewater / attached growth	Trickling filter BOD removal efficiency (NRC equation)
Wastewater / microbial kinetics	Monod specific growth rate
Wastewater / sludge handling	Volume of wet sludge from dry solids, Sludge volume after thickening or dewatering
Water quality / BOD kinetics	BOD exerted after a time (first-order), Ultimate BOD from a BOD test, Temperature correction of a rate constant (theta model), BOD rate constant and ultimate BOD fitted to BOD data

topic	capabilities
Water quality / BOD measurement	BOD from a dilution test (with optional seed correction)
Water quality / dissolved oxygen	Dissolved oxygen saturation concentration (Benson-Krause)
Water quality / lakes	Steady-state concentration in a well-mixed lake (first-order decay), Concentration in a well-mixed lake over time
Water quality / loads	Pollutant mass load of a flow
Water quality / mass balance	Concentration after mixing a discharge with a river, Flow-weighted mixing of several streams
Water quality / oxygen demand	Theoretical oxygen demand of an organic compound
Water quality / oxygen sag	Streeter-Phelps oxygen deficit and dissolved oxygen at a travel time, Dissolved oxygen at a distance downstream (Streeter-Phelps), Critical (minimum) dissolved oxygen of the oxygen sag
Water quality / reaeration	Stream reaeration coefficient (O'Connor-Dobbins, Churchill, Owens-Gibbs)
Water quality / rivers	First-order decay of a pollutant downstream (plug flow)
Water treatment / adsorption	Activated carbon dose from the Freundlich isotherm, Adsorbent dose from the Langmuir isotherm, Freundlich isotherm constants fitted to adsorption data, Langmuir isotherm constants fitted to adsorption data
Water treatment / basin hydraulics	Hydraulic detention time
Water treatment / disinfection	Disinfection CT value of a contact basin, Log inactivation from the CT ratio, Microbial inactivation by Chick's law (first order), Disinfection by the Chick-Watson law, Log removal and percentage removal, Chlorine dose and feed rate, Fraction of free chlorine as hypochlorous acid (pH)
Water treatment / filtration	Filtration (hydraulic loading) rate with one filter out of service, Clean-bed head loss through a granular filter (Ergun / Carman-Kozeny), Filter bed expansion during backwash
Water treatment / membranes	Membrane recovery, rejection, flux and concentrate, Osmotic pressure of a dilute solution (van 't Hoff)
Water treatment / mixing and flocculation	Mean velocity gradient G (rapid mixing and flocculation), Power needed for a target velocity gradient
Water treatment / sedimentation	Surface overflow rate of a circular clarifier, Sedimentation basin sizing from overflow rate and detention time, Weir loading rate of a circular clarifier, Stokes settling velocity of a discrete particle, Terminal settling velocity in any flow regime (drag coefficient iteration), Fraction of discrete particles removed in an ideal settling basin
Water treatment / water chemistry	Water hardness as CaCO3

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again

outcome	meaning	what to do
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
enveng_stability_class	Pasquill-Turner atmospheric stability class	u, condition	stability
enveng_briggs_dispersion_coefficients	Briggs dispersion coefficients sigma_y and sigma_z	x, cls, terrain	sigma_y, sigma_z

id	operation	inputs	outputs
enveng_gaussian_plume_concentration	Gaussian plume concentration at a receptor (ground reflection)	Q, u, sigma_y, sigma_z, H, y, z	C, C_ugm3
enveng_ground_centerline_concentration	Ground-level centreline concentration of a plume	Q, u, sigma_y, sigma_z, H	C, C_ugm3
enveng_plume_concentration_stability	Gaussian plume concentration from the stability class	Q, u, H, x, y, z, cls, terrain	C, C_ugm3, sigma_y, sigma_z
enveng_max_ground_concentration_ratio	Maximum ground-level concentration (constant sigma_y/sigma_z)	Q, u, H, k	C_max, C_max_ugm3, sigma_z_max
enveng_max_ground_concentration_briggs	Maximum ground-level concentration and its distance (Briggs coefficients)	Q, u, H, cls, terrain	x_max, C_max, C_max_ugm3, sigma_y, sigma_z
enveng_briggs_plume_rise	Briggs plume rise and effective stack height (neutral or unstable air)	vs, d, Ts, Ta, u, hs	F, x_f, dh, H
enveng_briggs_stable_plume_rise	Briggs plume rise in stable air	vs, d, Ts, Ta, u, dtheta, hs	F, s, dh, H
enveng_holland_plume_rise	Holland plume rise	vs, d, u, p, Ts, Ta, hs	dh, H
enveng_wind_power_law	Wind speed at height (power law)	u1, z1, z2, p	u2
enveng_ppm_to_mass_concentration	Gas concentration: ppm (by volume) to mass concentration	ppm, M, T, P	C, C_ugm3, C_mgm3
enveng_mass_concentration_to_ppm	Gas concentration: mass concentration to ppm (by volume)	C, M, T, P	ppm, ppb
enveng_box_model	Box model of an air basin (well-mixed, time-dependent)	q, L, u, Hm, Cb, C0, t	C, C_ss, C_ugm3
enveng_settling_chamber_efficiency	Gravity settling chamber collection efficiency	dp, rho_p, rho_g, mu, L, W, Q, N	vt, eta, Re
enveng_esp_efficiency	Electrostatic precipitator efficiency (Deutsch-Anderson)	w, A, Q	eta, penetration
enveng_esp_area	Electrostatic precipitator plate area for a target efficiency	w, Q, eta	A
enveng_cyclone_lapple	Cyclone cut diameter and fractional efficiency (Lapple)	mu, W, Ne, Vi, rho_p, rho_g, dp	d50, eta
enveng_so2_emission	Sulfur dioxide emission from fuel sulfur	m_fuel, S, eta	E, E_uncontrolled, E_kg_per_day

id	operation	inputs	outputs
enveng_plume_mixing_lid	Gaussian plume trapped below an inversion lid (multiple reflections)	Q, u, sigma_y, sigma_z, H, Lm, y, z	C, C_ugm3, C_mixed
enveng_stack_plume_ground_concentration	Stack plume: effective height, dispersion and ground-level concentration	Q, u, vs, d, Ts, Ta, hs, x, cls, terrain	H, sigma_y, sigma_z, C, C_ugm3
enveng_bod_exerted	BOD exerted after a time (first-order)	L0, k, t	BOD_t, L_t, BOD_t_mgL
enveng_ultimate_bod	Ultimate BOD from a BOD test	BOD_t, k, t	L0, L0_mgL
enveng_rate_constant_temperature	Temperature correction of a rate constant (theta model)	k20, theta, T	kT, kT_per_day
enveng_bod_dilution_test	BOD from a dilution test (with optional seed correction)	D1, D2, P, B1, B2, f	BOD, BOD_mgL
enveng_bod_rate_fit	BOD rate constant and ultimate BOD fitted to BOD data	t, y	k, k_per_day, L0, L0_mgL, rss
enveng_theoretical_oxygen_demand	Theoretical oxygen demand of an organic compound	formula, nitrification	thod, o2_per_mol, molar_mass
enveng_streeter_phelps_deficit	Streeter-Phelps oxygen deficit and dissolved oxygen at a travel time	kd, ka, L0, D0, t, DOsat	D, DO, D_mgL, DO_mgL
enveng_oxygen_sag_distance	Dissolved oxygen at a distance downstream (Streeter-Phelps)	kd, ka, L0, D0, x, v, DOsat	D, DO, D_mgL, DO_mgL, t
enveng_streeter_phelps_critical_point	Critical (minimum) dissolved oxygen of the oxygen sag	kd, ka, L0, D0, DOsat	t_c, t_c_days, D_c, DO_min, DO_min_mgL
enveng_reaeration_coefficient	Stream reaeration coefficient (O'Connor-Dobbins, Churchill, Owens-Gibbs)	method, u, H	ka, ka_per_day
enveng_do_saturation	Dissolved oxygen saturation concentration (Benson-Krause)	T, S	DOsat, DOsat_mgL
enveng_mixing_two_streams	Concentration after mixing a discharge with a river	Qw, Cw, Qr, Cr	C, C_mgL, Q
enveng_mixing_several_streams	Flow-weighted mixing of several streams	Q, C	C_mix, Q_total
enveng_lake_steady_state	Steady-state concentration in a well-mixed lake (first-order decay)	W, Q, k, V	C, C_mgL, tau

id	operation	inputs	outputs
enveng_lake_transient	Concentration in a well-mixed lake over time	W, Q, k, V, C0, t	C, C_mgL, C_ss
enveng_river_first_order_decay	First-order decay of a pollutant downstream (plug flow)	C0, k, x, u	C, C_mgL, t, fraction_remaining
enveng_pollutant_load	Pollutant mass load of a flow	Q, C	W, W_kg_per_day, W_t_per_year
enveng_detention_time	Hydraulic detention time	V, Q	theta, theta_h
enveng_overflow_rate_circular	Surface overflow rate of a circular clarifier	Q, D	A, v0, v0_m_per_day
enveng_sedimentation_basin_sizing	Sedimentation basin sizing from overflow rate and detention time	Q, v0, theta	A, V, depth, D
enveng_weir_loading_rate	Weir loading rate of a circular clarifier	Q, D	weir_length, WL, WL_m3_per_m_day
enveng_stokes_settling_velocity	Stokes settling velocity of a discrete particle	d, rho_p, rho_w, mu	vs, Re, vs_m_per_h
enveng_terminal_settling_velocity	Terminal settling velocity in any flow regime (drag coefficient iteration)	d, rho_p, rho_f, mu	vt, Re, Cd
enveng_ideal_basin_removal	Fraction of discrete particles removed in an ideal settling basin	vs, Q, A	v0, R
enveng_filtration_rate	Filtration (hydraulic loading) rate with one filter out of service	Q, A, N	v, v_m_per_h, v_one_out, v_one_out_m_per_h
enveng_filter_clean_bed_headloss	Clean-bed head loss through a granular filter (Ergun / Carman-Kozeny)	d, eps, psi, L, v, rho, mu	dP, h, Re
enveng_backwash_bed_expansion	Filter bed expansion during backwash	L, eps, vb, vs	eps_e, Le, expansion
enveng_velocity_gradient	Mean velocity gradient G (rapid mixing and flocculation)	P, mu, V, t	G, Gt
enveng_mixing_power	Power needed for a target velocity gradient	G, mu, V	P
enveng_disinfection_ct	Disinfection CT value of a contact basin	C, V, Q, BF	t10, CT, CT_mg_min_L
enveng_ct_log_inactivation	Log inactivation from the CT ratio	C, t10, CT999	CT, ratio, log_inactivation
enveng_chick_law	Microbial inactivation by Chick's law (first order)	N0, k, t	N, survival, log_removal, percent_removal
enveng_chick_watson	Disinfection by the Chick-Watson law	k, n, C, t	survival, log_inactivation
enveng_log_removal	Log removal and percentage removal	N0, N	log_removal, percent_removal

id	operation	inputs	outputs
enveng_chlorine_dose_feed	Chlorine dose and feed rate	Q, demand, residual, purity	dose, dose_mgL, feed, feed_kg_per_day, product_kg_per_day
enveng_hypochlorous_acid_fraction	Fraction of free chlorine as hypochlorous acid (pH)	pH, pKa, T	alpha_HOCl, alpha_OCl, pKa
enveng_water_hardness	Water hardness as CaCO3	Ca, Mg	total_hardness, calcium_hardness, magnesium_hardness, classification
enveng_freundlich_carbon_dose	Activated carbon dose from the Freundlich isotherm	C0, Ce, K, n	qe, dose, dose_mgL
enveng_langmuir_carbon_dose	Adsorbent dose from the Langmuir isotherm	C0, Ce, qmax, b	qe, dose, dose_mgL
enveng_freundlich_fit	Freundlich isotherm constants fitted to adsorption data	Ce, qe	K, inv_n, n, r2
enveng_langmuir_fit	Langmuir isotherm constants fitted to adsorption data	Ce, qe	qmax, b, r2
enveng_membrane_mass_balance	Membrane recovery, rejection, flux and concentrate	Qf, Qp, Cf, Cp, A	recovery, rejection, J, J_LMH, Qc, Cc, Cc_mgL
enveng_osmotic_pressure	Osmotic pressure of a dilute solution (van 't Hoff)	C, M, i, T	pi, pi_bar
enveng_food_to_microorganism_ratio	Food-to-microorganism ratio (F/M)	Q, S0, V, X	FM, FM_per_day
enveng_solids_retention_time	Solids retention time (mean cell residence time)	V, X, Qw, Xw, Qe, Xe	SRT, SRT_days
enveng_hrt_organic_loading	Hydraulic retention time and volumetric organic loading	Q, S0, V	HRT, HRT_h, OLR, OLR_kg_per_m3_day
enveng_monod_growth_rate	Monod specific growth rate	mu_max, Ks, S, kd	mu, mu_per_day, mu_net, mu_net_per_day
enveng_activated_sludge_cstr_design	Activated sludge (complete mix) effluent substrate and biomass (Lawrence-McCarty)	Y, k, Ks, kd, SRT, HRT, S0	S, S_mgL, X, X_mgL, Y_obs
enveng_minimum_srt	Minimum (washout) solids retention time	Y, k, Ks, kd, S0	SRT_min, SRT_min_days
enveng_sludge_production	Waste biomass production (observed yield)	Y, kd, SRT, Q, S0, S	Y_obs, Px, Px_kg_per_day
enveng_aeration_oxygen_requirement	Oxygen requirement of an activated-sludge process	Q, S0, S, f, Px	O2, O2_kg_per_day

id	operation	inputs	outputs
enveng_return_sludge_ratio	Return activated sludge ratio	X, Xr, Q	R, Qr
enveng_sludge_volume_index	Sludge volume index (SVI)	V30, MLSS	SVI, Xr_estimate, Xr_estimate_mgL
enveng_sludge_volume	Volume of wet sludge from dry solids	Ms, Ps, Ssl	V, V_m3_per_day
enveng_sludge_thickening	Sludge volume after thickening or dewatering	V1, P1, P2	V2, reduction
enveng_anaerobic_methane_yield	Methane production from COD removed (anaerobic digestion)	COD, Px, T, P	Q_CH4, Q_CH4_m3_per_day, Q_CH4_stp_m3_per_day
enveng_nrc_trickling_filter	Trickling filter BOD removal efficiency (NRC equation)	W, V, R	F, E, E_percent
enveng_field_oxygen_transfer	Field (actual) oxygen transfer rate of an aeration system	SOTR, alpha, beta, theta, CsT, Cs20, CL, T, Fp	AOTR, AOTR_kg_per_h, ratio
enveng_kla_two_point	Overall oxygen transfer coefficient KLa from two DO readings	Cs, C1, C2, t1, t2	KLa, KLa_per_h
enveng_kla_regression	KLa fitted to a reaeration test	t, C, Cs	KLa_per_min, KLa, KLa_per_h, r2
enveng_aeration_do_curve	Dissolved oxygen during aeration and the transfer rate	Cs, C0, KLa, t	C, C_mgL, OTR, OTR_mg_per_L_h
enveng_henry_constant_conversion	Henry's law constant conversion between forms	H, form, T	H_pa_m3_per_mol, H_atm_m3_per_mol, H_dimensionless, H_atm, H_mol_per_m3_pa, H_m_per_atm
enveng_henry_law_equilibrium	Dissolved concentration in equilibrium with a gas (Henry's law)	p, H, M	C, C_mass, C_mgL
enveng_henry_temperature_dependence	Temperature dependence of Henry's constant (van 't Hoff)	Hcp, B, T, Tref	Hcp_T, Hpc_T
enveng_two_film_transfer	Overall liquid-phase transfer coefficient and flux (two-film theory)	kl, kg, H, C, Cg	KL, flux, liquid_resistance_fraction
enveng_air_stripping_tower	Packed-tower air stripping: stripping factor, transfer units and height	Qw, Qa, H, Cin, Cout, HTU	R, NTU, Z, air_to_water_min
enveng_decibel_addition	Adding sound levels in decibels	L	L_total
enveng_background_noise_correction	Source level after removing background noise	Lt, Lb	L_source

id	operation	inputs	outputs
enveng_noise_distance_attenuation	Sound level at another distance (point or line source)	L1, r1, r2, source	L2
enveng_equivalent_sound_level	Equivalent continuous sound level Leq	L, t	Leq
enveng_day_night_level	Day-night (Ldn) and day-evening-night (Lden) sound levels	L, metric	L
enveng_sound_power_to_pressure	Sound pressure level from sound power level	Lw, r, Qd	Lp
enveng_occupational_noise_dose	Occupational noise dose and 8-hour TWA (OSHA, 5 dB exchange)	L, hours	dose_percent, TWA
enveng_solid_waste_generation	Municipal solid waste generation and collection trucks	pop, rate, rho, Vt	m, m_kg_per_day, m_t_per_year, V_m3_per_day, trucks_per_day
enveng_landfill_life	Landfill life with daily cover and waste growth	capacity, M, rho, cover, g	V_first_year, life_years
enveng_landfill_gas_generation	Landfill methane generation (first-order decay, constant acceptance)	M, L0, k, tc, t	Q_CH4, Q_CH4_m3_per_year, V_CH4, Q_LFG
enveng_dulong_heating_value	Heating value of waste from its elemental composition (modified Dulong)	C, H, O, S, N	HHV, HHV_kJ_per_kg, HHV_MJ_per_kg
enveng_waste_moisture_content	Moisture content of mixed solid waste	m, w	moisture_wet, dry_mass, moisture_dry

Pasquill-Turner atmospheric stability class, `enveng_stability_class`

The Pasquill stability class (A very unstable to F stable) from the surface wind speed and the daytime insolation or night-time cloud cover.

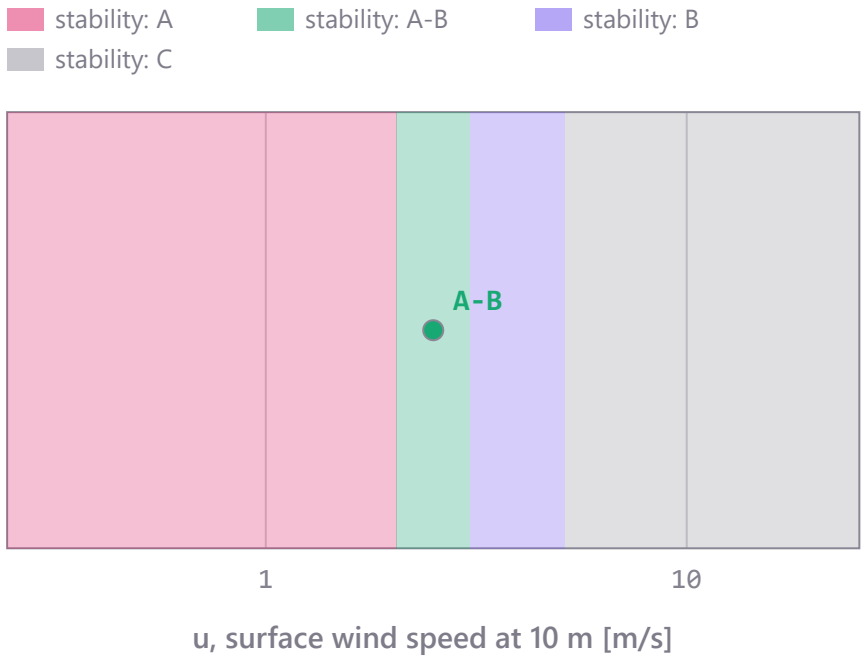
$$\text{class} = T(u,\ \text{insolation or cloud cover})$$

input	meaning	condition
`u`	surface wind speed at 10 m	
`condition`	daytime insolation (strong, moderate, slight), overcast (day or night), night_cloudy (at least $4/8$ low cloud) or night_clear (at most $3/8$ cloud)	

output	meaning
`stability`	Pasquill stability class (A-F; A-B etc. for intermediate cases)

- **Method:** Turner's table: wind-speed rows < 2, $2-3$, $3-5$, $5-6$, > 6 m/s against insolation or cloud-cover columns; heavy overcast is D
- **Assumes:** $u > 0$; no class is defined at night with u below 2 m/s

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Pasquill stability class (A-F; A-B etc. for intermediate cases) (stability) along u (surface wind speed at 10 m) from 0.25 m/s to 25 m/s (logarithmic axis): 81 runs of environmental-engineering-ai999x, each band coloured by the verdict returned (A, A-B, B, C); it changes near u = 2.04381 m/s, 3.05802 m/s, 5.13381 m/s. Other inputs as in the worked example: condition = strong. Marked: the worked example, u = 2.5 m/s → A-B.

```
environmental_engineering_ai999x.solve('enveng_stability_class', u=2.5, condition='strong')
# -> stability = A-B   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_stability_class', u=4.0, condition='night_clear')
# -> stability = E   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pasquill stability class: u = 2.5 m/s, condition = strong	SOLVED	stability = A-B
Pasquill stability class: u = 4 m/s, condition = night_clear	SOLVED	stability = E
Pasquill stability class: condition = strong	NEEDS_INPUT	enveng_stability_class needs u (surface wind speed at 10 m). Example: solve('enveng_stability_class', u='2.5 m/s', condi
Pasquill stability class: u = 2.5 m/s, condition = strong, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Stability class: u = 1.5 m/s, condition = night_clear	INVALID_INPUT	Pasquill-Turner atmospheric stability class: the table gives no class for a night with a surface wind below 2 m/s.
Atmospheric stability class: u = 9 km/h, condition = strong	SOLVED	stability = A-B

Briggs dispersion coefficients sigma_y and sigma_z, `enveng_briggs_dispersion_coefficients`

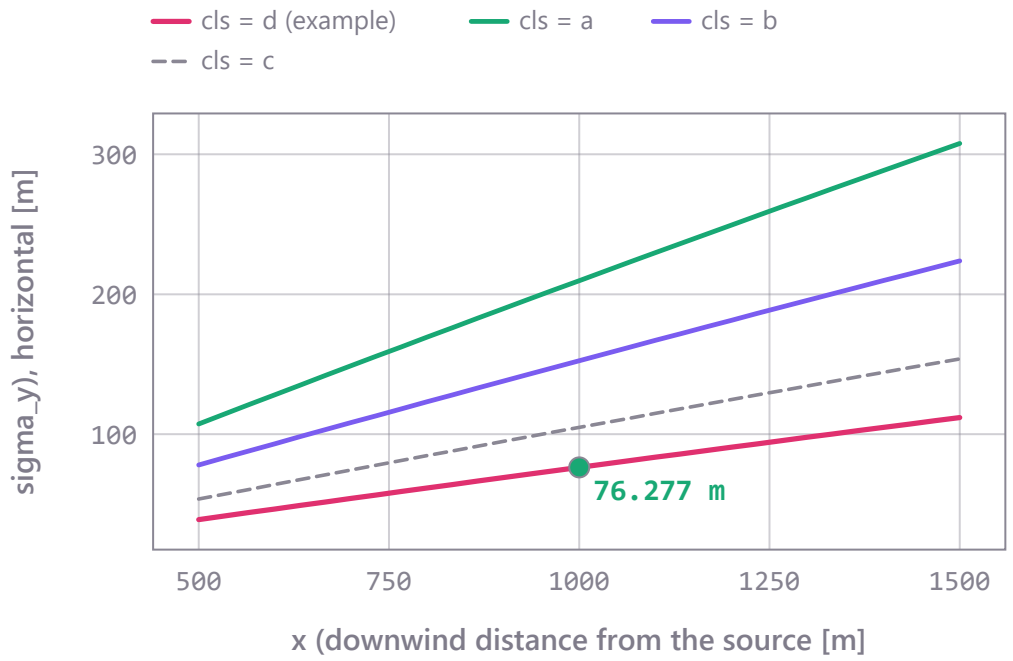
The horizontal and vertical plume spreads (Pasquill-Gifford curves in Briggs' formulas) at a downwind distance for a stability class, open-country or urban.

$$\sigma_y = \frac{a_y x}{\sqrt{1 + b_y x}}, \quad \sigma_z = a_z x \, (1 + b_z x)^{p_z}$$

input	meaning	condition
`x`	downwind distance from the source	
`cls`	Pasquill stability class A-F (urban: A and B share A-B, E and F share E-F)	
`terrain`	rural (open country) or urban coefficients	

output	meaning
`sigma_y`	horizontal (crosswind) dispersion coefficient
`sigma_z`	vertical dispersion coefficient

- **Method:** Briggs (1973) interpolation formulas $\sigma = a$ x (1 + b x)^p for each class and terrain
- **Assumes:** 100 $m \leq x \leq 10$ km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sigma_y (horizontal (crosswind) dispersion coefficient) against x (downwind distance from the source) from 500 m to 1500 m. One curve for each value of cls (Pasquill stability class A-F (urban: A and B share A-B, E and F share E-F)): d, a, b, c. Other inputs as in the worked example: terrain = rural. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, x = 1000 m, sigma_y = 76.277 m.

```
environmental_engineering_ai999x.solve('enveng_briggs_dispersion_coefficients', x=1000.0, cls='d',
terrain='rural')
# -> sigma_y = 76.277 m, sigma_z = 37.9473 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_briggs_dispersion_coefficients', x=2500.0, cls='b',
terrain='urban')
# -> sigma_y = 565.685 m, sigma_z = 1122.5 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Briggs dispersion coefficients: x = 1000 m, cls = d, terrain = rural	SOLVED	sigma_y = 76.277 m, sigma_z = 37.9473 m
Briggs dispersion coefficients: x = 2500 m, cls = b, terrain = urban	SOLVED	sigma_y = 565.685 m, sigma_z = 1122.5 m
Briggs dispersion coefficients: cls = d, terrain = rural	NEEDS_INPUT	enveng_briggs_dispersion_coefficients needs x (downwind distance from the source). Example: solve('enveng_briggs_dispers
Briggs dispersion coefficients: x = 1000 m, cls = d, terrain = rural, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Dispersion coefficients: x = 1 km, class = D, terrain = rural	SOLVED	sigma_y = 76.277 m, sigma_z = 37.9473 m
Briggs dispersion coefficients: x = 50 m, cls = d, terrain = rural	INVALID_INPUT	Briggs dispersion coefficients sigma_y and sigma_z: the Briggs coefficients hold for x between 100 m and 10 km.

Gaussian plume concentration at a receptor (ground reflection),

`enveng_gaussian_plume_concentration`

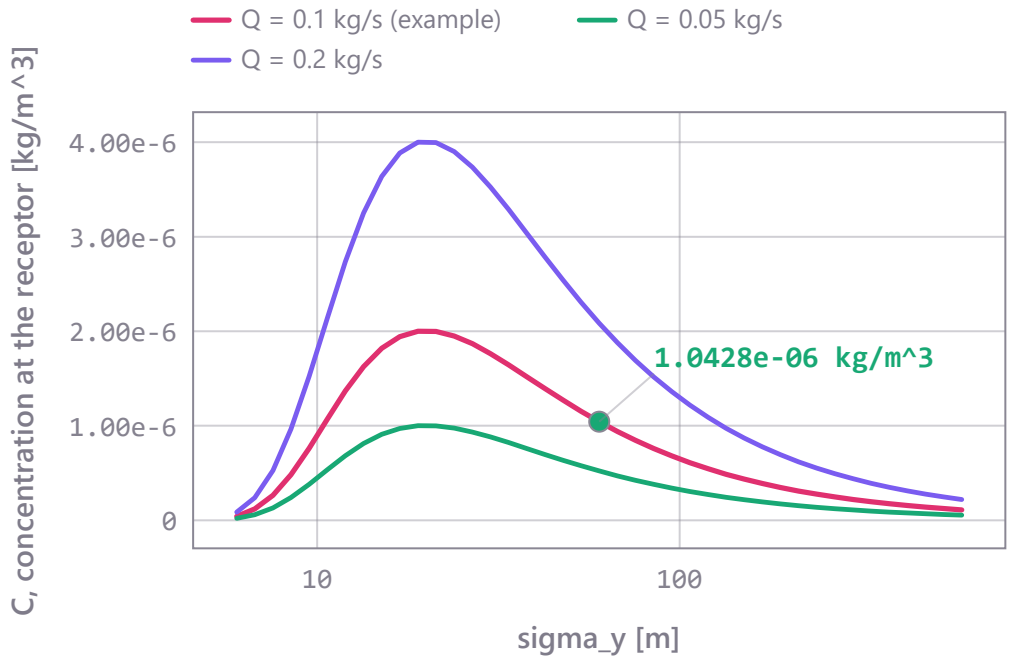
The steady concentration downwind of a continuous point source at crosswind distance y and height z, given the dispersion coefficients at the receptor's distance and the effective stack height, with full reflection at the ground.

$$C = \frac{Q}{2\pi u \sigma_y \sigma_z} e^{-\frac{y^2}{2\sigma_y^2}} \left[e^{-\frac{(z-H)^2}{2\sigma_z^2}} + e^{-\frac{(z+H)^2}{2\sigma_z^2}} \right]$$

input	meaning	condition
`Q`	pollutant emission rate	
`u`	mean wind speed at the stack height	
`sigma_y`	horizontal dispersion coefficient at the receptor distance	
`sigma_z`	vertical dispersion coefficient at the receptor distance	
`H`	effective stack height (stack height plus plume rise)	
`y`	crosswind distance from the plume centreline	
`z`	receptor height above the ground	

output	meaning
`C`	concentration at the receptor
`C_ugm3`	concentration in micrograms per cubic metre

- **Method:** C = Q/(2 pi u sy sz) exp(-y^2/2sy^2) [exp(-(z-H)^2/2sz^2) + exp(-(z+H)^2/2sz^2)]
- **Assumes:** Q, u, sigma_y, $sigma_z > 0$; H, $z \geq 0$; steady state, flat terrain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration at the receptor) against sigma_y (horizontal dispersion coefficient at the receptor distance) from 6 m to 600 m on a logarithmic axis. One curve for each value of Q (pollutant emission rate): 0.1 kg/s, 0.05 kg/s, 0.2 kg/s. Other inputs as in the worked example: u = 4.0 m/s, sigma_z = 30.0 m, H = 50.0 m, y = 20.0 m, z = 0.0 m. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, sigma_y = 60 m, C = 1.0428e-06 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_gaussian_plume_concentration', Q=0.1, u=4.0,
sigma_y=60.0, sigma_z=30.0, H=50.0, y=20.0, z=0.0)
# -> C = 1.04281e-06 kg/m^3, C_ugm3 = 1042.81   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_gaussian_plume_concentration', Q=0.5, u=6.0,
sigma_y=150.0, sigma_z=80.0, H=100.0, y=0.0, z=30.0)
# -> C = 1.04887e-06 kg/m^3, C_ugm3 = 1048.87   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gaussian plume concentration: Q = 100 g/s, u = 4 m/s, sigma_y = 60 m, sigma_z = 30 m, H = 50 m, y = 20 m, z = 0 m	SOLVED	C = 1.04281e-06 kg/m^3, C_ugm3 = 1042.81
Gaussian plume concentration: Q = 500 g/s, u = 6 m/s, sigma_y = 150 m, sigma_z = 80 m, H = 100 m, y = 0 m, z = 30 m	SOLVED	C = 1.04887e-06 kg/m^3, C_ugm3 = 1048.87
Gaussian plume concentration: u = 4 m/s, sigma_y = 60 m, sigma_z = 30 m, H = 50 m, y = 20 m, z = 0 m	NEEDS_INPUT	enveng_gaussian_plume_concentration needs Q (pollutant emission rate). Example: solve('enveng_gaussian_plume_concentrati
Gaussian plume concentration: Q = 100 g/s, u = 4 m/s, sigma_y = 60 m,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
sigma_z = 30 m, H = 50 m, y = 20 m, z = 0 m, zz = 1		environmental_engineering_ai999x.
Gaussian plume concentration: Q = 360 kg/h, u = 4 m/s, sigma_y = 60 m, sigma_z = 30 m, H = 50 m, y = 20 m, z = 0 m	SOLVED	C = 1.04281e-06 kg/m^3, C_ugm3 = 1042.81

Ground-level centreline concentration of a plume, `enveng_ground_centerline_concentration`

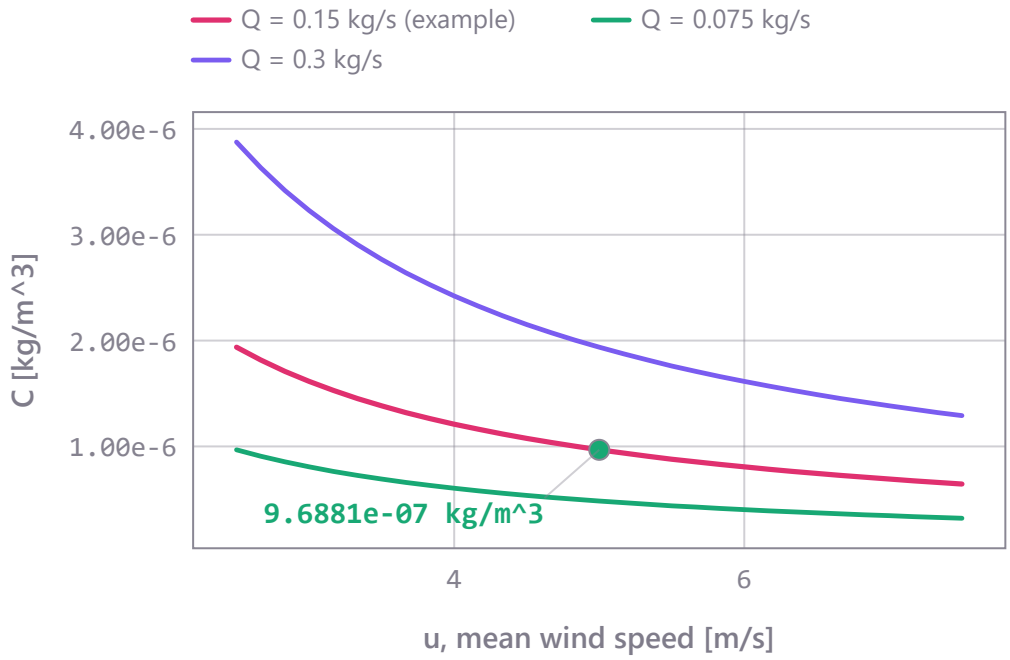
The ground-level concentration directly below the plume axis (y = 0, z = 0) from the emission rate, wind speed, dispersion coefficients and effective stack height (H = 0 for a ground-level source).

$$C(x,0,0) = \frac{Q}{\pi u \sigma_y \sigma_z} \exp\left(-\frac{H^2}{2\sigma_z^2}\right)$$

input	meaning	condition
`Q`	pollutant emission rate	
`u`	mean wind speed	
`sigma_y`	horizontal dispersion coefficient	
`sigma_z`	vertical dispersion coefficient	
`H`	effective stack height (0 for a ground-level source)	

output	meaning
`C`	ground-level centreline concentration
`C_ugm3`	concentration in micrograms per cubic metre

- **Method:** C = Q/(pi u sy sz) exp(-H^2/(2 sz^2))
- **Assumes:** Q, u, sigma_y, $sigma_z > 0$; $H \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (ground-level centreline concentration) against u (mean wind speed) from 2.5 m/s to 7.5 m/s. One curve for each value of Q (pollutant emission rate): 0.15 kg/s, 0.075 kg/s, 0.3 kg/s. Other inputs as in the worked example: sigma_y = 80.0 m, sigma_z = 40.0 m, H = 60.0 m. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, u = 5 m/s, C = 9.6881e-07 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_ground_centerline_concentration', Q=0.15, u=5.0,
sigma_y=80.0, sigma_z=40.0, H=60.0)
# -> C = 9.68813e-07 kg/m^3, C_ugm3 = 968.813   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_ground_centerline_concentration', Q=0.002, u=3.0,
sigma_y=20.0, sigma_z=12.0, H=0.0)
# -> C = 8.84194e-07 kg/m^3, C_ugm3 = 884.194   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ground-level centreline concentration: Q = 150 g/s, u = 5 m/s, sigma_y = 80 m, sigma_z = 40 m, H = 60 m	SOLVED	C = 9.68813e-07 kg/m^3, C_ugm3 = 968.813
Ground-level centreline concentration: Q = 2 g/s, u = 3 m/s, sigma_y = 20 m, sigma_z = 12 m, H = 0 m	SOLVED	C = 8.84194e-07 kg/m^3, C_ugm3 = 884.194
Ground-level centreline concentration: u = 5 m/s, sigma_y = 80 m, sigma_z = 40 m, H = 60 m	NEEDS_INPUT	enveng_ground_centerline_concentration needs Q (pollutant emission rate). Example: solve('enveng_ground_centerline_conce
Ground-level centreline concentration: Q = 150 g/s, u = 5 m/s, sigma_y = 80 m, sigma_z = 40 m, H = 60 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Gaussian plume concentration from the stability class, `enveng_plume_concentration_stability`

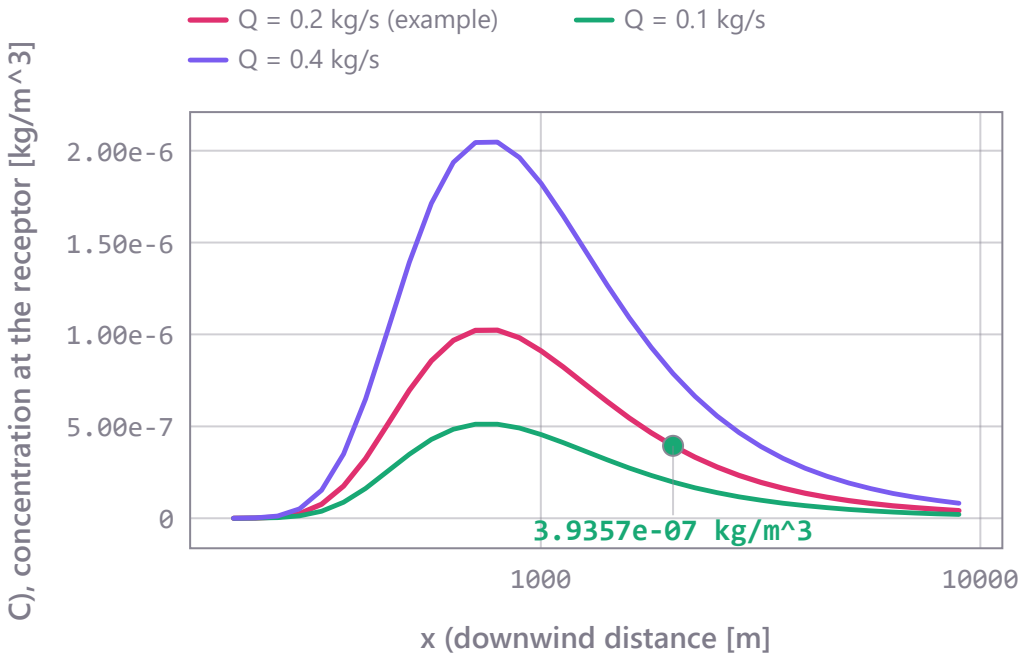
The concentration at a receptor (x, y, z) downwind of an elevated point source, with the dispersion coefficients taken from Briggs' formulas for the stability class and terrain.

$$C = \frac{Q}{2\pi u \sigma_y(x)\sigma_z(x)} e^{-\frac{y^2}{2\sigma_y^2}} \left[e^{-\frac{(z-H)^2}{2\sigma_z^2}} + e^{-\frac{(z+H)^2}{2\sigma_z^2}} \right]$$

input	meaning	condition
`Q`	pollutant emission rate	
`u`	mean wind speed at the stack height	
`H`	effective stack height	
`x`	downwind distance	
`y`	crosswind distance from the centreline	
`z`	receptor height	
`cls`	Pasquill stability class A-F	
`terrain`	rural or urban dispersion coefficients	

output	meaning
`C`	concentration at the receptor
`C_ugm3`	concentration in micrograms per cubic metre
`sigma_y`	horizontal dispersion coefficient at x
`sigma_z`	vertical dispersion coefficient at x

- **Method:** Briggs sigma_y(x), sigma_z(x) for the class, then the reflected Gaussian plume
- **Assumes:** 100 $m \leq x \leq 10$ km; Q, $u > 0$; H, $z \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration at the receptor) against x (downwind distance) from 200 m to 20000 m on a logarithmic axis. One curve for each value of Q (pollutant emission rate): 0.2 kg/s, 0.1 kg/s, 0.4 kg/s. Other inputs as in the worked example: u = 5.0 m/s, H = 80.0 m, y = 0.0 m, z = 0.0 m, cls = c, terrain = rural. 34 points per curve computed by environmental-engineering-ai999x; marked: the worked example, x = 2000 m, C = 3.9357e-07 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_plume_concentration_stability', Q=0.2, u=5.0,
H=80.0, x=2000.0, y=0.0, z=0.0, cls='c', terrain='rural')
# -> C = 3.93569e-07 kg/m^3, C_ugm3 = 393.569, sigma_y = 200.832 m, sigma_z = 135.225 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_plume_concentration_stability', Q=0.05, u=3.0,
H=40.0, x=800.0, y=30.0, z=2.0, cls='d', terrain='urban')
# -> C = 4.218e-07 kg/m^3, C_ugm3 = 421.8, sigma_y = 111.41 m, sigma_z = 100.579 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plume concentration with stability class: Q = 200 g/s, u = 5 m/s, H = 80 m, x = 2000 m, y = 0 m, z = 0 m, cls = c, terrain = rural	SOLVED	C = 3.93569e-07 kg/m^3, C_ugm3 = 393.569, sigma_y = 200.832 m, sigma_z = 135.225 m
Plume concentration with stability class: Q = 50 g/s, u = 3 m/s, H = 40 m, x = 800 m, y = 30 m, z = 2 m, cls = d, terrain = urban	SOLVED	C = 4.218e-07 kg/m^3, C_ugm3 = 421.8, sigma_y = 111.41 m, sigma_z = 100.579 m
Plume concentration with stability class: u = 5 m/s, H = 80 m, x = 2000 m, y = 0 m, z = 0 m, cls = c, terrain = rural	NEEDS_INPUT	enveng_plume_concentration_stability needs Q (pollutant emission rate). Example: solve('enveng_plume_concentration_stabi
Plume concentration with stability class: Q = 200 g/s, u = 5 m/s, H = 80 m, x = 2000 m, y = 0 m, z = 0 m, cls = c, terrain = rural, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Maximum ground-level concentration (constant sigma_y/sigma_z),

`enveng_max_ground_concentration_ratio`

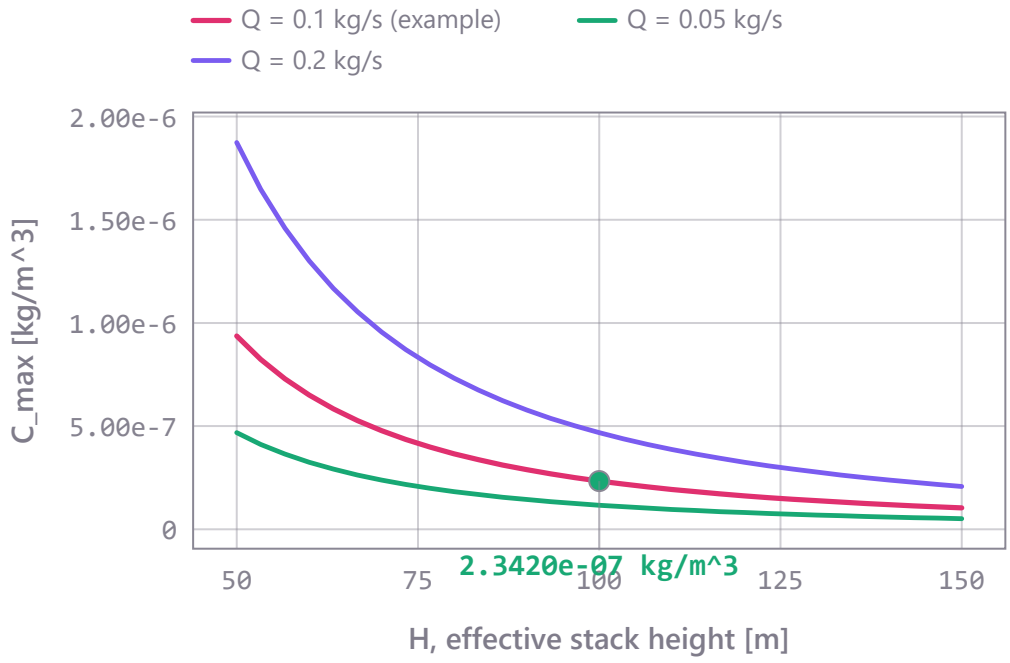
The largest ground-level centreline concentration of an elevated source when sigma_y/sigma_z is taken constant along the plume, and the vertical spread at which it occurs (sigma_z = H/sqrt 2).

$$C_{max} = \frac{2Q}{\pi e\, u H^2}\frac{\sigma_z}{\sigma_y}, \quad \sigma_z = \frac{H}{\sqrt{2}}$$

input	meaning	condition
`Q`	emission rate	
`u`	mean wind speed	
`H`	effective stack height	
`k`	ratio sigma_y/sigma_z (constant along the plume)	

output	meaning
`C_max`	maximum ground-level concentration
`C_max_ugm3`	maximum concentration in micrograms per cubic metre
`sigma_z_max`	vertical dispersion coefficient at the maximum

- **Method:** maximise C = Q/(pi u k sz^2) exp(-H^2/2sz^2) over sz: $sz = H/sqrt2$, Cmax = 2Q/(pi e u H^2 k)
- **Assumes:** Q, u, H, $k > 0$; sigma_y/sigma_z constant along the plume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C_max (maximum ground-level concentration) against H (effective stack height) from 50 m to 150 m. One curve for each value of Q (emission rate): 0.1 kg/s, 0.05 kg/s, 0.2 kg/s. Other inputs as in the worked example: u = 5.0 m/s, k = 2.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, H = 100 m, C_max = 2.3420e-07 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_max_ground_concentration_ratio', Q=0.1, u=5.0,
H=100.0, k=2.0)
# -> C_max = 2.34199e-07 kg/m^3, C_max_ugm3 = 234.199, sigma_z_max = 70.7107 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_max_ground_concentration_ratio', Q=1.2, u=3.5,
H=150.0, k=1.6)
# -> C_max = 2.23047e-06 kg/m^3, C_max_ugm3 = 2230.47, sigma_z_max = 106.066 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum ground-level concentration with constant sigma ratio: Q = 100 g/s, u = 5 m/s, H = 100 m, k = 2	SOLVED	C_max = 2.34199e-07 kg/m^3, C_max_ugm3 = 234.199, sigma_z_max = 70.7107 m
Maximum ground-level concentration with constant sigma ratio: Q = 1200 g/s, u = 3.5 m/s, H = 150 m, k = 1.6	SOLVED	C_max = 2.23047e-06 kg/m^3, C_max_ugm3 = 2230.47, sigma_z_max = 106.066 m
Maximum ground-level concentration with constant sigma ratio: u = 5 m/s, H = 100 m, k = 2	NEEDS_INPUT	enveng_max_ground_concentration_ratio needs Q (emission rate). Example: solve('enveng_max_ground_concentration_ratio', Q
Maximum ground-level concentration with constant sigma ratio: Q = 100 g/s, u = 5 m/s, H = 100 m, k = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Maximum ground-level concentration and its distance (Briggs coefficients),

`enveng_max_ground_concentration_briggs`

The distance of the largest ground-level centreline concentration of an elevated source and that concentration, found exactly on Briggs' dispersion curves for the stability class (no constant-ratio shortcut).

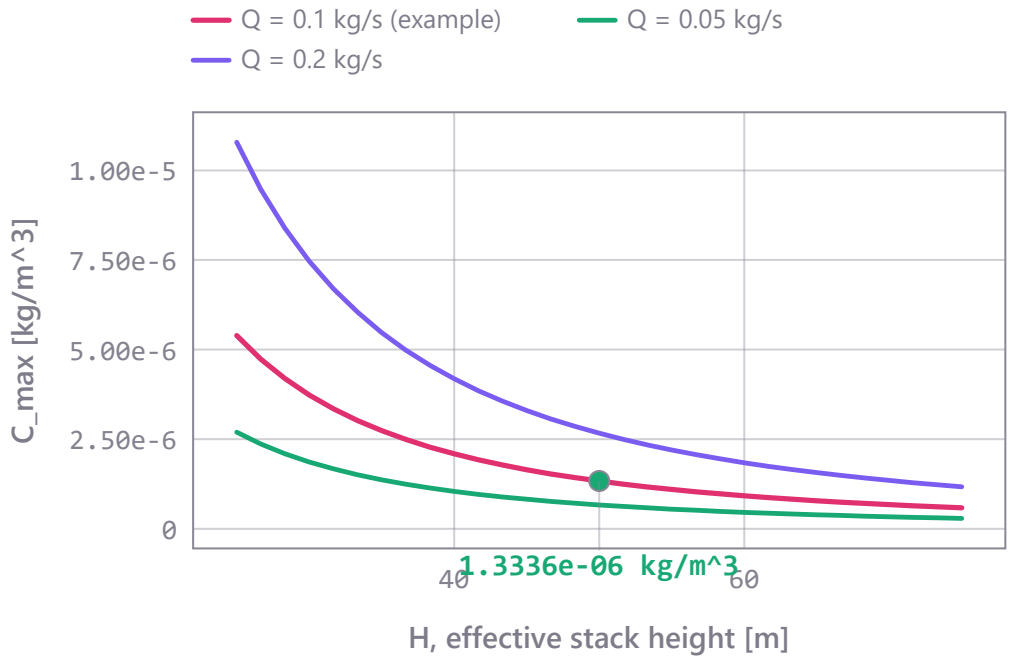
$$\frac{d\ln C}{dx} = -\frac{\sigma_y'}{\sigma_y} + \frac{\sigma_z'}{\sigma_z}\left(\frac{H^2}{\sigma_z^2} - 1\right) = 0$$

input	meaning	condition
`Q`	emission rate	
`u`	mean wind speed	
`H`	effective stack height	
`cls`	Pasquill stability class A-F	
`terrain`	rural or urban dispersion coefficients	

output	meaning
`x_max`	downwind distance of the maximum
`C_max`	maximum ground-level concentration

output	meaning
`C_max_ugm3`	maximum concentration in micrograms per cubic metre
`sigma_y`	sigma_y at the maximum
`sigma_z`	sigma_z at the maximum

- **Method:** root of d ln $C/dx = -sy'/sy + (sz'/sz)(H^2/sz^2 - 1)$ by a log-spaced scan and bisection
- **Assumes:** the maximum lies between 100 m and 10 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C_max (maximum ground-level concentration) against H (effective stack height) from 25 m to 75 m. One curve for each value of Q (emission rate): 0.1 kg/s, 0.05 kg/s, 0.2 kg/s. Other inputs as in the worked example: u = 5.0 m/s, cls = c, terrain = rural. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, H = 50 m, C_max = 1.3336e-06 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_max_ground_concentration_briggs', Q=0.1, u=5.0,
H=50.0, cls='c', terrain='rural')
# -> x_max = 459.393 m, C_max = 1.33357e-06 kg/m^3, C_max_ugm3 = 1333.57, sigma_y = 49.411 m,
sigma_z = 35.1712 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_max_ground_concentration_briggs', Q=0.3, u=4.0,
H=80.0, cls='d', terrain='urban')
# -> x_max = 431.272 m, C_max = 2.44679e-06 kg/m^3, C_max_ugm3 = 2446.79, sigma_y = 63.7254 m,
sigma_z = 56.8144 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum ground-level concentration with Briggs coefficients: Q = 100 g/s, u = 5 m/s, H = 50 m, cls = c, terrain = rural	SOLVED	x_max = 459.393 m, C_max = 1.33357e-06 kg/m^3, C_max_ugm3 = 1333.57, sigma_y = 49.411 m, sigma_z = 35.1712 m

question	outcome	result
Maximum ground-level concentration with Briggs coefficients: Q = 300 g/s, u = 4 m/s, H = 80 m, cls = d, terrain = urban	SOLVED	x_max = 431.272 m, C_max = 2.44679e-06 kg/m^3, C_max_ugm3 = 2446.79, sigma_y = 63.7254 m, sigma_z = 56.8144 m
Maximum ground-level concentration with Briggs coefficients: u = 5 m/s, H = 50 m, cls = c, terrain = rural	NEEDS_INPUT	enveng_max_ground_concentration_briggs needs Q (emission rate). Example: solve('enveng_max_ground_concentration_briggs',
Maximum ground-level concentration with Briggs coefficients: Q = 100 g/s, u = 5 m/s, H = 50 m, cls = c, terrain = rural, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Briggs plume rise and effective stack height (neutral or unstable air),

`enveng_briggs_plume_rise`

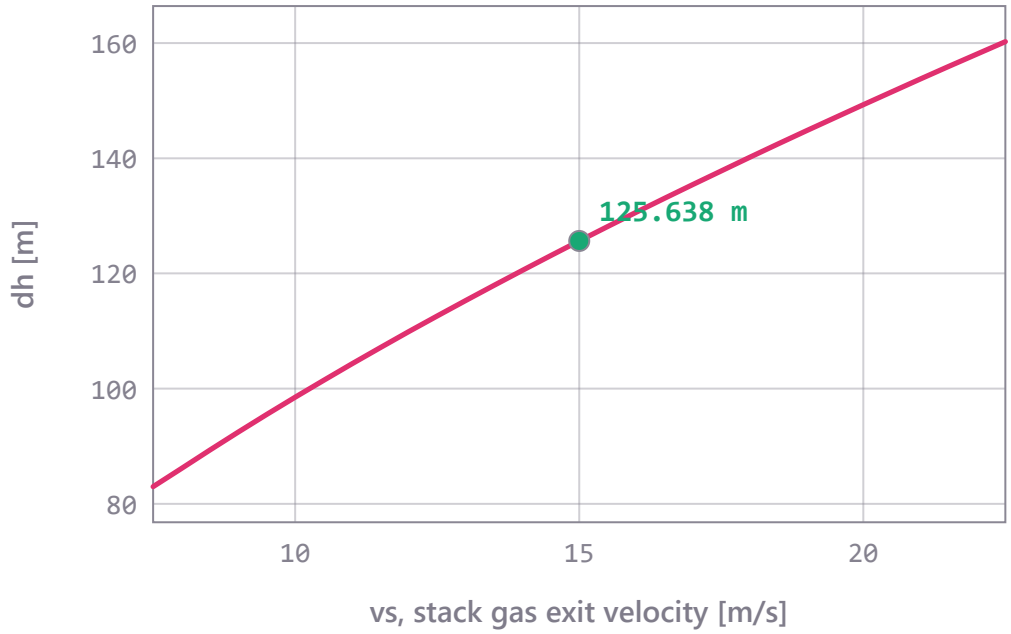
The buoyancy flux, the final plume rise by Briggs' $2/3$ law (neutral or unstable air) and the effective stack height.

$$F = \frac{g v_s d^2 (T_s - T_a)}{4 T_s}, \quad \Delta h = \frac{1.6 F^{1/3} x_f^{2/3}}{u}$$

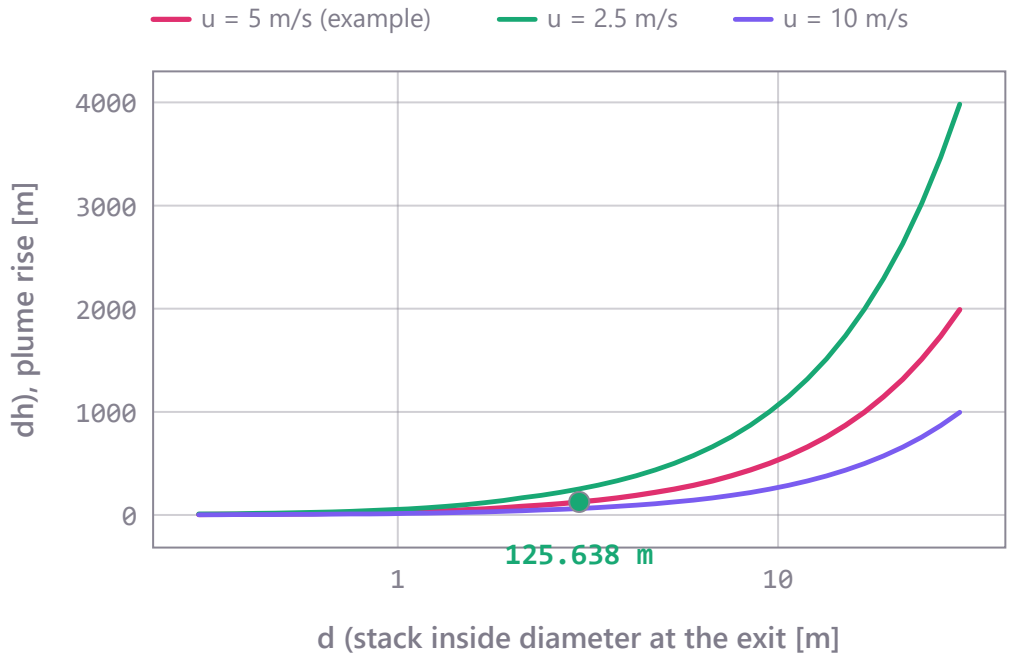
input	meaning	condition
`vs`	stack gas exit velocity	
`d`	stack inside diameter at the exit	
`Ts`	stack gas temperature	
`Ta`	ambient air temperature	
`u`	wind speed at the stack top	
`hs`	physical stack height	

output	meaning
`F`	buoyancy flux, m^4/s^3
`x_f`	distance to final rise
`dh`	plume rise
`H`	effective stack height hs + dh

- **Method:** $F = g$ vs d^2 (Ts - Ta)/(4 Ts); $x_f = 49\ F^{5/8}\ (F < 55)$ or 119 $F^{2/5}$; $dh = 1.6\ F^{1/3}\ x_f^{2/3}/u$
- **Assumes:** $Ts > Ta$; neutral or unstable air (classes A-D); buoyancy-dominated plume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dh (plume rise) against vs (stack gas exit velocity), with Ta = 288 K, Ts = 420 K, d = 3 m, hs = 100 m, u = 5 m/s. 31 points computed by environmental-engineering-ai999x; marked: vs = 15, dh = 125.638 m.



dh (plume rise) against d (stack inside diameter at the exit) from 0.3 m to 30 m on a logarithmic axis. One curve for each value of u (wind speed at the stack top): 5 m/s, 2.5 m/s, 10 m/s. Other inputs as in the worked example: vs = 15.0 m/s, Ts = 420.0 K, Ta = 288.0 K, hs = 100.0 m. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, d = 3 m, dh = 125.638 m.

```
environmental_engineering_ai999x.solve('enveng_briggs_plume_rise', vs=15.0, d=3.0, Ts=420.0,
Ta=288.0, u=5.0, hs=100.0)
# -> F = 104.021, x_f = 762.772 m, dh = 125.638 m, H = 225.638 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_briggs_plume_rise', vs=8.0, d=1.0, Ts=400.0,
Ta=293.15, u=3.0, hs=30.0)
# -> F = 5.2392, x_f = 137.955 m, dh = 24.7312 m, H = 54.7312 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Briggs plume rise: vs = 15 m/s, d = 3 m, Ts = 420 K, Ta = 288 K, u = 5 m/s, hs = 100 m	SOLVED	F = 104.021, x_f = 762.772 m, dh = 125.638 m, H = 225.638 m
Briggs plume rise: vs = 8 m/s, d = 1 m, Ts = 400 K, Ta = 293.15 K, u = 3 m/s, hs = 30 m	SOLVED	F = 5.2392, x_f = 137.955 m, dh = 24.7312 m, H = 54.7312 m
Briggs plume rise: d = 3 m, Ts = 420 K, Ta = 288 K, u = 5 m/s, hs = 100 m	NEEDS_INPUT	enveng_briggs_plume_rise needs vs (stack gas exit velocity). Example: solve('enveng_briggs_plume_rise', vs='15 m/s', d='
Briggs plume rise: vs = 15 m/s, d = 3 m, Ts = 420 K, Ta = 288 K, u = 5 m/s, hs = 100 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Briggs plume rise: vs = 15 m/s, d = 3 m, Ts = 146.85 degC, Ta = 14.85 degC, u = 5 m/s, hs = 100 m	SOLVED	F = 104.021, x_f = 762.772 m, dh = 125.638 m, H = 225.638 m

Briggs plume rise in stable air, `enveng_briggs_stable_plume_rise`

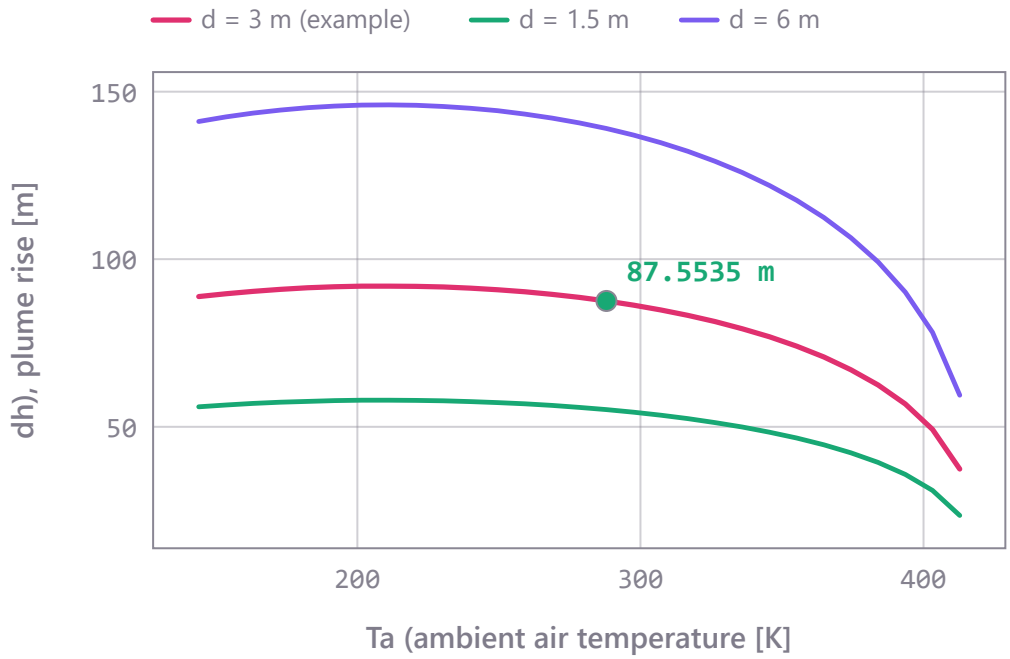
The final rise of a buoyant plume in stable air (classes E, F) from the buoyancy flux, wind speed and potential-temperature gradient, and the effective stack height.

$$\Delta h = 2.6\left(\frac{F}{us}\right)^{1/3}, \quad s = \frac{g}{T_a}\frac{\partial\theta}{\partial z}$$

input	meaning	condition
`vs`	stack gas exit velocity	
`d`	stack inside diameter at the exit	
`Ts`	stack gas temperature	
`Ta`	ambient air temperature	
`u`	wind speed at the stack top	
`dtheta`	potential temperature gradient d(theta)/dz	
`hs`	physical stack height	

output	meaning
`F`	buoyancy flux, m^4/s^3
`s`	stability parameter g/Ta dtheta/dz, $1/s^2$
`dh`	plume rise
`H`	effective stack height

- **Method:** $s = (g/Ta)$ dtheta/dz; $dh = 2.6$ (F/(u s))^$(1/3)$
- **Assumes:** $Ts > Ta$; stable air: dtheta/dz > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dh (plume rise) against Ta (ambient air temperature) from 144 K to 432 K. One curve for each value of d (stack inside diameter at the exit): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: vs = 15.0 m/s, Ts = 420.0 K, u = 4.0 m/s, dtheta = 0.02 K/m, hs = 100.0 m. 29 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Ta = 288 K, dh = 87.5535 m.

```
environmental_engineering_ai999x.solve('enveng_briggs_stable_plume_rise', vs=15.0, d=3.0, Ts=420.0,
Ta=288.0, u=4.0, dtheta=0.02, hs=100.0)
# -> F = 104.021, s = 0.000681017, dh = 87.5535 m, H = 187.554 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_briggs_stable_plume_rise', vs=10.0, d=1.5, Ts=450.0,
Ta=275.0, u=2.5, dtheta=0.035, hs=60.0)
# -> F = 21.452, s = 0.00124812, dh = 49.4384 m, H = 109.438 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Briggs stable plume rise: vs = 15 m/s, d = 3 m, Ts = 420 K, Ta = 288 K, u = 4 m/s, dtheta = 0.02 K/m, hs = 100 m	SOLVED	F = 104.021, s = 0.000681017, dh = 87.5535 m, H = 187.554 m
Briggs stable plume rise: vs = 10 m/s, d = 1.5 m, Ts = 450 K, Ta = 275 K, u = 2.5 m/s, dtheta = 0.035 K/m, hs = 60 m	SOLVED	F = 21.452, s = 0.00124812, dh = 49.4384 m, H = 109.438 m
Briggs stable plume rise: d = 3 m, Ts = 420 K, Ta = 288 K, u = 4 m/s, dtheta = 0.02 K/m, hs = 100 m	NEEDS_INPUT	enveng_briggs_stable_plume_rise needs vs (stack gas exit velocity). Example: solve('enveng_briggs_stable_plume_rise', vs
Briggs stable plume rise: vs = 15 m/s, d = 3 m, Ts = 420 K, Ta = 288 K, u = 4 m/s, dtheta = 0.02 K/m, hs = 100 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Holland plume rise, `enveng_holland_plume_rise`

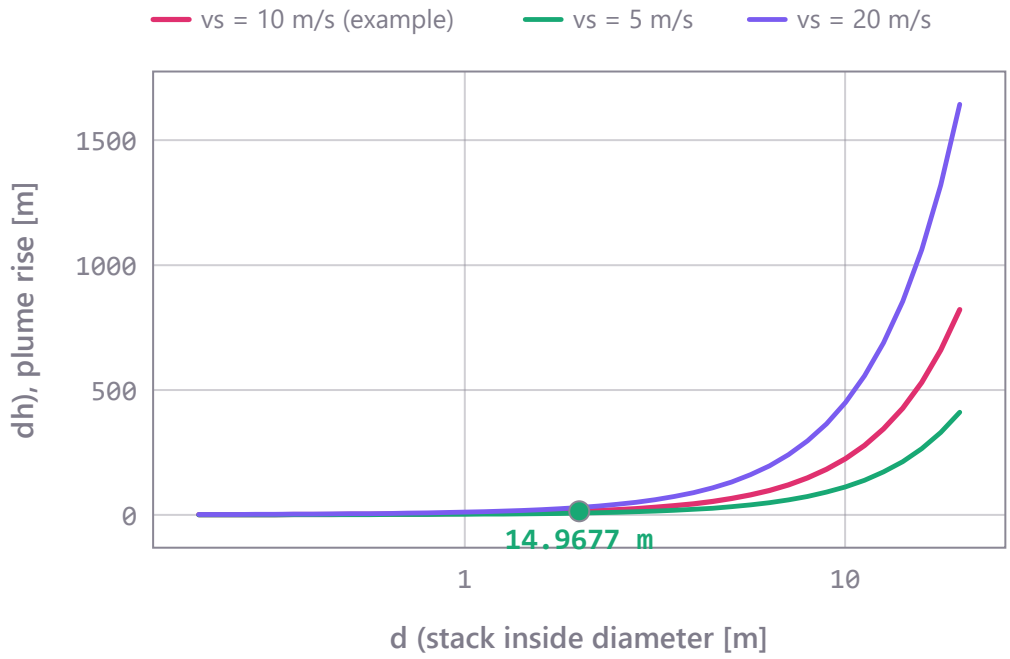
Plume rise by Holland's formula (momentum term plus a buoyancy term with the pressure in millibars) and the effective stack height.

$$\Delta h = \frac{v_s d}{u}\left(1.5 + 2.68 \times 10^{-3}\, p\, d\, \frac{T_s - T_a}{T_s}\right)$$

input	meaning	condition
`vs`	stack gas exit velocity	
`d`	stack inside diameter	
`u`	wind speed at the stack top	
`p`	atmospheric pressure	
`Ts`	stack gas temperature	
`Ta`	ambient air temperature	
`hs`	physical stack height	

output	meaning
`dh`	plume rise
`H`	effective stack height

- **Method:** $dh = vs$ d/u (1.5 + 2.68e-3 p[mbar] d (Ts - Ta)/Ts)
- **Assumes:** $Ts > Ta$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dh (plume rise) against d (stack inside diameter) from 0.2 m to 20 m on a logarithmic axis. One curve for each value of vs (stack gas exit velocity): 10 m/s, 5 m/s, 20 m/s. Other inputs as in the worked example: u = 4.0 m/s, p = 101325.0 Pa, Ts = 400.0 K, Ta = 290.0 K, hs = 50.0 m. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, d = 2 m, dh = 14.9677 m.

```
environmental_engineering_ai999x.solve('enveng_holland_plume_rise', vs=10.0, d=2.0, u=4.0,
p=101325.0, Ts=400.0, Ta=290.0, hs=50.0)
# -> dh = 14.9677 m, H = 64.9677 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_holland_plume_rise', vs=20.0, d=4.0, u=6.0,
p=95000.0, Ts=450.0, Ta=280.0, hs=150.0)
# -> dh = 71.2972 m, H = 221.297 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Holland plume rise: vs = 10 m/s, d = 2 m, u = 4 m/s, p = 101.325 kPa, Ts = 400 K, Ta = 290 K, hs = 50 m	SOLVED	dh = 14.9677 m, H = 64.9677 m
Holland plume rise: vs = 20 m/s, d = 4 m, u = 6 m/s, p = 95 kPa, Ts = 450 K, Ta = 280 K, hs = 150 m	SOLVED	dh = 71.2972 m, H = 221.297 m
Holland plume rise: d = 2 m, u = 4 m/s, p = 101.325 kPa, Ts = 400 K, Ta = 290 K, hs = 50 m	NEEDS_INPUT	enveng_holland_plume_rise needs vs (stack gas exit velocity). Example: solve('enveng_holland_plume_rise', vs='10 m/s', d
Holland plume rise: vs = 10 m/s, d = 2 m, u = 4 m/s, p = 101.325 kPa, Ts = 400 K, Ta = 290 K, hs = 50 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Wind speed at height (power law), `enveng_wind_power_law`

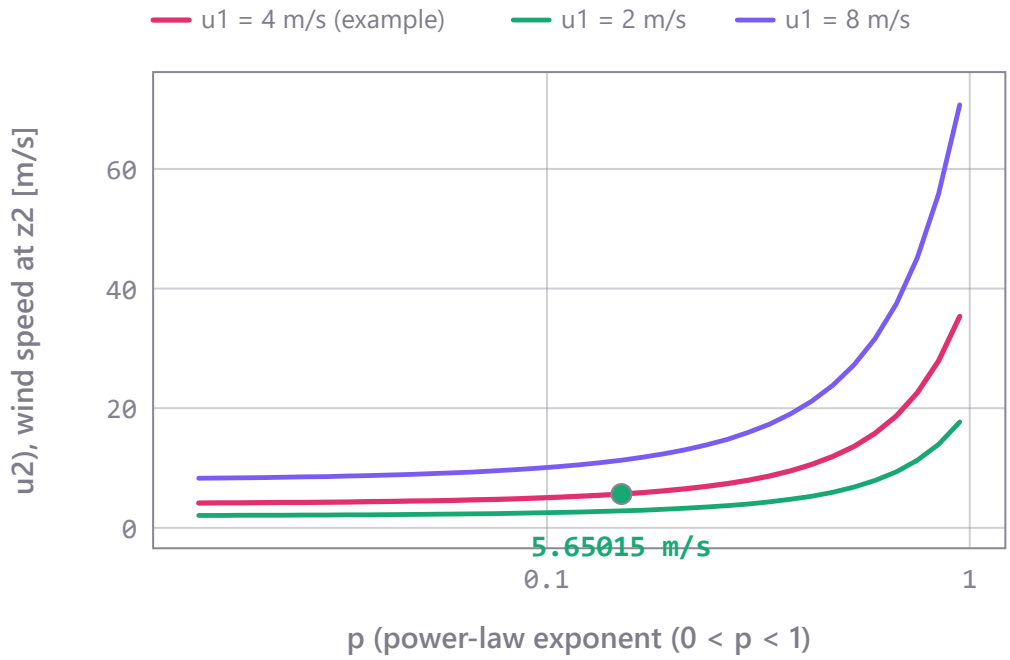
The wind speed at another height from a measured speed with the power-law profile $u_2 = u_1\,(z_2/z_1)^p$ (p from the stability class and terrain, e.g. 0.15 rural neutral).

$$u_2 = u_1\left(\frac{z_2}{z_1}\right)^p$$

input	meaning	condition
`u1`	measured wind speed	
`z1`	measurement height	
`z2`	height of interest (e.g. the stack top)	
`p`	power-law exponent $(0 < p < 1)$	

output	meaning
`u2`	wind speed at z2

- **Method:** $u_2 = u_1\,(z_2/z_1)^p$
- **Assumes:** $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



u2 (wind speed at z2) against p (power-law exponent (0 < p < 1)) from 0.015 to 1.5 on a logarithmic axis. One curve for each value of u1 (measured wind speed): 4 m/s, 2 m/s, 8 m/s. Other inputs as in the worked example: z1 = 10.0 m, z2 = 100.0 m. 37 points per curve computed by environmental-engineering-ai999x; marked: the worked example, p = 0.15, u2 = 5.65015 m/s.

```
environmental_engineering_ai999x.solve('enveng_wind_power_law', u1=4.0, z1=10.0, z2=100.0, p=0.15)
# -> u2 = 5.65015 m/s   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_wind_power_law', u1=2.5, z1=10.0, z2=60.0, p=0.35)
# -> u2 = 4.68051 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind speed at height: u1 = 4 m/s, z1 = 10 m, z2 = 100 m, p = 0.15	SOLVED	u2 = 5.65015 m/s
Wind speed at height: u1 = 2.5 m/s, z1 = 10 m, z2 = 60 m, p = 0.35	SOLVED	u2 = 4.68051 m/s
Wind speed at height: z1 = 10 m, z2 = 100 m, p = 0.15	NEEDS_INPUT	enveng_wind_power_law needs u1 (measured wind speed). Example: solve('enveng_wind_power_law', u1='4 m/s', z1='10 m', z2=
Wind speed at height: u1 = 4 m/s, z1 = 10 m, z2 = 100 m, p = 0.15, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Gas concentration: ppm (by volume) to mass concentration, `enveng_ppm_to_mass_concentration`

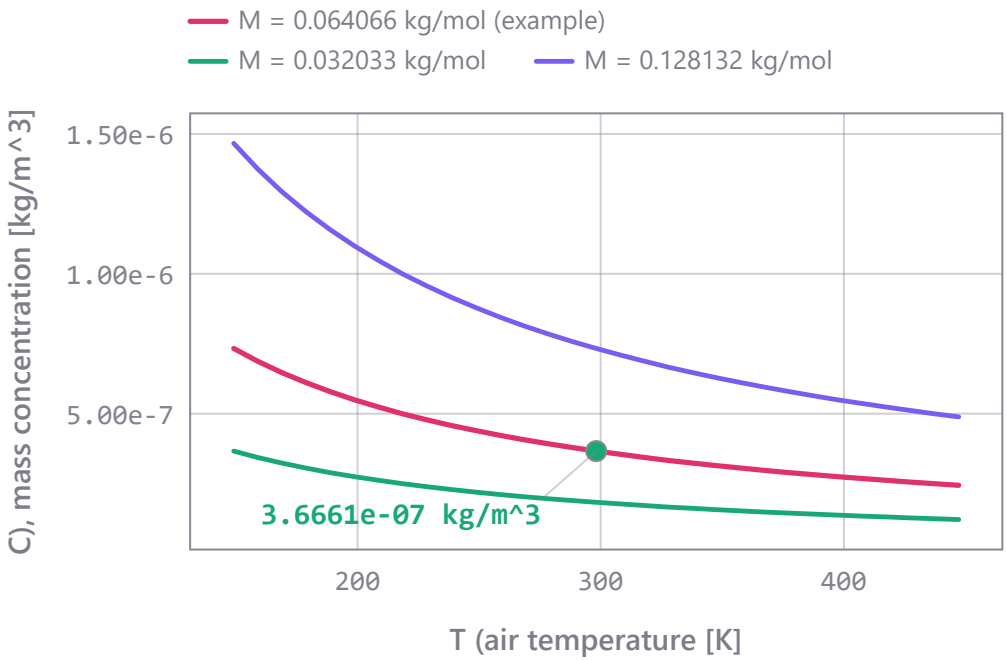
The mass concentration of a gaseous pollutant from its volume mixing ratio in ppm, its molar mass and the air temperature and pressure (ideal gas).

$$C = \mathrm{ppm} \times 10^{-6} \, \frac{PM}{RT}$$

input	meaning	condition
`ppm`	volume mixing ratio in parts per million	
`M`	molar mass of the pollutant	
`T`	air temperature	
`P`	air pressure	

output	meaning
`C`	mass concentration
`C_ugm3`	in micrograms per cubic metre
`C_mgm3`	in milligrams per cubic metre

- **Method:** C = ppm $1e-6$ P M/(R T)
- **Assumes:** ideal gas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (mass concentration) against T (air temperature) from 149.075 K to 447.225 K. One curve for each value of M (molar mass of the pollutant): 0.064066 kg/mol, 0.032033 kg/mol, 0.128132 kg/mol. Other inputs as in the worked example: ppm = 0.14, P = 101325.0 Pa. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, T = 298.15 K, C = 3.6661e-07 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_ppm_to_mass_concentration', ppm=0.14, M=0.064066,
T=298.15, P=101325.0)
# -> C = 3.66609e-07 kg/m^3, C_ugm3 = 366.609, C_mgm3 = 0.366609   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_ppm_to_mass_concentration', ppm=9.0, M=0.02801,
T=293.15, P=95000.0)
# -> C = 9.82551e-06 kg/m^3, C_ugm3 = 9825.51, C_mgm3 = 9.82551   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert ppm to mass concentration: ppm = 0.14 ppm, M = 64.066 g/mol, T = 298.15 K, P = 101.325 kPa	SOLVED	C = 3.66609e-07 kg/m^3, C_ugm3 = 366.609, C_mgm3 = 0.366609
Convert ppm to mass concentration: ppm = 9 ppm, M = 28.01 g/mol, T = 293.15 K, P = 95 kPa	SOLVED	C = 9.82551e-06 kg/m^3, C_ugm3 = 9825.51, C_mgm3 = 9.82551
Convert ppm to mass concentration: M = 64.066 g/mol, T = 298.15 K, P = 101.325 kPa	NEEDS_INPUT	enveng_ppm_to_mass_concentration needs ppm (volume mixing ratio in parts per million). Example: solve('enveng_ppm_to_mas
Convert ppm to mass concentration: ppm = 0.14 ppm, M = 64.066 g/mol, T = 298.15 K, P = 101.325 kPa, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Convert ppm to mass concentration: ppm = 0.14 ppm, M = 64.066 g/mol, T = 25 degC, P = 1 atm	SOLVED	C = 3.66609e-07 kg/m^3, C_ugm3 = 366.609, C_mgm3 = 0.366609

Gas concentration: mass concentration to ppm (by volume), `enveng_mass_concentration_to_ppm`

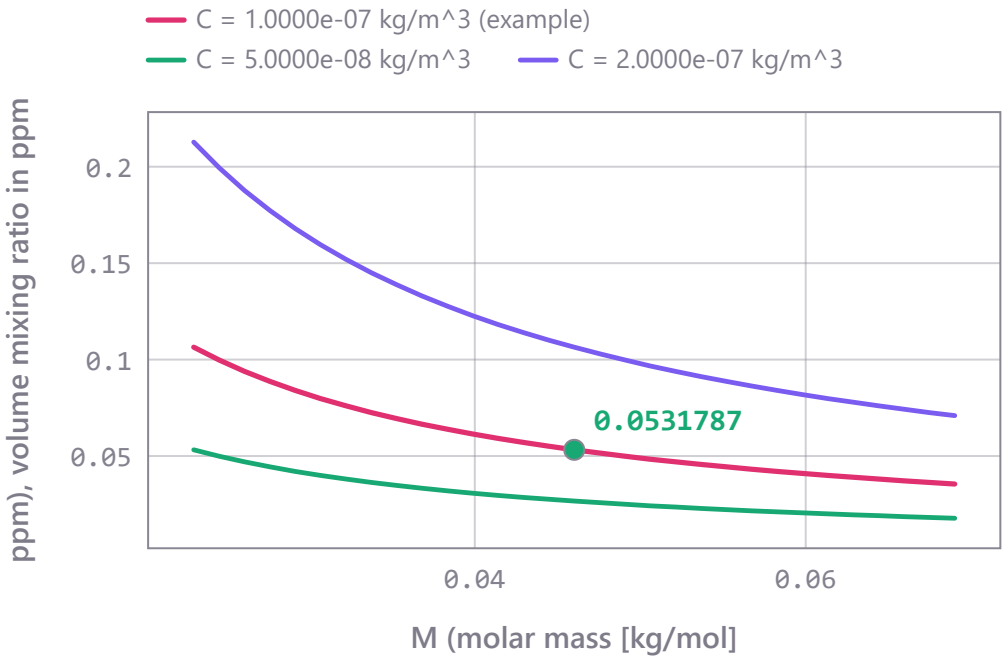
The volume mixing ratio (ppm and ppb) of a gaseous pollutant from its mass concentration, molar mass, temperature and pressure (ideal gas).

$$\mathrm{ppm} = \frac{CRT}{PM} \times 10^6$$

input	meaning	condition
`C`	mass concentration	
`M`	molar mass	
`T`	air temperature	
`P`	air pressure	

output	meaning
`ppm`	volume mixing ratio in ppm
`ppb`	volume mixing ratio in ppb

- **Method:** ppm = C R T/(P M) 1e6
- **Assumes:** ideal gas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ppm (volume mixing ratio in ppm) against M (molar mass) from 0.023003 kg/mol to 0.069009 kg/mol. One curve for each value of C (mass concentration): 1.0000e-07 kg/m^3, 5.0000e-08 kg/m^3, 2.0000e-07 kg/m^3. Other inputs as in the worked example: T = 298.15 K, P = 101325.0 Pa. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, M = 0.046006 kg/mol, ppm = 0.0531787.

```
environmental_engineering_ai999x.solve('enveng_mass_concentration_to_ppm', C=1e-07, M=0.046006,
T=298.15, P=101325.0)
# -> ppm = 0.0531787, ppb = 53.1787   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_mass_concentration_to_ppm', C=1e-05, M=0.02801,
T=288.15, P=90000.0)
# -> ppm = 9.5038, ppb = 9503.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert mass concentration to ppm: C = 100 ug/m^3, M = 46.006 g/mol, T = 298.15 K, P = 101.325 kPa	SOLVED	ppm = 0.0531787, ppb = 53.1787
Convert mass concentration to ppm: C = 10000 ug/m^3, M = 28.01 g/mol, T = 288.15 K, P = 90 kPa	SOLVED	ppm = 9.5038, ppb = 9503.8
Convert mass concentration to ppm: M = 46.006 g/mol, T = 298.15 K, P = 101.325 kPa	NEEDS_INPUT	enveng_mass_concentration_to_ppm needs C (mass concentration). Example: solve('enveng_mass_concentration_to_ppm', C='1e-
Convert mass concentration to ppm: C = 100 ug/m^3, M = 46.006 g/mol, T = 298.15 K, P = 101.325 kPa, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Box model of an air basin (well-mixed, time-dependent), `enveng_box_model`

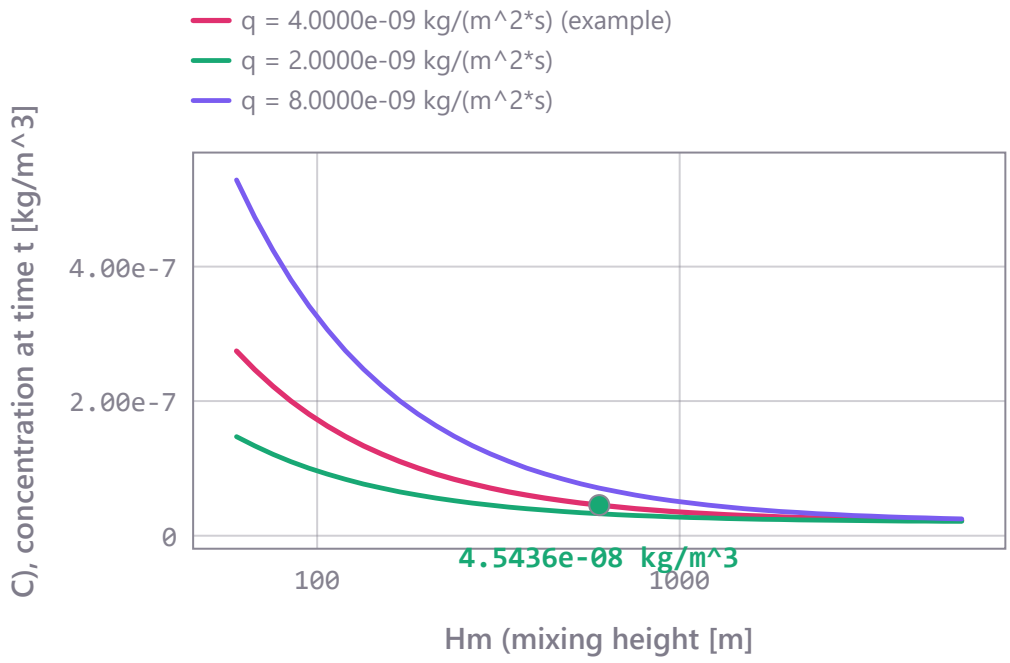
The concentration over a city treated as a well-mixed box: emission flux per area, wind through a box of length L under a mixing height, background air; the concentration after time t and at steady state.

$$C(t) = C_{ss} + (C_0 - C_{ss})e^{-ut/L}, \quad C_{ss} = C_b + \frac{qL}{uH_m}$$

input	meaning	condition
`q`	emission rate per unit ground area	
`L`	box length along the wind	
`u`	wind speed	
`Hm`	mixing height	
`Cb`	background (upwind) concentration	
`C0`	concentration at $t = 0$	
`t`	elapsed time	

output	meaning
`C`	concentration at time t
`C_ss`	steady-state concentration
`C_ugm3`	concentration at t in micrograms per cubic metre

- **Method:** $dC/dt = q/Hm - u$ (C - Cb)/L, so C = Css + (C0 - Css) e^(-u t/L) with Css = Cb + q L/(u Hm)
- **Assumes:** well-mixed box; constant wind, mixing height and emissions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration at time t) against Hm (mixing height) from 60 m to 6000 m on a logarithmic axis. One curve for each value of q (emission rate per unit ground area): 4.0000e-09 kg/(m^2*s), 2.0000e-09 kg/(m^2*s), 8.0000e-09 kg/(m^2*s). Other inputs as in the worked example: L = 15000.0 m, u = 3.0 m/s, Cb = 2e-08 kg/m^3, C0 = 2e-08 kg/m^3, t = 7200.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Hm = 600 m, C = 4.5436e-08 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_box_model', q=4e-09, L=15000.0, u=3.0, Hm=600.0,
Cb=2e-08, C0=2e-08, t=7200.0)
# -> C = 4.54357e-08 kg/m^3, C_ss = 5.33333e-08 kg/m^3, C_ugm3 = 45.4357   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_box_model', q=1e-08, L=20000.0, u=2.0, Hm=400.0,
Cb=1e-08, C0=5e-08, t=5400.0)
# -> C = 1.37623e-07 kg/m^3, C_ss = 2.6e-07 kg/m^3, C_ugm3 = 137.623   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Air basin box model: q = 4 ug/(m^2*s), L = 15000 m, u = 3 m/s, Hm = 600 m, Cb = 20 ug/m^3, C0 = 20 ug/m^3, t = 2 h	SOLVED	C = 4.54357e-08 kg/m^3, C_ss = 5.33333e-08 kg/m^3, C_ugm3 = 45.4357
Air basin box model: q = 10 ug/(m^2*s), L = 20000 m, u = 2 m/s, Hm = 400 m, Cb = 10 ug/m^3, C0 = 50 ug/m^3, t = 1.5 h	SOLVED	C = 1.37623e-07 kg/m^3, C_ss = 2.6e-07 kg/m^3, C_ugm3 = 137.623
Air basin box model: L = 15000 m, u = 3 m/s, Hm = 600 m, Cb = 20 ug/m^3, C0 = 20 ug/m^3, t = 2 h	NEEDS_INPUT	enveng_box_model needs q (emission rate per unit ground area). Example: solve('enveng_box_model', q='4e-09 kg/(m^2*s)',
Air basin box model: q = 4 ug/(m^2*s), L = 15000 m, u = 3 m/s, Hm = 600 m, Cb = 20 ug/m^3, C0 = 20 ug/m^3, t = 2 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Gravity settling chamber collection efficiency, `enveng_settling_chamber_efficiency`

The Stokes settling velocity of a particle in a gas and the collection efficiency of a horizontal-flow settling chamber with N trays (laminar plug-flow model).

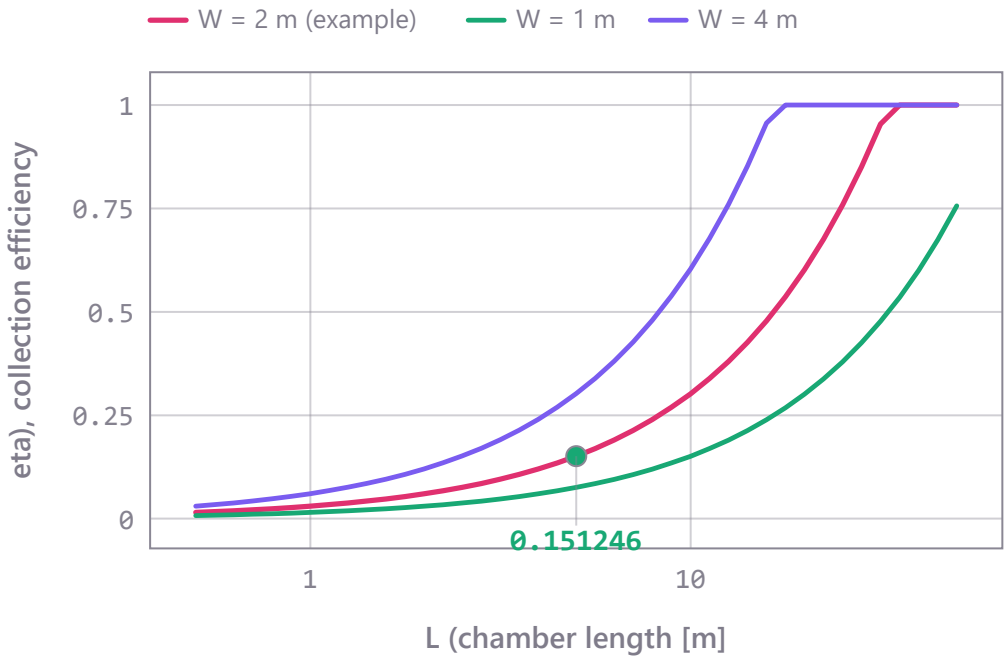
$$\eta = \frac{v_t L W N}{Q}, \quad v_t = \frac{g(\rho_p - \rho_g) d_p^2}{18\mu}$$

input	meaning	condition
`dp`	particle diameter	
`rho_p`	particle density	
`rho_g`	gas density	
`mu`	gas viscosity	
`L`	chamber length	
`W`	chamber width	
`Q`	gas flow rate	
`N`	number of trays (1 for a single chamber)	

output	meaning
`vt`	Stokes terminal settling velocity

output	meaning
`eta`	collection efficiency (fraction, at most 1)
`Re`	particle Reynolds number

- **Method:** $vt = g$ (rho_p - rho_g) dp^2/(18 mu); eta = min(1, vt L W N/Q)
- **Assumes:** Stokes regime $Re < 1$; laminar plug flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eta (collection efficiency (fraction, at most 1)) against L (chamber length) from 0.5 m to 50 m on a logarithmic axis. One curve for each value of W (chamber width): 2 m, 1 m, 4 m. Other inputs as in the worked example: dp = 5e-05 m, rho_p = 2000.0 kg/m^3, rho_g = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, Q = 10.0 m^3/s, N = 1. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, L = 5 m, eta = 0.151246.

```
environmental_engineering_ai999x.solve('enveng_settling_chamber_efficiency', dp=5e-05,
rho_p=2000.0, rho_g=1.2, mu=1.8e-05, L=5.0, W=2.0, Q=10.0, N=1)
# -> vt = 0.151246 m/s, eta = 0.151246, Re = 0.504155   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_settling_chamber_efficiency', dp=3e-05,
rho_p=2500.0, rho_g=1.2, mu=1.85e-05, L=6.0, W=3.0, Q=20.0, N=2)
# -> vt = 0.0662293 m/s, eta = 0.119213, Re = 0.128879   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Settling chamber efficiency: dp = 50 um, rho_p = 2000 kg/m^3, rho_g = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, L = 5 m, W = 2 m, Q = 10 m^3/s, N = 1	SOLVED	vt = 0.151246 m/s, eta = 0.151246, Re = 0.504155
Settling chamber efficiency: dp = 30 um, rho_p = 2500 kg/m^3, rho_g = 1.2 kg/m^3, mu = 1.85e-05 Pa*s, L = 6 m, W = 3 m, Q = 20 m^3/s, N = 2	SOLVED	vt = 0.0662293 m/s, eta = 0.119213, Re = 0.128879

question	outcome	result
Settling chamber efficiency: rho_p = 2000 kg/m^3, rho_g = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, L = 5 m, W = 2 m, Q = 10 m^3/s, N = 1	NEEDS_INPUT	enveng_settling_chamber_efficiency needs dp (particle diameter). Example: solve('enveng_settling_chamber_efficiency', dp
Settling chamber efficiency: dp = 50 um, rho_p = 2000 kg/m^3, rho_g = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, L = 5 m, W = 2 m, Q = 10 m^3/s, N = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Electrostatic precipitator efficiency (Deutsch-Anderson), `enveng_esp_efficiency`

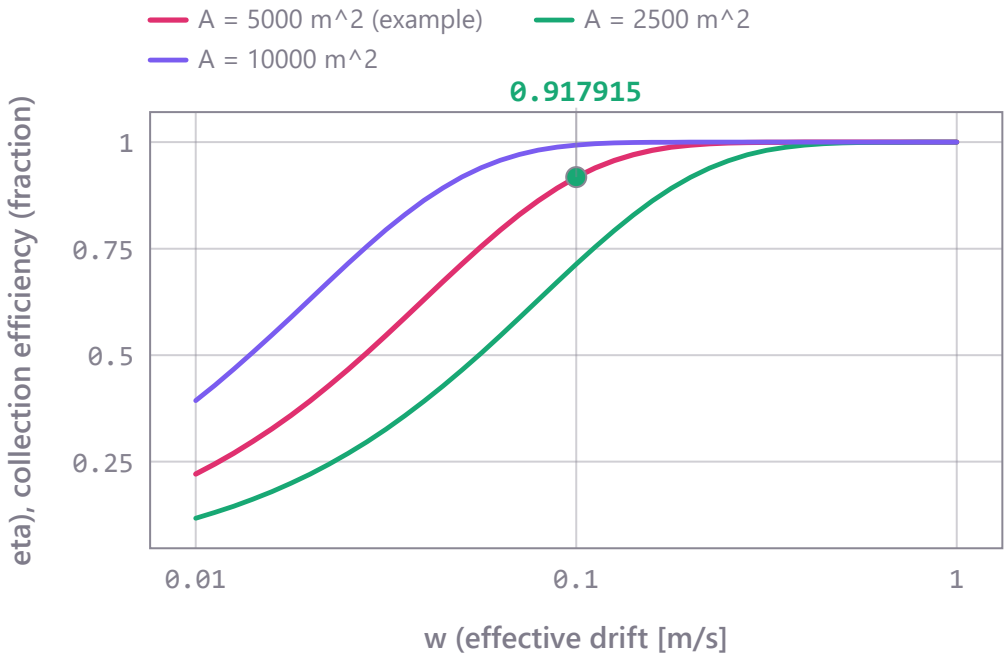
The collection efficiency and penetration of an electrostatic precipitator from the drift velocity, collecting area and gas flow.

$$\eta = 1 - e^{-wA/Q}$$

input	meaning	condition
`w`	effective drift (migration) velocity	
`A`	collecting plate area	
`Q`	gas flow rate	

output	meaning
`eta`	collection efficiency (fraction)
`penetration`	fraction passing $(1 - \eta)$

- **Method:** $\eta = 1 - exp$(-w A/Q)
- **Assumes:** w, A, $Q > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eta (collection efficiency (fraction)) against w (effective drift (migration) velocity) from 0.01 m/s to 1 m/s on a logarithmic axis. One curve for each value of A (collecting plate area): 5000 m^2, 2500 m^2, 10000 m^2. Other inputs as in the worked example: Q = 200.0 m^3/s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, w = 0.1 m/s, eta = 0.917915.

```
environmental_engineering_ai999x.solve('enveng_esp_efficiency', w=0.1, A=5000.0, Q=200.0)
# -> eta = 0.917915, penetration = 0.082085   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_esp_efficiency', w=0.06, A=2000.0, Q=100.0)
# -> eta = 0.698806, penetration = 0.301194   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Electrostatic precipitator efficiency: w = 0.1 m/s, A = 5000 m^2, Q = 200 m^3/s	SOLVED	eta = 0.917915, penetration = 0.082085
Electrostatic precipitator efficiency: w = 0.06 m/s, A = 2000 m^2, Q = 100 m^3/s	SOLVED	eta = 0.698806, penetration = 0.301194
Electrostatic precipitator efficiency: A = 5000 m^2, Q = 200 m^3/s	NEEDS_INPUT	enveng_esp_efficiency needs w (effective drift (migration) velocity). Example: solve('enveng_esp_efficiency', w='0.1 m/s
Electrostatic precipitator efficiency: w = 0.1 m/s, A = 5000 m^2, Q = 200 m^3/s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Electrostatic precipitator plate area for a target efficiency, `enveng_esp_area`

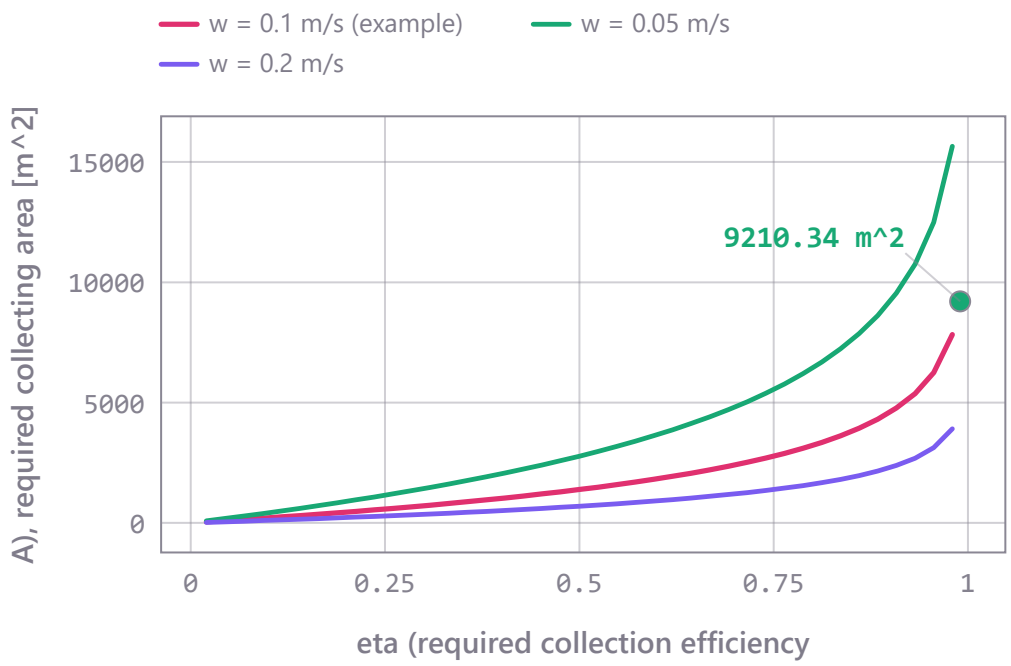
The collecting plate area an electrostatic precipitator needs to reach a stated efficiency (Deutsch-Anderson).

$$A = -\frac{Q}{w}\ln(1-\eta)$$

input	meaning	condition
`w`	effective drift velocity	
`Q`	gas flow rate	
`eta`	required collection efficiency (fraction or %)	

output	meaning
`A`	required collecting area

- **Method:** $A = -Q$ ln(1 - eta)/w
- **Assumes:** $0 < \eta < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (required collecting area) against eta (required collection efficiency (fraction or %)) from 0.02 to 0.98. One curve for each value of w (effective drift velocity): 0.1 m/s, 0.05 m/s, 0.2 m/s. Other inputs as in the worked example: Q = 200.0 m^3/s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, eta = 0.99, A = 9210.34 m^2.

```
environmental_engineering_ai999x.solve('enveng_esp_area', w=0.1, Q=200.0, eta=0.99)
# -> A = 9210.34 m^2   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_esp_area', w=0.08, Q=150.0, eta=0.95)
# -> A = 5617 m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Electrostatic precipitator area: w = 0.1 m/s, Q = 200 m^3/s, eta = 0.99	SOLVED	A = 9210.34 m^2
Electrostatic precipitator area: w = 0.08 m/s, Q = 150 m^3/s, eta = 0.95	SOLVED	A = 5617 m^2

question	outcome	result
Electrostatic precipitator area: Q = 200 m^3/s, eta = 0.99	NEEDS_INPUT	enveng_esp_area needs w (effective drift velocity). Example: solve('enveng_esp_area', w='0.1 m/s', Q='200 m^3/s', eta=0.
Electrostatic precipitator area: w = 0.1 m/s, Q = 200 m^3/s, eta = 0.99, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
ESP plate area: w = 0.1 m/s, Q = 200 m^3/s, eta = 99%	SOLVED	A = 9210.34 m^2

Cyclone cut diameter and fractional efficiency (Lapple), `enveng_cyclone_lapple`

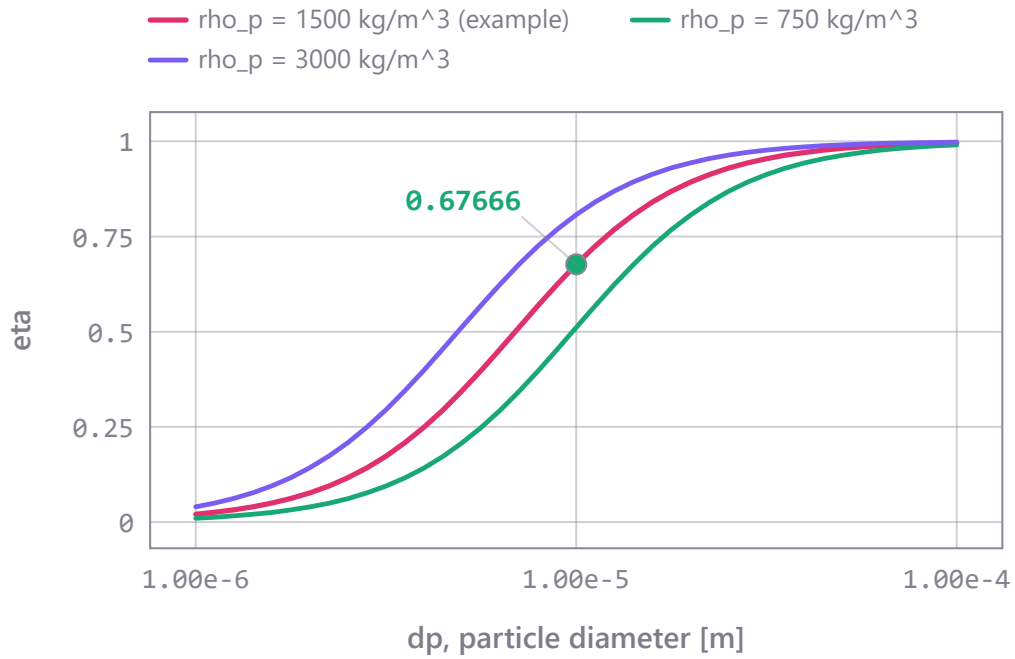
The cut diameter (50 % efficiency) of a cyclone from the gas viscosity, inlet width, effective number of turns, inlet velocity and densities, and the collection efficiency of a particle size by Lapple's curve.

$$d_{50} = \sqrt{\frac{9\mu W}{2\pi N_e V_i(\rho_p - \rho_g)}}, \quad \eta = \frac{1}{1 + (d_{50}/d_p)^2}$$

input	meaning	condition
`mu`	gas viscosity	
`W`	inlet width	
`Ne`	effective number of turns	
`Vi`	inlet gas velocity	
`rho_p`	particle density	
`rho_g`	gas density	
`dp`	particle diameter	

output	meaning
`d50`	cut diameter
`eta`	collection efficiency of particles of diameter dp

- **Method:** $d50 = sqrt$(9 mu W/(2 pi Ne Vi (rho_p - rho_g))); $\eta = 1/(1 + (d50/dp)^2)$
- **Assumes:** $rho_p > rho_g$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



eta (collection efficiency of particles of diameter dp) against dp (particle diameter) from 1.0000e-06 m to 0.0001 m on a logarithmic axis. One curve for each value of rho_p (particle density): 1500 kg/m^3, 750 kg/m^3, 3000 kg/m^3. Other inputs as in the worked example: mu = 1.8e-05 Pa*s, W = 0.25 m, Ne = 6.0, Vi = 15.0 m/s, rho_g = 1.2 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, dp = 1.0000e-05 m, eta = 0.67666.

```
environmental_engineering_ai999x.solve('enveng_cyclone_lapple', mu=1.8e-05, W=0.25, Ne=6.0,
Vi=15.0, rho_p=1500.0, rho_g=1.2, dp=1e-05)
# -> d50 = 6.91265e-06 m, eta = 0.67666   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_cyclone_lapple', mu=1.85e-05, W=0.4, Ne=5.0,
Vi=20.0, rho_p=2400.0, rho_g=1.1, dp=5e-06)
# -> d50 = 6.64724e-06 m, eta = 0.361346   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cyclone efficiency Lapple: mu = 1.8e-05 Pa*s, W = 0.25 m, Ne = 6, Vi = 15 m/s, rho_p = 1500 kg/m^3, rho_g = 1.2 kg/m^3, dp = 10 um	SOLVED	d50 = 6.91265e-06 m, eta = 0.67666
Cyclone efficiency Lapple: mu = 1.85e-05 Pa*s, W = 0.4 m, Ne = 5, Vi = 20 m/s, rho_p = 2400 kg/m^3, rho_g = 1.1 kg/m^3, dp = 5 um	SOLVED	d50 = 6.64724e-06 m, eta = 0.361346
Cyclone efficiency Lapple: W = 0.25 m, Ne = 6, Vi = 15 m/s, rho_p = 1500 kg/m^3, rho_g = 1.2 kg/m^3, dp = 10 um	NEEDS_INPUT	enveng_cyclone_lapple needs mu (gas viscosity). Example: solve('enveng_cyclone_lapple', mu='1.8e-05 Pa*s', W='0.25 m', N
Cyclone efficiency Lapple: mu = 1.8e-05 Pa*s, W = 0.25 m, Ne = 6, Vi = 15 m/s, rho_p = 1500 kg/m^3, rho_g = 1.2 kg/m^3, dp = 10 um, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sulfur dioxide emission from fuel sulfur, `enveng_so2_emission`

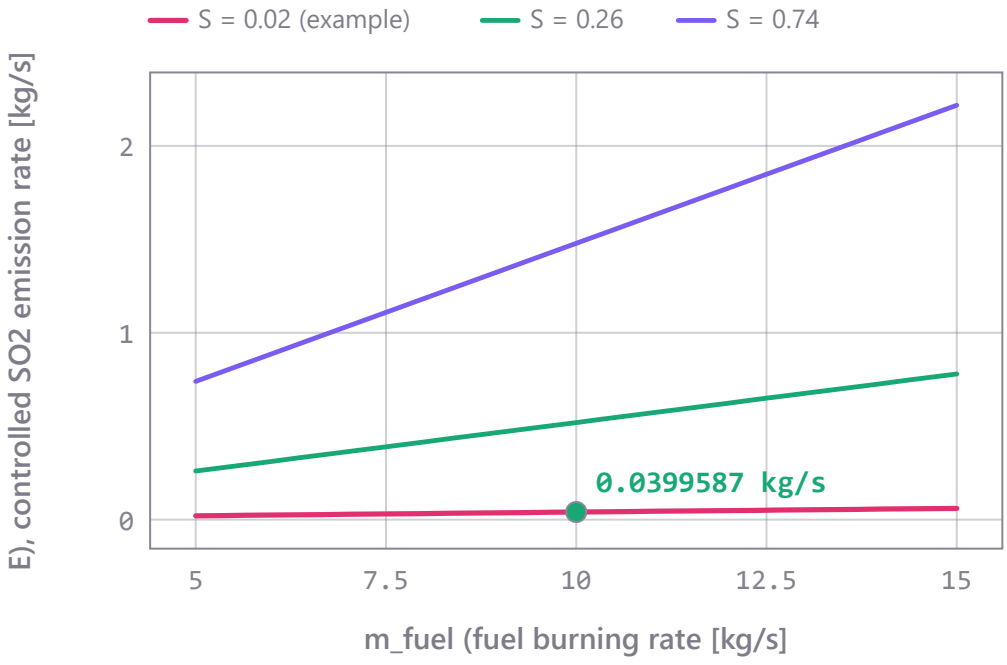
The SO2 emission rate from a fuel burning rate and its sulfur mass fraction (all sulfur to SO2), before and after a control device.

$$E = \dot{m}_{fuel}\, S\, \frac{M_{SO_2}}{M_S}(1-\eta)$$

input	meaning	condition
`m_fuel`	fuel burning rate	
`S`	sulfur mass fraction of the fuel (fraction or %)	
`eta`	SO2 control (removal) efficiency (fraction or %; 0 if uncontrolled)	

output	meaning
`E`	controlled SO2 emission rate
`E_uncontrolled`	uncontrolled SO2 emission rate
`E_kg_per_day`	controlled emission in kg per day

- **Method:** E = m_fuel S $(M_SO2/M_S)(1-\eta)$
- **Assumes:** all fuel sulfur oxidised to SO2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



E (controlled SO2 emission rate) against m_fuel (fuel burning rate) from 5 kg/s to 15 kg/s. One curve for each value of S (sulfur mass fraction of the fuel (fraction or %)): 0.02, 0.26, 0.74. Other inputs as in the worked example: eta = 0.9. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, m_fuel = 10 kg/s, E = 0.0399587 kg/s.

```
environmental_engineering_ai999x.solve('enveng_so2_emission', m_fuel=10.0, S=0.02, eta=0.9)
# -> E = 0.0399587 kg/s, E_uncontrolled = 0.399587 kg/s, E_kg_per_day = 3452.43   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_so2_emission', m_fuel=2.5, S=0.008, eta=0.0)
# -> E = 0.0399587 kg/s, E_uncontrolled = 0.0399587 kg/s, E_kg_per_day = 3452.43   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SO2 emission from fuel sulfur: m_fuel = 36 t/h, S = 0.02, eta = 0.9	SOLVED	E = 0.0399587 kg/s, E_uncontrolled = 0.399587 kg/s, E_kg_per_day = 3452.43
SO2 emission from fuel sulfur: m_fuel = 9 t/h, S = 0.008, eta = 0	SOLVED	E = 0.0399587 kg/s, E_uncontrolled = 0.0399587 kg/s, E_kg_per_day = 3452.43
SO2 emission from fuel sulfur: S = 0.02, eta = 0.9	NEEDS_INPUT	enveng_so2_emission needs m_fuel (fuel burning rate). Example: solve('enveng_so2_emission', m_fuel='10 kg/s', S=0.02, et
SO2 emission from fuel sulfur: m_fuel = 36 t/h, S = 0.02, eta = 0.9, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
SO2 emission from fuel sulfur: m_fuel = 36 t/h, S = 2%, eta = 90%	SOLVED	E = 0.0399587 kg/s, E_uncontrolled = 0.399587 kg/s, E_kg_per_day = 3452.43

Gaussian plume trapped below an inversion lid (multiple reflections), `enveng_plume_mixing_lid`

The concentration of a plume reflected both at the ground and at the base of an elevated inversion (mixing height), by the image-source series, with the fully mixed limit for comparison.

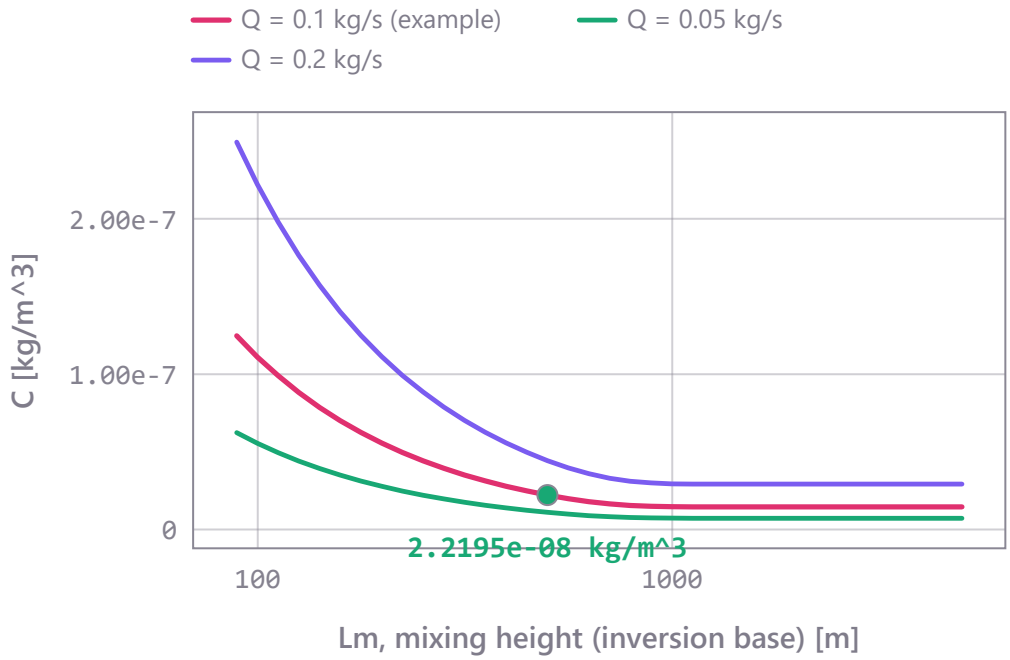
$$C = \frac{Q e^{-y^2/2\sigma_y^2}}{2\pi u \sigma_y \sigma_z} \sum_{n=-\infty}^{\infty} \left[e^{-\frac{(z-H+2nL)^2}{2\sigma_z^2}} + e^{-\frac{(z+H+2nL)^2}{2\sigma_z^2}} \right]$$

input	meaning	condition
`Q`	emission rate	
`u`	mean wind speed	
`sigma_y`	horizontal dispersion coefficient	
`sigma_z`	vertical dispersion coefficient	
`H`	effective stack height	
`Lm`	mixing height (inversion base)	
`y`	crosswind distance	
`z`	receptor height	

output	meaning
`C`	concentration with ground and lid reflections
`C_ugm3`	in micrograms per cubic metre
`C_mixed`	uniformly mixed limit Q/(sqrt(2 pi) u sy Lm) $e^{-y^2/2sy^2}$

- **Method:** image sources at z = +−$H + 2$ n Lm summed until the terms vanish
- **Assumes:** H, $z \leq Lm$; $sigma_z \leq 10$ Lm

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration with ground and lid reflections) against Lm (mixing height (inversion base)) from 50 m to 5000 m on a logarithmic axis. One curve for each value of Q (emission rate): 0.1 kg/s, 0.05 kg/s, 0.2 kg/s. Other inputs as in the worked example: u = 4.0 m/s, sigma_y = 900.0 m, sigma_z = 600.0 m, H = 80.0 m, y = 0.0 m, z = 0.0 m. 36 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Lm = 500 m, C = 2.2195e-08 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_plume_mixing_lid', Q=0.1, u=4.0, sigma_y=900.0,
sigma_z=600.0, H=80.0, Lm=500.0, y=0.0, z=0.0)
# -> C = 2.21953e-08 kg/m^3, C_ugm3 = 22.1953, C_mixed = 2.21635e-08 kg/m^3   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_plume_mixing_lid', Q=0.2, u=3.0, sigma_y=400.0,
sigma_z=250.0, H=120.0, Lm=800.0, y=100.0, z=10.0)
# -> C = 1.83185e-07 kg/m^3, C_ugm3 = 183.185, C_mixed = 8.05559e-08 kg/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plume below an inversion lid: Q = 100 g/s, u = 4 m/s, sigma_y = 900 m, sigma_z = 600 m, H = 80 m, Lm = 500 m, y = 0 m, z = 0 m	SOLVED	C = 2.21953e-08 kg/m^3, C_ugm3 = 22.1953, C_mixed = 2.21635e-08 kg/m^3
Plume below an inversion lid: Q = 200 g/s, u = 3 m/s, sigma_y = 400 m, sigma_z = 250 m, H = 120 m, Lm = 800 m, y = 100 m, z = 10 m	SOLVED	C = 1.83185e-07 kg/m^3, C_ugm3 = 183.185, C_mixed = 8.05559e-08 kg/m^3
Plume below an inversion lid: u = 4 m/s, sigma_y = 900 m, sigma_z = 600 m, H = 80 m, Lm = 500 m, y = 0 m, z = 0 m	NEEDS_INPUT	enveng_plume_mixing_lid needs Q (emission rate). Example: solve('enveng_plume_mixing_lid', Q='0.1 kg/s', u='4 m/s', sigm
Plume below an inversion lid: Q = 100 g/s, u = 4 m/s, sigma_y = 900 m,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

question	outcome	result
sigma_z = 600 m, H = 80 m, Lm = 500 m, y = 0 m, z = 0 m, zz = 1		

Stack plume: effective height, dispersion and ground-level concentration,

`enveng_stack_plume_ground_concentration`

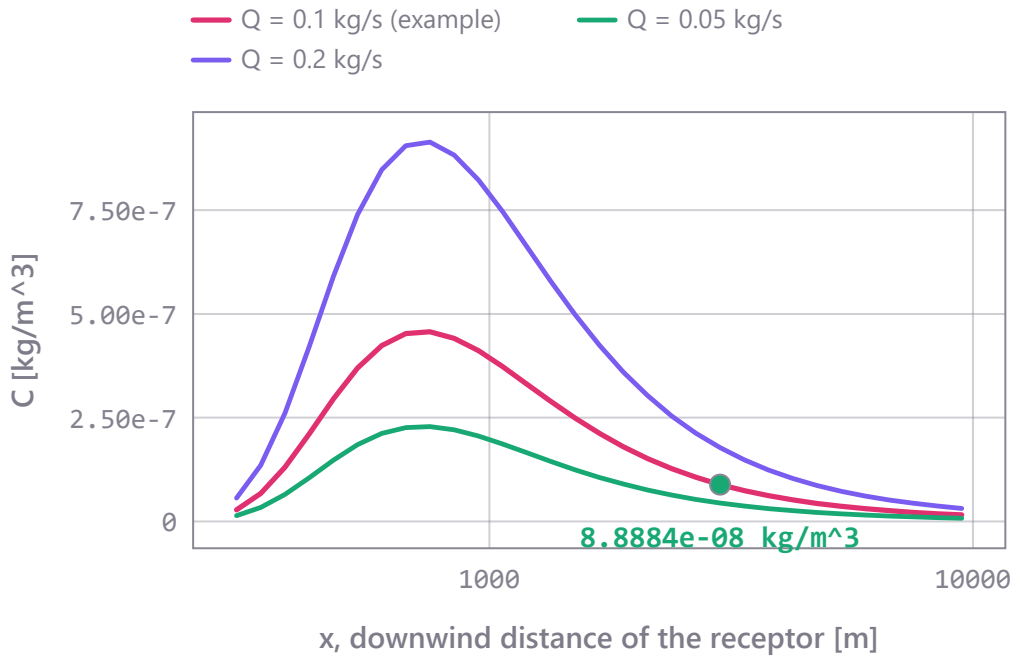
The whole chain for a buoyant stack in neutral or unstable air: Briggs plume rise to the effective height, Briggs dispersion coefficients at the receptor distance, then the ground-level centreline concentration.

$$C = \frac{Q}{\pi u \sigma_y \sigma_z} \exp\left(-\frac{(h_s + \Delta h)^2}{2\sigma_z^2}\right)$$

input	meaning	condition
`Q`	emission rate	
`u`	wind speed at the stack top	
`vs`	stack gas exit velocity	
`d`	stack inside diameter at the exit	
`Ts`	stack gas temperature	
`Ta`	ambient air temperature	
`hs`	physical stack height	
`x`	downwind distance of the receptor	
`cls`	Pasquill stability class A-D	
`terrain`	rural or urban dispersion coefficients	

output	meaning
`H`	effective stack height
`sigma_y`	sigma_y at x
`sigma_z`	sigma_z at x
`C`	ground-level centreline concentration at x
`C_ugm3`	in micrograms per cubic metre

- **Method:** Briggs final rise (2/3 law) -> H; Briggs sigma_y, sigma_z at x; C = Q/(pi u sy sz) exp(-H^2/2sz^2)
- **Assumes:** $Ts > Ta$; class A-D; 100 $m \leq x \leq 10$ km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (ground-level centreline concentration at x) against x (downwind distance of the receptor) from 300 m to 30000 m on a logarithmic axis. One curve for each value of Q (emission rate): 0.1 kg/s, 0.05 kg/s, 0.2 kg/s. Other inputs as in the worked example: u = 6.0 m/s, vs = 10.0 m/s, d = 1.5 m, Ts = 400.0 K, Ta = 290.0 K, hs = 50.0 m, cls = c, terrain = rural. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, x = 3000 m, C = 8.8884e-08 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_stack_plume_ground_concentration', Q=0.1, u=6.0,
vs=10.0, d=1.5, Ts=400.0, Ta=290.0, hs=50.0, x=3000.0, cls='c', terrain='rural')
# -> H = 77.4472 m, sigma_y = 289.429 m, sigma_z = 189.737 m, C = 8.88845e-08 kg/m^3, C_ugm3 =
88.8845   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_stack_plume_ground_concentration', Q=0.05, u=4.0,
vs=8.0, d=1.0, Ts=380.0, Ta=285.0, hs=30.0, x=2000.0, cls='d', terrain='urban')
# -> H = 47.6492 m, sigma_y = 238.514 m, sigma_z = 221.359 m, C = 7.36354e-08 kg/m^3, C_ugm3 =
73.6354   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stack plume ground-level concentration: Q = 100 g/s, u = 6 m/s, vs = 10 m/s, d = 1.5 m, Ts = 400 K, Ta = 290 K, hs = 50 m, x = 3000 m, cls = c, terrain = rural	SOLVED	H = 77.4472 m, sigma_y = 289.429 m, sigma_z = 189.737 m, C = 8.88845e-08 kg/m^3, C_ugm3 = 88.8845
Stack plume ground-level concentration: Q = 50 g/s, u = 4 m/s, vs = 8 m/s, d = 1 m, Ts = 380 K, Ta = 285 K, hs = 30 m, x = 2000 m, cls = d, terrain = urban	SOLVED	H = 47.6492 m, sigma_y = 238.514 m, sigma_z = 221.359 m, C = 7.36354e-08 kg/m^3, C_ugm3 = 73.6354
Stack plume ground-level concentration: u = 6 m/s, vs = 10 m/s, d = 1.5 m, Ts = 400 K, Ta = 290 K, hs = 50 m, x = 3000 m, cls = c, terrain = rural	NEEDS_INPUT	enveng_stack_plume_ground_concentration needs Q (emission rate). Example: solve('enveng_stack_plume_ground_concentration

question	outcome	result
Stack plume ground-level concentration: Q = 100 g/s, u = 6 m/s, vs = 10 m/s, d = 1.5 m, Ts = 400 K, Ta = 290 K, hs = 50 m, x = 3000 m, cls = c, terrain = rural, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

BOD exerted after a time (first-order), `enveng_bod_exerted`

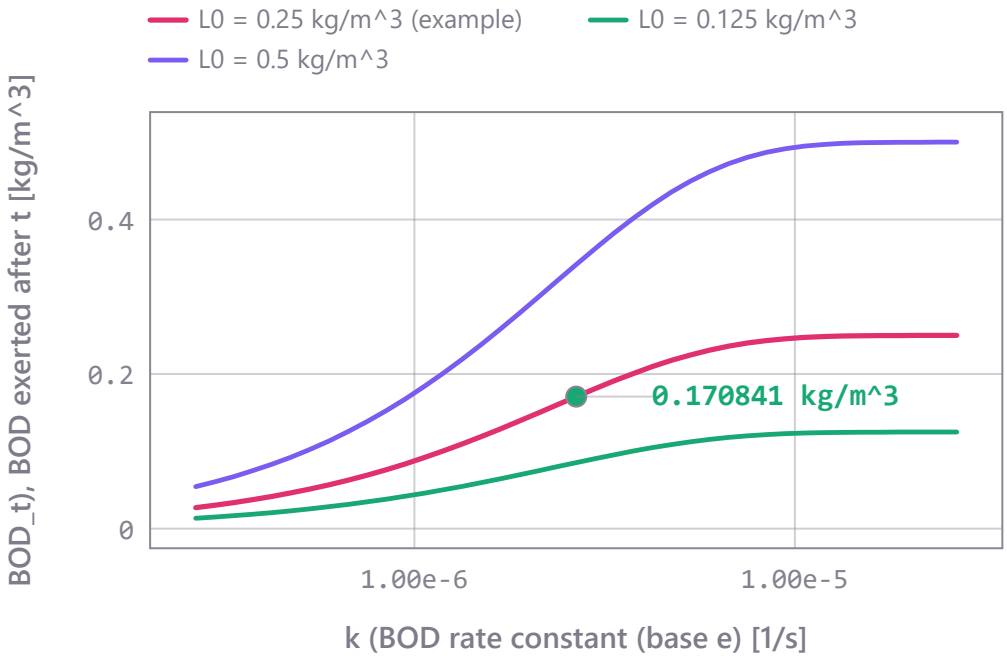
The BOD exerted after time t and the oxygen demand remaining, from the ultimate BOD and the first-order rate constant.

$$BOD_t = L_0\left(1 - e^{-kt}\right)$$

input	meaning	condition
`L0`	ultimate (carbonaceous) BOD	
`k`	BOD rate constant (base e)	
`t`	incubation or travel time	

output	meaning
`BOD_t`	BOD exerted after t
`L_t`	BOD remaining after t
`BOD_t_mgL`	BOD exerted in mg/L

- **Method:** BOD_t = L0 (1 - e^(-k t)), $L_t = L_0$ e^(-k t)
- **Assumes:** first-order kinetics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



BOD_t (BOD exerted after t) against k (BOD rate constant (base e)) from 2.6620e-07 1/s to 2.6620e-05 1/s on a logarithmic axis. One curve for each value of L0 (ultimate (carbonaceous) BOD): 0.25 kg/m^3, 0.125 kg/m^3, 0.5 kg/m^3. Other inputs as in the worked example: t = 432000.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 2.6620e-06 1/s, BOD_t = 0.170841 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_bod_exerted', L0=0.25, k=2.662037037037037e-06,
t=432000.0)
# -> BOD_t = 0.170841 kg/m^3, L_t = 0.0791592 kg/m^3, BOD_t_mgL = 170.841   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_bod_exerted', L0=0.03, k=1.1574074074074074e-06,
t=259200.0)
# -> BOD_t = 0.00777545 kg/m^3, L_t = 0.0222245 kg/m^3, BOD_t_mgL = 7.77545   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BOD exerted: L0 = 250 mg/L, k = 0.23 1/d, t = 5 d	SOLVED	BOD_t = 0.170841 kg/m^3, L_t = 0.0791592 kg/m^3, BOD_t_mgL = 170.841
BOD exerted: L0 = 30 mg/L, k = 0.1 1/d, t = 3 d	SOLVED	BOD_t = 0.00777545 kg/m^3, L_t = 0.0222245 kg/m^3, BOD_t_mgL = 7.77545
BOD exerted: k = 0.23 1/d, t = 5 d	NEEDS_INPUT	enveng_bod_exerted needs L0 (ultimate (carbonaceous) BOD). Example: solve('enveng_bod_exerted', L0='0.25 kg/m^3', k='2.6
BOD exerted: L0 = 250 mg/L, k = 0.23 1/d, t = 5 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
BOD exerted: L0 = 250 mg/L, k = 0.23 1/day, t = 120 h	SOLVED	BOD_t = 0.170841 kg/m^3, L_t = 0.0791592 kg/m^3, BOD_t_mgL = 170.841

Ultimate BOD from a BOD test, `enveng_ultimate_bod`

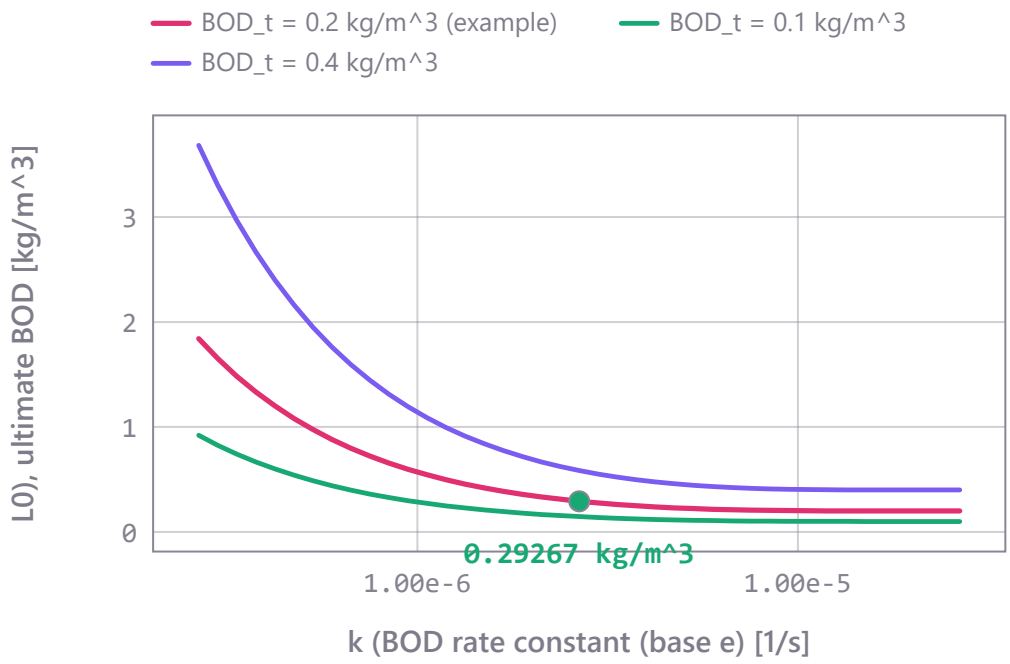
The ultimate carbonaceous BOD from a BOD measured after t days (e.g. BOD5) and the rate constant.

$$L_0 = \frac{BOD_t}{1 - e^{-kt}}$$

input	meaning	condition
`BOD_t`	BOD measured after time t (e.g. BOD5)	
`k`	BOD rate constant (base e)	
`t`	incubation time	

output	meaning
`L0`	ultimate BOD
`L0_mgL`	ultimate BOD in mg/L

- **Method:** L0 = BOD_t/(1 - e^(-k t))
- **Assumes:** first-order kinetics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L0 (ultimate BOD) against k (BOD rate constant (base e)) from 2.6620e-07 1/s to 2.6620e-05 1/s on a logarithmic axis. One curve for each value of BOD_t (BOD measured after time t (e.g. BOD5)): 0.2 kg/m^3, 0.1 kg/m^3, 0.4 kg/m^3. Other inputs as in the worked example: t = 432000.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 2.6620e-06 1/s, L0 = 0.29267 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_ultimate_bod', BOD_t=0.2, k=2.662037037037037e-06,
t=432000.0)
# -> L0 = 0.29267 kg/m^3, L0_mgL = 292.67   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_ultimate_bod', BOD_t=0.012, k=1.736111111111111e-06,
t=432000.0)
# -> L0 = 0.0227431 kg/m^3, L0_mgL = 22.7431   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ultimate BOD: BOD_t = 200 mg/L, k = 0.23 1/d, t = 5 d	SOLVED	L0 = 0.29267 kg/m^3, L0_mgL = 292.67
Ultimate BOD: BOD_t = 12 mg/L, k = 0.15 1/d, t = 5 d	SOLVED	L0 = 0.0227431 kg/m^3, L0_mgL = 22.7431
Ultimate BOD: k = 0.23 1/d, t = 5 d	NEEDS_INPUT	enveng_ultimate_bod needs BOD_t (BOD measured after time t (e.g. BOD5)). Example: solve('enveng_ultimate_bod', BOD_t='0.
Ultimate BOD: BOD_t = 200 mg/L, k = 0.23 1/d, t = 5 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Temperature correction of a rate constant (theta model), `enveng_rate_constant_temperature`

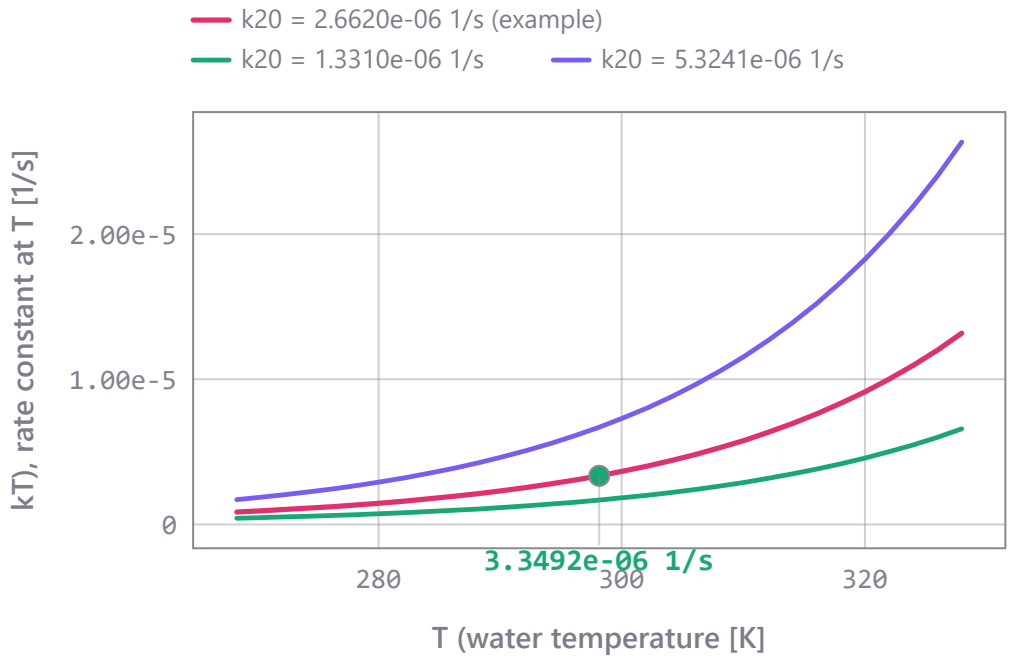
A first-order rate constant (BOD, reaeration, decay) at temperature T from its 20 degC value and the temperature coefficient theta.

$$k_T = k_{20}\,\theta^{\,T-20}$$

input	meaning	condition
`k20`	rate constant at 20 degC	
`theta`	temperature coefficient (1.047 BOD, 1.024 reaeration)	
`T`	water temperature	

output	meaning
`kT`	rate constant at T
`kT_per_day`	rate constant in $1/d$

- **Method:** $kT = k20\,\theta^{T-20}$
- **Assumes:** $0.9 < \theta < 1.2$; -5 degC <= T <= 60 degC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



kT (rate constant at T) against T (water temperature) from 268.335 K to 327.965 K. One curve for each value of k20 (rate constant at 20 degC): 2.6620e-06 1/s, 1.3310e-06 1/s, 5.3241e-06 1/s. Other inputs as in the worked example: theta = 1.047. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, T = 298.15 K, kT = 3.3492e-06 1/s.

```
environmental_engineering_ai999x.solve('enveng_rate_constant_temperature', k20=2.662037037037037e-06, theta=1.047, T=298.15)
# -> kT = 3.34925e-06 1/s, kT_per_day = 0.289375   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_rate_constant_temperature', k20=4.6296296296296296e-06, theta=1.024, T=283.15)
# -> kT = 3.65213e-06 1/s, kT_per_day = 0.315544   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rate constant temperature correction: k20 = 0.23 1/d, theta = 1.047, T = 25 degC	SOLVED	kT = 3.34925e-06 1/s, kT_per_day = 0.289375
Rate constant temperature correction: k20 = 0.4 1/d, theta = 1.024, T = 10 degC	SOLVED	kT = 3.65213e-06 1/s, kT_per_day = 0.315544
Rate constant temperature correction: theta = 1.047, T = 25 degC	NEEDS_INPUT	enveng_rate_constant_temperature needs k20 (rate constant at 20 degC). Example: solve('enveng_rate_constant_temperature'
Rate constant temperature correction: k20 = 0.23 1/d, theta = 1.047, T = 25 degC, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

BOD from a dilution test (with optional seed correction), `enveng_bod_dilution_test`

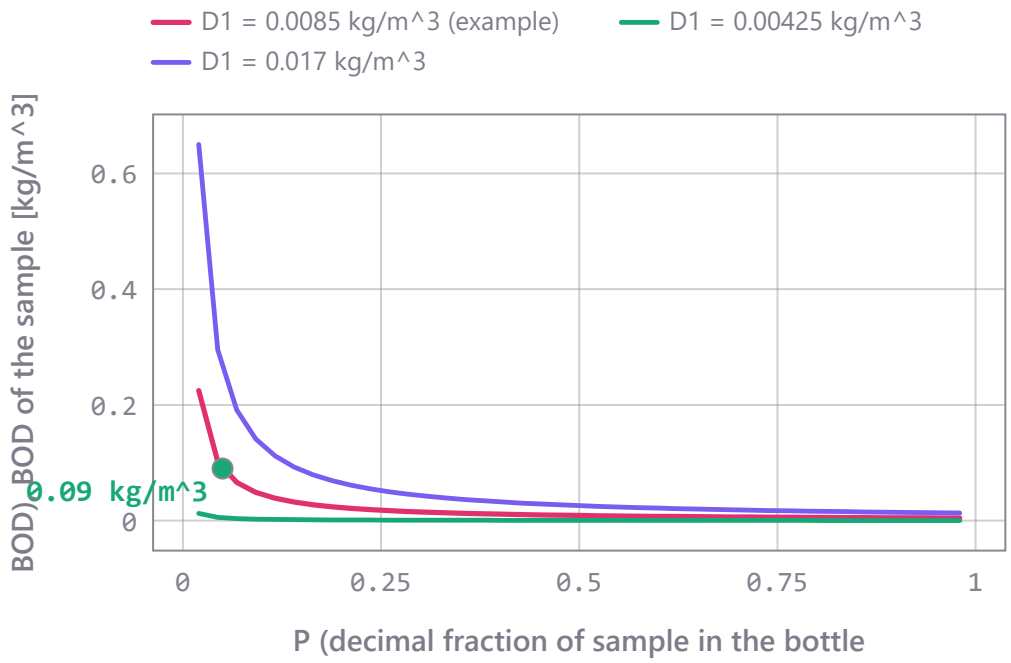
The BOD of a sample from the dissolved oxygen of the diluted sample before and after incubation and the sample fraction, with the seed correction when the dilution water is seeded.

$$BOD = \frac{(D_1 - D_2) - (B_1 - B_2)f}{P}$$

input	meaning	condition
`D1`	DO of the diluted sample immediately after preparation	
`D2`	DO of the diluted sample after incubation	
`P`	decimal fraction of sample in the bottle (sample volume / bottle volume)	
`B1`	DO of the seed control before incubation	
`B2`	DO of the seed control after incubation	
`f`	ratio of seed in the sample bottle to seed in the control	

output	meaning
`BOD`	BOD of the sample
`BOD_mgL`	BOD in mg/L

- **Method:** BOD = (($D_1 - D_2$) − ($B_1 - B_2$) f)/P (unseeded: BOD = (D1 - D2)/P)
- **Assumes:** $D_1 > D_2$; B1, B2 and f given together or not at all
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



BOD (BOD of the sample) against P (decimal fraction of sample in the bottle (sample volume / bottle volume)) from 0.02 to 0.98. One curve for each value of D1 (DO of the diluted sample immediately after preparation): 0.0085 kg/m^3, 0.00425 kg/m^3, 0.017 kg/m^3. Other inputs as in the worked example: D2 = 0.004 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, P = 0.05, BOD = 0.09 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_bod_dilution_test', D1=0.0085, D2=0.004, P=0.05)
# -> BOD = 0.09 kg/m^3, BOD_mgL = 90   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_bod_dilution_test', D1=0.0088, D2=0.0036, P=0.02,
B1=0.0089, B2=0.0083, f=0.98)
# -> BOD = 0.2306 kg/m^3, BOD_mgL = 230.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BOD dilution test: D1 = 8.5 mg/L, D2 = 4 mg/L, P = 0.05	SOLVED	BOD = 0.09 kg/m^3, BOD_mgL = 90
BOD dilution test: D1 = 8.8 mg/L, D2 = 3.6 mg/L, P = 0.02, B1 = 8.9 mg/L, B2 = 8.3 mg/L, f = 0.98	SOLVED	BOD = 0.2306 kg/m^3, BOD_mgL = 230.6
BOD dilution test: D2 = 4 mg/L, P = 0.05	NEEDS_INPUT	enveng_bod_dilution_test needs D1 (DO of the diluted sample immediately after preparation). Example: solve('enveng_bod_d
BOD dilution test: D1 = 8.5 mg/L, D2 = 4 mg/L, P = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
BOD dilution test: D1 = 8.5 mg/L, D2 = 4 mg/L, P = 5%	SOLVED	BOD = 0.09 kg/m^3, BOD_mgL = 90
BOD dilution test: D1 = 8.5 mg/L, D2 = 4 mg/L, P = 0.05, B1 = 8.9 mg/L	INVALID_INPUT	BOD from a dilution test (with optional seed correction): a seeded test needs B1, B2 and the seed fraction f together.

BOD rate constant and ultimate BOD fitted to BOD data, `enveng_bod_rate_fit`

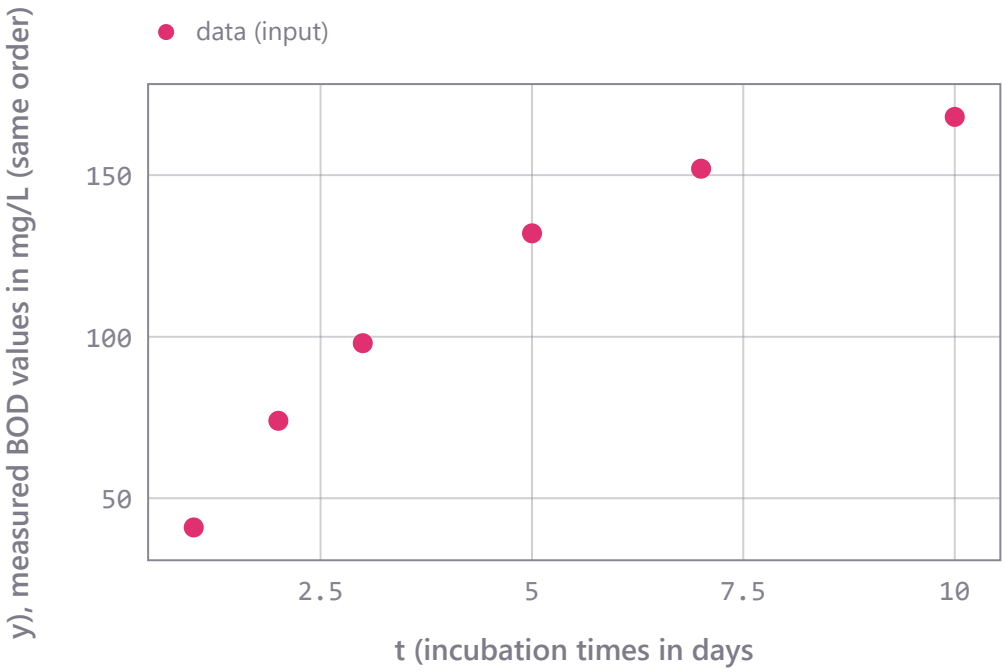
The first-order rate constant k and ultimate BOD L0 that best fit a series of BOD measurements (nonlinear least squares of BOD = L0 (1 - e^(-k t))).

$$\min_{k,L_0} \sum_i \left[y_i - L_0 \left(1 - e^{-k t_i}\right) \right]^2$$

input	meaning	condition
`t`	incubation times in days	
`y`	measured BOD values in mg/L (same order)	

output	meaning
`k`	fitted rate constant
`k_per_day`	fitted rate constant in $1/d$
`L0`	fitted ultimate BOD
`L0_mgL`	fitted ultimate BOD in mg/L
`rss`	residual sum of squares in $(mg/L)^2$

- **Method:** for each k the best L0 is sum(y f)/sum(f^2) with f = 1 - e^(-k t); k is the root of the stationarity condition sum((y - L0 f) t e^(-k t)) = 0 (scan and bisection)
- **Assumes:** at least three distinct positive times; a single rate constant between 0.001 and 20 per day fits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (measured BOD values in mg/L (same order)) against t (incubation times in days) as given in the worked example (6 points). Also returned: k = 3.0024e-06 1/s, k_per_day = 0.259411, L0 = 0.181572 kg/m^3, L0_mgL = 181.572, rss = 0.532714.

```
environmental_engineering_ai999x.solve('enveng_bod_rate_fit', t=[1.0, 2.0, 3.0, 5.0, 7.0, 10.0], y=
[41.0, 74.0, 98.0, 132.0, 152.0, 168.0])
# -> k = 3.00244e-06 1/s, k_per_day = 0.259411, L0 = 0.181572 kg/m^3, L0_mgL = 181.572, rss =
0.532714   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_bod_rate_fit', t=[2.0, 4.0, 6.0, 8.0, 10.0], y=
[11.0, 18.0, 22.0, 24.0, 26.0])
# -> k = 2.91964e-06 1/s, k_per_day = 0.252257, L0 = 0.0280754 kg/m^3, L0_mgL = 28.0754, rss =
0.201669   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BOD rate constant fit: t = [1, 2, 3, 5, 7, 10], y = [41, 74, 98, 132, 152, 168]	SOLVED	k = 3.00244e-06 1/s, k_per_day = 0.259411, L0 = 0.181572 kg/m^3, L0_mgL = 181.572, rss = 0.532714
BOD rate constant fit: t = [2, 4, 6, 8, 10], y = [11, 18, 22, 24, 26]	SOLVED	k = 2.91964e-06 1/s, k_per_day = 0.252257, L0 = 0.0280754 kg/m^3, L0_mgL = 28.0754, rss = 0.201669
BOD rate constant fit: y = [41, 74, 98, 132, 152, 168]	NEEDS_INPUT	enveng_bod_rate_fit needs t (incubation times in days). Example: solve('enveng_bod_rate_fit', t=[1.0, 2.0, 3.0, 5.0, 7.0
BOD rate constant fit: t = [1, 2, 3, 5, 7, 10], y = [41, 74, 98, 132, 152, 168], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Theoretical oxygen demand of an organic compound, `enveng_theoretical_oxygen_demand`

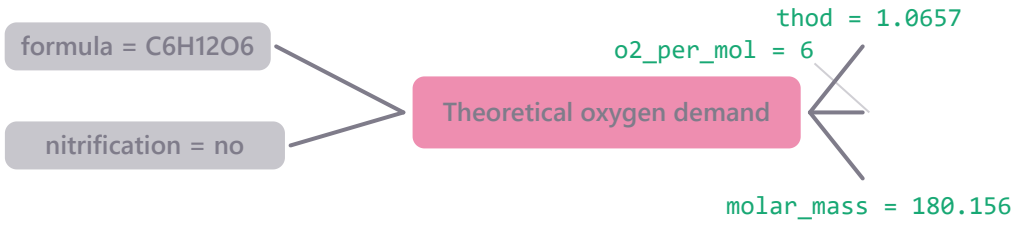
The theoretical oxygen demand (g O2 per g compound) of a compound CaHbOcNd oxidised to CO2 and water, with its nitrogen released as ammonia or, with nitrification, oxidised to nitrate.

$$\mathrm{ThOD} = \frac{\left(a + \frac{b}{4} - \frac{c}{2} - \frac{3d}{4}\right) M_{O_2}}{M}$$

input	meaning	condition
`formula`	molecular formula with C, H, O, N only (e.g. C6H12O6)	
`nitrification`	include the oxygen to nitrify ammonia (yes) or not (no)	

output	meaning
`thod`	theoretical oxygen demand, g O2 per g compound
`o2_per_mol`	mol O2 per mol compound
`molar_mass`	molar mass, g/mol

- **Method:** CaHbOcNd + (a + b/4 - c/2 - 3d/4) O2 -> a $CO2 + (b - 3d)/2\ H2O + d$ NH3; nitrification adds 2d O2
- **Assumes:** elements C, H, O, N only; $b \geq 3d$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example of Theoretical oxygen demand of an organic compound as inputs and results: stated formula = C6H12O6, nitrification = no; returned by environmental-engineering-ai999x: thod = 1.0657, o2_per_mol = 6, molar_mass = 180.156.

```
environmental_engineering_ai999x.solve('enveng_theoretical_oxygen_demand', formula='C6H12O6',
nitrification='no')
# -> thod = 1.0657, o2_per_mol = 6, molar_mass = 180.156   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_theoretical_oxygen_demand', formula='C5H7O2N',
nitrification='yes')
# -> thod = 1.98022, o2_per_mol = 7, molar_mass = 113.115   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Theoretical oxygen demand: formula = C6H12O6, nitrification = no	SOLVED	thod = 1.0657, o2_per_mol = 6, molar_mass = 180.156
Theoretical oxygen demand: formula = C5H7O2N, nitrification = yes	SOLVED	thod = 1.98022, o2_per_mol = 7, molar_mass = 113.115

question	outcome	result
Theoretical oxygen demand: nitrification = no	NEEDS_INPUT	enveng_theoretical_oxygen_demand needs formula (molecular formula with C, H, O, N only (e.g. C6H12O6)). Example: solve('
Theoretical oxygen demand: formula = C6H12O6, nitrification = no, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Streeter-Phelps oxygen deficit and dissolved oxygen at a travel time,

`enveng_streeter_phelps_deficit`

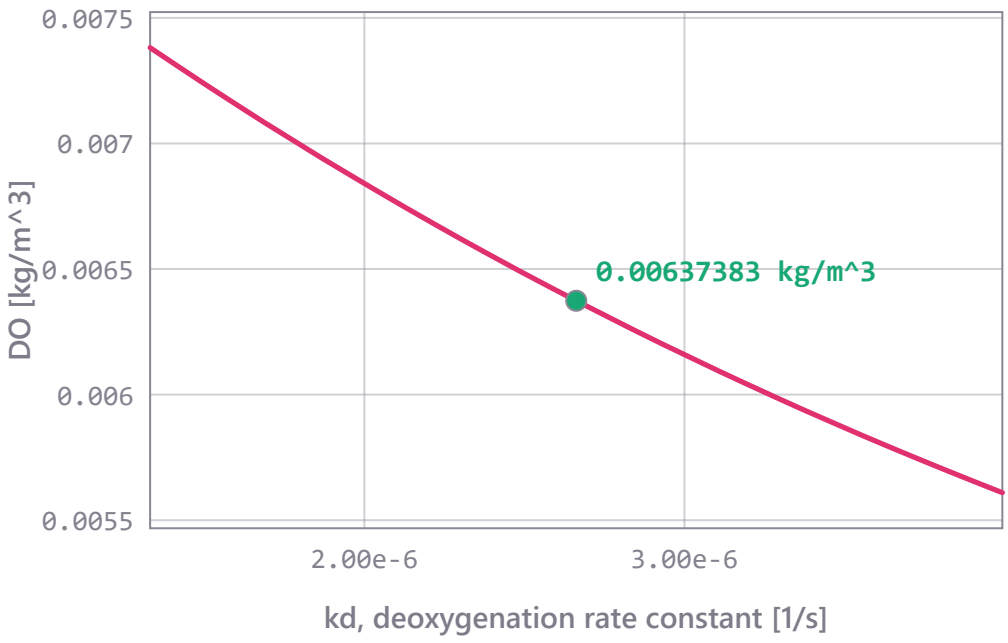
The dissolved-oxygen deficit and the dissolved oxygen of a river at a travel time downstream of a BOD discharge (Streeter-Phelps oxygen sag).

$$D = \frac{k_d L_0}{k_a - k_d}\left(e^{-k_d t} - e^{-k_a t}\right) + D_0 e^{-k_a t}$$

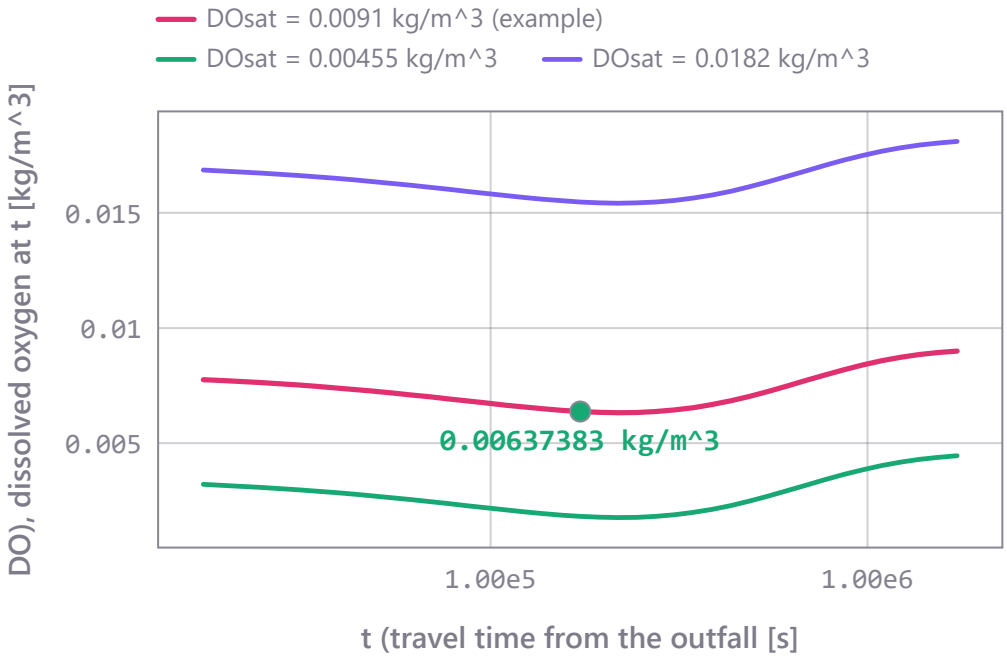
input	meaning	condition
`kd`	deoxygenation rate constant	
`ka`	reaeration rate constant	
`L0`	ultimate BOD of the mixed river water at the outfall	
`D0`	oxygen deficit at the outfall (DOsat - DO)	
`t`	travel time from the outfall	
`DOsat`	saturation dissolved oxygen	

output	meaning
`D`	oxygen deficit at t
`DO`	dissolved oxygen at t
`D_mgL`	deficit in mg/L
`DO_mgL`	dissolved oxygen in mg/L

- **Method:** $D = kd\,L_0/(ka - kd)$ (e^(-kd t) $- e^($ $-$ ka t)) + D0 e^(-ka t); for $ka = kd$, D = (kd L0 t + D0) e^(-kd t)
- **Assumes:** first-order deoxygenation and reaeration; plug flow; deficit stays below saturation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



DO (dissolved oxygen at t) against kd (deoxygenation rate constant), with D0 = 0.001 kg/m^3, DOsat = 0.0091 kg/m^3, L0 = 0.01 kg/m^3, ka = 5.32e-6 1/s, t = 1.73e5 s. 31 points computed by environmental-engineering-ai999x; marked: kd = 2.66204e-06, DO = 0.00637383 kg/m^3.



DO (dissolved oxygen at t) against t (travel time from the outfall) from 17280 s to 1.728e+06 s on a logarithmic axis. One curve for each value of DOsat (saturation dissolved oxygen): 0.0091 kg/m^3, 0.00455 kg/m^3, 0.0182 kg/m^3. Other inputs as in the worked example: kd = 2.662037037037037e-06 1/s, ka = 5.324074074074074e-06 1/s, L0 = 0.01 kg/m^3, D0 = 0.001 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, t = 172800 s, DO = 0.00637383 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_streeter_phelps_deficit', kd=2.662037037037037e-06,
ka=5.324074074074074e-06, L0=0.01, D0=0.001, t=172800.0, DOsat=0.0091)
# -> D = 0.00272617 kg/m^3, DO = 0.00637383 kg/m^3, D_mgL = 2.72617, DO_mgL = 6.37383   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_streeter_phelps_deficit', kd=3.472222222222222e-06,
ka=3.472222222222222e-06, L0=0.008, D0=0.0015, t=129600.0, DOsat=0.0084)
# -> D = 0.0032519 kg/m^3, DO = 0.0051481 kg/m^3, D_mgL = 3.2519, DO_mgL = 5.1481   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Streeter-Phelps oxygen deficit: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, t = 2 d, DOsat = 9.1 mg/L	SOLVED	D = 0.00272617 kg/m^3, DO = 0.00637383 kg/m^3, D_mgL = 2.72617, DO_mgL = 6.37383
Streeter-Phelps oxygen deficit: kd = 0.3 1/d, ka = 0.3 1/d, L0 = 8 mg/L, D0 = 1.5 mg/L, t = 1.5 d, DOsat = 8.4 mg/L	SOLVED	D = 0.0032519 kg/m^3, DO = 0.0051481 kg/m^3, D_mgL = 3.2519, DO_mgL = 5.1481
Streeter-Phelps oxygen deficit: ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, t = 2 d, DOsat = 9.1 mg/L	NEEDS_INPUT	enveng_streeter_phelps_deficit needs kd (deoxygenation rate constant). Example: solve('enveng_streeter_phelps_deficit',
Streeter-Phelps oxygen deficit: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, t = 2 d, DOsat = 9.1 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Streeter-Phelps oxygen deficit: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, t = 48 h, DOsat = 9.1 mg/L	SOLVED	D = 0.00272617 kg/m^3, DO = 0.00637383 kg/m^3, D_mgL = 2.72617, DO_mgL = 6.37383

Dissolved oxygen at a distance downstream (Streeter-Phelps), `enveng_oxygen_sag_distance`

The oxygen deficit and dissolved oxygen at a distance downstream of an outfall from the river velocity (travel time x/v) and the Streeter-Phelps sag.

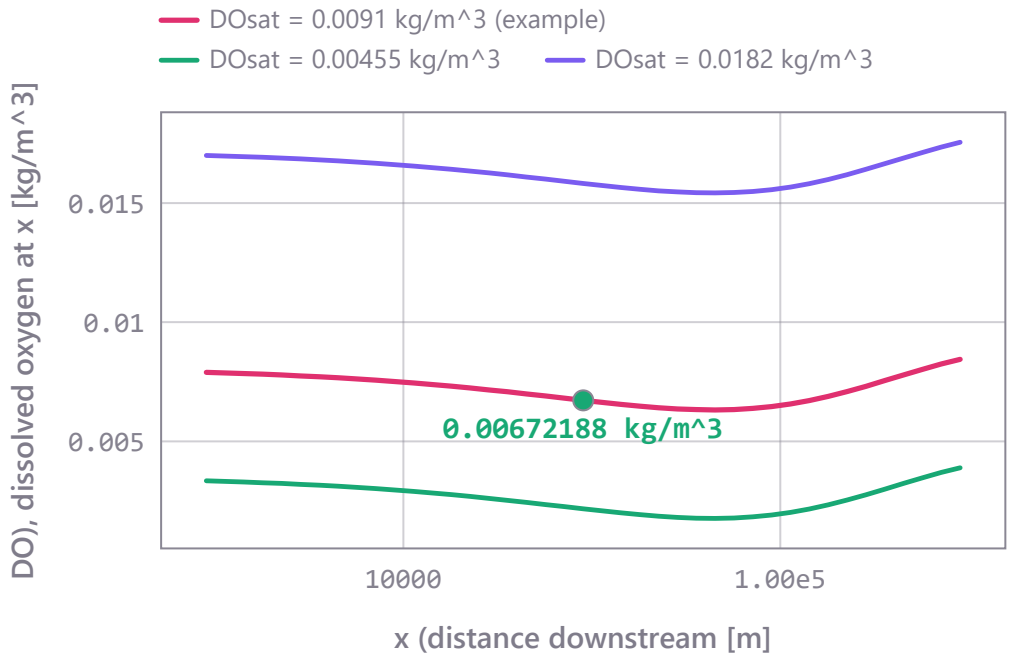
$$D(x) = \frac{k_d L_0}{k_a - k_d}\left(e^{-k_d x/v} - e^{-k_a x/v}\right) + D_0 e^{-k_a x/v}$$

input	meaning	condition
`kd`	deoxygenation rate constant	
`ka`	reaeration rate constant	
`L0`	ultimate BOD of the mixed river water at the outfall	
`D0`	oxygen deficit at the outfall (DOsat - DO)	
`x`	distance downstream	
`v`	mean river velocity	
`DOsat`	saturation dissolved oxygen	

output	meaning
`D`	oxygen deficit at x
`DO`	dissolved oxygen at x

output	meaning
`D_mgL`	deficit in mg/L
`DO_mgL`	dissolved oxygen in mg/L
`t`	travel time

- **Method:** $t = x/v$, then the Streeter-Phelps deficit
- **Assumes:** plug flow at constant velocity; deficit stays below saturation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



DO (dissolved oxygen at x) against x (distance downstream) from 3000 m to 300000 m on a logarithmic axis. One curve for each value of DOsat (saturation dissolved oxygen): 0.0091 kg/m^3, 0.00455 kg/m^3, 0.0182 kg/m^3. Other inputs as in the worked example: kd = 2.662037037037037e-06 1/s, ka = 5.324074074074074e-06 1/s, L0 = 0.01 kg/m^3, D0 = 0.001 kg/m^3, v = 0.3 m/s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, x = 30000 m, DO = 0.00672188 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_oxygen_sag_distance', kd=2.662037037037037e-06,
ka=5.324074074074074e-06, L0=0.01, D0=0.001, x=30000.0, v=0.3, DOsat=0.0091)
# -> D = 0.00237812 kg/m^3, DO = 0.00672188 kg/m^3, D_mgL = 2.37812, DO_mgL = 6.72188, t = 100000 s
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_oxygen_sag_distance', kd=4.050925925925926e-06,
ka=8.101851851851852e-06, L0=0.012, D0=0.0005, x=15000.0, v=0.2, DOsat=0.0088)
# -> D = 0.00259262 kg/m^3, DO = 0.00620738 kg/m^3, D_mgL = 2.59262, DO_mgL = 6.20738, t = 75000 s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Oxygen sag at a distance: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, x = 30 km, v = 0.3 m/s, DOsat = 9.1 mg/L	SOLVED	D = 0.00237812 kg/m^3, DO = 0.00672188 kg/m^3, D_mgL = 2.37812, DO_mgL = 6.72188, t = 100000 s

question	outcome	result
Oxygen sag at a distance: kd = 0.35 1/d, ka = 0.7 1/d, L0 = 12 mg/L, D0 = 0.5 mg/L, x = 15 km, v = 0.2 m/s, DOsat = 8.8 mg/L	SOLVED	D = 0.00259262 kg/m^3, DO = 0.00620738 kg/m^3, D_mgL = 2.59262, DO_mgL = 6.20738, t = 75000 s
Oxygen sag at a distance: ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, x = 30 km, v = 0.3 m/s, DOsat = 9.1 mg/L	NEEDS_INPUT	enveng_oxygen_sag_distance needs kd (deoxygenation rate constant). Example: solve('enveng_oxygen_sag_distance', kd='2.66
Oxygen sag at a distance: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, x = 30 km, v = 0.3 m/s, DOsat = 9.1 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Critical (minimum) dissolved oxygen of the oxygen sag, `enveng_streeter_phelps_critical_point`

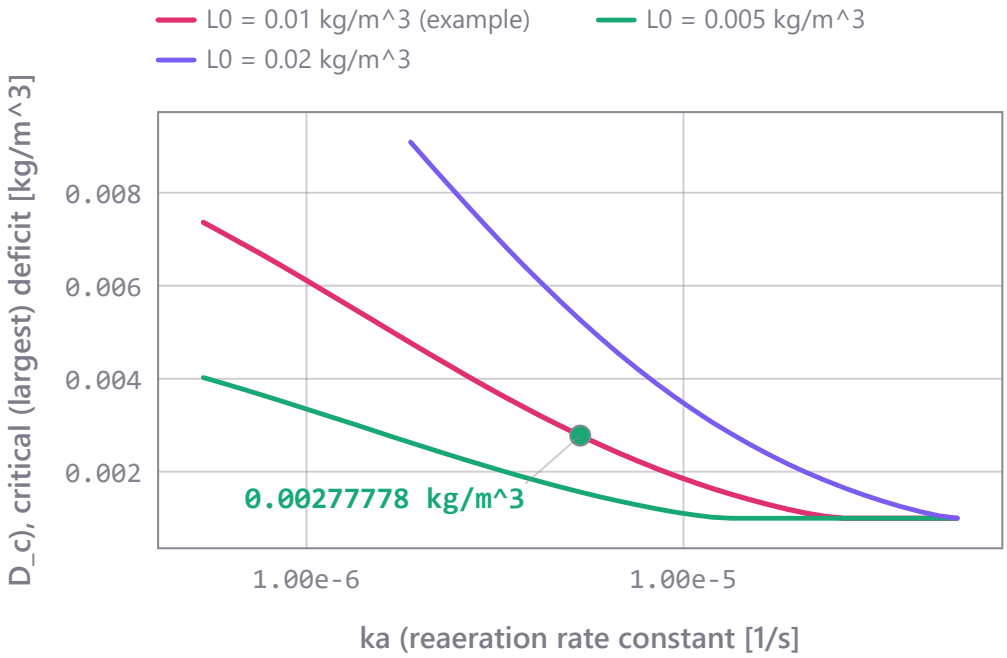
The critical time at which the oxygen deficit of a river is largest, the critical deficit and the minimum dissolved oxygen (Streeter-Phelps).

$$t_c = \frac{1}{k_a - k_d} \ln\left[\frac{k_a}{k_d}\left(1 - \frac{D_0(k_a - k_d)}{k_d L_0}\right)\right], \quad D_c = \frac{k_d}{k_a} L_0 e^{-k_d t_c}$$

input	meaning	condition
`kd`	deoxygenation rate constant	
`ka`	reaeration rate constant	
`L0`	ultimate BOD of the mixed river water at the outfall	
`D0`	oxygen deficit at the outfall (DOsat - DO)	
`DOsat`	saturation dissolved oxygen	

output	meaning
`t_c`	critical travel time
`t_c_days`	critical time in days
`D_c`	critical (largest) deficit
`DO_min`	minimum dissolved oxygen
`DO_min_mgL`	minimum dissolved oxygen in mg/L

- **Method:** $t_c = ln$[(ka/kd)(1 - D0 (ka - kd)/(kd L0))]/(ka - kd); $D_c = (kd/ka)$ L0 e^(-kd t_c)
- **Assumes:** the critical deficit stays below saturation; if the deficit falls from the outfall on, $t_c = 0$ and $D_c = D_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



D_c (critical (largest) deficit) against ka (reaeration rate constant) from 5.3241e-07 1/s to 5.3241e-05 1/s on a logarithmic axis. One curve for each value of L0 (ultimate BOD of the mixed river water at the outfall): 0.01 kg/m^3, 0.005 kg/m^3, 0.02 kg/m^3. Other inputs as in the worked example: kd = 2.662037037037037e-06 1/s, D0 = 0.001 kg/m^3, DOsat = 0.0091 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, ka = 5.3241e-06 1/s, D_c = 0.00277778 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_streeter_phelps_critical_point',
kd=2.662037037037037e-06, ka=5.324074074074074e-06, L0=0.01, D0=0.001, DOsat=0.0091)
# -> t_c = 220803 s, t_c_days = 2.55559, D_c = 0.00277778 kg/m^3, DO_min = 0.00632222 kg/m^3,
DO_min_mgL = 6.32222   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_streeter_phelps_critical_point',
kd=4.050925925925926e-06, ka=8.101851851851852e-06, L0=0.012, D0=0.002, DOsat=0.0088)
# -> t_c = 126101 s, t_c_days = 1.4595, D_c = 0.0036 kg/m^3, DO_min = 0.0052 kg/m^3, DO_min_mgL =
5.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Streeter-Phelps critical point: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, DOsat = 9.1 mg/L	SOLVED	t_c = 220803 s, t_c_days = 2.55559, D_c = 0.00277778 kg/m^3, DO_min = 0.00632222 kg/m^3, DO_min_mgL = 6.32222
Streeter-Phelps critical point: kd = 0.35 1/d, ka = 0.7 1/d, L0 = 12 mg/L, D0 = 2 mg/L, DOsat = 8.8 mg/L	SOLVED	t_c = 126101 s, t_c_days = 1.4595, D_c = 0.0036 kg/m^3, DO_min = 0.0052 kg/m^3, DO_min_mgL = 5.2
Streeter-Phelps critical point: ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, DOsat = 9.1 mg/L	NEEDS_INPUT	enveng_streeter_phelps_critical_point needs kd (deoxygenation rate constant). Example: solve('enveng_streeter_phelps_cri
Streeter-Phelps critical point: kd = 0.23 1/d, ka = 0.46 1/d, L0 = 10 mg/L, D0 = 1 mg/L, DOsat = 9.1 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Stream reaeration coefficient (O'Connor-Dobbins, Churchill, Owens-Gibbs),

`enveng_reaeration_coefficient`

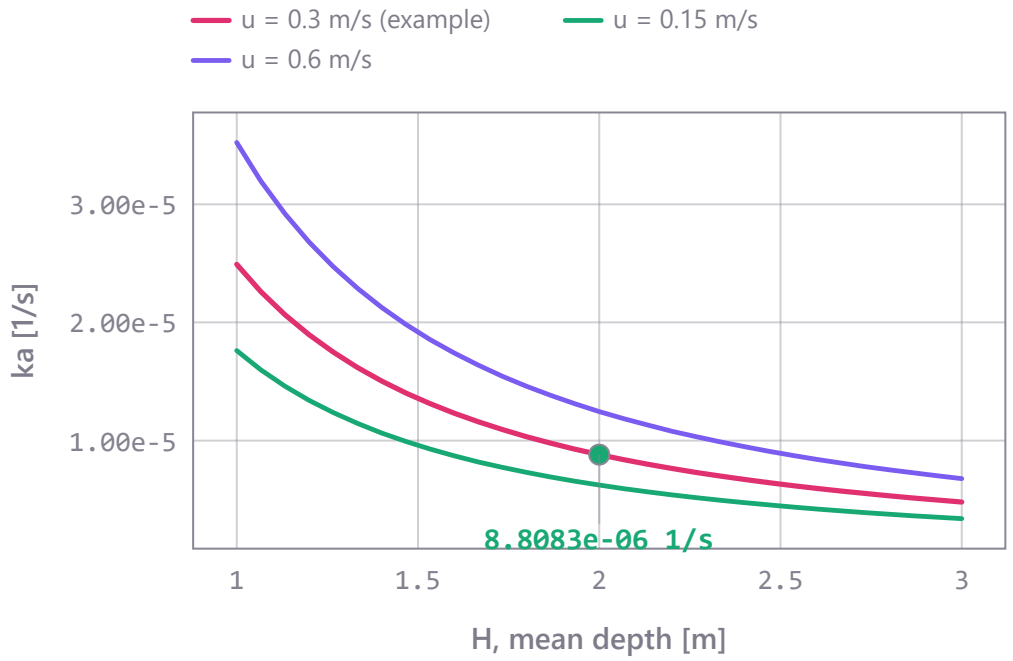
The reaeration rate constant of a stream at 20 degC from its mean velocity and depth by a chosen empirical formula.

$$k_a = \frac{3.93\,u^{0.5}}{H^{1.5}}\ \text{(O'Connor-Dobbins)}$$

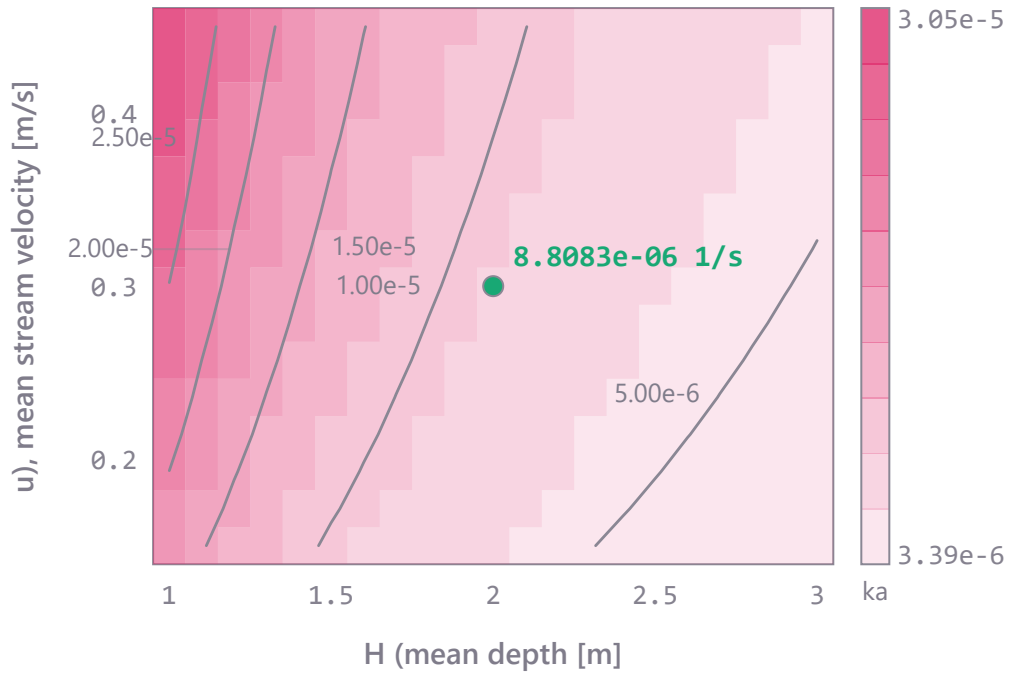
input	meaning	condition
`method`	o_connor_dobbins, churchill or owens_gibbs	
`u`	mean stream velocity	
`H`	mean depth	

output	meaning
`ka`	reaeration rate constant at 20 degC
`ka_per_day`	in $1/d$

- **Method:** $ka = 3.93\,u^{0.5}/H^{1.5}$ (O'Connor-Dobbins), 5.026 $u/H^{1.67}$ (Churchill), 5.32 $u^{0.67}/H^{1.85}$ (Owens-Gibbs), per day
- **Assumes:** u in m/s and H in m inside the formula constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ka (reaeration rate constant at 20 degC) against H (mean depth) from 1 m to 3 m. One curve for each value of u (mean stream velocity): 0.3 m/s, 0.15 m/s, 0.6 m/s. Other inputs as in the worked example: method = o_connor_dobbins. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, H = 2 m, ka = 8.8083e-06 1/s.



ka (reaeration rate constant at 20 degC) over H (mean depth) from 1 m to 3 m and u (mean stream velocity) from 0.15 m/s to 0.45 m/s: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 3.3903e-06 1/s to dark 3.0513e-05 1/s); iso-lines at ka = 5.00e-6, 1.00e-5, 1.50e-5, 2.00e-5, 2.50e-5 1/s. Other inputs as in the worked example: method = o_connor_dobbins. Marked: the worked example, H = 2 m, u = 0.3 m/s, ka = 8.8083e-06 1/s.

```
environmental_engineering_ai999x.solve('enveng_reaeration_coefficient', method='o_connor_dobbins',
u=0.3, H=2.0)
# -> ka = 8.80835e-06 1/s, ka_per_day = 0.761041   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_reaeration_coefficient', method='churchill', u=0.8,
H=1.5)
# -> ka = 2.36443e-05 1/s, ka_per_day = 2.04287   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reaeration coefficient: method = o_connor_dobbins, u = 0.3 m/s, H = 2 m	SOLVED	ka = 8.80835e-06 1/s, ka_per_day = 0.761041
Reaeration coefficient: method = churchill, u = 0.8 m/s, H = 1.5 m	SOLVED	ka = 2.36443e-05 1/s, ka_per_day = 2.04287
Reaeration coefficient: u = 0.3 m/s, H = 2 m	NEEDS_INPUT	enveng_reaeration_coefficient needs method (o_connor_dobbins, churchill or owens_gibbs). Example: solve('enveng_reaerati
Reaeration coefficient: method = o_connor_dobbins, u = 0.3 m/s, H = 2 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Dissolved oxygen saturation concentration (Benson-Krause), `enveng_do_saturation`

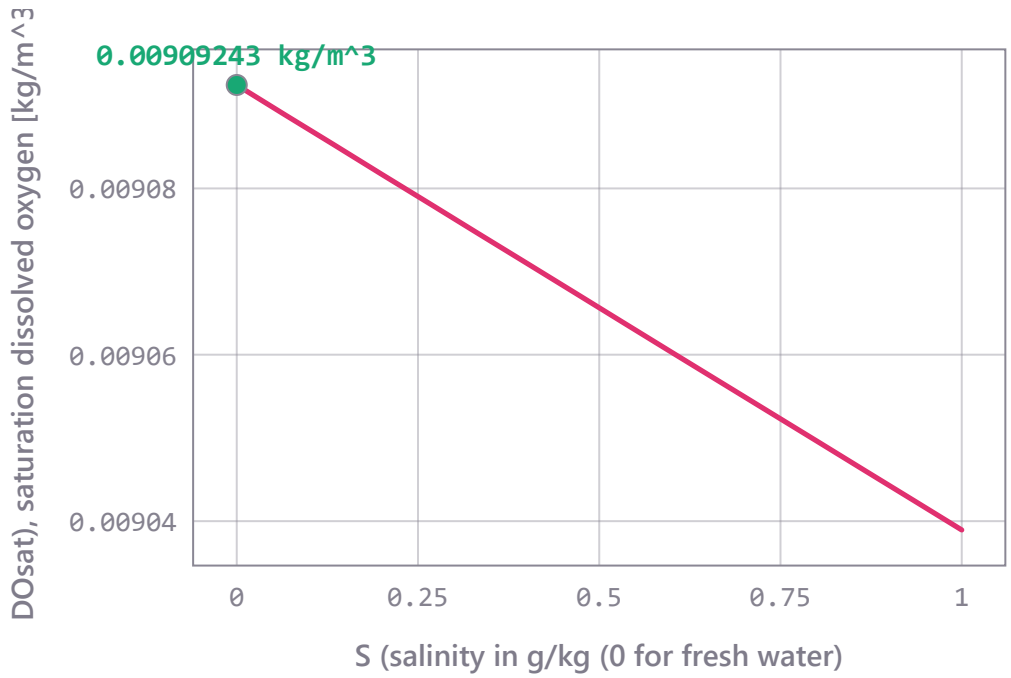
The saturation concentration of dissolved oxygen in water in equilibrium with moist air at 1 atm, from the temperature and salinity (Benson and Krause equation as adopted in Standard Methods).

$$\ln C_s = -139.34411 + \frac{1.575701 \times 10^5}{T} - \frac{6.642308 \times 10^7}{T^2} + \frac{1.2438 \times 10^{10}}{T^3} - \frac{8.621949 \times 10^{11}}{T^4} - S($$

input	meaning	condition
`T`	water temperature	
`S`	salinity in g/kg (0 for fresh water)	

output	meaning
`DOsat`	saturation dissolved oxygen
`DOsat_mgL`	in mg/L

- **Method:** ln $C = -139.34411 + 1.575701e5/T - 6.642308e7/T^2 + 1.2438e10/T^3 - 8.621949e11/T^4 - S$ (1.7674e-2 - 10.754/T + 2140.7/T^2)
- **Assumes:** $0 - 40$ degC; $0 - 40$ g/kg salinity; 1 atm moist air
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



DOsat (saturation dissolved oxygen) against S (salinity in g/kg (0 for fresh water)) from -1 to 1. Other inputs as in the worked example: T = 293.15 K. 16 points computed by environmental-engineering-ai999x; marked: the worked example, S = 0, DOsat = 0.00909243 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_do_saturation', T=293.15, S=0.0)
# -> DOsat = 0.00909243 kg/m^3, DOsat_mgL = 9.09243   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_do_saturation', T=283.15, S=35.0)
# -> DOsat = 0.00902426 kg/m^3, DOsat_mgL = 9.02426   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dissolved oxygen saturation: T = 20 degC, S = 0 g/kg	SOLVED	DOsat = 0.00909243 kg/m^3, DOsat_mgL = 9.09243
Dissolved oxygen saturation: T = 10 degC, S = 35 g/kg	SOLVED	DOsat = 0.00902426 kg/m^3, DOsat_mgL = 9.02426
Dissolved oxygen saturation: S = 0 g/kg	NEEDS_INPUT	enveng_do_saturation needs T (water temperature). Example: solve('enveng_do_saturation', T='293.15 K', S=0).
Dissolved oxygen saturation: T = 20 degC, S = 0 g/kg, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Concentration after mixing a discharge with a river, `enveng_mixing_two_streams`

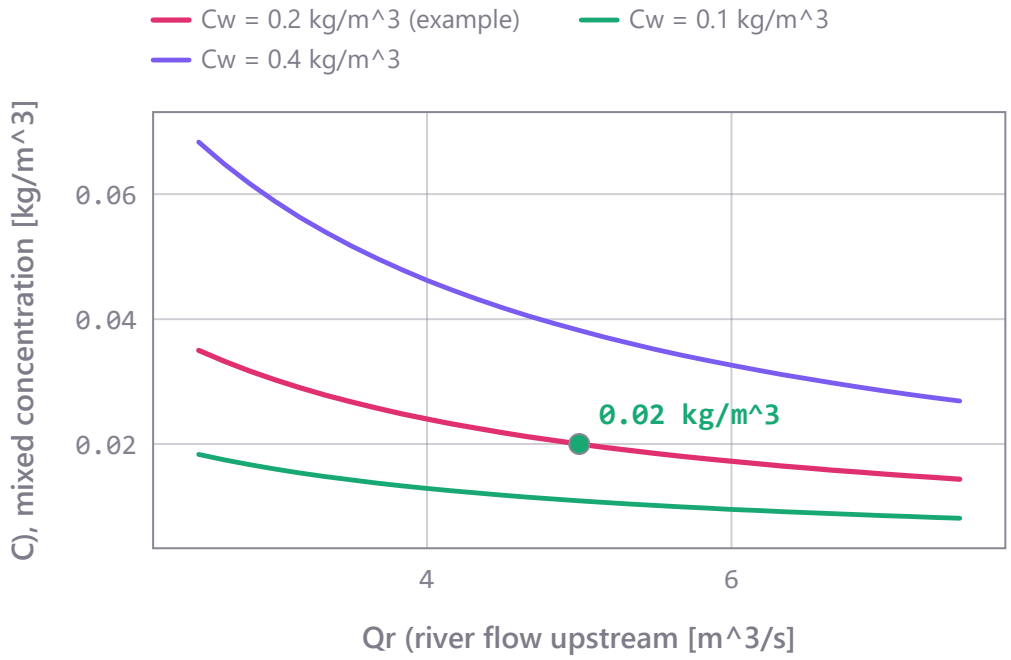
The fully mixed concentration (BOD, DO, a pollutant) and flow just below a wastewater outfall from the flows and concentrations of the discharge and the river.

$$C = \frac{Q_w C_w + Q_r C_r}{Q_w + Q_r}$$

input	meaning	condition
`Qw`	wastewater (discharge) flow	
`Cw`	concentration in the wastewater	
`Qr`	river flow upstream	
`Cr`	concentration in the river upstream	

output	meaning
`C`	mixed concentration
`C_mgL`	in mg/L
`Q`	combined flow

- **Method:** C = (Qw Cw + Qr Cr)/(Qw + Qr)
- **Assumes:** complete mixing; conservative over the mixing zone
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (mixed concentration) against Qr (river flow upstream) from 2.5 m^3/s to 7.5 m^3/s. One curve for each value of Cw (concentration in the wastewater): 0.2 kg/m^3, 0.1 kg/m^3, 0.4 kg/m^3. Other inputs as in the worked example: Qw = 0.5 m^3/s, Cr = 0.002 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Qr = 5 m^3/s, C = 0.02 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_mixing_two_streams', Qw=0.5, Cw=0.2, Qr=5.0,
Cr=0.002)
# -> C = 0.02 kg/m^3, C_mgL = 20, Q = 5.5 m^3/s   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_mixing_two_streams', Qw=1.2, Cw=0.03, Qr=10.0,
Cr=0.0015)
# -> C = 0.00455357 kg/m^3, C_mgL = 4.55357, Q = 11.2 m^3/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mixing of discharge and river: Qw = 0.5 m^3/s, Cw = 200 mg/L, Qr = 5 m^3/s, Cr = 2 mg/L	SOLVED	C = 0.02 kg/m^3, C_mgL = 20, Q = 5.5 m^3/s
Mixing of discharge and river: Qw = 1.2 m^3/s, Cw = 30 mg/L, Qr = 10 m^3/s, Cr = 1.5 mg/L	SOLVED	C = 0.00455357 kg/m^3, C_mgL = 4.55357, Q = 11.2 m^3/s
Mixing of discharge and river: Cw = 200 mg/L, Qr = 5 m^3/s, Cr = 2 mg/L	NEEDS_INPUT	enveng_mixing_two_streams needs Qw (wastewater (discharge) flow). Example: solve('enveng_mixing_two_streams', Qw='0.5 m^
Mixing of discharge and river: Qw = 0.5 m^3/s, Cw = 200 mg/L, Qr = 5 m^3/s, Cr = 2 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Mixing of discharge and river: Qw = 1800 m^3/h, Cw = 200 mg/L, Qr = 5 m^3/s, Cr = 2 mg/L	SOLVED	C = 0.02 kg/m^3, C_mgL = 20, Q = 5.5 m^3/s

Flow-weighted mixing of several streams, `enveng_mixing_several_streams`

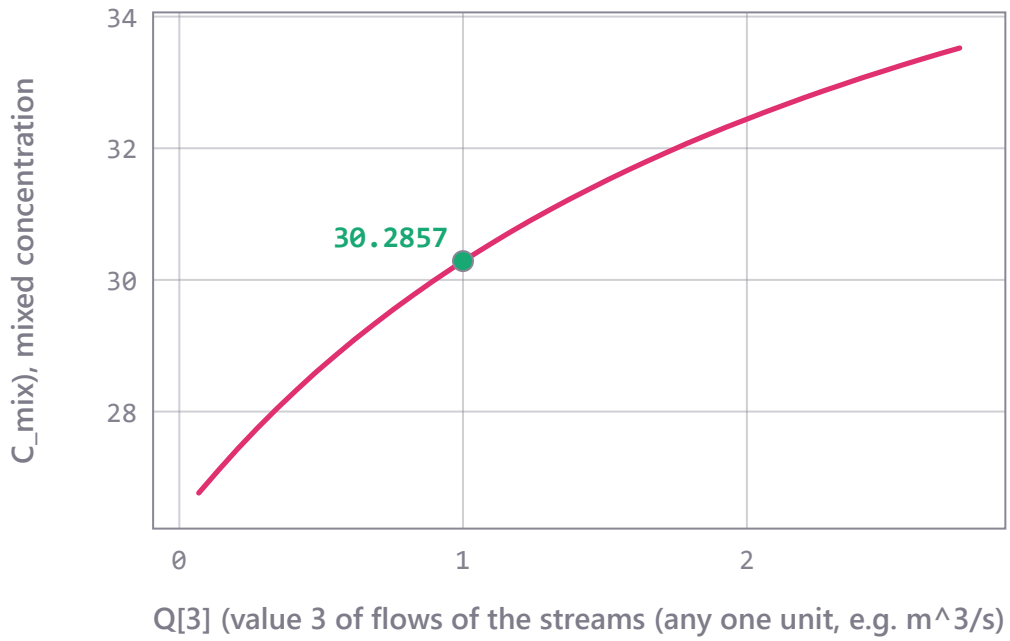
The flow-weighted concentration (or temperature) of several streams combined, and the total flow; flows in any one unit, concentrations in any one unit.

$$C = \frac{\sum_i Q_i C_i}{\sum_i Q_i}$$

input	meaning	condition
`Q`	flows of the streams (any one unit, e.g. m^3/s)	
`C`	concentrations of the streams (any one unit, e.g. mg/L)	

output	meaning
`C_mix`	mixed concentration, in the unit of C
`Q_total`	total flow, in the unit of Q

- **Method:** C = sum(Q_i C_i)/sum(Q_i)
- **Assumes:** complete mixing; conservative substance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C_mix (mixed concentration, in the unit of C) as value 3 of Q (flows of the streams (any one unit, e.g. m^3/s)) moves from 0 to 2.75, the other values as given ([2.0, 0.5, 1.0]); 40 runs of environmental-engineering-ai999x. Other inputs: C = [3.0, 120.0, 40.0]. Marked: the worked example, Q[3] = 1, C_mix = 30.2857.

```
environmental_engineering_ai999x.solve('enveng_mixing_several_streams', Q=[2.0, 0.5, 1.0], C=[3.0, 120.0, 40.0])
# -> C_mix = 30.2857, Q_total = 3.5   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_mixing_several_streams', Q=[10.0, 4.0], C=[12.0, 25.0])
# -> C_mix = 15.7143, Q_total = 14   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flow-weighted mixing: Q = [2, 0.5, 1], C = [3, 120, 40]	SOLVED	C_mix = 30.2857, Q_total = 3.5
Flow-weighted mixing: Q = [10, 4], C = [12, 25]	SOLVED	C_mix = 15.7143, Q_total = 14
Flow-weighted mixing: C = [3, 120, 40]	NEEDS_INPUT	enveng_mixing_several_streams needs Q (flows of the streams (any one unit, e.g. m^3/s)). Example: solve('enveng_mixing_s
Flow-weighted mixing: Q = [2, 0.5, 1], C = [3, 120, 40], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Steady-state concentration in a well-mixed lake (first-order decay), `enveng_lake_steady_state`

The steady concentration of a pollutant in a lake treated as a completely mixed reactor with inflow load, outflow and first-order decay, and the hydraulic residence time.

$$C = \frac{W}{Q + kV}$$

input	meaning	condition
`W`	total pollutant input (load)	
`Q`	outflow rate	
`k`	first-order decay constant (0 if conservative)	
`V`	lake volume	

output	meaning
`C`	steady-state concentration
`C_mgL`	in mg/L
`tau`	hydraulic residence time V/Q

- **Method:** $0 = W - Q$ C - k C V, C = W/(Q + k V)
- **Assumes:** completely mixed; steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (steady-state concentration) against k (first-order decay constant (0 if conservative)) from 2.3148e-07 1/s to 2.3148e-05 1/s on a logarithmic axis. One curve for each value of W (total pollutant input (load)): 0.00115741 kg/s, 0.000578704 kg/s, 0.00231481 kg/s. Other inputs as in the worked example: Q = 10.0 m^3/s, V = 10000000.0 m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 2.3148e-06 1/s, C = 3.4916e-05 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_lake_steady_state', W=0.0011574074074074073, Q=10.0,
k=2.3148148148148148e-06, V=10000000.0)
# -> C = 3.49162e-05 kg/m^3, C_mgL = 0.0349162, tau = 1e+06 s   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_lake_steady_state', W=0.0002314814814814815, Q=3.0,
k=5.787037037037037e-07, V=5000000.0)
# -> C = 3.92773e-05 kg/m^3, C_mgL = 0.0392773, tau = 1.66667e+06 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lake steady-state concentration: W = 100 kg/d, Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3	SOLVED	C = 3.49162e-05 kg/m^3, C_mgL = 0.0349162, tau = 1e+06 s
Lake steady-state concentration: W = 20 kg/d, Q = 3 m^3/s, k = 0.05 1/d, V = 5000000 m^3	SOLVED	C = 3.92773e-05 kg/m^3, C_mgL = 0.0392773, tau = 1.66667e+06 s
Lake steady-state concentration: Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3	NEEDS_INPUT	enveng_lake_steady_state needs W (total pollutant input (load)). Example: solve('enveng_lake_steady_state', W='0.0011574
Lake steady-state concentration: W = 100 kg/d, Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Concentration in a well-mixed lake over time, `enveng_lake_transient`

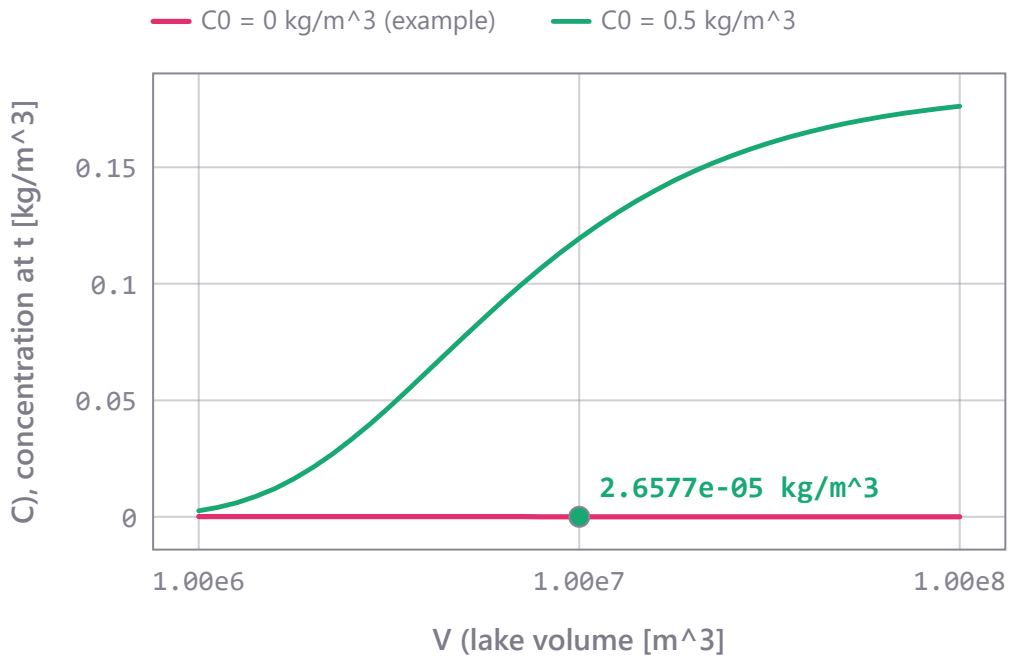
The concentration in a completely mixed lake a time t after a change in load, from the initial concentration, the load, outflow, volume and first-order decay.

$$C(t) = C_{ss} + (C_0 - C_{ss})e^{-(Q/V+k)t}$$

input	meaning	condition
`W`	pollutant load after the change	
`Q`	outflow rate	
`k`	first-order decay constant	
`V`	lake volume	
`C0`	concentration at $t = 0$	
`t`	elapsed time	

output	meaning
`C`	concentration at t
`C_mgL`	in mg/L
`C_ss`	new steady state

- **Method:** V $dC/dt = W - Q$ C - k C V, so C = Css + (C0 - Css) e^(-(Q/V + k) t)
- **Assumes:** completely mixed; constant flow, volume and load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration at t) against V (lake volume) from 1e+06 m^3 to 1.0000e+08 m^3 on a logarithmic axis. One curve for each value of C0 (concentration at t = 0): 0 kg/m^3, 0.5 kg/m^3. Other inputs as in the worked example: W = 0.0011574074074074073 kg/s, Q = 10.0 m^3/s, k = 2.3148148148148148e-06 1/s, t = 432000.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, V = 1.0000e+07 m^3, C = 2.6577e-05 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_lake_transient', W=0.0011574074074074073, Q=10.0,
k=2.3148148148148148e-06, V=10000000.0, C0=0.0, t=432000.0)
# -> C = 2.65771e-05 kg/m^3, C_mgL = 0.0265771, C_ss = 3.49162e-05 kg/m^3   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_lake_transient', W=0.00011574074074074075, Q=5.0,
k=1.1574074074074074e-06, V=20000000.0, C0=0.01, t=1728000.0)
# -> C = 0.00088236 kg/m^3, C_mgL = 0.88236, C_ss = 4.11184e-06 kg/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lake concentration over time: W = 100 kg/d, Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3, C0 = 0 mg/L, t = 5 d	SOLVED	C = 2.65771e-05 kg/m^3, C_mgL = 0.0265771, C_ss = 3.49162e-05 kg/m^3
Lake concentration over time: W = 10 kg/d, Q = 5 m^3/s, k = 0.1 1/d, V = 20000000 m^3, C0 = 10 mg/L, t = 20 d	SOLVED	C = 0.00088236 kg/m^3, C_mgL = 0.88236, C_ss = 4.11184e-06 kg/m^3
Lake concentration over time: Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3, C0 = 0 mg/L, t = 5 d	NEEDS_INPUT	enveng_lake_transient needs W (pollutant load after the change). Example: solve('enveng_lake_transient', W='0.00115741 k
Lake concentration over time: W = 100 kg/d, Q = 10 m^3/s, k = 0.2 1/d, V = 10000000 m^3, C0 = 0 mg/L, t = 5 d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

First-order decay of a pollutant downstream (plug flow), `enveng_river_first_order_decay`

The concentration of a decaying pollutant (BOD, coliforms, a chemical) a distance downstream in a river in plug flow, and the travel time.

$$C = C_0\, e^{-kx/u}$$

input	meaning	condition
`C0`	concentration at the start (after mixing)	
`k`	first-order decay constant	
`x`	distance downstream	
`u`	mean velocity	

output	meaning
`C`	concentration at x
`C_mgL`	in mg/L
`t`	travel time x/u
`fraction_remaining`	C/C_0

- **Method:** $C = C_0$ e^(-k x/u)
- **Assumes:** plug flow; first-order decay
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (concentration at x) against k (first-order decay constant) from 3.4722e-07 1/s to 3.4722e-05 1/s on a logarithmic axis. One curve for each value of C0 (concentration at the start (after mixing)): 0.02 kg/m^3, 0.01 kg/m^3, 0.04 kg/m^3. Other inputs as in the worked example: x = 40000.0 m, u = 0.25 m/s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 3.4722e-06 1/s, C = 0.0114751 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_river_first_order_decay', C0=0.02,
k=3.472222222222222e-06, x=40000.0, u=0.25)
# -> C = 0.0114751 kg/m^3, C_mgL = 11.4751, t = 160000 s, fraction_remaining = 0.573753   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_river_first_order_decay', C0=0.005,
k=1.1574074074074073e-05, x=10000.0, u=0.5)
# -> C = 0.00396679 kg/m^3, C_mgL = 3.96679, t = 20000 s, fraction_remaining = 0.793357   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
River first-order decay: C0 = 20 mg/L, k = 0.3 1/d, x = 40 km, u = 0.25 m/s	SOLVED	C = 0.0114751 kg/m^3, C_mgL = 11.4751, t = 160000 s, fraction_remaining = 0.573753
River first-order decay: C0 = 5 mg/L, k = 1 1/d, x = 10 km, u = 0.5 m/s	SOLVED	C = 0.00396679 kg/m^3, C_mgL = 3.96679, t = 20000 s, fraction_remaining = 0.793357
River first-order decay: k = 0.3 1/d, x = 40 km, u = 0.25 m/s	NEEDS_INPUT	enveng_river_first_order_decay needs C0 (concentration at the start (after mixing)). Example: solve('enveng_river_first_
River first-order decay: C0 = 20 mg/L, k = 0.3 1/d, x = 40 km, u = 0.25 m/s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Pollutant mass load of a flow, `enveng_pollutant_load`

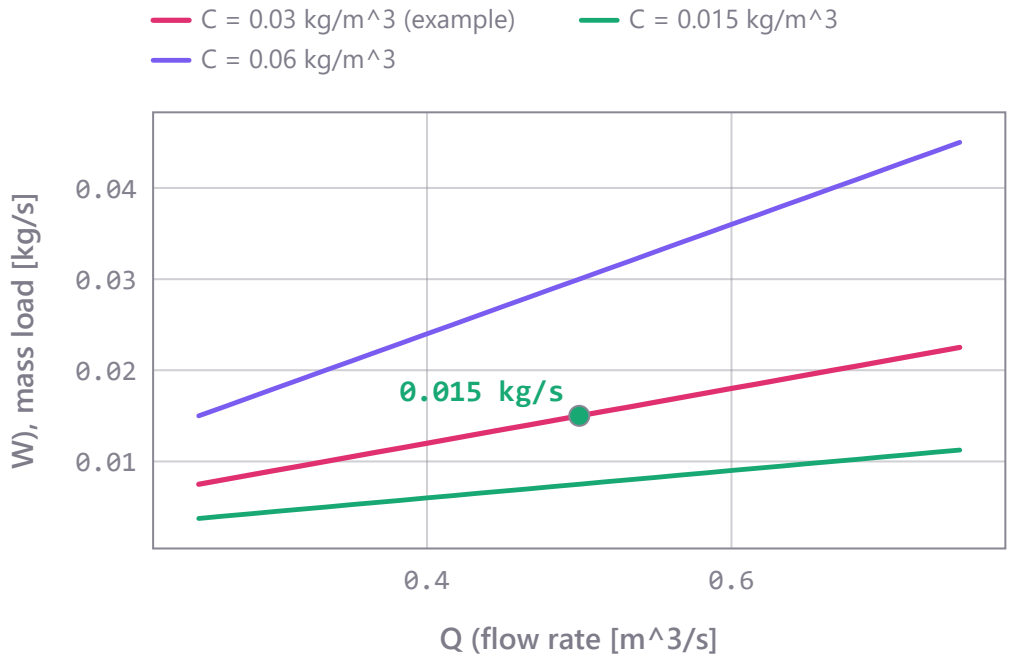
The mass load (rate) carried by a flow at a concentration, in kg/s, kg per day and tonnes per year.

$$W = Q\,C$$

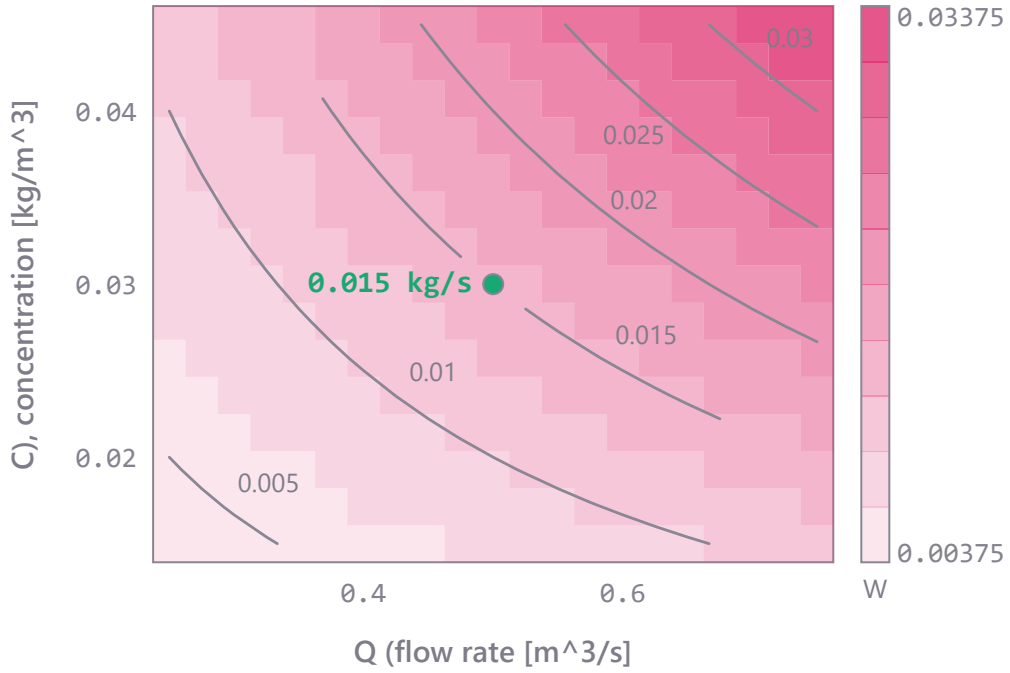
input	meaning	condition
`Q`	flow rate	
`C`	concentration	

output	meaning
`W`	mass load
`W_kg_per_day`	load in kg per day
`W_t_per_year`	load in tonnes per (365.25-day) year

- **Method:** $W = Q$ C
- **Assumes:** $Q > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



W (mass load) against Q (flow rate) from 0.25 m^3/s to 0.75 m^3/s. One curve for each value of C (concentration): 0.03 kg/m^3, 0.015 kg/m^3, 0.06 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Q = 0.5 m^3/s, W = 0.015 kg/s.



W (mass load) over Q (flow rate) from 0.25 m^3/s to 0.75 m^3/s and C (concentration) from 0.015 kg/m^3 to 0.045 kg/m^3: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 0.00375 kg/s to dark 0.03375 kg/s); iso-lines at W = 0.005, 0.01, 0.015, 0.02, 0.025, 0.03 kg/s. Marked: the worked example, Q = 0.5 m^3/s, C = 0.03 kg/m^3, W = 0.015 kg/s.

```
environmental_engineering_ai999x.solve('enveng_pollutant_load', Q=0.5, C=0.03)
# -> W = 0.015 kg/s, W_kg_per_day = 1296, W_t_per_year = 473.364   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_pollutant_load', Q=2.0, C=0.0004)
# -> W = 0.0008 kg/s, W_kg_per_day = 69.12, W_t_per_year = 25.2461   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pollutant load: Q = 0.5 m^3/s, C = 30 mg/L	SOLVED	W = 0.015 kg/s, W_kg_per_day = 1296, W_t_per_year = 473.364
Pollutant load: Q = 2 m^3/s, C = 0.4 mg/L	SOLVED	W = 0.0008 kg/s, W_kg_per_day = 69.12, W_t_per_year = 25.2461
Pollutant load: C = 30 mg/L	NEEDS_INPUT	enveng_pollutant_load needs Q (flow rate). Example: solve('enveng_pollutant_load', Q='0.5 m^3/s', C='0.03 kg/m^3').
Pollutant load: Q = 0.5 m^3/s, C = 30 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Pollutant load: Q = 43200 m^3/d, C = 30 mg/L	SOLVED	W = 0.015 kg/s, W_kg_per_day = 1296, W_t_per_year = 473.364

Hydraulic detention time, `enveng_detention_time`

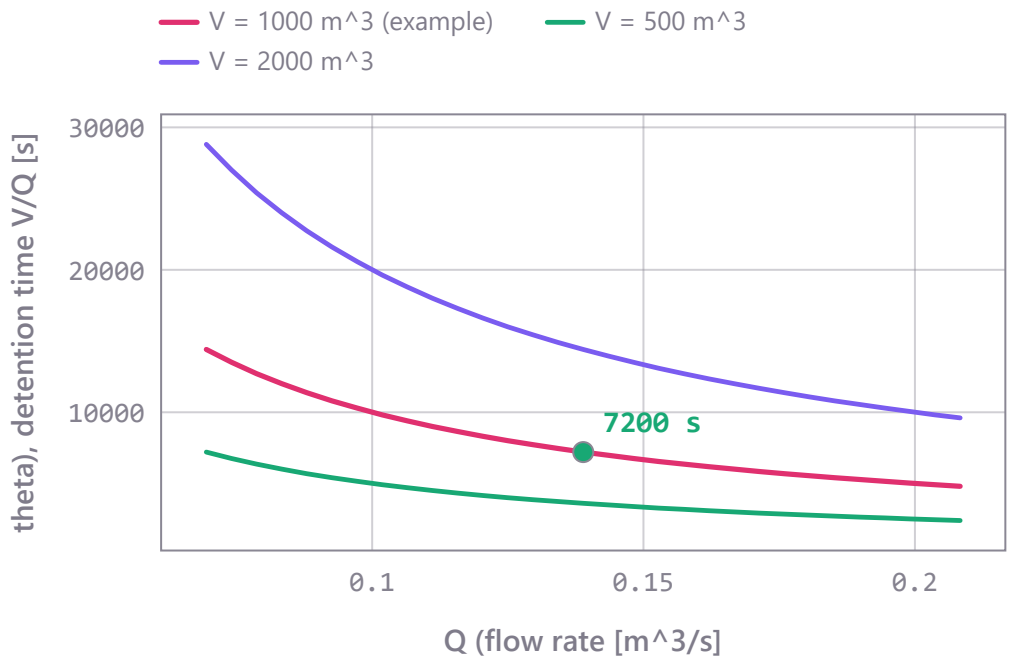
The hydraulic detention (residence) time of a tank or basin from its volume and the flow.

$$\theta = \frac{V}{Q}$$

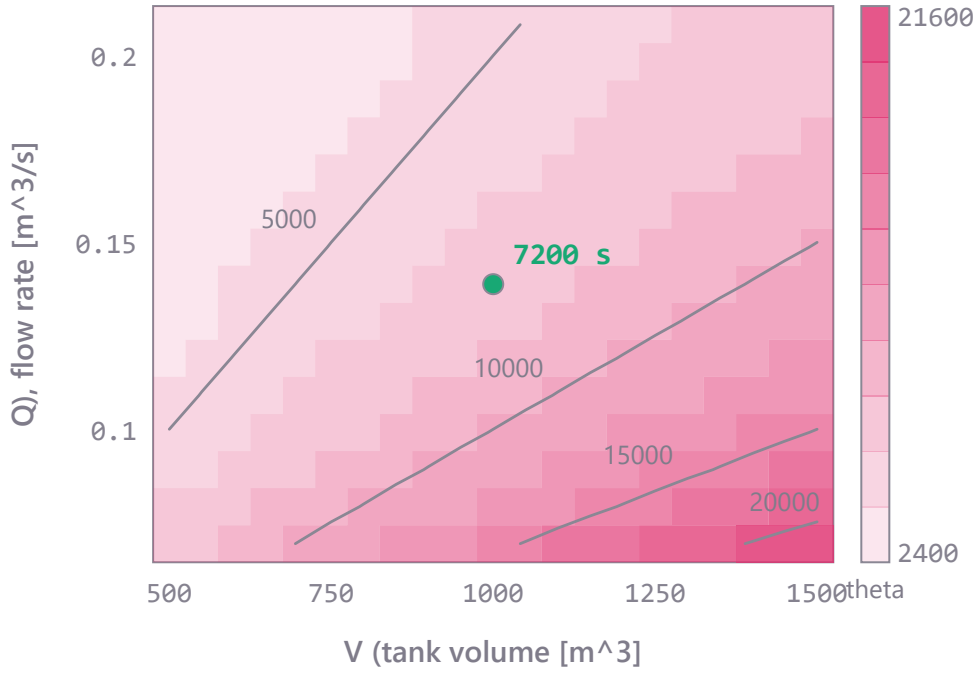
input	meaning	condition
`V`	tank volume	
`Q`	flow rate	

output	meaning
`theta`	detention time V/Q
`theta_h`	detention time in hours

- **Method:** $\theta = V/Q$
- **Assumes:** V, $Q > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta (detention time V/Q) against Q (flow rate) from 0.0694444 m^3/s to 0.208333 m^3/s. One curve for each value of V (tank volume): 1000 m^3, 500 m^3, 2000 m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Q = 0.138889 m^3/s, theta = 7200 s.



theta (detention time V/Q) over V (tank volume) from 500 m^3 to 1500 m^3 and Q (flow rate) from 0.0694444 m^3/s to 0.208333 m^3/s: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 2400 s to dark 21600 s); iso-lines at theta = 5000, 10000, 15000, 20000 s. Marked: the worked example, V = 1000 m^3, Q = 0.138889 m^3/s, theta = 7200 s.

```
environmental_engineering_ai999x.solve('enveng_detention_time', V=1000.0, Q=0.1388888888888889)
# -> theta = 7200 s, theta_h = 2   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_detention_time', V=250.0, Q=0.05)
# -> theta = 5000 s, theta_h = 1.38889   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Detention time: V = 1000 m^3, Q = 12000 m^3/d	SOLVED	theta = 7200 s, theta_h = 2
Detention time: V = 250 m^3, Q = 4320 m^3/d	SOLVED	theta = 5000 s, theta_h = 1.38889
Detention time: Q = 12000 m^3/d	NEEDS_INPUT	enveng_detention_time needs V (tank volume). Example: solve('enveng_detention_time', V='1000 m^3', Q='0.138889 m^3/s').
Detention time: V = 1000 m^3, Q = 12000 m^3/d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Detention time: V = 1000 m^3, Q = 500 m^3/h	SOLVED	theta = 7200 s, theta_h = 2

Surface overflow rate of a circular clarifier, `enveng_overflow_rate_circular`

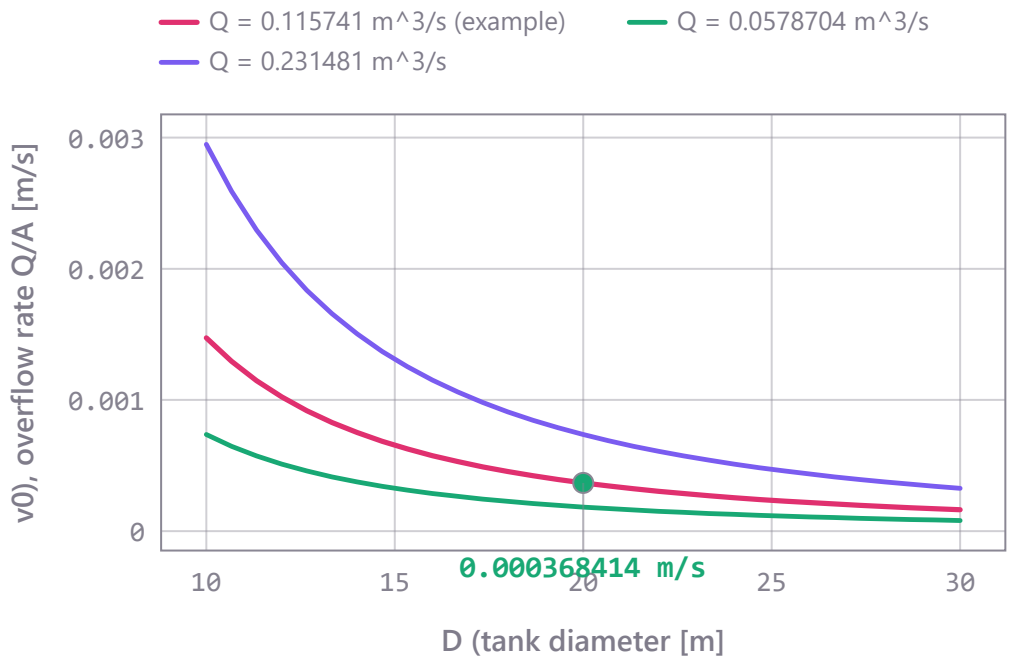
The surface area and the surface overflow rate (surface loading) of a circular settling tank from the flow and the tank diameter.

$$v_0 = \frac{Q}{\pi D^2/4}$$

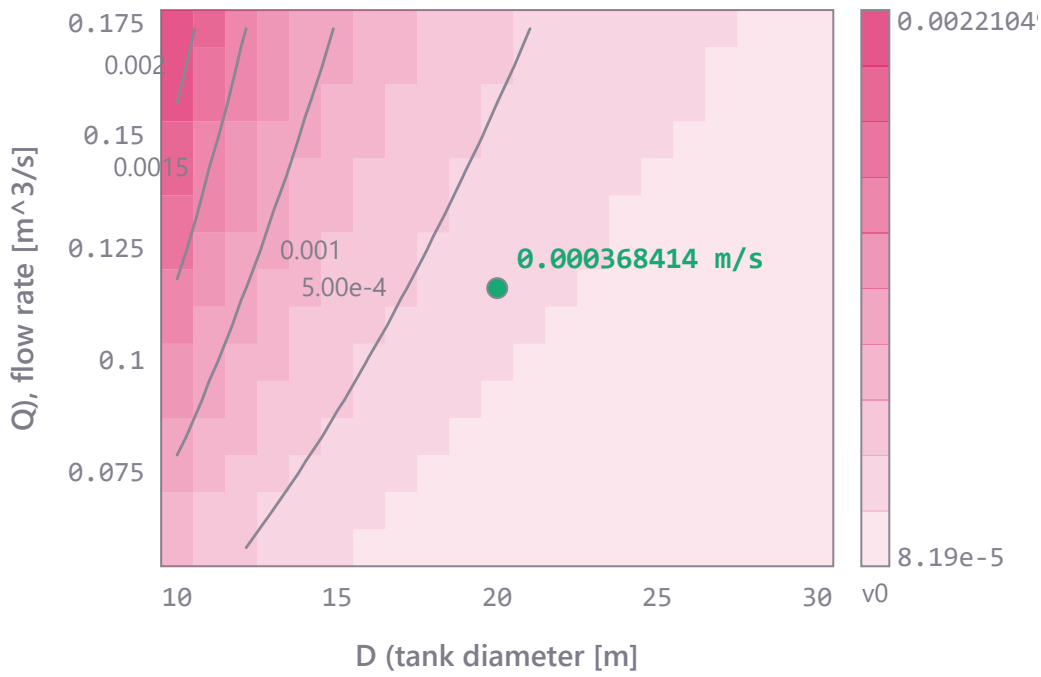
input	meaning	condition
`Q`	flow rate	
`D`	tank diameter	

output	meaning
`A`	surface area
`v0`	overflow rate Q/A
`v0_m_per_day`	overflow rate in m^3/(m^2 d)

- **Method:** $A = pi\ D^2/4$; $v_0 = Q/A$
- **Assumes:** Q, $D > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v0 (overflow rate Q/A) against D (tank diameter) from 10 m to 30 m. One curve for each value of Q (flow rate): 0.115741 m^3/s, 0.0578704 m^3/s, 0.231481 m^3/s. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, D = 20 m, v0 = 0.000368414 m/s.



v0 (overflow rate Q/A) over D (tank diameter) from 10 m to 30 m and Q (flow rate) from 0.0578704 m^3/s to 0.173611 m^3/s: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 8.1870e-05 m/s to dark 0.00221049 m/s); iso-lines at v0 = 5.00e-4, 0.001, 0.0015, 0.002 m/s. Marked: the worked example, D = 20 m, Q = 0.115741 m^3/s, v0 = 0.000368414 m/s.

```
environmental_engineering_ai999x.solve('enveng_overflow_rate_circular', Q=0.11574074074074074,
D=20.0)
# -> A = 314.159 m^2, v0 = 0.000368414 m/s, v0_m_per_day = 31.831   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_overflow_rate_circular', Q=0.5, D=35.0)
# -> A = 962.113 m^2, v0 = 0.00051969 m/s, v0_m_per_day = 44.9012   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Overflow rate of a circular clarifier: Q = 10000 m^3/d, D = 20 m	SOLVED	A = 314.159 m^2, v0 = 0.000368414 m/s, v0_m_per_day = 31.831
Overflow rate of a circular clarifier: Q = 43200 m^3/d, D = 35 m	SOLVED	A = 962.113 m^2, v0 = 0.00051969 m/s, v0_m_per_day = 44.9012
Overflow rate of a circular clarifier: D = 20 m	NEEDS_INPUT	enveng_overflow_rate_circular needs Q (flow rate). Example: solve('enveng_overflow_rate_circular', Q='0.115741 m^3/s', D
Overflow rate of a circular clarifier: Q = 10000 m^3/d, D = 20 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sedimentation basin sizing from overflow rate and detention time,

`enveng_sedimentation_basin_sizing`

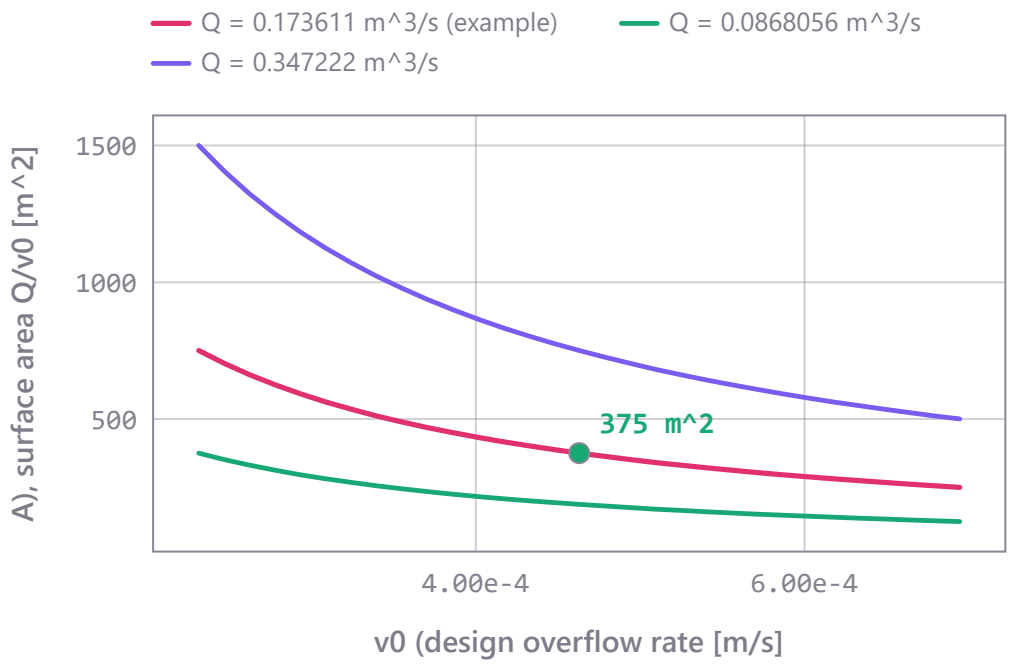
The surface area, volume, side-water depth and (circular) diameter of a settling basin from the design flow, the design overflow rate and the detention time.

$$A = \frac{Q}{v_0},\ V = Q\theta,\ h = v_0\theta$$

input	meaning	condition
`Q`	design flow	
`v0`	design overflow rate (surface loading)	
`theta`	design detention time	

output	meaning
`A`	surface area Q/v_0
`V`	volume Q theta
`depth`	depth $V/A = v_0$ theta
`D`	diameter of one circular basin

- **Method:** $A = Q/v_0$; $V = Q$ theta; depth = V/A; $D = sqrt$(4A/pi)
- **Assumes:** one basin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A (surface area Q/v0) against v0 (design overflow rate (surface loading)) from 0.000231481 m/s to 0.000694444 m/s. One curve for each value of Q (design flow): 0.173611 m^3/s, 0.0868056 m^3/s, 0.347222 m^3/s. Other inputs as in the worked example: theta = 7200.0 s. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, v0 = 0.000462963 m/s, A = 375 m^2.

```
environmental_engineering_ai999x.solve('enveng_sedimentation_basin_sizing', Q=0.1736111111111111,
v0=0.000462962962962963, theta=7200.0)
# -> A = 375 m^2, V = 1250 m^3, depth = 3.33333 m, D = 21.851 m   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sedimentation_basin_sizing', Q=0.46296296296296297,
v0=0.0003472222222222224, theta=10800.0)
# -> A = 1333.33 m^2, V = 5000 m^3, depth = 3.75 m, D = 41.2026 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sedimentation basin sizing: Q = 15000 m^3/d, v0 = 40 m/d, theta = 2 h	SOLVED	A = 375 m^2, V = 1250 m^3, depth = 3.33333 m, D = 21.851 m
Sedimentation basin sizing: Q = 40000 m^3/d, v0 = 30 m/d, theta = 3 h	SOLVED	A = 1333.33 m^2, V = 5000 m^3, depth = 3.75 m, D = 41.2026 m
Sedimentation basin sizing: v0 = 40 m/d, theta = 2 h	NEEDS_INPUT	enveng_sedimentation_basin_sizing needs Q (design flow). Example: solve('enveng_sedimentation_basin_sizing', Q='0.173611
Sedimentation basin sizing: Q = 15000 m^3/d, v0 = 40 m/d, theta = 2 h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Weir loading rate of a circular clarifier, `enveng_weir_loading_rate`

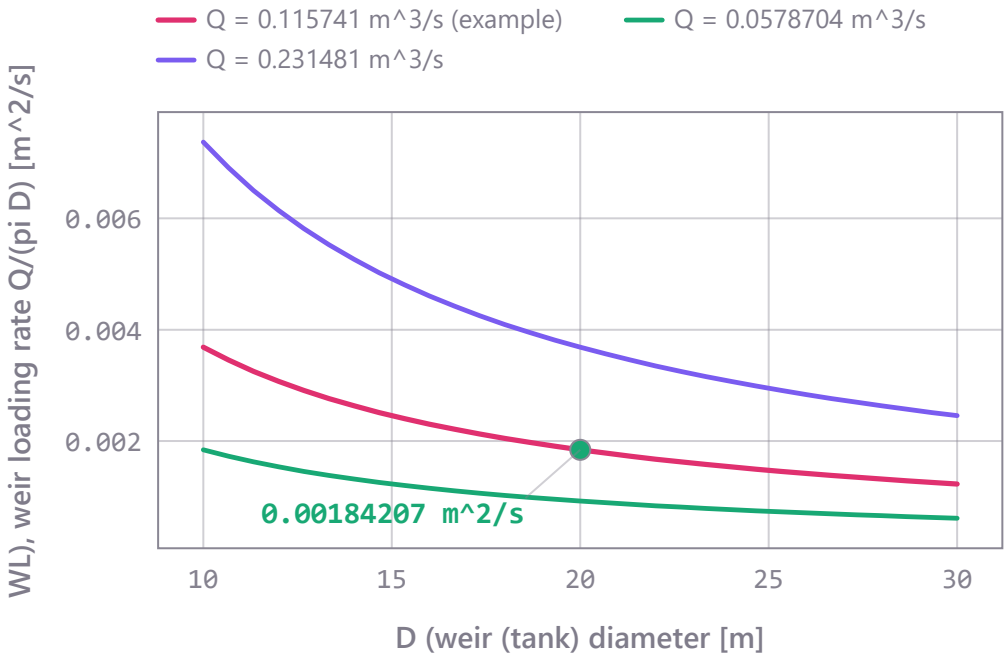
The weir length and weir loading rate of a circular clarifier with a peripheral weir.

$$WL = \frac{Q}{\pi D}$$

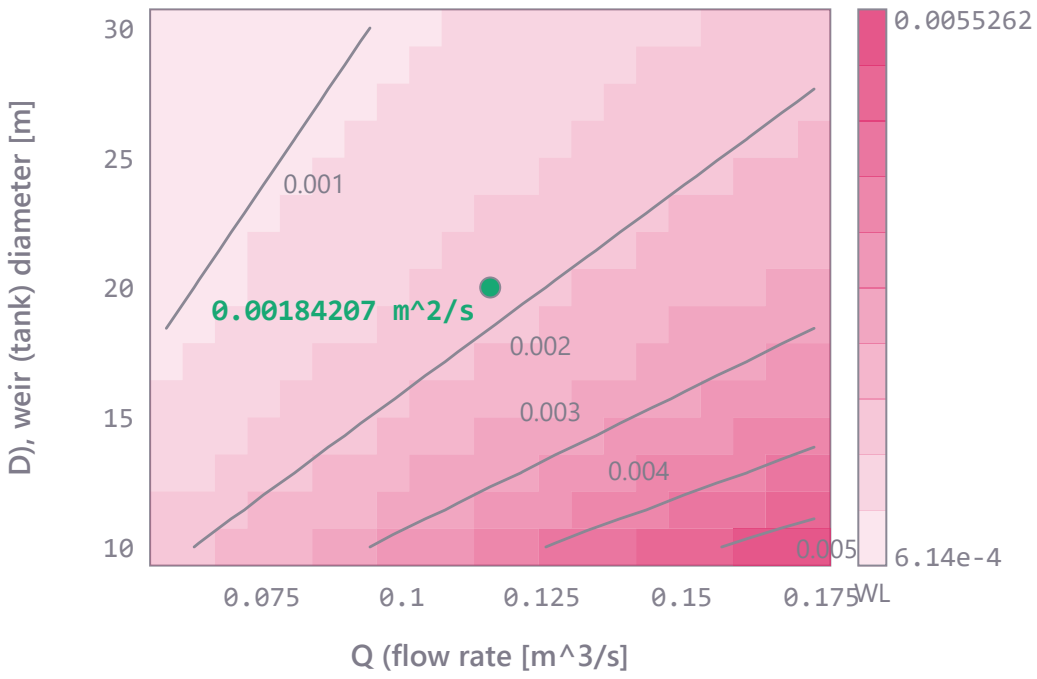
input	meaning	condition
`Q`	flow rate	
`D`	weir (tank) diameter	

output	meaning
`weir_length`	weir length pi D
`WL`	weir loading rate Q/(pi D)
`WL_m3_per_m_day`	weir loading in m^3/(m d)

- **Method:** WL = Q/(pi D)
- **Assumes:** peripheral weir
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



WL (weir loading rate Q/(pi D)) against D (weir (tank) diameter) from 10 m to 30 m. One curve for each value of Q (flow rate): 0.115741 m^3/s, 0.0578704 m^3/s, 0.231481 m^3/s. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, D = 20 m, WL = 0.00184207 m^2/s.



WL (weir loading rate Q/(pi D)) over Q (flow rate) from 0.0578704 m^3/s to 0.173611 m^3/s and D (weir (tank) diameter) from 10 m to 30 m: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 0.000614024 m^2/s to dark 0.00552621 m^2/s); iso-lines at WL = 0.001, 0.002, 0.003, 0.004, 0.005 m^2/s. Marked: the worked example, Q = 0.115741 m^3/s, D = 20 m, WL = 0.00184207 m^2/s.

```
environmental_engineering_ai999x.solve('enveng_weir_loading_rate', Q=0.11574074074074074, D=20.0)
# -> weir_length = 62.8319 m, WL = 0.00184207 m^2/s, WL_m3_per_m_day = 159.155   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_weir_loading_rate', Q=0.3472222222222222, D=32.0)
# -> weir_length = 100.531 m, WL = 0.00345388 m^2/s, WL_m3_per_m_day = 298.416   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weir loading rate: Q = 10000 m^3/d, D = 20 m	SOLVED	weir_length = 62.8319 m, WL = 0.00184207 m^2/s, WL_m3_per_m_day = 159.155
Weir loading rate: Q = 30000 m^3/d, D = 32 m	SOLVED	weir_length = 100.531 m, WL = 0.00345388 m^2/s, WL_m3_per_m_day = 298.416
Weir loading rate: D = 20 m	NEEDS_INPUT	enveng_weir_loading_rate needs Q (flow rate). Example: solve('enveng_weir_loading_rate', Q='0.115741 m^3/s', D='20 m').
Weir loading rate: Q = 10000 m^3/d, D = 20 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Stokes settling velocity of a discrete particle, `enveng_stokes_settling_velocity`

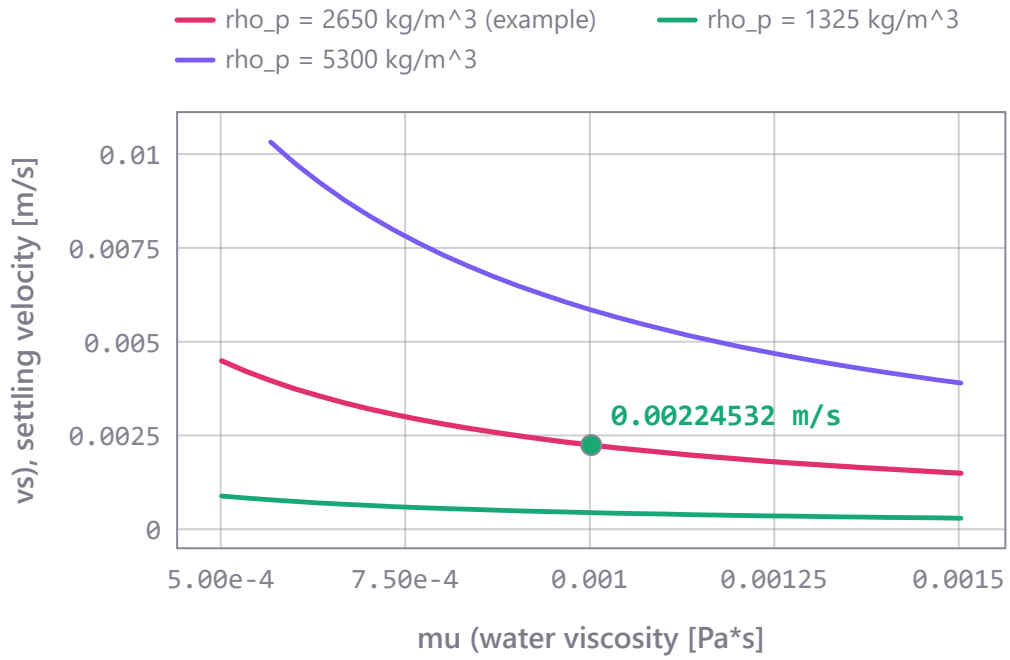
The terminal settling velocity of a small spherical particle in water by Stokes' law, with the particle Reynolds number that confirms laminar settling.

$$v_s = \frac{g(\rho_p - \rho_w)d^2}{18\mu}$$

input	meaning	condition
`d`	particle diameter	
`rho_p`	particle density	
`rho_w`	water density	
`mu`	water viscosity	

output	meaning
`vs`	settling velocity
`Re`	particle Reynolds number
`vs_m_per_h`	in m/h

- **Method:** $vs = g$ (rho_p - rho_w) d^2/(18 mu)
- **Assumes:** $Re \leq 1$; $rho_p > rho_w$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



vs (settling velocity) against mu (water viscosity) from 0.000501 Pa*s to 0.001503 Pa*s. One curve for each value of rho_p (particle density): 2650 kg/m^3, 1325 kg/m^3, 5300 kg/m^3. Other inputs as in the worked example: d = 5e-05 m, rho_w = 998.2 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, mu = 0.001002 Pa*s, vs = 0.00224532 m/s.

```
environmental_engineering_ai999x.solve('enveng_stokes_settling_velocity', d=5e-05, rho_p=2650.0, rho_w=998.2, mu=0.001002)
# -> vs = 0.00224532 m/s, Re = 0.11184, vs_m_per_h = 8.08315   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_stokes_settling_velocity', d=2e-05, rho_p=1200.0, rho_w=999.7, mu=0.001307)
# -> vs = 3.33975e-05 m/s, Re = 0.000510902, vs_m_per_h = 0.120231   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stokes settling velocity: d = 0.05 mm, rho_p = 2650 kg/m^3, rho_w = 998.2 kg/m^3, mu = 0.001002 Pa*s	SOLVED	vs = 0.00224532 m/s, Re = 0.11184, vs_m_per_h = 8.08315
Stokes settling velocity: d = 0.02 mm, rho_p = 1200 kg/m^3, rho_w = 999.7 kg/m^3, mu = 0.001307 Pa*s	SOLVED	vs = 3.33975e-05 m/s, Re = 0.000510902, vs_m_per_h = 0.120231
Stokes settling velocity: rho_p = 2650 kg/m^3, rho_w = 998.2 kg/m^3, mu = 0.001002 Pa*s	NEEDS_INPUT	enveng_stokes_settling_velocity needs d (particle diameter). Example: solve('enveng_stokes_settling_velocity', d='5e-05
Stokes settling velocity: d = 0.05 mm, rho_p = 2650 kg/m^3, rho_w = 998.2 kg/m^3, mu = 0.001002 Pa*s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Terminal settling velocity in any flow regime (drag coefficient iteration),

`enveng_terminal_settling_velocity`

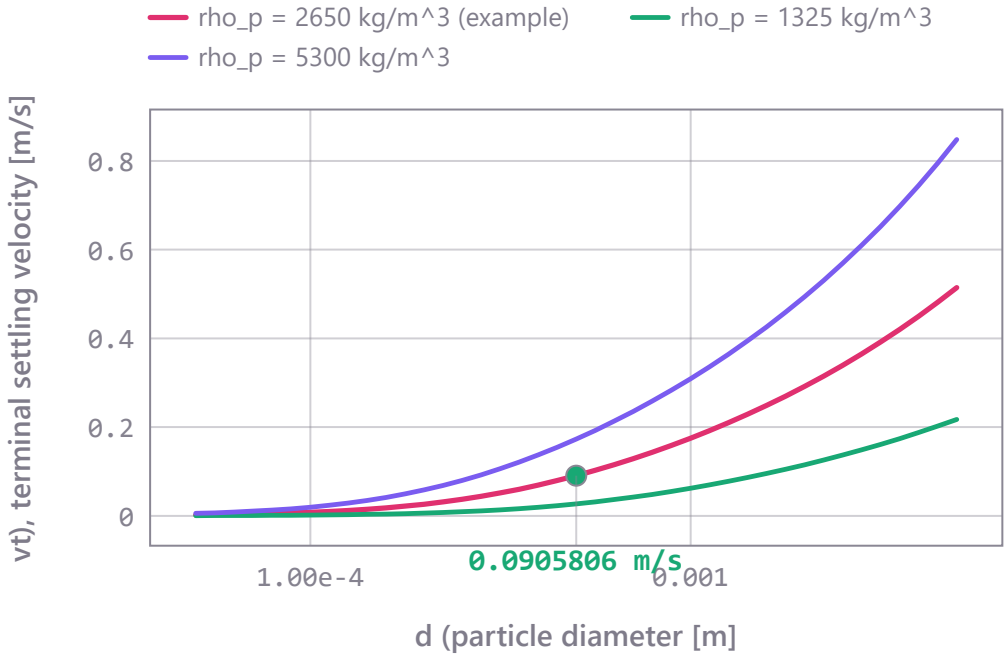
The terminal settling velocity of a sphere in a fluid for laminar, transitional or turbulent settling, solving the force balance with the drag law $Cd = 24/Re + 3/sqrt$(Re) + 0.34.

$$v_t = \sqrt{\frac{4gd(\rho_p - \rho_f)}{3C_D\rho_f}}, \quad C_D = \frac{24}{Re} + \frac{3}{\sqrt{Re}} + 0.34$$

input	meaning	condition
`d`	particle diameter	
`rho_p`	particle density	
`rho_f`	fluid density	
`mu`	fluid viscosity	

output	meaning
`vt`	terminal settling velocity
`Re`	particle Reynolds number
`Cd`	drag coefficient

- **Method:** vt^2 Cd(Re) = (4/3) g d (rho_p - rho_f)/rho_f with $Cd = \frac{24}{Re} + \frac{3}{\sqrt{Re}} + 0.34$, solved by bisection below the Stokes velocity
- **Assumes:** $Re \leq 2e5$; $rho_p > rho_f$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



vt (terminal settling velocity) against d (particle diameter) from 5.0000e-05 m to 0.005 m on a logarithmic axis. One curve for each value of rho_p (particle density): 2650 kg/m^3, 1325 kg/m^3, 5300 kg/m^3. Other inputs as in the worked example: rho_f = 998.2 kg/m^3, mu = 0.001002 Pa*s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, d = 0.0005 m, vt = 0.0905806 m/s.

```
environmental_engineering_ai999x.solve('enveng_terminal_settling_velocity', d=0.0005, rho_p=2650.0,
rho_f=998.2, mu=0.001002)
# -> vt = 0.0905806 m/s, Re = 45.1185, Cd = 1.31856   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_terminal_settling_velocity', d=0.002, rho_p=1500.0,
rho_f=998.2, mu=0.001)
# -> vt = 0.148638 m/s, Re = 296.742, Cd = 0.595032   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Terminal settling velocity: d = 0.5 mm, rho_p = 2650 kg/m^3, rho_f = 998.2 kg/m^3, mu = 0.001002 Pa*s	SOLVED	vt = 0.0905806 m/s, Re = 45.1185, Cd = 1.31856
Terminal settling velocity: d = 2 mm, rho_p = 1500 kg/m^3, rho_f = 998.2 kg/m^3, mu = 0.001 Pa*s	SOLVED	vt = 0.148638 m/s, Re = 296.742, Cd = 0.595032
Terminal settling velocity: rho_p = 2650 kg/m^3, rho_f = 998.2 kg/m^3, mu = 0.001002 Pa*s	NEEDS_INPUT	enveng_terminal_settling_velocity needs d (particle diameter). Example: solve('enveng_terminal_settling_velocity', d='0.
Terminal settling velocity: d = 0.5 mm, rho_p = 2650 kg/m^3, rho_f = 998.2 kg/m^3, mu = 0.001002 Pa*s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Fraction of discrete particles removed in an ideal settling basin, `enveng_ideal_basin_removal`

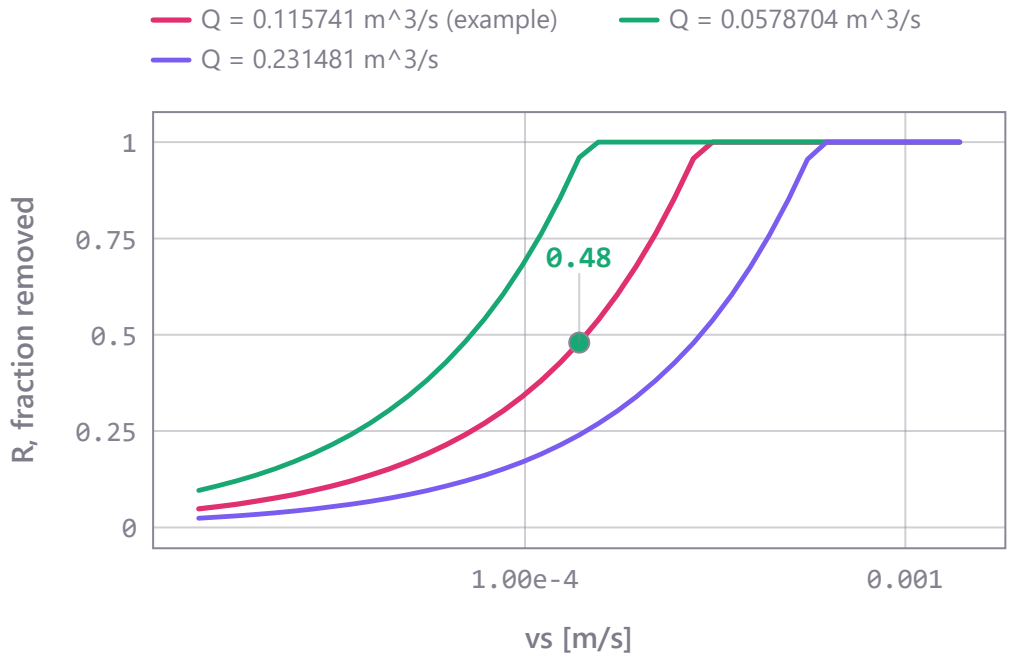
The overflow rate of an ideal (Camp) settling basin and the fraction of particles of one settling velocity it removes (all if $vs \geq v_0$, otherwise vs/v0).

$$R = \min\left(1, \frac{v_s}{v_0}\right)$$

input	meaning	condition
`vs`	settling velocity of the particles	
`Q`	flow	
`A`	basin surface area	

output	meaning
`v0`	overflow rate Q/A
`R`	fraction removed

- **Method:** $v_0 = Q/A$; R = min(1, vs/v0)
- **Assumes:** ideal basin, discrete (type I) settling
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (fraction removed) against vs (settling velocity of the particles) from 1.3889e-05 m/s to 0.00138889 m/s on a logarithmic axis. One curve for each value of Q (flow): 0.115741 m^3/s, 0.0578704 m^3/s, 0.231481 m^3/s. Other inputs as in the worked example: A = 400.0 m^2. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, vs = 0.000138889 m/s, R = 0.48.

```
environmental_engineering_ai999x.solve('enveng_ideal_basin_removal', vs=0.0001388888888888889,
Q=0.11574074074074074, A=400.0)
# -> v0 = 0.000289352 m/s, R = 0.48   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_ideal_basin_removal', vs=0.0005555555555555556,
Q=0.23148148148148148, A=300.0)
# -> v0 = 0.000771605 m/s, R = 0.72   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ideal settling basin removal: vs = 0.5 m/h, Q = 10000 m^3/d, A = 400 m^2	SOLVED	v0 = 0.000289352 m/s, R = 0.48
Ideal settling basin removal: vs = 2 m/h, Q = 20000 m^3/d, A = 300 m^2	SOLVED	v0 = 0.000771605 m/s, R = 0.72
Ideal settling basin removal: Q = 10000 m^3/d, A = 400 m^2	NEEDS_INPUT	enveng_ideal_basin_removal needs vs (settling velocity of the particles). Example: solve('enveng_ideal_basin_removal', v
Ideal settling basin removal: vs = 0.5 m/h, Q = 10000 m^3/d, A = 400 m^2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Filtration (hydraulic loading) rate with one filter out of service, `enveng_filtration_rate`

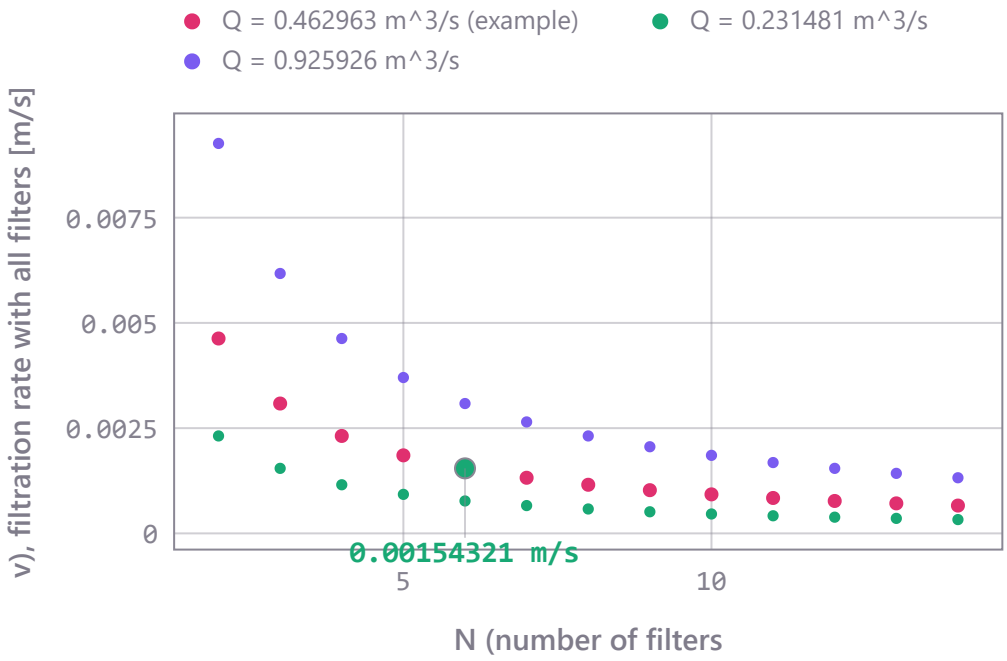
The filtration rate of a bank of identical filters, with all in service and with one out of service (for backwashing).

$$v = \frac{Q}{NA}, \quad v' = \frac{Q}{(N-1)A}$$

input	meaning	condition
`Q`	plant flow	
`A`	area of one filter	
`N`	number of filters	

output	meaning
`v`	filtration rate with all filters
`v_m_per_h`	in m/h
`v_one_out`	rate with one filter out of service
`v_one_out_m_per_h`	in m/h

- **Method:** v = Q/(N A); v' = Q/((N − 1) A)
- **Assumes:** $N \geq 2$ identical filters
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (filtration rate with all filters) against N (number of filters) from 1 to 14. One curve for each value of Q (plant flow): 0.462963 m^3/s, 0.231481 m^3/s, 0.925926 m^3/s. Other inputs as in the worked example: A = 50.0 m^2. 13 points per curve computed by environmental-engineering-ai999x; marked: the worked example, N = 6, v = 0.00154321 m/s.

```
environmental_engineering_ai999x.solve('enveng_filtration_rate', Q=0.46296296296296297, A=50.0,
N=6)
# -> v = 0.00154321 m/s, v_m_per_h = 5.55556, v_one_out = 0.00185185 m/s, v_one_out_m_per_h =
6.66667   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_filtration_rate', Q=0.1388888888888889, A=30.0, N=4)
# -> v = 0.00115741 m/s, v_m_per_h = 4.16667, v_one_out = 0.00154321 m/s, v_one_out_m_per_h =
5.55556   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Filtration rate: Q = 40000 m^3/d, A = 50 m^2, N = 6	SOLVED	v = 0.00154321 m/s, v_m_per_h = 5.55556, v_one_out = 0.00185185 m/s, v_one_out_m_per_h = 6.66667
Filtration rate: Q = 12000 m^3/d, A = 30 m^2, N = 4	SOLVED	v = 0.00115741 m/s, v_m_per_h = 4.16667, v_one_out = 0.00154321 m/s, v_one_out_m_per_h = 5.55556
Filtration rate: A = 50 m^2, N = 6	NEEDS_INPUT	enveng_filtration_rate needs Q (plant flow). Example: solve('enveng_filtration_rate', Q='0.462963 m^3/s', A='50 m^2', N=
Filtration rate: Q = 40000 m^3/d, A = 50 m^2, N = 6, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Clean-bed head loss through a granular filter (Ergun / Carman-Kozeny),

`enveng_filter_clean_bed_headloss`

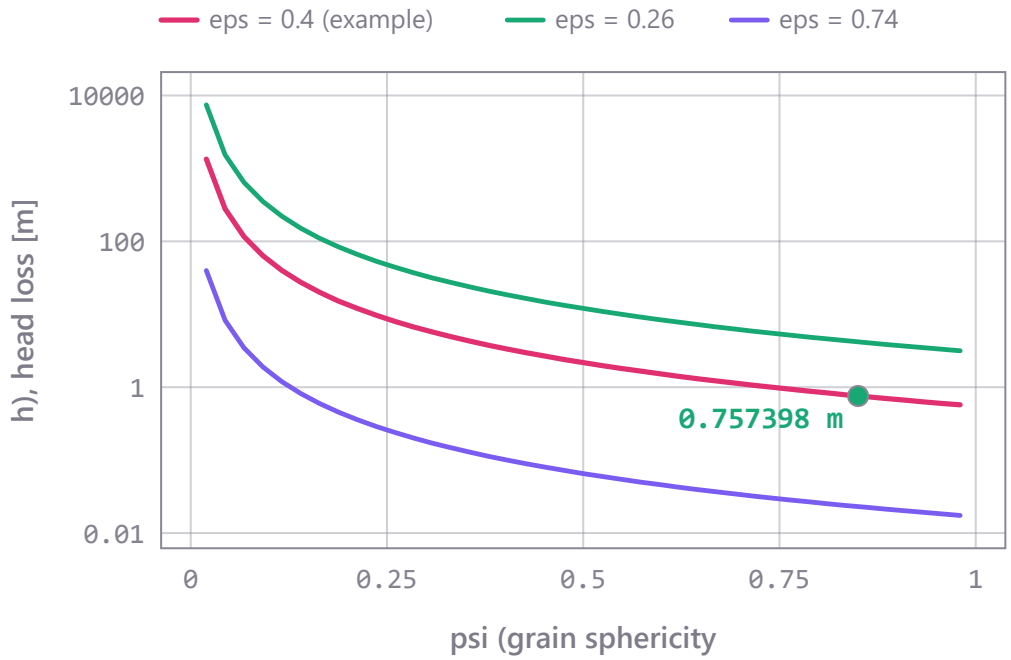
The head loss through a clean granular filter bed from the grain size, sphericity, porosity, depth and filtration rate (Ergun equation, the Carman-Kozeny form with friction factor 150/Re + 1.75).

$$\frac{h}{L} = \frac{150\mu(1-\varepsilon)^2 v}{\rho g \varepsilon^3 (\psi d)^2} + \frac{1.75(1-\varepsilon)v^2}{g\varepsilon^3 \psi d}$$

input	meaning	condition
`d`	grain diameter	
`eps`	bed porosity (fraction)	
`psi`	grain sphericity (fraction, 1 for spheres)	
`L`	bed depth	
`v`	filtration (approach) velocity	
`rho`	water density	
`mu`	water viscosity	

output	meaning
`dP`	pressure drop
`h`	head loss
`Re`	bed Reynolds number rho v psi d/(mu (1 - eps))

- **Method:** $dP/L = 150$ mu (1-eps)^2 v/(eps^3 (psi d)^2) + 1.75 rho (1-eps) v^2/(eps^3 psi d); h = dP/(rho g)
- **Assumes:** 0 < eps < 1; $0 < \psi \leq 1$; clean bed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



h (head loss) against psi (grain sphericity (fraction, 1 for spheres)) from 0.02 to 0.98, logarithmic output axis. One curve for each value of eps (bed porosity (fraction)): 0.4, 0.26, 0.74. Other inputs as in the worked example: d = 0.0005 m, L = 0.7 m, v = 0.0022222222222222222 m/s, rho = 998.2 kg/m^3, mu = 0.001002 Pa*s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, psi = 0.85, h = 0.757398 m.

```
environmental_engineering_ai999x.solve('enveng_filter_clean_bed_headloss', d=0.0005, eps=0.4,
psi=0.85, L=0.7, v=0.0022222222222222222, rho=998.2, mu=0.001002)
# -> dP = 7414.17 Pa, h = 0.757398 m, Re = 1.5681   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_filter_clean_bed_headloss', d=0.001, eps=0.45,
psi=0.75, L=1.0, v=0.0033333333333333335, rho=999.7, mu=0.001307)
# -> dP = 4013.09 Pa, h = 0.409344 m, Re = 3.47673   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Filter clean-bed head loss: d = 0.5 mm, eps = 0.4, psi = 0.85, L = 0.7 m, v = 8 m/h, rho = 998.2 kg/m^3, mu = 0.001002 Pa*s	SOLVED	dP = 7414.17 Pa, h = 0.757398 m, Re = 1.5681
Filter clean-bed head loss: d = 1 mm, eps = 0.45, psi = 0.75, L = 1 m, v = 12 m/h, rho = 999.7 kg/m^3, mu = 0.001307 Pa*s	SOLVED	dP = 4013.09 Pa, h = 0.409344 m, Re = 3.47673
Filter clean-bed head loss: eps = 0.4, psi = 0.85, L = 0.7 m, v = 8 m/h, rho = 998.2 kg/m^3, mu = 0.001002 Pa*s	NEEDS_INPUT	enveng_filter_clean_bed_headloss needs d (grain diameter). Example: solve('enveng_filter_clean_bed_headloss', d='0.0005
Filter clean-bed head loss: d = 0.5 mm, eps = 0.4, psi = 0.85, L = 0.7 m, v = 8 m/h, rho = 998.2 kg/m^3, mu = 0.001002 Pa*s, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Filter bed expansion during backwash, `enveng_backwash_bed_expansion`

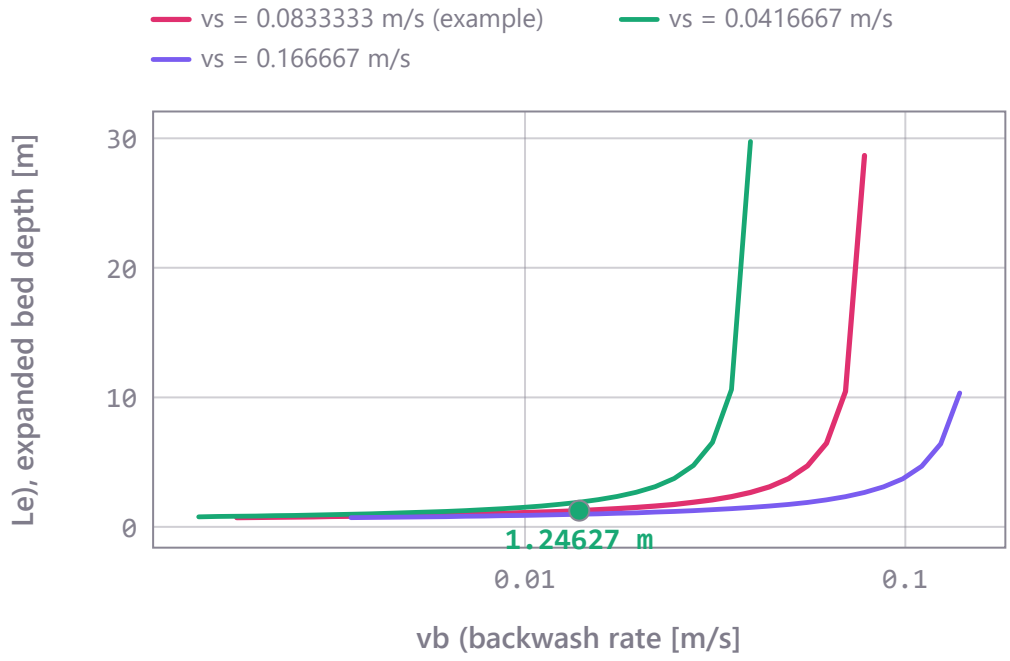
The expanded porosity, expanded depth and percentage expansion of a filter bed fluidised by backwash, from the backwash rate and the grains' settling velocity (expanded porosity (vb/vs)^0.22).

$$\varepsilon_e = \left(\frac{v_b}{v_s}\right)^{0.22}, \quad L_e = \frac{L(1-\varepsilon)}{1-\varepsilon_e}$$

input	meaning	condition
`L`	unexpanded bed depth	
`eps`	unexpanded porosity (fraction)	
`vb`	backwash rate	
`vs`	settling velocity of the grains	

output	meaning
`eps_e`	expanded porosity
`Le`	expanded bed depth
`expansion`	fractional expansion $Le/L - 1$

- **Method:** eps_e = (vb/vs)^0.22; $Le = L$ (1 - eps)/(1 - eps_e)
- **Assumes:** the bed is fluidised: eps < (vb/vs)^0.22 < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Le (expanded bed depth) against vb (backwash rate) from 0.00138889 m/s to 0.138889 m/s on a logarithmic axis. One curve for each value of vs (settling velocity of the grains): 0.0833333 m/s, 0.0416667 m/s, 0.166667 m/s. Other inputs as in the worked example: L = 0.7 m, eps = 0.42. 34 points per curve computed by environmental-engineering-ai999x; marked: the worked example, vb = 0.0138889 m/s, Le = 1.24627 m.

```
environmental_engineering_ai999x.solve('enveng_backwash_bed_expansion', L=0.7, eps=0.42,
vb=0.013888888888888888, vs=0.08333333333333333)
# -> eps_e = 0.674228, Le = 1.24627 m, expansion = 0.780386   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_backwash_bed_expansion', L=1.0, eps=0.4,
vb=0.008333333333333333, vs=0.06944444444444445)
# -> eps_e = 0.62722, Le = 1.60953 m, expansion = 0.609528   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Backwash bed expansion: L = 0.7 m, eps = 0.42, vb = 50 m/h, vs = 300 m/h	SOLVED	eps_e = 0.674228, Le = 1.24627 m, expansion = 0.780386
Backwash bed expansion: L = 1 m, eps = 0.4, vb = 30 m/h, vs = 250 m/h	SOLVED	eps_e = 0.62722, Le = 1.60953 m, expansion = 0.609528
Backwash bed expansion: eps = 0.42, vb = 50 m/h, vs = 300 m/h	NEEDS_INPUT	enveng_backwash_bed_expansion needs L (unexpanded bed depth). Example: solve('enveng_backwash_bed_expansion', L='0.7 m',
Backwash bed expansion: L = 0.7 m, eps = 0.42, vb = 50 m/h, vs = 300 m/h, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Mean velocity gradient G (rapid mixing and flocculation), `enveng_velocity_gradient`

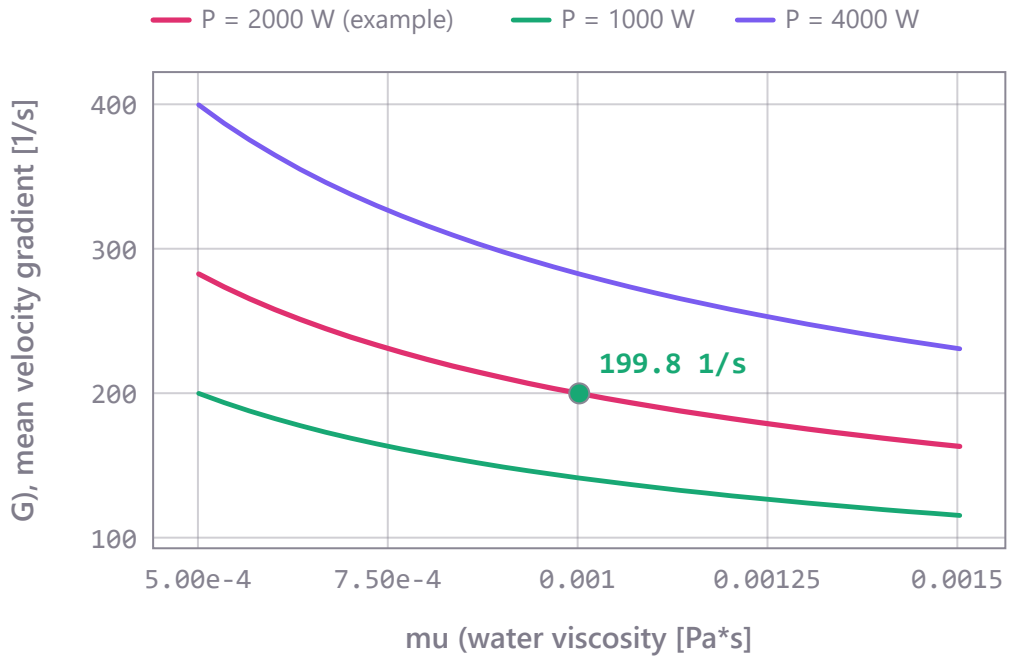
The root-mean-square velocity gradient G of a mixing or flocculation tank from the power dissipated, the viscosity and the volume, and the Camp number G t when the detention time is given.

$$G = \sqrt{\frac{P}{\mu V}}$$

input	meaning	condition
`P`	power dissipated in the water	
`mu`	water viscosity	
`V`	tank volume	
`t`	detention time (for G t)	

output	meaning
`G`	mean velocity gradient
`Gt`	Camp number G t (dimensionless)

- **Method:** $G = sqrt$(P/(mu V)); Camp number G t
- **Assumes:** P, mu, $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G (mean velocity gradient) against mu (water viscosity) from 0.000501 Pa*s to 0.001503 Pa*s. One curve for each value of P (power dissipated in the water): 2000 W, 1000 W, 4000 W. Other inputs as in the worked example: V = 50.0 m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, mu = 0.001002 Pa*s, G = 199.8 1/s.

```
environmental_engineering_ai999x.solve('enveng_velocity_gradient', P=2000.0, mu=0.001002, V=50.0)
# -> G = 199.8 1/s    (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_velocity_gradient', P=500.0, mu=0.001307, V=400.0,
t=1800.0)
# -> G = 30.9255 1/s, Gt = 55666    (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Velocity gradient: P = 2000 W, mu = 0.001002 Pa*s, V = 50 m^3	SOLVED	G = 199.8 1/s
Velocity gradient: P = 500 W, mu = 0.001307 Pa*s, V = 400 m^3, t = 30 min	SOLVED	G = 30.9255 1/s, Gt = 55666
Velocity gradient: mu = 0.001002 Pa*s, V = 50 m^3	NEEDS_INPUT	enveng_velocity_gradient needs P (power dissipated in the water). Example: solve('enveng_velocity_gradient', P='2000 W',
Velocity gradient: P = 2000 W, mu = 0.001002 Pa*s, V = 50 m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Power needed for a target velocity gradient, `enveng_mixing_power`

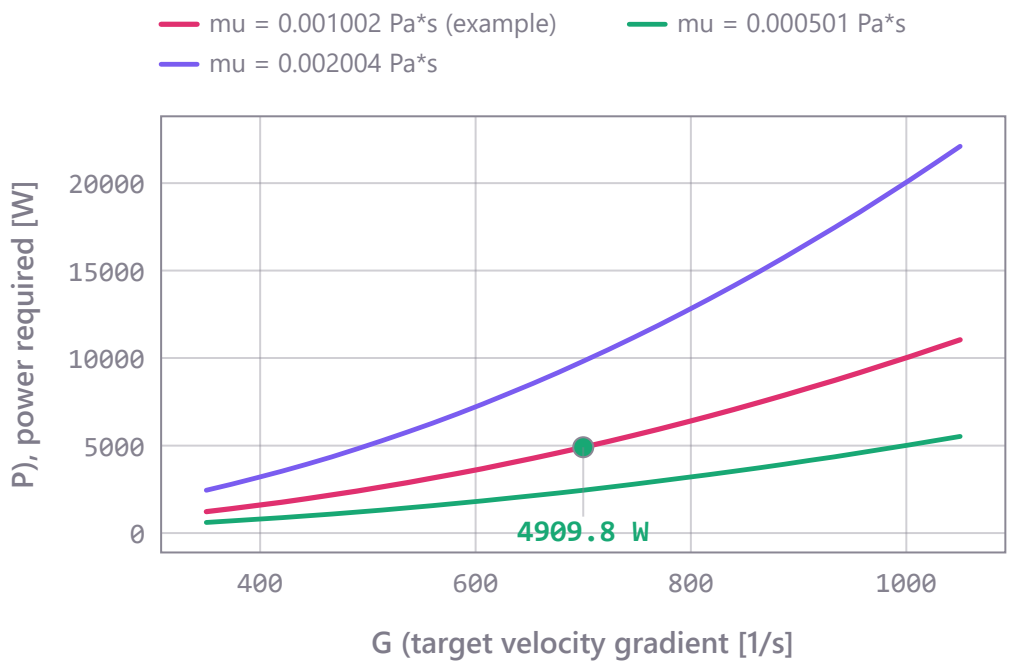
The power that must be dissipated in a tank to give a target mean velocity gradient G.

$$P = G^2 \mu V$$

input	meaning	condition
`G`	target velocity gradient	
`mu`	water viscosity	
`V`	tank volume	

output	meaning
`P`	power required

- **Method:** $P = G^2$ mu V
- **Assumes:** G, mu, $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (power required) against G (target velocity gradient) from 350 1/s to 1050 1/s. One curve for each value of mu (water viscosity): 0.001002 Pa*s, 0.000501 Pa*s, 0.002004 Pa*s. Other inputs as in the worked example: V = 10.0 m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, G = 700 1/s, P = 4909.8 W.

```
environmental_engineering_ai999x.solve('enveng_mixing_power', G=700.0, mu=0.001002, V=10.0)
# -> P = 4909.8 W   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_mixing_power', G=40.0, mu=0.001307, V=500.0)
# -> P = 1045.6 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mixing power: G = 700 1/s, mu = 0.001002 Pa*s, V = 10 m^3	SOLVED	P = 4909.8 W
Mixing power: G = 40 1/s, mu = 0.001307 Pa*s, V = 500 m^3	SOLVED	P = 1045.6 W

question	outcome	result
Mixing power: mu = 0.001002 Pa*s, V = 10 m^3	NEEDS_INPUT	enveng_mixing_power needs G (target velocity gradient). Example: solve('enveng_mixing_power', G='700 1/s', mu='0.001002
Mixing power: G = 700 1/s, mu = 0.001002 Pa*s, V = 10 m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Disinfection CT value of a contact basin, `enveng_disinfection_ct`

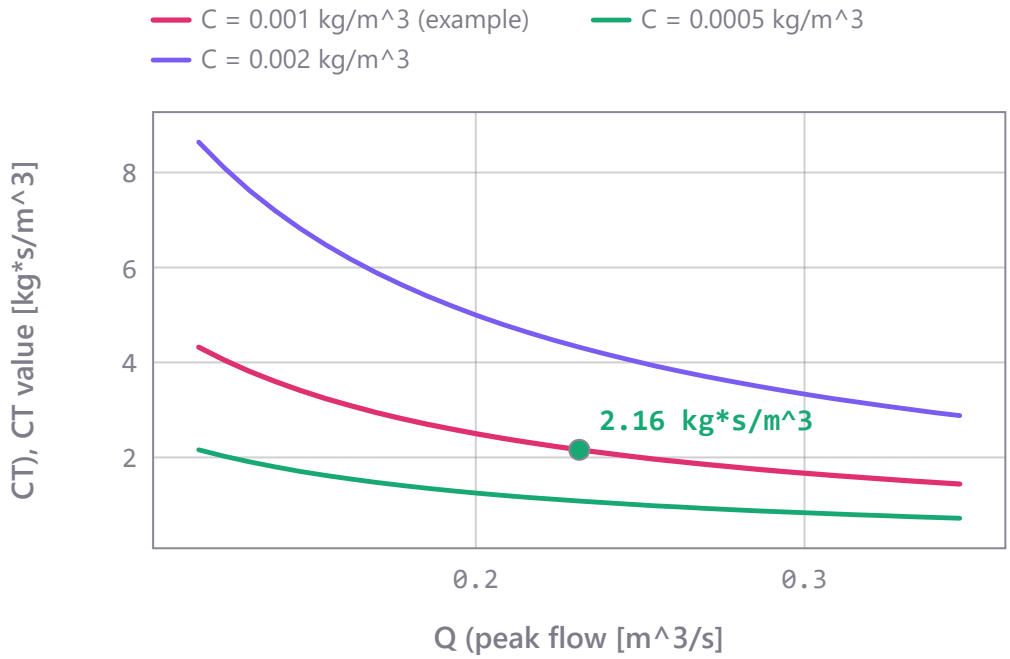
The effective contact time t10 of a basin (baffling factor times the theoretical detention time) and the CT value of a disinfectant residual.

$$CT = C\,t_{10} = C\,BF\,\frac{V}{Q}$$

input	meaning	condition
`C`	disinfectant residual at the outlet	
`V`	basin volume	
`Q`	peak flow	
`BF`	baffling factor $t10/T$ (0.1 unbaffled to 1 plug flow)	

output	meaning
`t10`	effective contact time
`CT`	CT value
`CT_mg_min_L`	CT in mg min/L

- **Method:** $t10 = BF$ V/Q; $CT = C$ t10
- **Assumes:** $0 < BF \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



CT (CT value) against Q (peak flow) from 0.115741 m^3/s to 0.347222 m^3/s. One curve for each value of C (disinfectant residual at the outlet): 0.001 kg/m^3, 0.0005 kg/m^3, 0.002 kg/m^3. Other inputs as in the worked example: V = 1000.0 m^3, BF = 0.5. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Q = 0.231481 m^3/s, CT = 2.16 kg*s/m^3.

```
environmental_engineering_ai999x.solve('enveng_disinfection_ct', C=0.001, V=1000.0,
Q=0.23148148148148148, BF=0.5)
# -> t10 = 2160 s, CT = 2.16 kg*s/m^3, CT_mg_min_L = 36   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_disinfection_ct', C=0.0008, V=2500.0,
Q=0.46296296296296297, BF=0.7)
# -> t10 = 3780 s, CT = 3.024 kg*s/m^3, CT_mg_min_L = 50.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Disinfection CT: C = 1 mg/L, V = 1000 m^3, Q = 20000 m^3/d, BF = 0.5	SOLVED	t10 = 2160 s, CT = 2.16 kg*s/m^3, CT_mg_min_L = 36
Disinfection CT: C = 0.8 mg/L, V = 2500 m^3, Q = 40000 m^3/d, BF = 0.7	SOLVED	t10 = 3780 s, CT = 3.024 kg*s/m^3, CT_mg_min_L = 50.4
Disinfection CT: V = 1000 m^3, Q = 20000 m^3/d, BF = 0.5	NEEDS_INPUT	enveng_disinfection_ct needs C (disinfectant residual at the outlet). Example: solve('enveng_disinfection_ct', C='0.001
Disinfection CT: C = 1 mg/L, V = 1000 m^3, Q = 20000 m^3/d, BF = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Log inactivation from the CT ratio, `enveng_ct_log_inactivation`

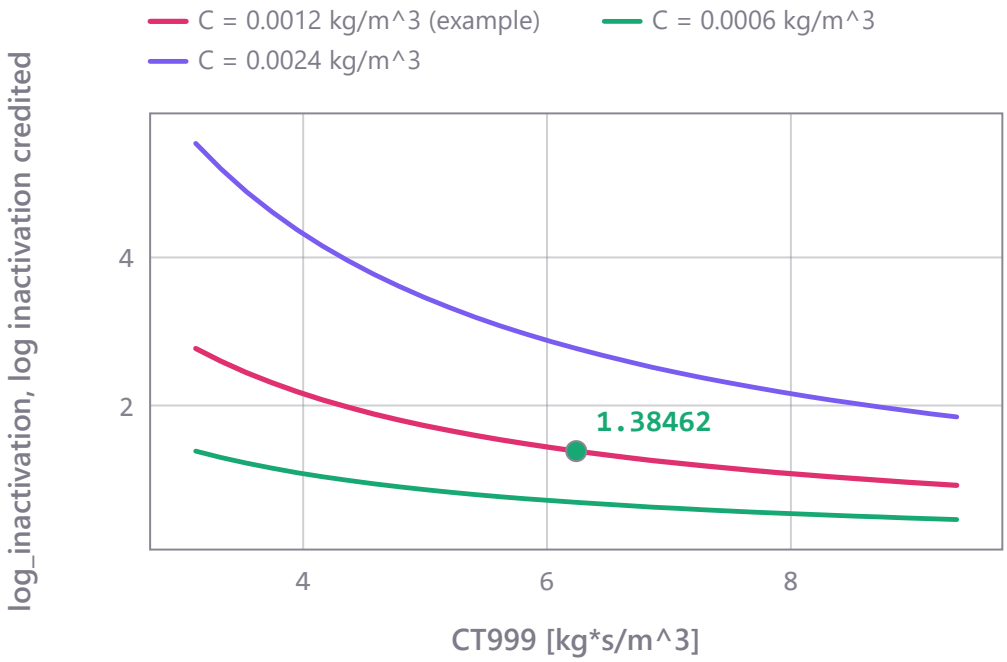
The CT achieved, the ratio to the CT required for $3 - log$ (99.9 %) inactivation from the regulatory tables, and the log inactivation credited (3 times the ratio).

$$\log \text{inactivation} = 3\,\frac{Ct_{10}}{CT_{99.9}}$$

input	meaning	condition
`C`	disinfectant residual	
`t10`	effective contact time	
`CT999`	CT required for $3-log$ inactivation (from the tables)	

output	meaning
`CT`	CT achieved
`ratio`	CT achieved / CT for $3-log$
`log_inactivation`	log inactivation credited

- **Method:** log inactivation = 3 $CT/CT_{9}9.9$
- **Assumes:** first-order (Chick-Watson) inactivation with $n = 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



log_inactivation (log inactivation credited) against CT999 (CT required for 3-log inactivation (from the tables)) from 3.12 kg*s/m^3 to 9.36 kg*s/m^3. One curve for each value of C (disinfectant residual): 0.0012 kg/m^3, 0.0006 kg/m^3, 0.0024 kg/m^3. Other inputs as in the worked example: t10 = 2400.0 s. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, CT999 = 6.24 kg*s/m^3, log_inactivation = 1.38462.

```
environmental_engineering_ai999x.solve('enveng_ct_log_inactivation', C=0.0012, t10=2400.0,
CT999=6.24)
# -> CT = 2.88 kg*s/m^3, ratio = 0.461538, log_inactivation = 1.38462   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_ct_log_inactivation', C=0.0005, t10=5400.0,
CT999=4.2)
# -> CT = 2.7 kg*s/m^3, ratio = 0.642857, log_inactivation = 1.92857   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
CT log inactivation: C = 1.2 mg/L, t10 = 40 min, CT999 = 104 mg*min/L	SOLVED	CT = 2.88 kg*s/m^3, ratio = 0.461538, log_inactivation = 1.38462
CT log inactivation: C = 0.5 mg/L, t10 = 90 min, CT999 = 70 mg*min/L	SOLVED	CT = 2.7 kg*s/m^3, ratio = 0.642857, log_inactivation = 1.92857
CT log inactivation: t10 = 40 min, CT999 = 104 mg*min/L	NEEDS_INPUT	enveng_ct_log_inactivation needs C (disinfectant residual). Example: solve('enveng_ct_log_inactivation', C='0.0012 kg/m^
CT log inactivation: C = 1.2 mg/L, t10 = 40 min, CT999 = 104 mg*min/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Microbial inactivation by Chick's law (first order), `enveng_chick_law`

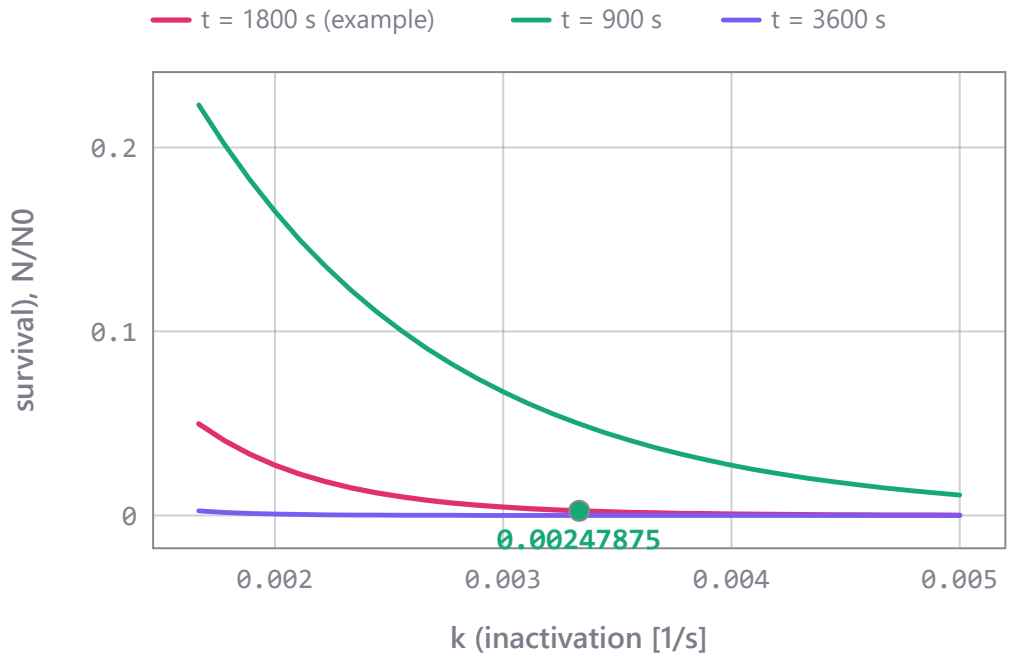
The surviving organisms, survival fraction, log removal and percentage removal after a contact time under first-order (Chick's law) inactivation or die-off.

$$\frac{N}{N_0} = e^{-kt}$$

input	meaning	condition
`N0`	initial organism concentration (count per volume, e.g. per 100 mL)	
`k`	inactivation (die-off) rate constant	
`t`	contact time	

output	meaning
`N`	surviving organisms, same unit as N0
`survival`	N/N_0
`log_removal`	log10 removal
`percent_removal`	percentage removed

- **Method:** $N = N_0$ e^(-k t); log removal = k t/ln 10
- **Assumes:** first-order kinetics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



survival (N/N0) against k (inactivation (die-off) rate constant) from 0.00166667 1/s to 0.005 1/s. One curve for each value of t (contact time): 1800 s, 900 s, 3600 s. Other inputs as in the worked example: N0 = 1000000.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 0.00333333 1/s, survival = 0.00247875.

```
environmental_engineering_ai999x.solve('enveng_chick_law', N0=1000000.0, k=0.0033333333333333335,
t=1800.0)
# -> N = 2478.75, survival = 0.00247875, log_removal = 2.60577, percent_removal = 99.7521
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_chick_law', N0=5000.0, k=0.0008333333333333334,
t=1200.0)
# -> N = 1839.4, survival = 0.367879, log_removal = 0.434294, percent_removal = 63.2121   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chick's law inactivation: N0 = 1000000, k = 0.2 1/min, t = 30 min	SOLVED	N = 2478.75, survival = 0.00247875, log_removal = 2.60577, percent_removal = 99.7521
Chick's law inactivation: N0 = 5000, k = 0.05 1/min, t = 20 min	SOLVED	N = 1839.4, survival = 0.367879, log_removal = 0.434294, percent_removal = 63.2121
Chick's law inactivation: k = 0.2 1/min, t = 30 min	NEEDS_INPUT	enveng_chick_law needs N0 (initial organism concentration (count per volume, e.g. per 100 mL)). Example: solve('enveng_c
Chick's law inactivation: N0 = 1000000, k = 0.2 1/min, t = 30 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Disinfection by the Chick-Watson law, `enveng_chick_watson`

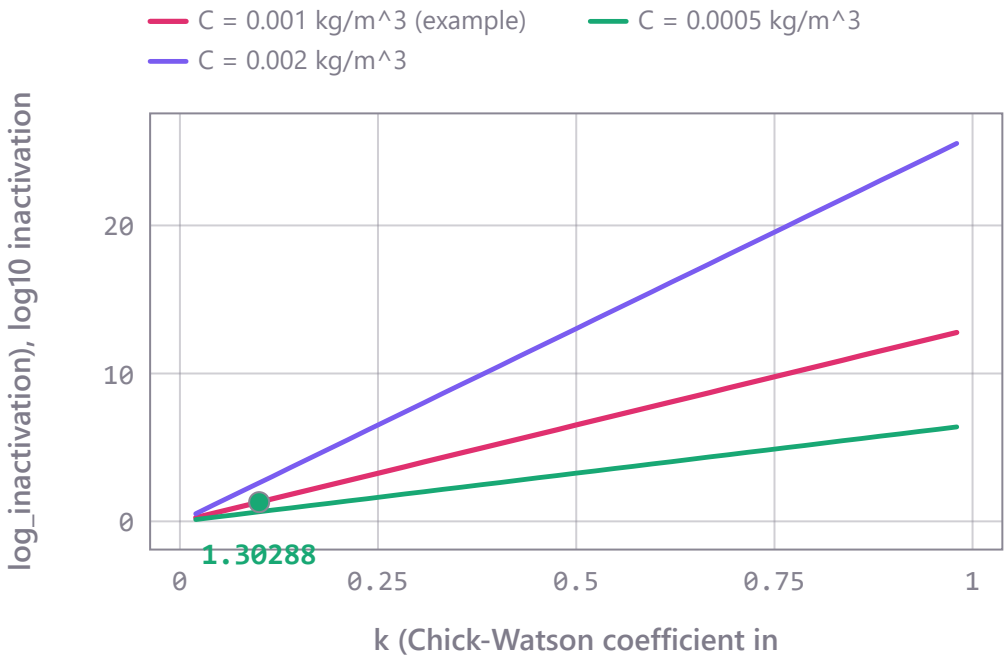
The survival fraction and log inactivation of organisms exposed to a disinfectant concentration C for time t under ln(N/N0) = -k C^n t (k in $(L/mg)^n$ per minute, C in mg/L, t in minutes inside the law).

$$\ln \frac{N}{N_0} = -k C^n t$$

input	meaning	condition
`k`	Chick-Watson coefficient in $(L/mg)^n$ per minute	
`n`	coefficient of dilution (1 for most disinfectants)	
`C`	disinfectant concentration	
`t`	contact time	

output	meaning
`survival`	N/N_0
`log_inactivation`	log10 inactivation

- **Method:** ln(N/N0) = -k C^n t with C in mg/L and t in minutes
- **Assumes:** constant concentration; $0.3 \leq n \leq 3$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



log_inactivation (log10 inactivation) against k (Chick-Watson coefficient in (L/mg)^n per minute) from 0.02 to 0.98. One curve for each value of C (disinfectant concentration): 0.001 kg/m^3, 0.0005 kg/m^3, 0.002 kg/m^3. Other inputs as in the worked example: n = 1.0, t = 1800.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 0.1, log_inactivation = 1.30288.

```
environmental_engineering_ai999x.solve('enveng_chick_watson', k=0.1, n=1.0, C=0.001, t=1800.0)
# -> survival = 0.0497871, log_inactivation = 1.30288   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_chick_watson', k=0.3, n=0.8, C=0.002, t=600.0)
# -> survival = 0.0053895, log_inactivation = 2.26845   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chick-Watson inactivation: k = 0.1, n = 1, C = 1 mg/L, t = 30 min	SOLVED	survival = 0.0497871, log_inactivation = 1.30288
Chick-Watson inactivation: k = 0.3, n = 0.8, C = 2 mg/L, t = 10 min	SOLVED	survival = 0.0053895, log_inactivation = 2.26845
Chick-Watson inactivation: n = 1, C = 1 mg/L, t = 30 min	NEEDS_INPUT	enveng_chick_watson needs k (Chick-Watson coefficient in (L/mg)^n per minute). Example: solve('enveng_chick_watson', k=0
Chick-Watson inactivation: k = 0.1, n = 1, C = 1 mg/L, t = 30 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Log removal and percentage removal, `enveng_log_removal`

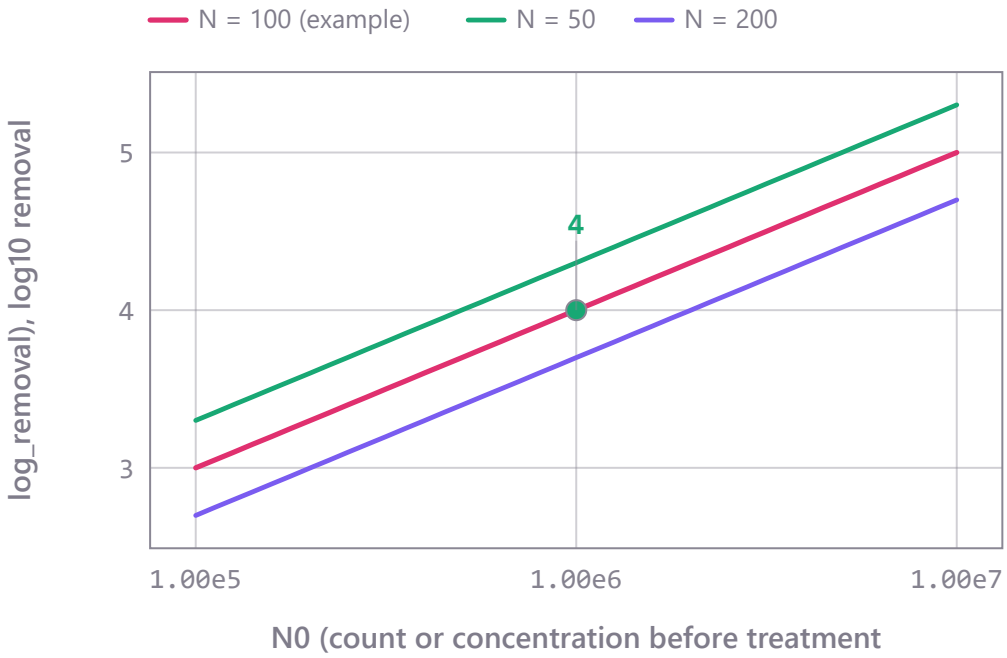
The log10 removal (log reduction value) and the percentage removal from the counts or concentrations before and after treatment.

$$LR = \log_{10} \frac{N_0}{N}$$

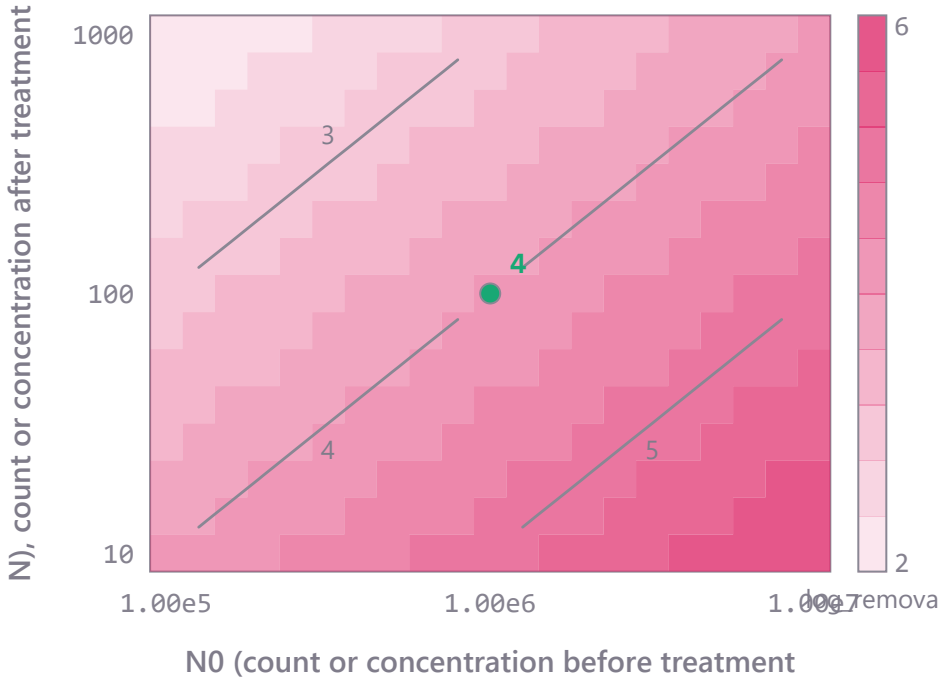
input	meaning	condition
`N0`	count or concentration before treatment	
`N`	count or concentration after treatment (same unit)	

output	meaning
`log_removal`	log10 removal
`percent_removal`	percentage removed

- **Method:** $LR = log10$(N0/N); percent = (1 - N/N0) 100
- **Assumes:** $0 < N < N_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



log_removal (log10 removal) against N0 (count or concentration before treatment) from 100000 to 1.0000e+07 on a logarithmic axis. One curve for each value of N (count or concentration after treatment (same unit)): 100, 50, 200. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, N0 = 1e+06, log_removal = 4.



log_removal (log10 removal) over N0 (count or concentration before treatment) from 100000 to 1.0000e+07 and N (count or concentration after treatment (same unit)) from 10 to 1000 (logarithmic axes): each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 2 to dark 6); iso-lines at log_removal = 3, 4, 5. Marked: the worked example, N0 = 1e+06, N = 100, log_removal = 4.

```
environmental_engineering_ai999x.solve('enveng_log_removal', N0=1000000.0, N=100.0)
# -> log_removal = 4, percent_removal = 99.99   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_log_removal', N0=250000.0, N=4000.0)
# -> log_removal = 1.79588, percent_removal = 98.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Log removal: N0 = 1000000, N = 100	SOLVED	log_removal = 4, percent_removal = 99.99
Log removal: N0 = 250000, N = 4000	SOLVED	log_removal = 1.79588, percent_removal = 98.4
Log removal: N = 100	NEEDS_INPUT	enveng_log_removal needs N0 (count or concentration before treatment). Example: solve('enveng_log_removal', N0=1e+06, N=
Log removal: N0 = 1000000, N = 100, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Chlorine dose and feed rate, `enveng_chlorine_dose_feed`

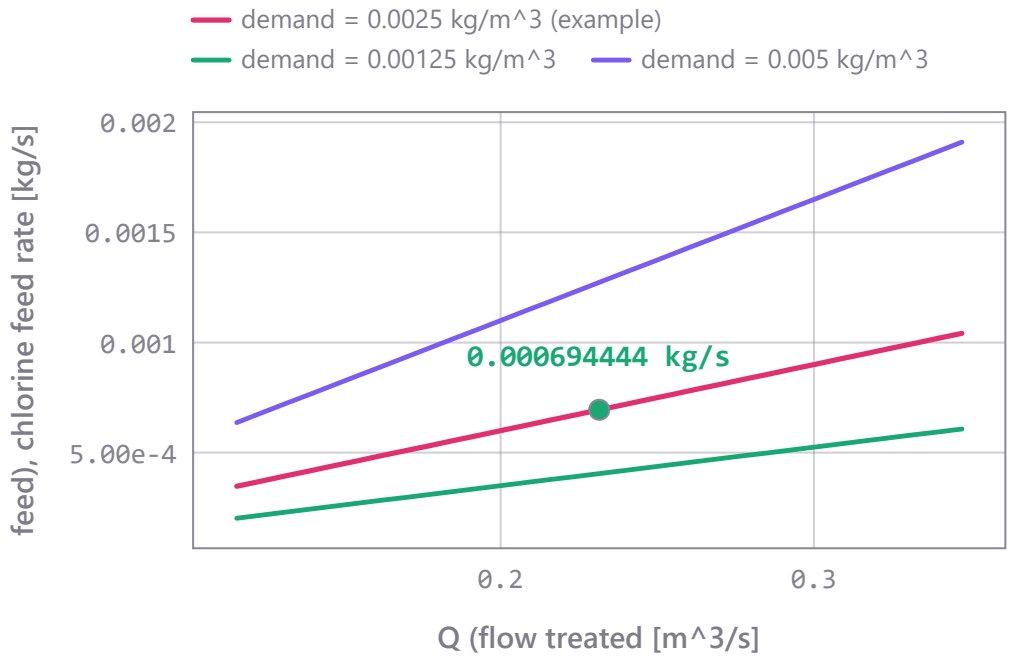
The chlorine dose (demand plus residual), the chlorine feed rate for a flow and the feed of a product with a stated available-chlorine fraction (e.g. 65 % calcium hypochlorite).

$$\dot{m} = Q\,(\text{demand} + \text{residual}), \quad \dot{m}_{product} = \frac{\dot{m}}{p}$$

input	meaning	condition
`Q`	flow treated	
`demand`	chlorine demand	
`residual`	required free chlorine residual	
`purity`	available chlorine fraction of the product (1 for chlorine gas; fraction or %)	

output	meaning
`dose`	chlorine dose
`dose_mgL`	dose in mg/L
`feed`	chlorine feed rate
`feed_kg_per_day`	chlorine feed in kg/d
`product_kg_per_day`	product feed in kg/d

- **Method:** dose = demand + residual; feed = Q dose; product = feed/purity
- **Assumes:** 0 < purity <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



feed (chlorine feed rate) against Q (flow treated) from 0.115741 m^3/s to 0.347222 m^3/s. One curve for each value of demand (chlorine demand): 0.0025 kg/m^3, 0.00125 kg/m^3, 0.005 kg/m^3. Other inputs as in the worked example: residual = 0.0005 kg/m^3, purity = 1.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Q = 0.231481 m^3/s, feed = 0.000694444 kg/s.

```
environmental_engineering_ai999x.solve('enveng_chlorine_dose_feed', Q=0.23148148148148148,
demand=0.0025, residual=0.0005, purity=1.0)
# -> dose = 0.003 kg/m^3, dose_mgL = 3, feed = 0.000694444 kg/s, feed_kg_per_day = 60,
product_kg_per_day = 60   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_chlorine_dose_feed', Q=0.05787037037037037,
demand=0.0018, residual=0.0007, purity=0.65)
# -> dose = 0.0025 kg/m^3, dose_mgL = 2.5, feed = 0.000144676 kg/s, feed_kg_per_day = 12.5,
product_kg_per_day = 19.2308   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chlorine dose and feed rate: Q = 20000 m^3/d, demand = 2.5 mg/L, residual = 0.5 mg/L, purity = 1	SOLVED	dose = 0.003 kg/m^3, dose_mgL = 3, feed = 0.000694444 kg/s, feed_kg_per_day = 60, product_kg_per_day = 60
Chlorine dose and feed rate: Q = 5000 m^3/d, demand = 1.8 mg/L, residual = 0.7 mg/L, purity = 0.65	SOLVED	dose = 0.0025 kg/m^3, dose_mgL = 2.5, feed = 0.000144676 kg/s, feed_kg_per_day = 12.5, product_kg_per_day = 19.2308
Chlorine dose and feed rate: demand = 2.5 mg/L, residual = 0.5 mg/L, purity = 1	NEEDS_INPUT	enveng_chlorine_dose_feed needs Q (flow treated). Example: solve('enveng_chlorine_dose_feed', Q='0.231481 m^3/s', demand
Chlorine dose and feed rate: Q = 20000 m^3/d, demand = 2.5 mg/L, residual = 0.5 mg/L, purity = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Chlorine feed rate: Q = 5000 m^3/d, demand = 1.8 mg/L, residual = 0.7 mg/L, purity = 65%	SOLVED	dose = 0.0025 kg/m^3, dose_mgL = 2.5, feed = 0.000144676 kg/s, feed_kg_per_day = 12.5, product_kg_per_day = 19.2308

Fraction of free chlorine as hypochlorous acid (pH), `enveng_hypochlorous_acid_fraction`

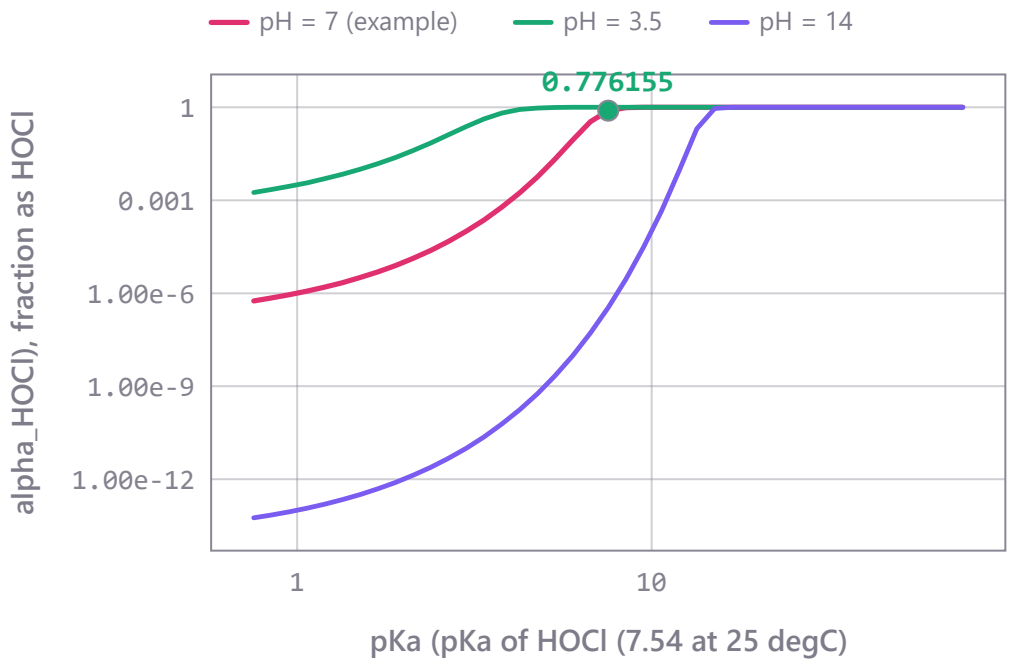
The fractions of free chlorine present as HOCl and as OCl- at a pH, from the acid dissociation constant (given, or from the temperature by Morris' correlation).

$$\alpha_{HOCl} = \frac{1}{1 + 10^{\,pH - pK_a}}$$

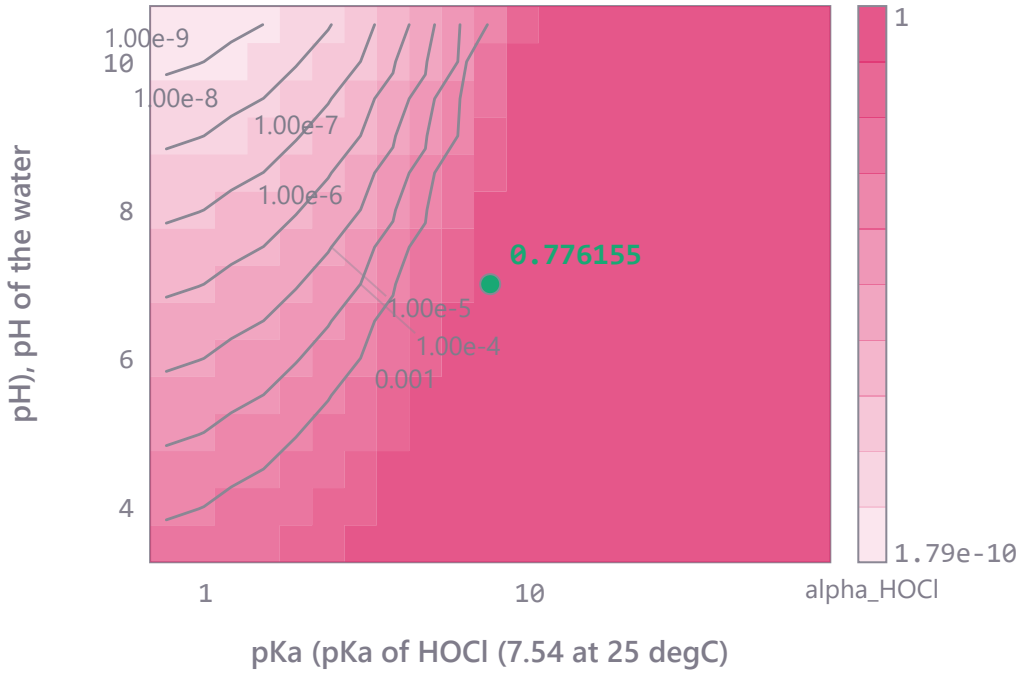
input	meaning	condition
`pH`	pH of the water	
`pKa`	pKa of HOCl (7.54 at 25 degC)	
`T`	water temperature (to compute pKa instead)	

output	meaning
`alpha_HOCl`	fraction as HOCl
`alpha_OCl`	fraction as OCl-
`pKa`	pKa used

- **Method:** alpha_HOCl = 1/(1 + 10^(pH - pKa)); pKa = 3000/T - 10.0686 + 0.0253 T (Morris)
- **Assumes:** either pKa or T; $0 - 35$ degC for the correlation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



alpha_HOCl (fraction as HOCl) against pKa (pKa of HOCl (7.54 at 25 degC)) from 0.754 to 75.4 on a logarithmic axis, logarithmic output axis. One curve for each value of pH (pH of the water): 7, 3.5, 14. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, pKa = 7.54, alpha_HOCl = 0.776155.



alpha_HOCl (fraction as HOCl) over pKa (pKa of HOCl (7.54 at 25 degC)) from 0.754 to 75.4 and pH (pH of the water) from 3.5 to 10.5 (logarithmic horizontal axis): each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 1.7947e-10 to dark 1, logarithmic shading); iso-lines at alpha_HOCl = 1.00e-9, 1.00e-8, 1.00e-7, 1.00e-6, 1.00e-5, 1.00e-4, 0.001. Marked: the worked example, pKa = 7.54, pH = 7, alpha_HOCl = 0.776155.

```
environmental_engineering_ai999x.solve('enveng_hypochlorous_acid_fraction', pH=7.0, pKa=7.54)
# -> alpha_HOCl = 0.776155, alpha_OCl = 0.223845, pKa = 7.54   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_hypochlorous_acid_fraction', pH=8.0, T=293.15)
# -> alpha_HOCl = 0.276273, alpha_OCl = 0.723727, pKa = 7.58176   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hypochlorous acid fraction: pH = 7, pKa = 7.54	SOLVED	alpha_HOCl = 0.776155, alpha_OCl = 0.223845, pKa = 7.54
Hypochlorous acid fraction: pH = 8, T = 20 degC	SOLVED	alpha_HOCl = 0.276273, alpha_OCl = 0.723727, pKa = 7.58176
Hypochlorous acid fraction: pKa = 7.54	NEEDS_INPUT	enveng_hypochlorous_acid_fraction needs pH (pH of the water). Example: solve('enveng_hypochlorous_acid_fraction', pH=7,
Hypochlorous acid fraction: pH = 7, pKa = 7.54, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Water hardness as CaCO3, `enveng_water_hardness`

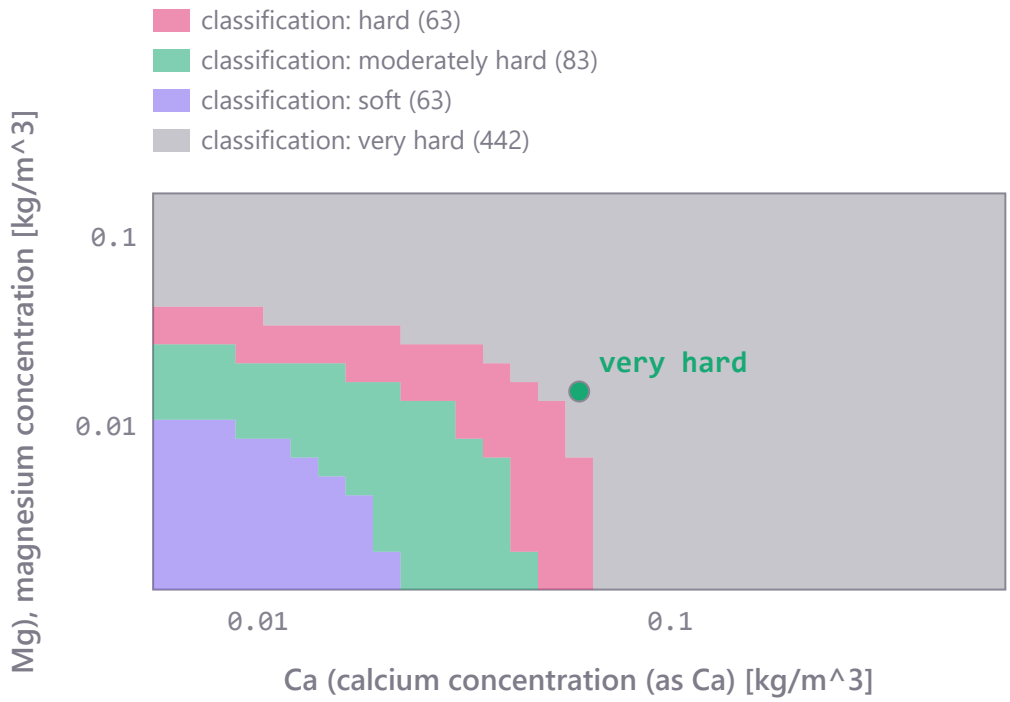
Calcium, magnesium and total hardness expressed as mg/L of CaCO3 from the calcium and magnesium concentrations, with the usual classification (soft to very hard).

$$TH = Ca\,\frac{50.04}{20.04} + Mg\,\frac{50.04}{12.15}$$

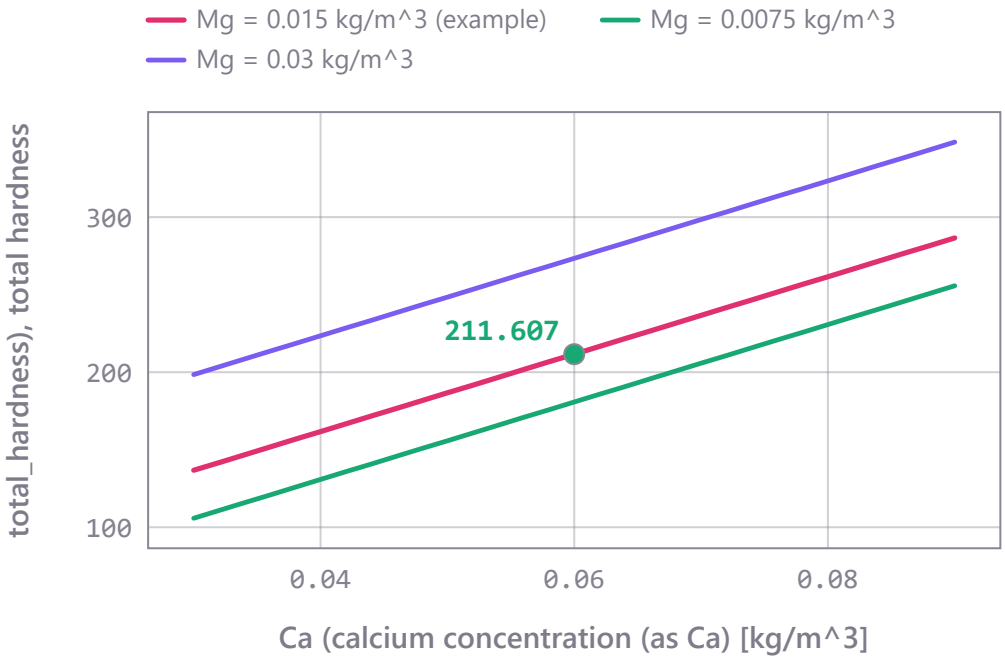
input	meaning	condition
`Ca`	calcium concentration (as Ca)	
`Mg`	magnesium concentration (as Mg)	

output	meaning
`total_hardness`	total hardness, mg/L as CaCO3
`calcium_hardness`	calcium hardness, mg/L as CaCO3
`magnesium_hardness`	magnesium hardness, mg/L as CaCO3
`classification`	soft (< 60), moderately hard $(60 - 120)$, hard $(120 - 180)$, very hard (>= 180)

- **Method:** hardness = C (EW_CaCO3/EW_ion), EW_CaCO3 = 50.04, $EW_Ca = 20.04$, $EW_Mg = 12.15$
- **Assumes:** only Ca and Mg contribute
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



soft (< 60), moderately hard (60-120), hard (120-180), very hard (>= 180) (classification) over Ca (calcium concentration (as Ca)) from 0.006 kg/m^3 to 0.6 kg/m^3 and Mg (magnesium concentration (as Mg)) from 0.0015 kg/m^3 to 0.15 kg/m^3 (logarithmic axes): each of the 31 × 21 cells is one run of environmental-engineering-ai999x, coloured by the verdict it returned, hard: 63 cells, moderately hard: 83 cells, soft: 63 cells, very hard: 442 cells. Marked: the worked example, Ca = 0.06 kg/m^3, Mg = 0.015 kg/m^3 → very hard.



total_hardness (total hardness, mg/L as CaCO3) against Ca (calcium concentration (as Ca)) from 0.03 kg/m^3 to 0.09 kg/m^3. One curve for each value of Mg (magnesium concentration (as Mg)): 0.015 kg/m^3, 0.0075 kg/m^3, 0.03 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Ca = 0.06 kg/m^3, total_hardness = 211.607.

```
environmental_engineering_ai999x.solve('enveng_water_hardness', Ca=0.06, Mg=0.015)
# -> total_hardness = 211.607, calcium_hardness = 149.838, magnesium_hardness = 61.7693,
classification = very hard   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_water_hardness', Ca=0.012, Mg=0.004)
# -> total_hardness = 46.4395, calcium_hardness = 29.9676, magnesium_hardness = 16.4718,
classification = soft   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Water hardness: Ca = 60 mg/L, Mg = 15 mg/L	SOLVED	total_hardness = 211.607, calcium_hardness = 149.838, magnesium_hardness = 61.7693, classification = very hard
Water hardness: Ca = 12 mg/L, Mg = 4 mg/L	SOLVED	total_hardness = 46.4395, calcium_hardness = 29.9676, magnesium_hardness = 16.4718, classification = soft
Water hardness: Mg = 15 mg/L	NEEDS_INPUT	enveng_water_hardness needs Ca (calcium concentration (as Ca)). Example: solve('enveng_water_hardness', Ca='0.06 kg/m^3'
Water hardness: Ca = 60 mg/L, Mg = 15 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Activated carbon dose from the Freundlich isotherm, `enveng_freundlich_carbon_dose`

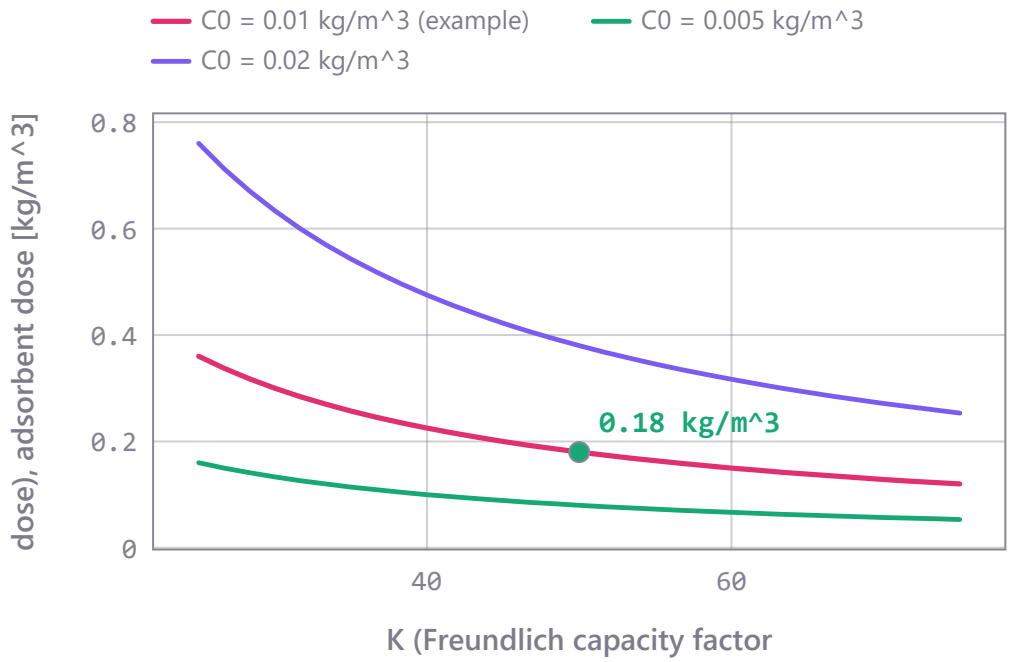
The equilibrium adsorbent loading from the Freundlich isotherm $q = K\,Ce^{1/n}$ (q in mg/g, Ce in mg/L) and the batch carbon dose that lowers C0 to Ce.

$$q_e = KC_e^{1/n}, \quad \frac{m}{V} = \frac{C_0 - C_e}{q_e}$$

input	meaning	condition
`C0`	initial concentration	
`Ce`	target equilibrium concentration	
`K`	Freundlich capacity factor, (mg/g)$(L/mg)^{1/n}$	
`n`	Freundlich exponent n (isotherm exponent 1/n)	

output	meaning
`qe`	equilibrium loading, mg/g
`dose`	adsorbent dose
`dose_mgL`	dose in mg/L

- **Method:** $qe = K\,Ce^{1/n}$; dose = (C0 - Ce)/qe
- **Assumes:** $Ce < C_0$; batch equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dose (adsorbent dose) against K (Freundlich capacity factor, (mg/g)(L/mg)^(1/n)) from 25 to 75. One curve for each value of C0 (initial concentration): 0.01 kg/m^3, 0.005 kg/m^3, 0.02 kg/m^3. Other inputs as in the worked example: Ce = 0.001 kg/m^3, n = 2.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, K = 50, dose = 0.18 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_freundlich_carbon_dose', C0=0.01, Ce=0.001, K=50.0,
n=2.0)
# -> qe = 50, dose = 0.18 kg/m^3, dose_mgL = 180   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_freundlich_carbon_dose', C0=0.005, Ce=0.0005,
K=120.0, n=3.0)
# -> qe = 95.2441, dose = 0.047247 kg/m^3, dose_mgL = 47.247   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Freundlich carbon dose: C0 = 10 mg/L, Ce = 1 mg/L, K = 50, n = 2	SOLVED	qe = 50, dose = 0.18 kg/m^3, dose_mgL = 180
Freundlich carbon dose: C0 = 5 mg/L, Ce = 0.5 mg/L, K = 120, n = 3	SOLVED	qe = 95.2441, dose = 0.047247 kg/m^3, dose_mgL = 47.247
Freundlich carbon dose: Ce = 1 mg/L, K = 50, n = 2	NEEDS_INPUT	enveng_freundlich_carbon_dose needs C0 (initial concentration). Example: solve('enveng_freundlich_carbon_dose', C0='0.01
Freundlich carbon dose: C0 = 10 mg/L, Ce = 1 mg/L, K = 50, n = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Adsorbent dose from the Langmuir isotherm, `enveng_langmuir_carbon_dose`

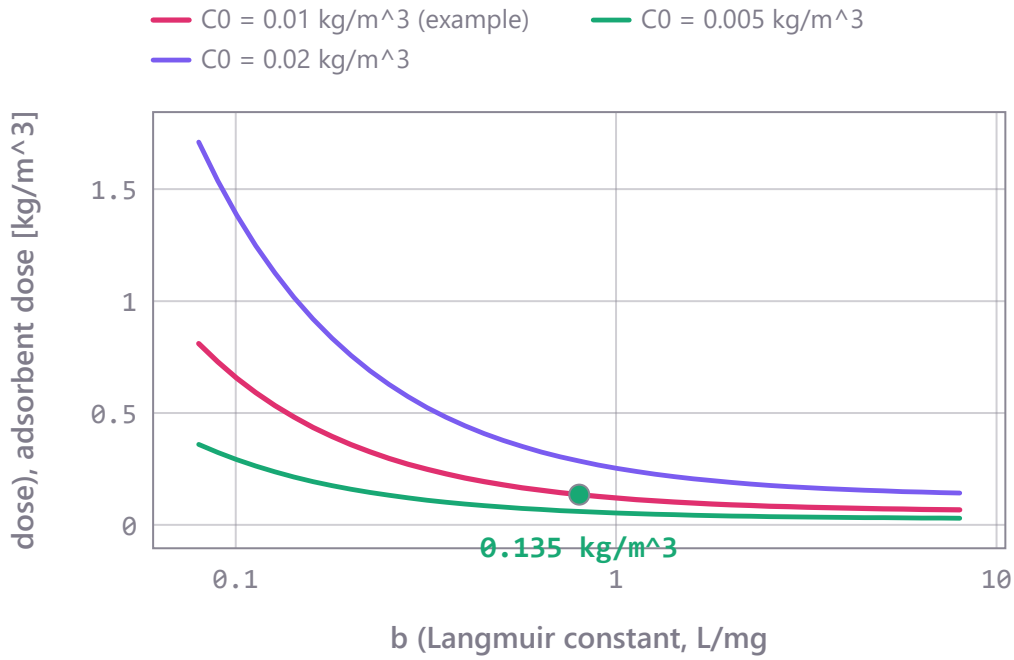
The equilibrium loading from the Langmuir isotherm q = qmax b Ce/(1 + b Ce) (q in mg/g, Ce in mg/L) and the batch adsorbent dose that lowers C0 to Ce.

$$q_e = \frac{q_{max} b C_e}{1 + b C_e}, \quad \frac{m}{V} = \frac{C_0 - C_e}{q_e}$$

input	meaning	condition
`C0`	initial concentration	
`Ce`	target equilibrium concentration	
`qmax`	Langmuir monolayer capacity, mg/g	
`b`	Langmuir constant, L/mg	

output	meaning
`qe`	equilibrium loading, mg/g
`dose`	adsorbent dose
`dose_mgL`	dose in mg/L

- **Method:** qe = qmax b Ce/(1 + b Ce); dose = (C0 - Ce)/qe
- **Assumes:** $Ce < C_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dose (adsorbent dose) against b (Langmuir constant, L/mg) from 0.08 to 8 on a logarithmic axis. One curve for each value of C0 (initial concentration): 0.01 kg/m^3, 0.005 kg/m^3, 0.02 kg/m^3. Other inputs as in the worked example: Ce = 0.001 kg/m^3, qmax = 150.0. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, b = 0.8, dose = 0.135 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_langmuir_carbon_dose', C0=0.01, Ce=0.001,
qmax=150.0, b=0.8)
# -> qe = 66.6667, dose = 0.135 kg/m^3, dose_mgL = 135   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_langmuir_carbon_dose', C0=0.02, Ce=0.004,
qmax=300.0, b=0.3)
# -> qe = 163.636, dose = 0.0977778 kg/m^3, dose_mgL = 97.7778   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Langmuir adsorbent dose: C0 = 10 mg/L, Ce = 1 mg/L, qmax = 150, b = 0.8	SOLVED	qe = 66.6667, dose = 0.135 kg/m^3, dose_mgL = 135
Langmuir adsorbent dose: C0 = 20 mg/L, Ce = 4 mg/L, qmax = 300, b = 0.3	SOLVED	qe = 163.636, dose = 0.0977778 kg/m^3, dose_mgL = 97.7778
Langmuir adsorbent dose: Ce = 1 mg/L, qmax = 150, b = 0.8	NEEDS_INPUT	enveng_langmuir_carbon_dose needs C0 (initial concentration). Example: solve('enveng_langmuir_carbon_dose', C0='0.01 kg/
Langmuir adsorbent dose: C0 = 10 mg/L, Ce = 1 mg/L, qmax = 150, b = 0.8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Freundlich isotherm constants fitted to adsorption data, `enveng_freundlich_fit`

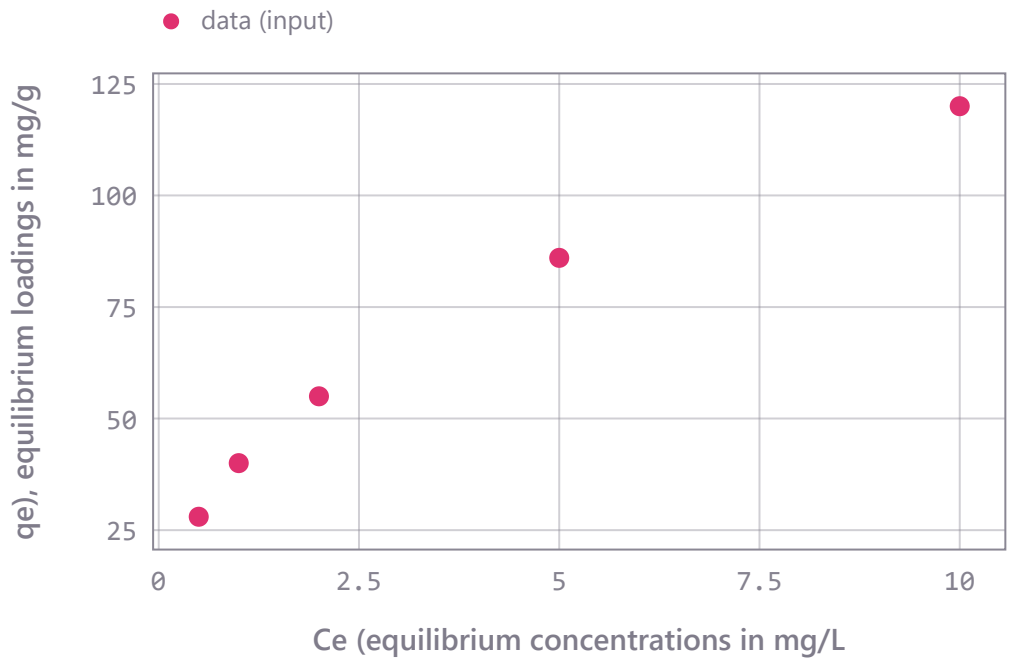
The Freundlich constants K and $1/n$ fitted by linear regression of log q against log Ce, with the coefficient of determination.

$$\log q_e = \log K + \frac{1}{n}\log C_e$$

input	meaning	condition
`Ce`	equilibrium concentrations in mg/L	
`qe`	equilibrium loadings in mg/g (same order)	

output	meaning
`K`	Freundlich K, (mg/g)$(L/mg)^{1/n}$
`inv_n`	isotherm exponent $1/n$
`n`	Freundlich n
`r2`	coefficient of determination of the log-log fit

- **Method:** least squares of log10 $q = log10\ K + (1/n)$ log10 Ce
- **Assumes:** at least three positive points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data qe (equilibrium loadings in mg/g (same order)) against Ce (equilibrium concentrations in mg/L) as given in the worked example (5 points). Also returned: K = 39.4762, inv_n = 0.483576, n = 2.06793, r2 = 0.999809.

```
environmental_engineering_ai999x.solve('enveng_freundlich_fit', Ce=[0.5, 1.0, 2.0, 5.0, 10.0], qe=
[28.0, 40.0, 55.0, 86.0, 120.0])
# -> K = 39.4762, inv_n = 0.483576, n = 2.06793, r2 = 0.999809   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_freundlich_fit', Ce=[1.2, 3.5, 8.0, 20.0], qe=[12.0,
19.5, 27.0, 41.0])
# -> K = 11.1446, inv_n = 0.433303, n = 2.30785, r2 = 0.999278   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Freundlich isotherm fit: Ce = [0.5, 1, 2, 5, 10], qe = [28, 40, 55, 86, 120]	SOLVED	K = 39.4762, inv_n = 0.483576, n = 2.06793, r2 = 0.999809
Freundlich isotherm fit: Ce = [1.2, 3.5, 8, 20], qe = [12, 19.5, 27, 41]	SOLVED	K = 11.1446, inv_n = 0.433303, n = 2.30785, r2 = 0.999278
Freundlich isotherm fit: qe = [28, 40, 55, 86, 120]	NEEDS_INPUT	enveng_freundlich_fit needs Ce (equilibrium concentrations in mg/L). Example: solve('enveng_freundlich_fit', Ce=[0.5, 1.
Freundlich isotherm fit: Ce = [0.5, 1, 2, 5, 10], qe = [28, 40, 55, 86, 120], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Langmuir isotherm constants fitted to adsorption data, `enveng_langmuir_fit`

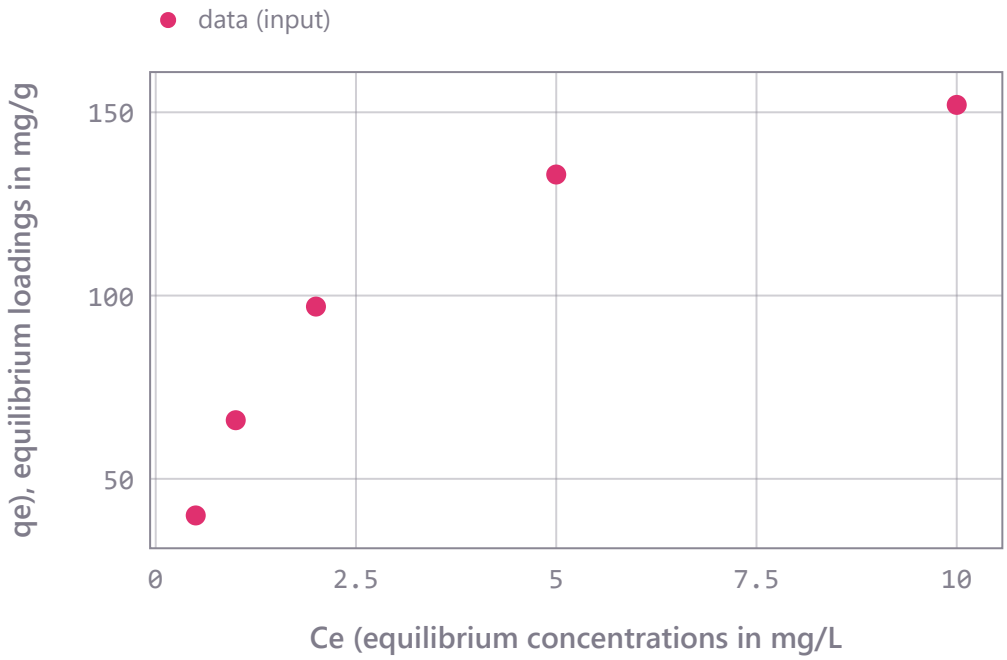
The Langmuir capacity qmax and constant b fitted by linear regression of Ce/q against Ce (linearised Langmuir form), with the coefficient of determination.

$$\frac{C_e}{q_e} = \frac{1}{q_{max} b} + \frac{C_e}{q_{max}}$$

input	meaning	condition
`Ce`	equilibrium concentrations in mg/L	
`qe`	equilibrium loadings in mg/g (same order)	

output	meaning
`qmax`	monolayer capacity, mg/g
`b`	Langmuir constant, L/mg
`r2`	coefficient of determination of the linearised fit

- **Method:** least squares of Ce/q = 1/(qmax b) + Ce/qmax
- **Assumes:** positive slope and intercept of Ce/q against Ce
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data qe (equilibrium loadings in mg/g (same order)) against Ce (equilibrium concentrations in mg/L) as given in the worked example (5 points). Also returned: qmax = 177.9, b = 0.589676, r2 = 0.999973.

```
environmental_engineering_ai999x.solve('enveng_langmuir_fit', Ce=[0.5, 1.0, 2.0, 5.0, 10.0], qe=
[40.0, 66.0, 97.0, 133.0, 152.0])
# -> qmax = 177.9, b = 0.589676, r2 = 0.999973   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_langmuir_fit', Ce=[2.0, 6.0, 15.0, 40.0], qe=[20.0,
42.0, 66.0, 85.0])
# -> qmax = 102.998, b = 0.118021, r2 = 0.999894   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Langmuir isotherm fit: Ce = [0.5, 1, 2, 5, 10], qe = [40, 66, 97, 133, 152]	SOLVED	qmax = 177.9, b = 0.589676, r2 = 0.999973
Langmuir isotherm fit: Ce = [2, 6, 15, 40], qe = [20, 42, 66, 85]	SOLVED	qmax = 102.998, b = 0.118021, r2 = 0.999894
Langmuir isotherm fit: qe = [40, 66, 97, 133, 152]	NEEDS_INPUT	enveng_langmuir_fit needs Ce (equilibrium concentrations in mg/L). Example: solve('enveng_langmuir_fit', Ce=[0.5, 1.0, 2
Langmuir isotherm fit: Ce = [0.5, 1, 2, 5, 10], qe = [40, 66, 97, 133, 152], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Membrane recovery, rejection, flux and concentrate, `enveng_membrane_mass_balance`

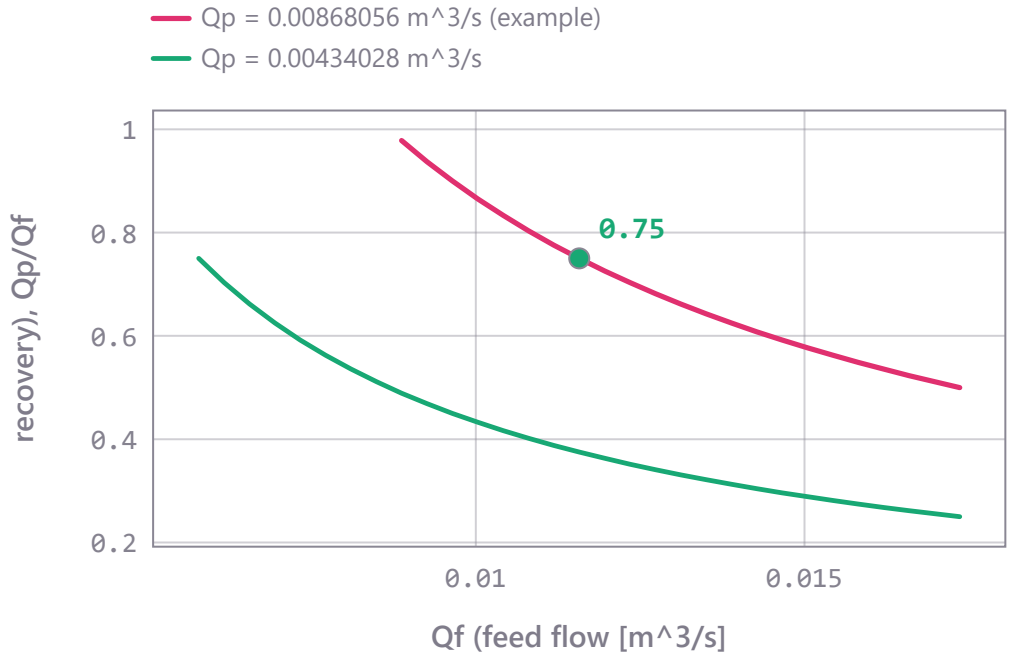
The water recovery, solute rejection, permeate flux (also in L/(m^2 h)) and the concentrate flow and concentration of a membrane unit from the feed and permeate flows and concentrations and the membrane area.

$$r = \frac{Q_p}{Q_f},\ R = 1 - \frac{C_p}{C_f},\ C_c = \frac{Q_f C_f - Q_p C_p}{Q_f - Q_p}$$

input	meaning	condition
`Qf`	feed flow	
`Qp`	permeate flow	
`Cf`	feed concentration	
`Cp`	permeate concentration	
`A`	membrane area	

output	meaning
`recovery`	Qp/Qf
`rejection`	$1 - Cp/Cf$
`J`	permeate flux
`J_LMH`	flux in L/(m^2 h)
`Qc`	concentrate flow
`Cc`	concentrate concentration
`Cc_mgL`	concentrate concentration in mg/L

- **Method:** $r = Qp/Qf$; $R = 1 - Cp/Cf$; $J = Qp/A$; $Qc = Qf - Qp$; Cc = (Qf Cf - Qp Cp)/Qc
- **Assumes:** $Qp < Qf$; $Cp < Cf$; steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



recovery (Qp/Qf) against Qf (feed flow) from 0.00578704 m^3/s to 0.0173611 m^3/s. One curve for each value of Qp (permeate flow): 0.00868056 m^3/s, 0.00434028 m^3/s. Other inputs as in the worked example: Cf = 2.0 kg/m^3, Cp = 0.05 kg/m^3, A = 1500.0 m^2. 23 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Qf = 0.0115741 m^3/s, recovery = 0.75.

```
environmental_engineering_ai999x.solve('enveng_membrane_mass_balance', Qf=0.011574074074074073,
Qp=0.008680555555555556, Cf=2.0, Cp=0.05, A=1500.0)
# -> recovery = 0.75, rejection = 0.975, J = 5.78704e-06 m/s, J_LMH = 20.8333, Qc = 0.00289352
m^3/s, Cc = 7.85 kg/m^3, Cc_mgL = 7850   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_membrane_mass_balance', Qf=0.05787037037037037,
Qp=0.026041666666666668, Cf=35.0, Cp=0.2, A=6000.0)
# -> recovery = 0.45, rejection = 0.994286, J = 4.34028e-06 m/s, J_LMH = 15.625, Qc = 0.0318287
m^3/s, Cc = 63.4727 kg/m^3, Cc_mgL = 63472.7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Membrane mass balance: Qf = 1000 m^3/d, Qp = 750 m^3/d, Cf = 2000 mg/L, Cp = 50 mg/L, A = 1500 m^2	SOLVED	recovery = 0.75, rejection = 0.975, J = 5.78704e-06 m/s, J_LMH = 20.8333, Qc = 0.00289352 m^3/s, Cc = 7.85 kg/m^3, Cc_mgL = 7850
Membrane mass balance: Qf = 5000 m^3/d, Qp = 2250 m^3/d, Cf = 35000 mg/L, Cp = 200 mg/L, A = 6000 m^2	SOLVED	recovery = 0.45, rejection = 0.994286, J = 4.34028e-06 m/s, J_LMH = 15.625, Qc = 0.0318287 m^3/s, Cc = 63.4727 kg/m^3, Cc_mgL = 63472.7
Membrane mass balance: Qp = 750 m^3/d, Cf = 2000 mg/L, Cp = 50 mg/L, A = 1500 m^2	NEEDS_INPUT	enveng_membrane_mass_balance needs Qf (feed flow). Example: solve('enveng_membrane_mass_balance', Qf='0.0115741 m^3/s',
Membrane mass balance: Qf = 1000 m^3/d, Qp = 750 m^3/d, Cf = 2000 mg/L, Cp = 50 mg/L, A = 1500 m^2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Osmotic pressure of a dilute solution (van 't Hoff), `enveng_osmotic_pressure`

The osmotic pressure of a dilute salt solution from its mass concentration, molar mass, van 't Hoff factor and temperature.

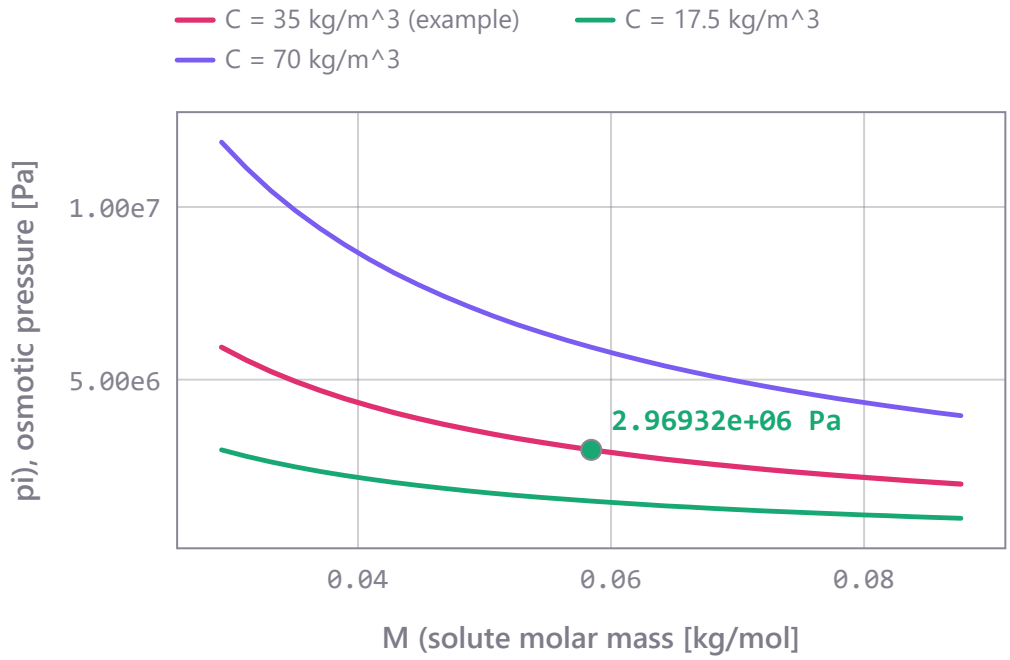
$$\pi = i \frac{C}{M} RT$$

input	meaning	condition
`C`	solute mass concentration	
`M`	solute molar mass	
`i`	van 't Hoff factor (ions per formula unit, 2 for NaCl)	
`T`	temperature	

output	meaning
`pi`	osmotic pressure
`pi_bar`	osmotic pressure in bar

- **Method:** $pi = i$ (C/M) R T
- **Assumes:** dilute (ideal) solution

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pi (osmotic pressure) against M (solute molar mass) from 0.02922 kg/mol to 0.08766 kg/mol. One curve for each value of C (solute mass concentration): 35 kg/m^3, 17.5 kg/m^3, 70 kg/m^3. Other inputs as in the worked example: i = 2.0, T = 298.15 K. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, M = 0.05844 kg/mol, pi = 2.96932e+06 Pa.

```
environmental_engineering_ai999x.solve('enveng_osmotic_pressure', C=35.0, M=0.05844, i=2.0,
T=298.15)
# -> pi = 2.96932e+06 Pa, pi_bar = 29.6932   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_osmotic_pressure', C=2.0, M=0.11098, i=3.0,
T=293.15)
# -> pi = 131774 Pa, pi_bar = 1.31774   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Osmotic pressure: C = 35 g/L, M = 58.44 g/mol, i = 2, T = 298.15 K	SOLVED	pi = 2.96932e+06 Pa, pi_bar = 29.6932
Osmotic pressure: C = 2 g/L, M = 110.98 g/mol, i = 3, T = 293.15 K	SOLVED	pi = 131774 Pa, pi_bar = 1.31774
Osmotic pressure: M = 58.44 g/mol, i = 2, T = 298.15 K	NEEDS_INPUT	enveng_osmotic_pressure needs C (solute mass concentration). Example: solve('enveng_osmotic_pressure', C='35 kg/m^3', M=
Osmotic pressure: C = 35 g/L, M = 58.44 g/mol, i = 2, T = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Food-to-microorganism ratio (F/M), `enveng_food_to_microorganism_ratio`

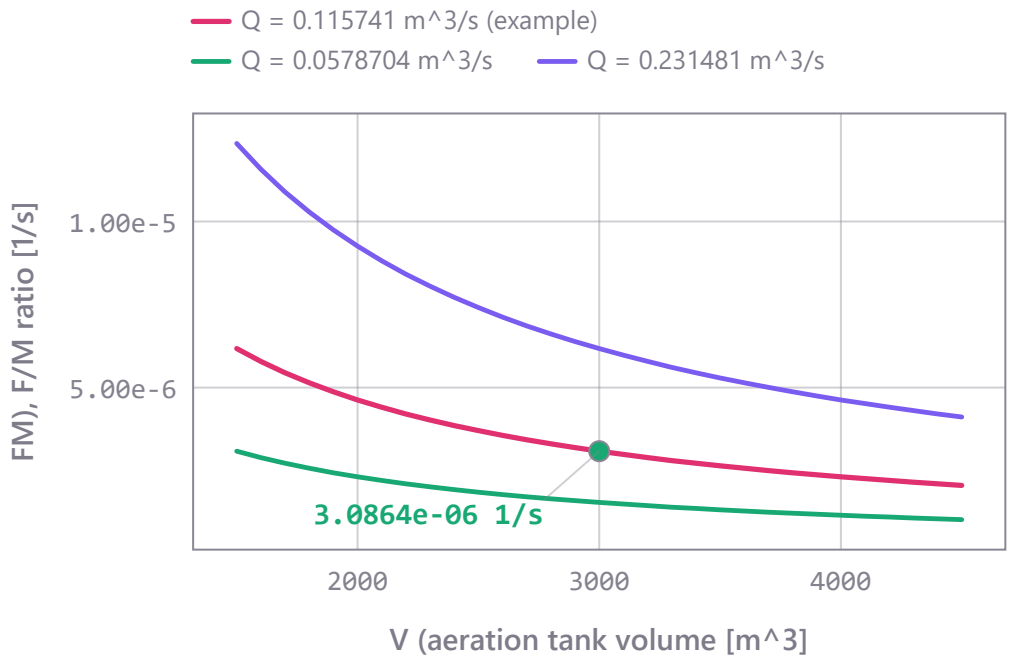
The food-to-microorganism ratio of an aeration tank from the flow, influent BOD, tank volume and mixed-liquor volatile suspended solids.

$$\frac{F}{M} = \frac{QS_0}{VX}$$

input	meaning	condition
`Q`	influent flow	
`S0`	influent BOD (or COD)	
`V`	aeration tank volume	
`X`	mixed liquor volatile suspended solids (MLVSS)	

output	meaning
`FM`	F/M ratio
`FM_per_day`	F/M in kg BOD/(kg MLVSS d)

- **Method:** $F/M = Q$ S0/(V X)
- **Assumes:** Q, S0, V, $X > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



FM (F/M ratio) against V (aeration tank volume) from 1500 m^3 to 4500 m^3. One curve for each value of Q (influent flow): 0.115741 m^3/s, 0.0578704 m^3/s, 0.231481 m^3/s. Other inputs as in the worked example: S0 = 0.2 kg/m^3, X = 2.5 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, V = 3000 m^3, FM = 3.0864e-06 1/s.

```
environmental_engineering_ai999x.solve('enveng_food_to_microorganism_ratio', Q=0.11574074074074074,
S0=0.2, V=3000.0, X=2.5)
# -> FM = 3.08642e-06 1/s, FM_per_day = 0.266667   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_food_to_microorganism_ratio',
Q=0.046296296296296294, S0=0.15, V=1500.0, X=2.0)
# -> FM = 2.31481e-06 1/s, FM_per_day = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Food-to-microorganism ratio: Q = 10000 m^3/d, S0 = 200 mg/L, V = 3000 m^3, X = 2500 mg/L	SOLVED	FM = 3.08642e-06 1/s, FM_per_day = 0.266667
Food-to-microorganism ratio: Q = 4000 m^3/d, S0 = 150 mg/L, V = 1500 m^3, X = 2000 mg/L	SOLVED	FM = 2.31481e-06 1/s, FM_per_day = 0.2
Food-to-microorganism ratio: S0 = 200 mg/L, V = 3000 m^3, X = 2500 mg/L	NEEDS_INPUT	enveng_food_to_microorganism_ratio needs Q (influent flow). Example: solve('enveng_food_to_microorganism_ratio', Q='0.11
Food-to-microorganism ratio: Q = 10000 m^3/d, S0 = 200 mg/L, V = 3000 m^3, X = 2500 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Solids retention time (mean cell residence time), `enveng_solids_retention_time`

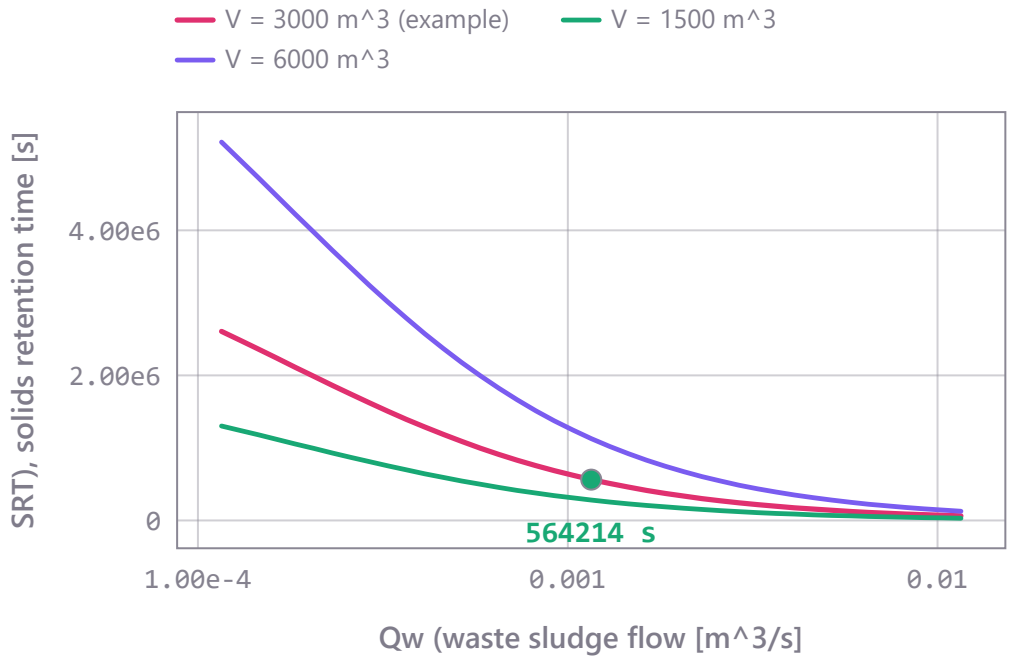
The solids retention time (sludge age) of an activated-sludge system from the biomass in the tank and the biomass leaving in the waste sludge and the effluent.

$$\theta_c = \frac{VX}{Q_w X_w + Q_e X_e}$$

input	meaning	condition
`V`	aeration tank volume	
`X`	MLVSS in the tank	
`Qw`	waste sludge flow	
`Xw`	VSS in the waste sludge	
`Qe`	effluent flow	
`Xe`	VSS in the effluent	

output	meaning
`SRT`	solids retention time
`SRT_days`	SRT in days

- **Method:** SRT = V X/(Qw Xw + Qe Xe)
- **Assumes:** steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



SRT (solids retention time) against Qw (waste sludge flow) from 0.000115741 m^3/s to 0.0115741 m^3/s on a logarithmic axis. One curve for each value of V (aeration tank volume): 3000 m^3, 1500 m^3, 6000 m^3. Other inputs as in the worked example: X = 2.5 kg/m^3, Xw = 10.0 kg/m^3, Qe = 0.11458333333333333 m^3/s, Xe = 0.015 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Qw = 0.00115741 m^3/s, SRT = 564214 s.

```
environmental_engineering_ai999x.solve('enveng_solids_retention_time', V=3000.0, X=2.5,
Qw=0.0011574074074074073, Xw=10.0, Qe=0.11458333333333333, Xe=0.015)
# -> SRT = 564214 s, SRT_days = 6.53026   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_solids_retention_time', V=1200.0, X=3.0,
Qw=0.0006944444444444445, Xw=8.0, Qe=0.04560185185185185, Xe=0.01)
# -> SRT = 598845 s, SRT_days = 6.93107   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solids retention time: V = 3000 m^3, X = 2500 mg/L, Qw = 100 m^3/d, Xw = 10000 mg/L, Qe = 9900 m^3/d, Xe = 15 mg/L	SOLVED	SRT = 564214 s, SRT_days = 6.53026
Solids retention time: V = 1200 m^3, X = 3000 mg/L, Qw = 60 m^3/d, Xw = 8000 mg/L, Qe = 3940 m^3/d, Xe = 10 mg/L	SOLVED	SRT = 598845 s, SRT_days = 6.93107
Solids retention time: X = 2500 mg/L, Qw = 100 m^3/d, Xw = 10000 mg/L, Qe = 9900 m^3/d, Xe = 15 mg/L	NEEDS_INPUT	enveng_solids_retention_time needs V (aeration tank volume). Example: solve('enveng_solids_retention_time', V='3000 m^3'
Solids retention time: V = 3000 m^3, X = 2500 mg/L, Qw = 100 m^3/d, Xw = 10000 mg/L, Qe = 9900 m^3/d, Xe = 15 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Hydraulic retention time and volumetric organic loading, `enveng_hrt_organic_loading`

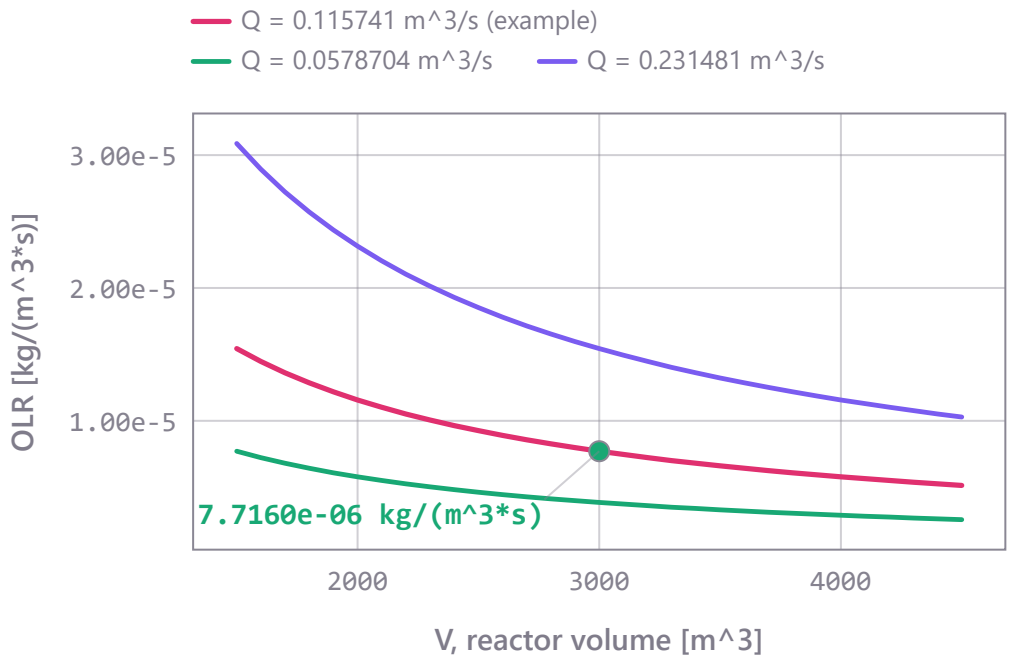
The hydraulic retention time of a reactor and its volumetric organic (BOD) loading rate from the flow, influent BOD and volume.

$$\theta = \frac{V}{Q}, \quad L_v = \frac{Q S_0}{V}$$

input	meaning	condition
`Q`	influent flow	
`S0`	influent BOD	
`V`	reactor volume	

output	meaning
`HRT`	hydraulic retention time
`HRT_h`	HRT in hours
`OLR`	volumetric organic loading rate
`OLR_kg_per_m3_day`	loading in kg BOD/(m^3 d)

- **Method:** HRT = V/Q; OLR = Q S_0/V
- **Assumes:** Q, S0, $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



OLR (volumetric organic loading rate) against V (reactor volume) from 1500 m^3 to 4500 m^3. One curve for each value of Q (influent flow): 0.115741 m^3/s, 0.0578704 m^3/s, 0.231481 m^3/s. Other inputs as in the worked example: S0 = 0.2 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, V = 3000 m^3, OLR = 7.7160e-06 kg/(m^3*s).

```
environmental_engineering_ai999x.solve('enveng_hrt_organic_loading', Q=0.11574074074074074, S0=0.2,
V=3000.0)
# -> HRT = 25920 s, HRT_h = 7.2, OLR = 7.71605e-06 kg/(m^3*s), OLR_kg_per_m3_day = 0.666667
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_hrt_organic_loading', Q=0.028935185185185185,
S0=0.35, V=800.0)
# -> HRT = 27648 s, HRT_h = 7.68, OLR = 1.26591e-05 kg/(m^3*s), OLR_kg_per_m3_day = 1.09375
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydraulic retention time and organic loading: Q = 10000 m^3/d, S0 = 200 mg/L, V = 3000 m^3	SOLVED	HRT = 25920 s, HRT_h = 7.2, OLR = 7.71605e-06 kg/(m^3*s), OLR_kg_per_m3_day = 0.666667
Hydraulic retention time and organic loading: Q = 2500 m^3/d, S0 = 350 mg/L, V = 800 m^3	SOLVED	HRT = 27648 s, HRT_h = 7.68, OLR = 1.26591e-05 kg/(m^3*s), OLR_kg_per_m3_day = 1.09375
Hydraulic retention time and organic loading: S0 = 200 mg/L, V = 3000 m^3	NEEDS_INPUT	enveng_hrt_organic_loading needs Q (influent flow). Example: solve('enveng_hrt_organic_loading', Q='0.115741 m^3/s', S0=
Hydraulic retention time and organic loading: Q = 10000 m^3/d, S0 = 200 mg/L, V = 3000 m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Monod specific growth rate, `enveng_monod_growth_rate`

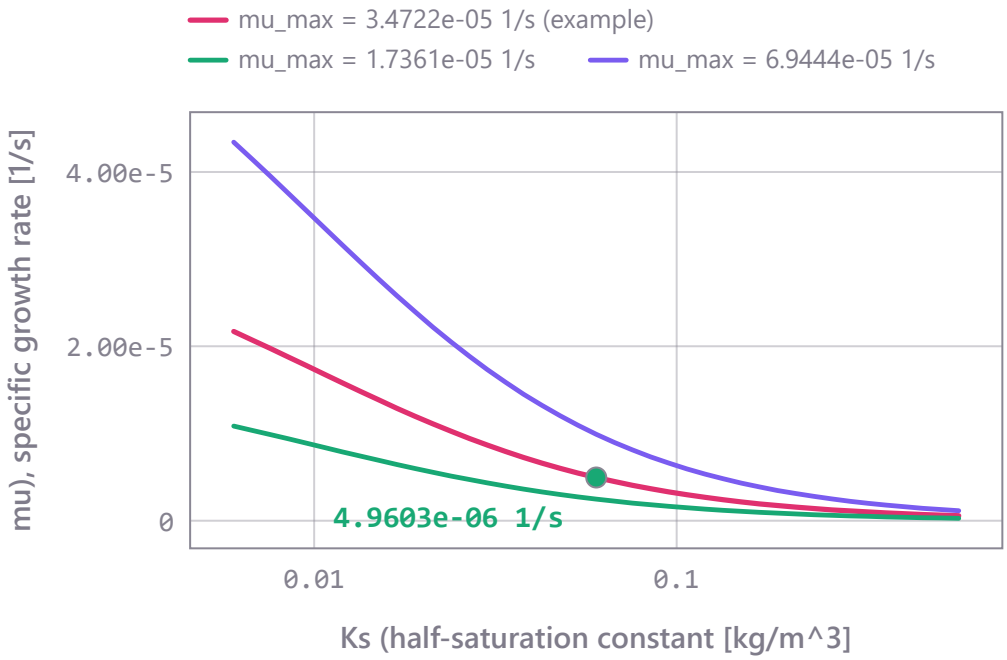
The specific growth rate of microorganisms at a substrate concentration by the Monod equation, and the net rate when the endogenous decay coefficient is given.

$$\mu = \frac{\mu_{max} S}{K_s + S}$$

input	meaning	condition
`mu_max`	maximum specific growth rate	
`Ks`	half-saturation constant	
`S`	substrate concentration	
`kd`	endogenous decay coefficient	

output	meaning
`mu`	specific growth rate
`mu_per_day`	in $1/d$
`mu_net`	net specific growth rate mu - kd
`mu_net_per_day`	net rate in $1/d$

- **Method:** mu = mu_max S/(Ks + S); mu_net = mu - kd
- **Assumes:** Monod kinetics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



mu (specific growth rate) against Ks (half-saturation constant) from 0.006 kg/m^3 to 0.6 kg/m^3 on a logarithmic axis. One curve for each value of mu_max (maximum specific growth rate): 3.4722e-05 1/s, 1.7361e-05 1/s, 6.9444e-05 1/s. Other inputs as in the worked example: S = 0.01 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Ks = 0.06 kg/m^3, mu = 4.9603e-06 1/s.

```
environmental_engineering_ai999x.solve('enveng_monod_growth_rate', mu_max=3.472222222222222e-05,
Ks=0.06, S=0.01)
# -> mu = 4.96032e-06 1/s, mu_per_day = 0.428571   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_monod_growth_rate', mu_max=5.787037037037037e-05,
Ks=0.04, S=0.1, kd=1.1574074074074074e-06)
# -> mu = 4.1336e-05 1/s, mu_per_day = 3.57143, mu_net = 4.01786e-05 1/s, mu_net_per_day = 3.47143
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Monod growth rate: mu_max = 3 1/d, Ks = 60 mg/L, S = 10 mg/L	SOLVED	mu = 4.96032e-06 1/s, mu_per_day = 0.428571
Monod growth rate: mu_max = 5 1/d, Ks = 40 mg/L, S = 100 mg/L, kd = 0.1 1/d	SOLVED	mu = 4.1336e-05 1/s, mu_per_day = 3.57143, mu_net = 4.01786e-05 1/s, mu_net_per_day = 3.47143
Monod growth rate: Ks = 60 mg/L, S = 10 mg/L	NEEDS_INPUT	enveng_monod_growth_rate needs mu_max (maximum specific growth rate). Example: solve('enveng_monod_growth_rate', mu_max=
Monod growth rate: mu_max = 3 1/d, Ks = 60 mg/L, S = 10 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Activated sludge (complete mix) effluent substrate and biomass (Lawrence-McCarty), `enveng_activated_sludge_cstr_design`

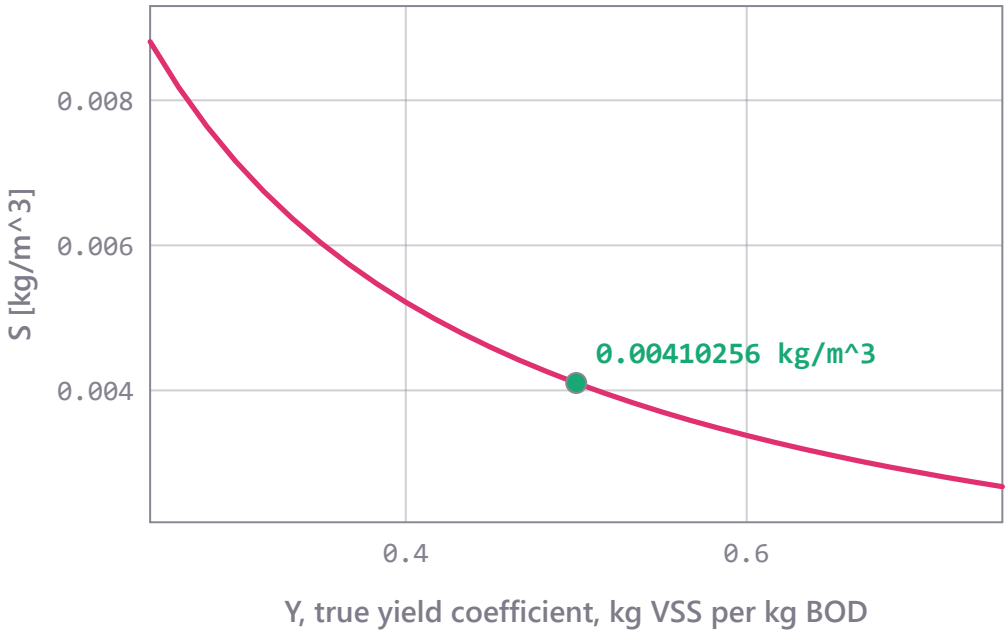
The effluent soluble substrate, the mixed-liquor biomass and the observed yield of a complete-mix activated-sludge reactor with recycle, from the kinetic coefficients, the SRT and HRT and the influent substrate.

$$S = \frac{K_s(1 + k_d\theta_c)}{\theta_c(Yk - k_d) - 1}, \quad X = \frac{\theta_c Y(S_0 - S)}{\theta(1 + k_d\theta_c)}$$

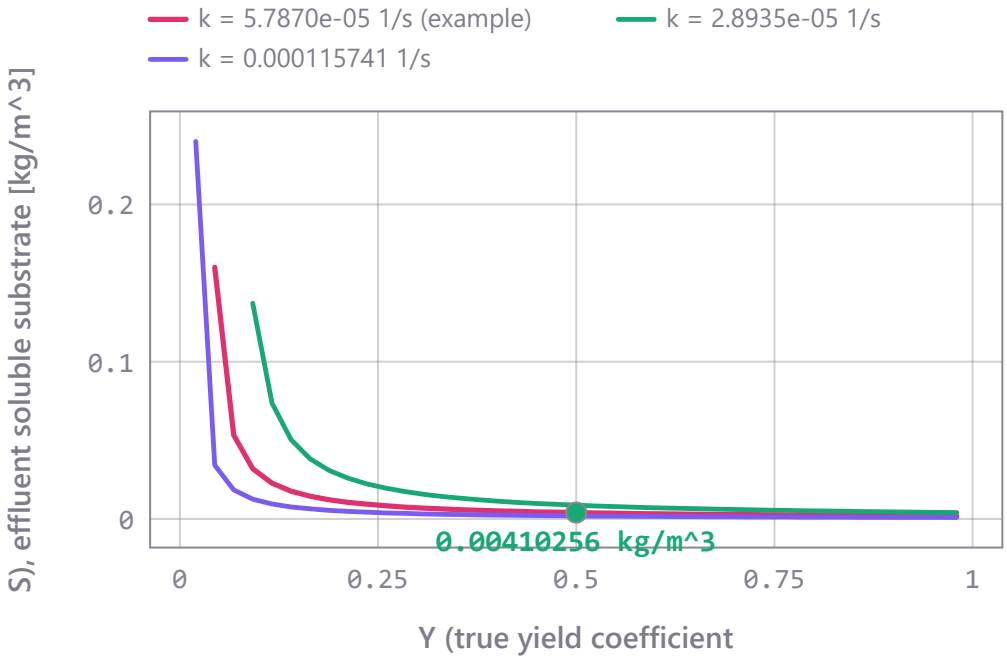
input	meaning	condition
`Y`	true yield coefficient, kg VSS per kg BOD	
`k`	maximum specific substrate utilisation rate	
`Ks`	half-saturation constant	
`kd`	endogenous decay coefficient	
`SRT`	solids retention time	
`HRT`	hydraulic retention time	
`S0`	influent substrate (BOD)	

output	meaning
`S`	effluent soluble substrate
`S_mgL`	in mg/L
`X`	biomass concentration in the reactor
`X_mgL`	in mg/L
`Y_obs`	observed yield Y/(1 + kd SRT)

- **Method:** $S = Ks$ (1 + kd SRT)/(SRT (Y k - kd) - 1); X = (SRT/HRT) Y (S0 - S)/(1 + kd SRT)
- **Assumes:** steady state, complete mix; SRT above the washout limit; $S < S_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (effluent soluble substrate) against Y (true yield coefficient, kg VSS per kg BOD), with HRT = 21600 s, Ks = 0.06 kg/m^3, S0 = 0.25 kg/m^3, SRT = 8.64e5 s, k = 5.79e-5 1/s, kd = 6.94e-7 1/s. 31 points computed by environmental-engineering-ai999x; marked: Y = 0.5, S = 0.00410256 kg/m^3.



S (effluent soluble substrate) against Y (true yield coefficient, kg VSS per kg BOD) from 0.02 to 0.98. One curve for each value of k (maximum specific substrate utilisation rate): 5.7870e-05 1/s, 2.8935e-05 1/s, 0.000115741 1/s. Other inputs as in the worked example: Ks = 0.06 kg/m^3, kd = 6.944444444444444e-07 1/s, SRT = 864000.0 s, HRT = 21600.0 s, S0 = 0.25 kg/m^3. 40 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Y = 0.5, S = 0.00410256 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_activated_sludge_cstr_design', Y=0.5,
k=5.787037037037037e-05, Ks=0.06, kd=6.944444444444444e-07, SRT=864000.0, HRT=21600.0, S0=0.25)
# -> S = 0.00410256 kg/m^3, S_mgL = 4.10256, X = 3.07372 kg/m^3, X_mgL = 3073.72, Y_obs = 0.3125
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_activated_sludge_cstr_design', Y=0.6,
k=9.259259259259259e-05, Ks=0.04, kd=9.259259259259259e-07, SRT=518400.0, HRT=14400.0, S0=0.18)
# -> S = 0.00216691 kg/m^3, S_mgL = 2.16691, X = 2.5954 kg/m^3, X_mgL = 2595.4, Y_obs = 0.405405
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Activated sludge complete-mix design: Y = 0.5, k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, SRT = 10 d, HRT = 6 h, S0 = 250 mg/L	SOLVED	S = 0.00410256 kg/m^3, S_mgL = 4.10256, X = 3.07372 kg/m^3, X_mgL = 3073.72, Y_obs = 0.3125
Activated sludge complete-mix design: Y = 0.6, k = 8 1/d, Ks = 40 mg/L, kd = 0.08 1/d, SRT = 6 d, HRT = 4 h, S0 = 180 mg/L	SOLVED	S = 0.00216691 kg/m^3, S_mgL = 2.16691, X = 2.5954 kg/m^3, X_mgL = 2595.4, Y_obs = 0.405405
Activated sludge complete-mix design: k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, SRT = 10 d, HRT = 6 h, S0 = 250 mg/L	NEEDS_INPUT	enveng_activated_sludge_cstr_design needs Y (true yield coefficient, kg VSS per kg BOD). Example: solve('enveng_activate
Activated sludge complete-mix design: Y = 0.5, k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, SRT = 10 d, HRT = 6 h, S0 = 250 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Minimum (washout) solids retention time, `enveng_minimum_srt`

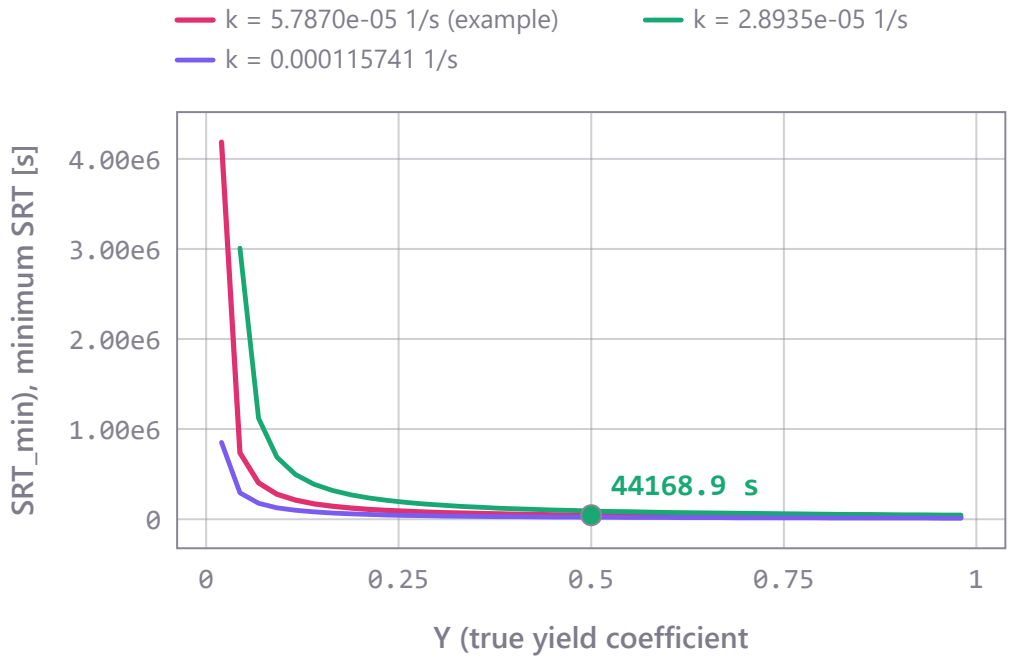
The washout SRT of a complete-mix biological reactor: below it the organisms are lost faster than they grow on the influent substrate.

$$\frac{1}{\theta_{c,min}} = \frac{YkS_0}{K_s + S_0} - k_d$$

input	meaning	condition
`Y`	true yield coefficient, kg VSS per kg BOD	
`k`	maximum specific substrate utilisation rate	
`Ks`	half-saturation constant	
`kd`	endogenous decay coefficient	
`S0`	influent substrate	

output	meaning
`SRT_min`	minimum SRT
`SRT_min_days`	minimum SRT in days

- **Method:** 1/SRT_min = Y k $S_0/(Ks + S_0) - kd$
- **Assumes:** Y k $S_0/(Ks + S_0) > kd$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



SRT_min (minimum SRT) against Y (true yield coefficient, kg VSS per kg BOD) from 0.02 to 0.98. One curve for each value of k (maximum specific substrate utilisation rate): 5.7870e-05 1/s, 2.8935e-05 1/s, 0.000115741 1/s. Other inputs as in the worked example: Ks = 0.06 kg/m^3, kd = 6.944444444444444e-07 1/s, S0 = 0.25 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Y = 0.5, SRT_min = 44168.9 s.

```
environmental_engineering_ai999x.solve('enveng_minimum_srt', Y=0.5, k=5.787037037037037e-05,
Ks=0.06, kd=6.944444444444444e-07, S0=0.25)
# -> SRT_min = 44168.9 s, SRT_min_days = 0.511214   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_minimum_srt', Y=0.6, k=9.259259259259259e-05,
Ks=0.04, kd=9.259259259259259e-07, S0=0.18)
# -> SRT_min = 22457.5 s, SRT_min_days = 0.259924   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Minimum solids retention time: Y = 0.5, k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, S0 = 250 mg/L	SOLVED	SRT_min = 44168.9 s, SRT_min_days = 0.511214
Minimum solids retention time: Y = 0.6, k = 8 1/d, Ks = 40 mg/L, kd = 0.08 1/d, S0 = 180 mg/L	SOLVED	SRT_min = 22457.5 s, SRT_min_days = 0.259924
Minimum solids retention time: k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, S0 = 250 mg/L	NEEDS_INPUT	enveng_minimum_srt needs Y (true yield coefficient, kg VSS per kg BOD). Example: solve('enveng_minimum_srt', Y=0.5, k='5
Minimum solids retention time: Y = 0.5, k = 5 1/d, Ks = 60 mg/L, kd = 0.06 1/d, S0 = 250 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Waste biomass production (observed yield), `enveng_sludge_production`

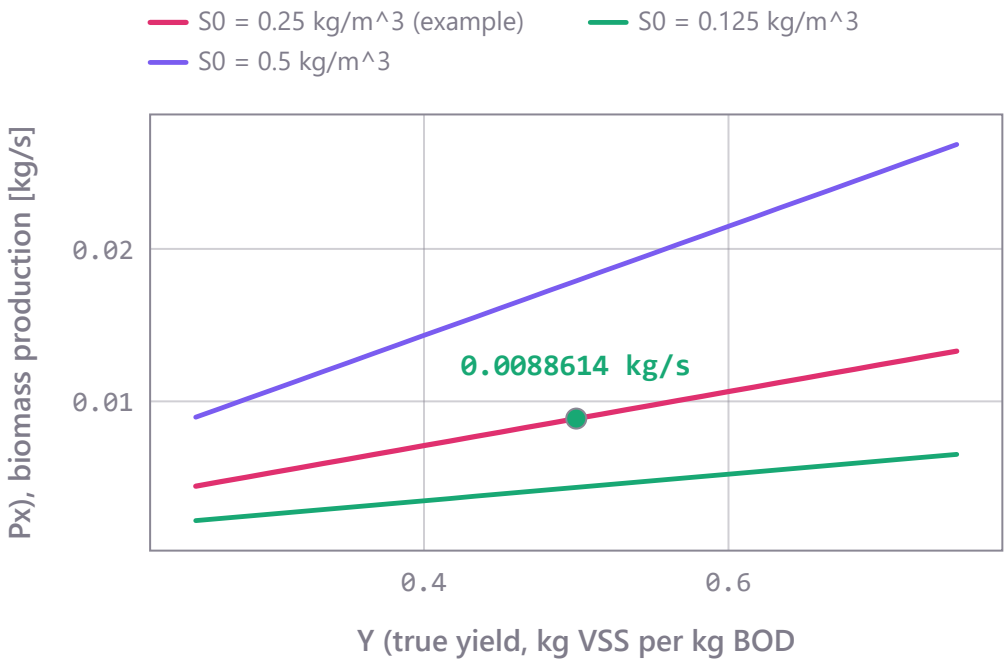
The observed yield and the daily mass of biological solids produced (to be wasted) from the kinetic yield, decay coefficient, SRT, flow and substrate removed.

$$P_x = \frac{Y}{1 + k_d \theta_c} Q(S_0 - S)$$

input	meaning	condition
`Y`	true yield, kg VSS per kg BOD	
`kd`	endogenous decay coefficient	
`SRT`	solids retention time	
`Q`	flow	
`S0`	influent substrate	
`S`	effluent substrate	

output	meaning
`Y_obs`	observed yield
`Px`	biomass production
`Px_kg_per_day`	in kg VSS/d

- **Method:** Y_obs = Y/(1 + kd SRT); Px = Y_obs Q $(S_0 - S)$
- **Assumes:** $S < S_0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Px (biomass production) against Y (true yield, kg VSS per kg BOD) from 0.25 to 0.75. One curve for each value of S0 (influent substrate): 0.25 kg/m^3, 0.125 kg/m^3, 0.5 kg/m^3. Other inputs as in the worked example: kd = 6.944444444444444e-07 1/s, SRT = 864000.0 s, Q = 0.11574074074074074 m^3/s, S = 0.005 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Y = 0.5, Px = 0.0088614 kg/s.

```
environmental_engineering_ai999x.solve('enveng_sludge_production', Y=0.5, kd=6.944444444444444e-07,
SRT=864000.0, Q=0.11574074074074074, S0=0.25, S=0.005)
# -> Y_obs = 0.3125, Px = 0.0088614 kg/s, Px_kg_per_day = 765.625   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sludge_production', Y=0.6, kd=5.787037037037037e-07,
SRT=691200.0, Q=0.046296296296296294, S0=0.2, S=0.01)
# -> Y_obs = 0.428571, Px = 0.00376984 kg/s, Px_kg_per_day = 325.714   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sludge production: Y = 0.5, kd = 0.06 1/d, SRT = 10 d, Q = 10000 m^3/d, S0 = 250 mg/L, S = 5 mg/L	SOLVED	Y_obs = 0.3125, Px = 0.0088614 kg/s, Px_kg_per_day = 765.625
Sludge production: Y = 0.6, kd = 0.05 1/d, SRT = 8 d, Q = 4000 m^3/d, S0 = 200 mg/L, S = 10 mg/L	SOLVED	Y_obs = 0.428571, Px = 0.00376984 kg/s, Px_kg_per_day = 325.714
Sludge production: kd = 0.06 1/d, SRT = 10 d, Q = 10000 m^3/d, S0 = 250 mg/L, S = 5 mg/L	NEEDS_INPUT	enveng_sludge_production needs Y (true yield, kg VSS per kg BOD). Example: solve('enveng_sludge_production', Y=0.5, kd='
Sludge production: Y = 0.5, kd = 0.06 1/d, SRT = 10 d, Q = 10000 m^3/d, S0 = 250 mg/L, S = 5 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Oxygen requirement of an activated-sludge process, `enveng_aeration_oxygen_requirement`

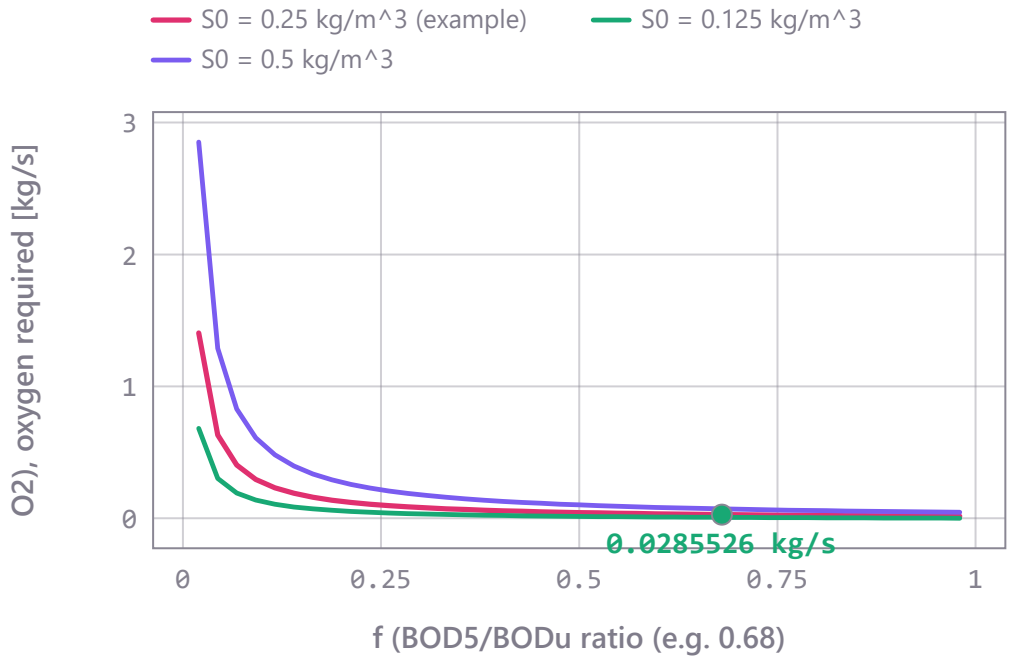
The carbonaceous oxygen requirement: ultimate BOD removed (BOD5 removed divided by the BOD5/BODu ratio) less 1.42 times the biomass wasted.

$$\dot{m}_{O_2} = \frac{Q(S_0 - S)}{f} - 1.42 P_x$$

input	meaning	condition
`Q`	flow	
`S0`	influent BOD5	
`S`	effluent soluble BOD5	
`f`	BOD5/BODu ratio (e.g. 0.68)	
`Px`	biomass wasted (VSS)	

output	meaning
`O2`	oxygen required
`O2_kg_per_day`	in kg O_2/d

- **Method:** $O_2 = Q\,(S_0 - S)/f - 1.42$ Px
- **Assumes:** carbonaceous demand only (no nitrification)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



O2 (oxygen required) against f (BOD5/BODu ratio (e.g. 0.68)) from 0.02 to 0.98. One curve for each value of S0 (influent BOD5): 0.25 kg/m^3, 0.125 kg/m^3, 0.5 kg/m^3. Other inputs as in the worked example: Q = 0.11574074074074074 m^3/s, S = 0.005 kg/m^3, Px = 0.009259259259259259 kg/s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, f = 0.68, O2 = 0.0285526 kg/s.

```
environmental_engineering_ai999x.solve('enveng_aeration_oxygen_requirement', Q=0.11574074074074074,
S0=0.25, S=0.005, f=0.68, Px=0.009259259259259259)
# -> O2 = 0.0285526 kg/s, O2_kg_per_day = 2466.94   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_aeration_oxygen_requirement',
Q=0.046296296296296294, S0=0.2, S=0.01, f=0.7, Px=0.0028935185185185184)
# -> O2 = 0.00845734 kg/s, O2_kg_per_day = 730.714   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aeration oxygen requirement: Q = 10000 m^3/d, S0 = 250 mg/L, S = 5 mg/L, f = 0.68, Px = 800 kg/d	SOLVED	O2 = 0.0285526 kg/s, O2_kg_per_day = 2466.94
Aeration oxygen requirement: Q = 4000 m^3/d, S0 = 200 mg/L, S = 10 mg/L, f = 0.7, Px = 250 kg/d	SOLVED	O2 = 0.00845734 kg/s, O2_kg_per_day = 730.714
Aeration oxygen requirement: S0 = 250 mg/L, S = 5 mg/L, f = 0.68, Px = 800 kg/d	NEEDS_INPUT	enveng_aeration_oxygen_requirement needs Q (flow). Example: solve('enveng_aeration_oxygen_requirement', Q='0.115741 m^3/
Aeration oxygen requirement: Q = 10000 m^3/d, S0 = 250 mg/L, S = 5 mg/L, f = 0.68, Px = 800 kg/d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Return activated sludge ratio, `enveng_return_sludge_ratio`

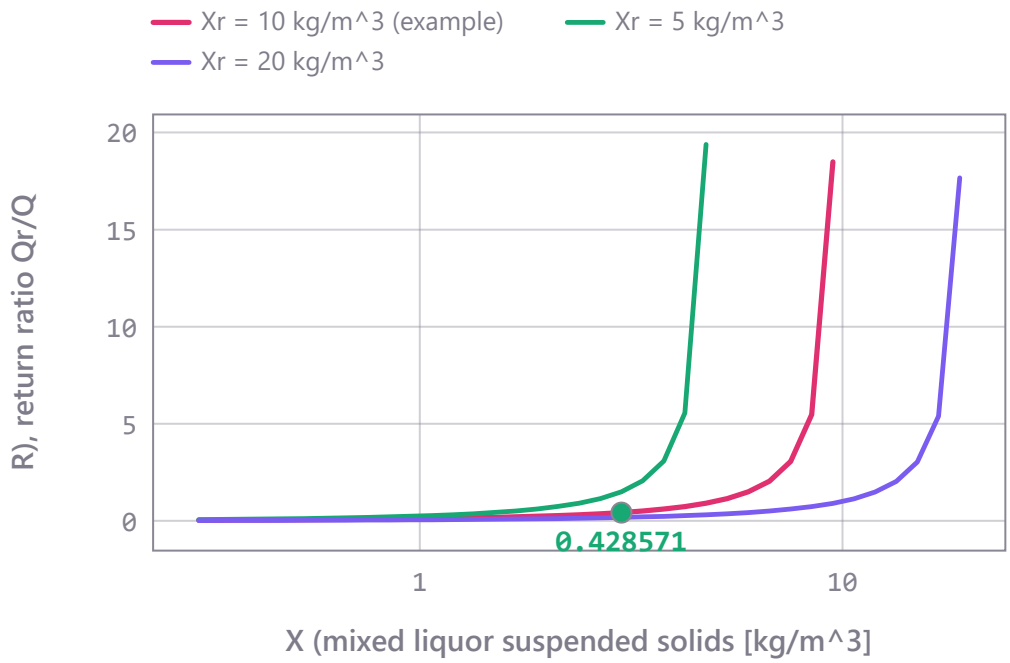
The return sludge ratio and flow needed to hold the mixed-liquor concentration, from a solids balance around the aeration tank (influent solids neglected).

$$R = \frac{Q_r}{Q} = \frac{X}{X_r - X}$$

input	meaning	condition
`X`	mixed liquor suspended solids	
`Xr`	return sludge concentration	
`Q`	influent flow	

output	meaning
`R`	return ratio Qr/Q
`Qr`	return sludge flow

- **Method:** (Q + Qr) $X = Qr$ Xr, so $R = X/(Xr - X)$
- **Assumes:** $Xr > X$; influent solids negligible
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (return ratio Qr/Q) against X (mixed liquor suspended solids) from 0.3 kg/m^3 to 30 kg/m^3 on a logarithmic axis. One curve for each value of Xr (return sludge concentration): 10 kg/m^3, 5 kg/m^3, 20 kg/m^3. Other inputs as in the worked example: Q = 0.11574074074074074 m^3/s. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, X = 3 kg/m^3, R = 0.428571.

```
environmental_engineering_ai999x.solve('enveng_return_sludge_ratio', X=3.0, Xr=10.0,
Q=0.11574074074074074)
# -> R = 0.428571, Qr = 0.0496032 m^3/s   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_return_sludge_ratio', X=2.5, Xr=8.0,
Q=0.05787037037037037)
# -> R = 0.454545, Qr = 0.0263047 m^3/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Return sludge ratio: X = 3000 mg/L, Xr = 10000 mg/L, Q = 10000 m^3/d	SOLVED	R = 0.428571, Qr = 0.0496032 m^3/s
Return sludge ratio: X = 2500 mg/L, Xr = 8000 mg/L, Q = 5000 m^3/d	SOLVED	R = 0.454545, Qr = 0.0263047 m^3/s
Return sludge ratio: Xr = 10000 mg/L, Q = 10000 m^3/d	NEEDS_INPUT	enveng_return_sludge_ratio needs X (mixed liquor suspended solids). Example: solve('enveng_return_sludge_ratio', X='3 kg
Return sludge ratio: X = 3000 mg/L, Xr = 10000 mg/L, Q = 10000 m^3/d, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sludge volume index (SVI), `enveng_sludge_volume_index`

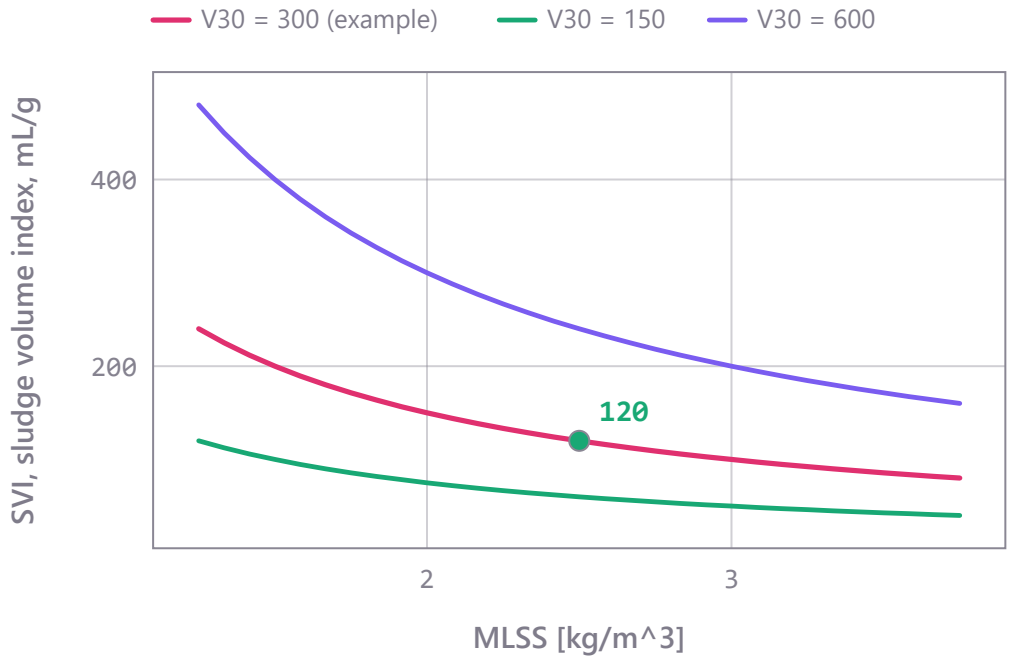
The sludge volume index from the 30-minute settled volume and the mixed-liquor suspended solids, with the customary estimate of the return sludge concentration (10^6/SVI mg/L).

$$SVI = \frac{V_{30}}{MLSS}$$

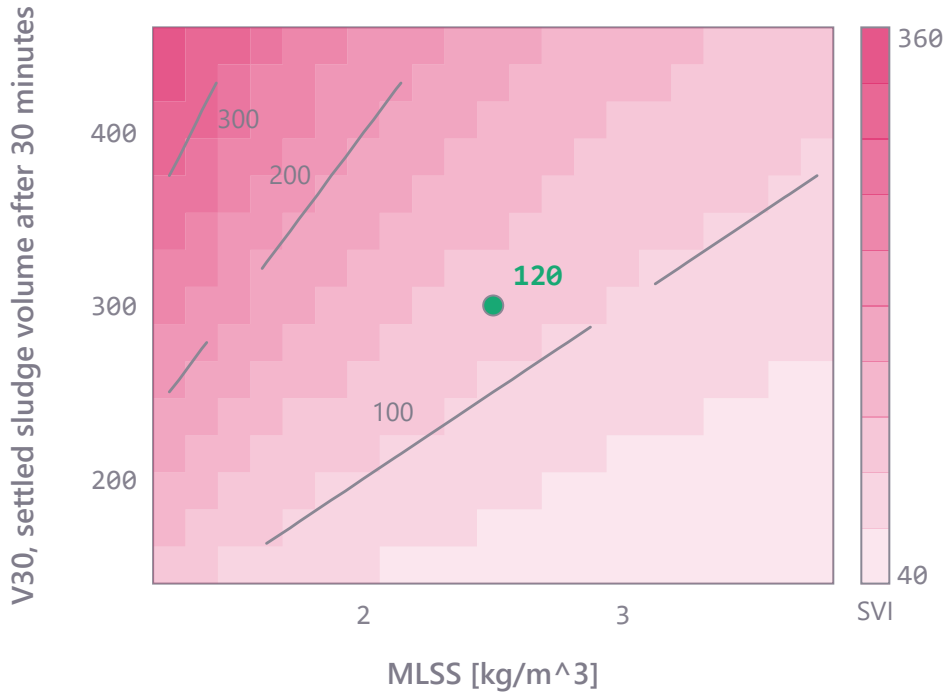
input	meaning	condition
`V30`	settled sludge volume after 30 minutes, mL/L	
`MLSS`	mixed liquor suspended solids	

output	meaning
`SVI`	sludge volume index, mL/g
`Xr_estimate`	estimated maximum return sludge concentration
`Xr_estimate_mgL`	estimate in mg/L

- **Method:** SVI = V30 (mL/L)/MLSS (g/L)
- **Assumes:** $V30 \leq 1000$ mL/L
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



SVI (sludge volume index, mL/g) against MLSS (mixed liquor suspended solids) from 1.25 kg/m^3 to 3.75 kg/m^3. One curve for each value of V30 (settled sludge volume after 30 minutes, mL/L): 300, 150, 600. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, MLSS = 2.5 kg/m^3, SVI = 120.



SVI (sludge volume index, mL/g) over MLSS (mixed liquor suspended solids) from 1.25 kg/m^3 to 3.75 kg/m^3 and V30 (settled sludge volume after 30 minutes, mL/L) from 150 to 450: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 40 to dark 360); iso-lines at SVI = 100, 200, 300. Marked: the worked example, MLSS = 2.5 kg/m^3, V30 = 300, SVI = 120.

```
environmental_engineering_ai999x.solve('enveng_sludge_volume_index', V30=300.0, MLSS=2.5)
# -> SVI = 120, Xr_estimate = 8.33333 kg/m^3, Xr_estimate_mgL = 8333.33   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sludge_volume_index', V30=450.0, MLSS=3.2)
# -> SVI = 140.625, Xr_estimate = 7.11111 kg/m^3, Xr_estimate_mgL = 7111.11   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sludge volume index: V30 = 300 mL/L, MLSS = 2500 mg/L	SOLVED	SVI = 120, Xr_estimate = 8.33333 kg/m^3, Xr_estimate_mgL = 8333.33
Sludge volume index: V30 = 450 mL/L, MLSS = 3200 mg/L	SOLVED	SVI = 140.625, Xr_estimate = 7.11111 kg/m^3, Xr_estimate_mgL = 7111.11
Sludge volume index: MLSS = 2500 mg/L	NEEDS_INPUT	enveng_sludge_volume_index needs V30 (settled sludge volume after 30 minutes, mL/L). Example: solve('enveng_sludge_volum
Sludge volume index: V30 = 300 mL/L, MLSS = 2500 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Volume of wet sludge from dry solids, `enveng_sludge_volume`

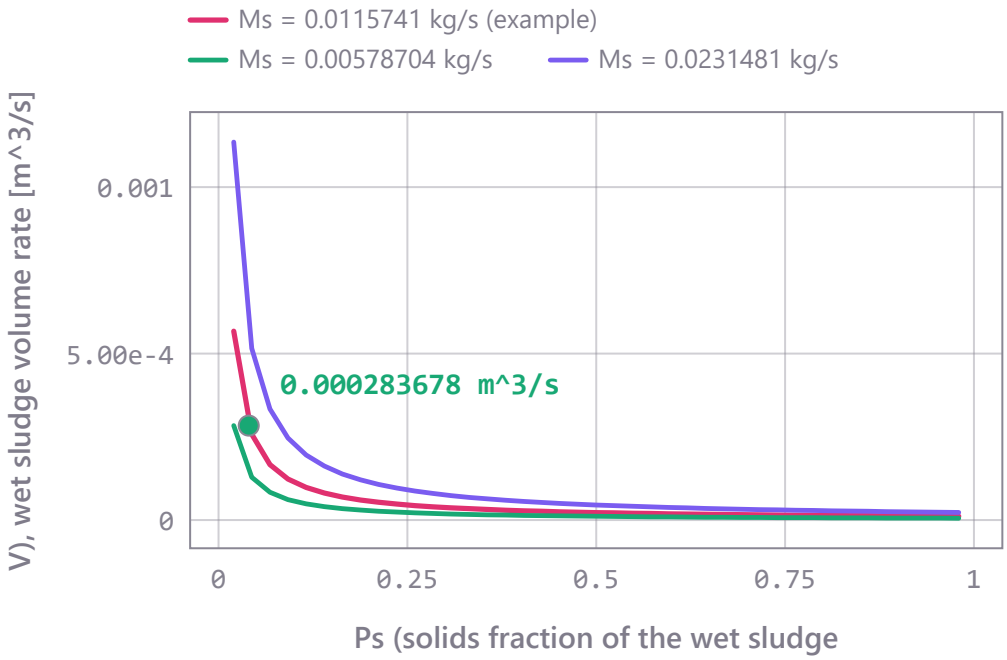
The volume (flow) of wet sludge from the dry-solids mass rate, the solids fraction and the specific gravity of the sludge (water 1000 kg/m^3).

$$V = \frac{M_s}{\rho_w S_{sl} P_s}$$

input	meaning	condition
`Ms`	dry solids mass rate	
`Ps`	solids fraction of the wet sludge (fraction or %)	
`Ssl`	specific gravity of the wet sludge	

output	meaning
`V`	wet sludge volume rate
`V_m3_per_day`	in m^3/d

- **Method:** V = Ms/(rho_w Ssl Ps)
- **Assumes:** $0 < Ps \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V (wet sludge volume rate) against Ps (solids fraction of the wet sludge (fraction or %)) from 0.02 to 0.98. One curve for each value of Ms (dry solids mass rate): 0.0115741 kg/s, 0.00578704 kg/s, 0.0231481 kg/s. Other inputs as in the worked example: Ssl = 1.02. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Ps = 0.04, V = 0.000283678 m^3/s.

```
environmental_engineering_ai999x.solve('enveng_sludge_volume', Ms=0.011574074074074073, Ps=0.04,
Ssl=1.02)
# -> V = 0.000283678 m^3/s, V_m3_per_day = 24.5098   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sludge_volume', Ms=0.04050925925925926, Ps=0.22,
Ssl=1.05)
# -> V = 0.000175365 m^3/s, V_m3_per_day = 15.1515   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wet sludge volume: Ms = 1000 kg/d, Ps = 0.04, Ssl = 1.02	SOLVED	V = 0.000283678 m^3/s, V_m3_per_day = 24.5098
Wet sludge volume: Ms = 3500 kg/d, Ps = 0.22, Ssl = 1.05	SOLVED	V = 0.000175365 m^3/s, V_m3_per_day = 15.1515
Wet sludge volume: Ps = 0.04, Ssl = 1.02	NEEDS_INPUT	enveng_sludge_volume needs Ms (dry solids mass rate). Example: solve('enveng_sludge_volume', Ms='0.0115741 kg/s', Ps=0.0
Wet sludge volume: Ms = 1000 kg/d, Ps = 0.04, Ssl = 1.02, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sludge volume after thickening or dewatering, `enveng_sludge_thickening`

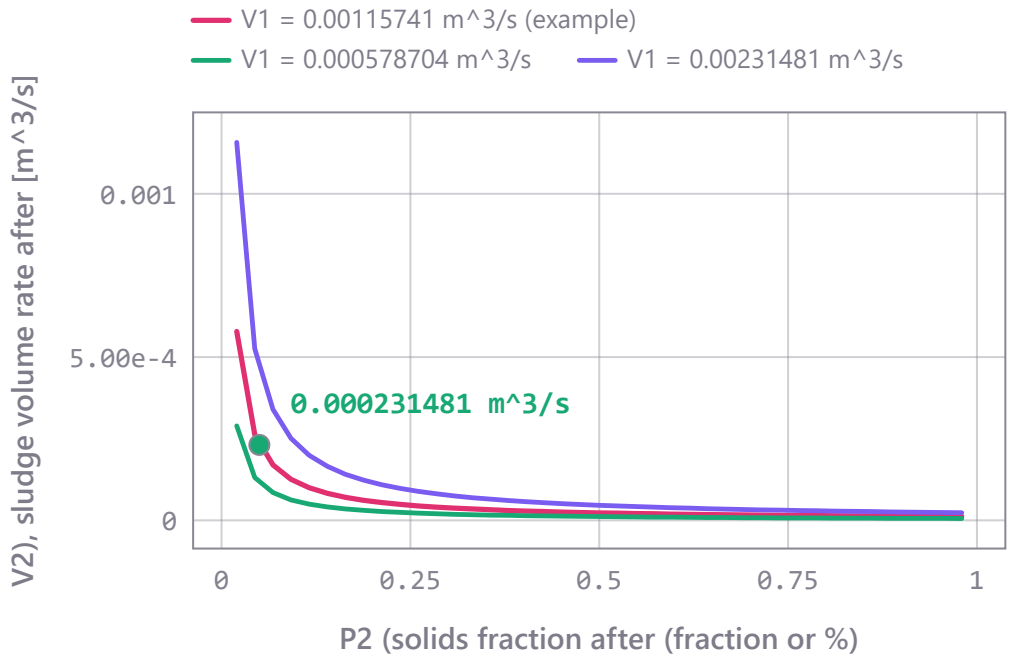
The sludge volume after thickening or dewatering from one solids fraction to another (same specific gravity), and the fractional volume reduction.

$$V_2 = V_1 \frac{P_1}{P_2}$$

input	meaning	condition
`V1`	sludge volume rate before	
`P1`	solids fraction before (fraction or %)	
`P2`	solids fraction after (fraction or %)	

output	meaning
`V2`	sludge volume rate after
`reduction`	fractional volume reduction

- **Method:** $V_2 = V_1\ P_1/P_2$
- **Assumes:** $P_2 > P_1$; same specific gravity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V2 (sludge volume rate after) against P2 (solids fraction after (fraction or %)) from 0.02 to 0.98. One curve for each value of V1 (sludge volume rate before): 0.00115741 m^3/s, 0.000578704 m^3/s, 0.00231481 m^3/s. Other inputs as in the worked example: P1 = 0.01. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, P2 = 0.05, V2 = 0.000231481 m^3/s.

```
environmental_engineering_ai999x.solve('enveng_sludge_thickening', V1=0.0011574074074074073,
P1=0.01, P2=0.05)
# -> V2 = 0.000231481 m^3/s, reduction = 0.8   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sludge_thickening', V1=0.000462962962962963,
P1=0.04, P2=0.2)
# -> V2 = 9.25926e-05 m^3/s, reduction = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sludge thickening: V1 = 100 m^3/d, P1 = 0.01, P2 = 0.05	SOLVED	V2 = 0.000231481 m^3/s, reduction = 0.8
Sludge thickening: V1 = 40 m^3/d, P1 = 0.04, P2 = 0.2	SOLVED	V2 = 9.25926e-05 m^3/s, reduction = 0.8
Sludge thickening: P1 = 0.01, P2 = 0.05	NEEDS_INPUT	enveng_sludge_thickening needs V1 (sludge volume rate before). Example: solve('enveng_sludge_thickening', V1='0.00115741
Sludge thickening: V1 = 100 m^3/d, P1 = 0.01, P2 = 0.05, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Sludge thickening: V1 = 100 m^3/d, P1 = 1%, P2 = 5%	SOLVED	V2 = 0.000231481 m^3/s, reduction = 0.8

Methane production from COD removed (anaerobic digestion), `enveng_anaerobic_methane_yield`

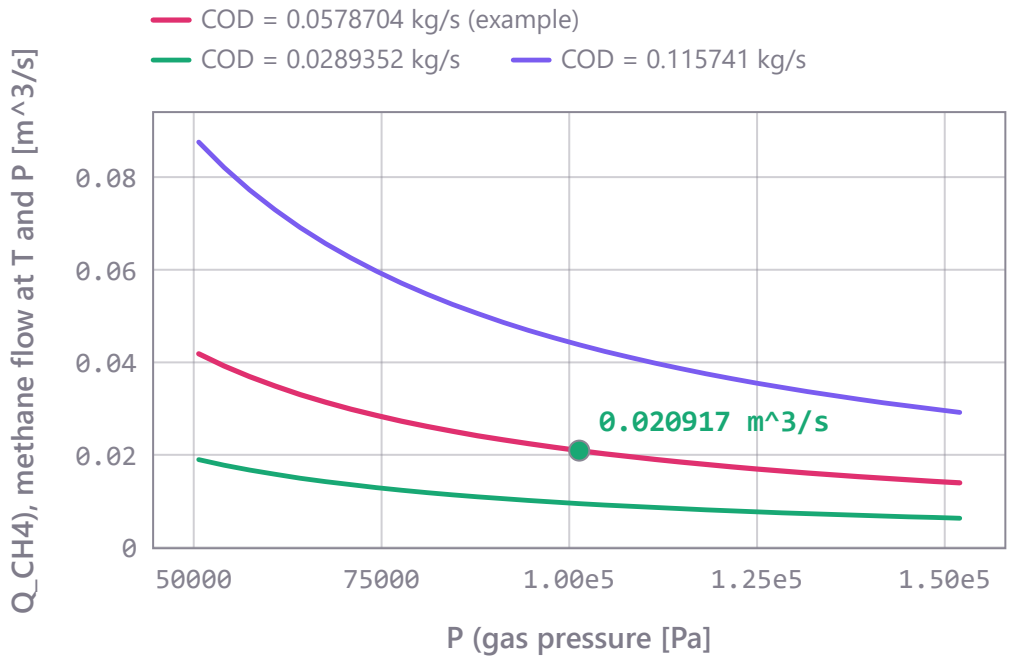
The methane gas flow from an anaerobic process from the COD removed less the COD converted to biomass (1.42 kg COD per kg VSS), at a stated temperature and pressure (64 g COD per mole of methane).

$$Q_{CH_4} = \frac{\mathrm{COD} - 1.42 P_x}{64\ \mathrm{g/mol}} \frac{RT}{P}$$

input	meaning	condition
`COD`	COD removed	
`Px`	biomass (VSS) produced	
`T`	gas temperature	
`P`	gas pressure	

output	meaning
`Q_CH4`	methane flow at T and P
`Q_CH4_m3_per_day`	in m^3/d at T and P
`Q_CH4_stp_m3_per_day`	in m^3/d at 0 degC and 1 atm

- **Method:** n = (COD - 1.42 Px)/(64 g/mol); $Q = n$ R T/P
- **Assumes:** ideal gas; COD balance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q_CH4 (methane flow at T and P) against P (gas pressure) from 50662.5 Pa to 151988 Pa. One curve for each value of COD (COD removed): 0.0578704 kg/s, 0.0289352 kg/s, 0.115741 kg/s. Other inputs as in the worked example: Px = 0.003472222222222222 kg/s, T = 308.15 K. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, P = 101325 Pa, Q_CH4 = 0.020917 m^3/s.

```
environmental_engineering_ai999x.solve('enveng_anaerobic_methane_yield', COD=0.05787037037037037,
Px=0.003472222222222222, T=308.15, P=101325.0)
# -> Q_CH4 = 0.020917 m^3/s, Q_CH4_m3_per_day = 1807.22, Q_CH4_stp_m3_per_day = 1601.96   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_anaerobic_methane_yield', COD=0.013888888888888888,
Px=0.0006944444444444445, T=298.15, P=100000.0)
# -> Q_CH4 = 0.00499791 m^3/s, Q_CH4_m3_per_day = 431.82, Q_CH4_stp_m3_per_day = 390.438   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Methane production from COD: COD = 5000 kg/d, Px = 300 kg/d, T = 308.15 K, P = 101.325 kPa	SOLVED	Q_CH4 = 0.020917 m^3/s, Q_CH4_m3_per_day = 1807.22, Q_CH4_stp_m3_per_day = 1601.96
Methane production from COD: COD = 1200 kg/d, Px = 60 kg/d, T = 298.15 K, P = 100 kPa	SOLVED	Q_CH4 = 0.00499791 m^3/s, Q_CH4_m3_per_day = 431.82, Q_CH4_stp_m3_per_day = 390.438
Methane production from COD: Px = 300 kg/d, T = 308.15 K, P = 101.325 kPa	NEEDS_INPUT	enveng_anaerobic_methane_yield needs COD (COD removed). Example: solve('enveng_anaerobic_methane_yield', COD='0.0578704
Methane production from COD: COD = 5000 kg/d, Px = 300 kg/d, T = 308.15 K, P = 101.325 kPa, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Trickling filter BOD removal efficiency (NRC equation), `enveng_nrc_trickling_filter`

The BOD removal efficiency of a single-stage stone-media trickling filter (with its settling tank) by the NRC equation, from the BOD load, the filter volume and the recirculation ratio.

$$E = \frac{100}{1 + 0.4432\sqrt{W/(VF)}}, \quad F = \frac{1+R}{(1+R/10)^2}$$

input	meaning	condition
`W`	BOD load to the filter (excluding recycle)	
`V`	filter media volume	
`R`	recirculation ratio Qr/Q	

output	meaning
`F`	recirculation factor $(1+R)/(1+R/10)^2$
`E`	BOD removal efficiency (fraction)
`E_percent`	efficiency in per cent

- **Method:** $F = (1+R)/(1+R/10)^2$; E = 1/(1 + 0.4432 sqrt(W/(V F)))
- **Assumes:** first-stage stone filter at 20 degC; W in kg/d and V in m^3 inside the formula constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



E (BOD removal efficiency (fraction)) against R (recirculation ratio Qr/Q) from 0.2 to 20 on a logarithmic axis. One curve for each value of W (BOD load to the filter (excluding recycle)): 0.00925926 kg/s, 0.00462963 kg/s, 0.0185185 kg/s. Other inputs as in the worked example: V = 1000.0 m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, R = 2, E = 0.784535.

```
environmental_engineering_ai999x.solve('enveng_nrc_trickling_filter', W=0.009259259259259259,
V=1000.0, R=2.0)
# -> F = 2.08333, E = 0.784535, E_percent = 78.4535   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_nrc_trickling_filter', W=0.017361111111111112,
V=1200.0, R=1.0)
# -> F = 1.65289, E = 0.721804, E_percent = 72.1804   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
NRC trickling filter efficiency: W = 800 kg/d, V = 1000 m^3, R = 2	SOLVED	F = 2.08333, E = 0.784535, E_percent = 78.4535
NRC trickling filter efficiency: W = 1500 kg/d, V = 1200 m^3, R = 1	SOLVED	F = 1.65289, E = 0.721804, E_percent = 72.1804
NRC trickling filter efficiency: V = 1000 m^3, R = 2	NEEDS_INPUT	enveng_nrc_trickling_filter needs W (BOD load to the filter (excluding recycle)). Example: solve('enveng_nrc_trickling_f
NRC trickling filter efficiency: W = 800 kg/d, V = 1000 m^3, R = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Field (actual) oxygen transfer rate of an aeration system, `enveng_field_oxygen_transfer`

The actual oxygen transfer rate in the process from the standard (clean water, 20 degC, zero DO) rate, corrected by alpha, beta, the temperature coefficient, the site saturation and the operating DO.

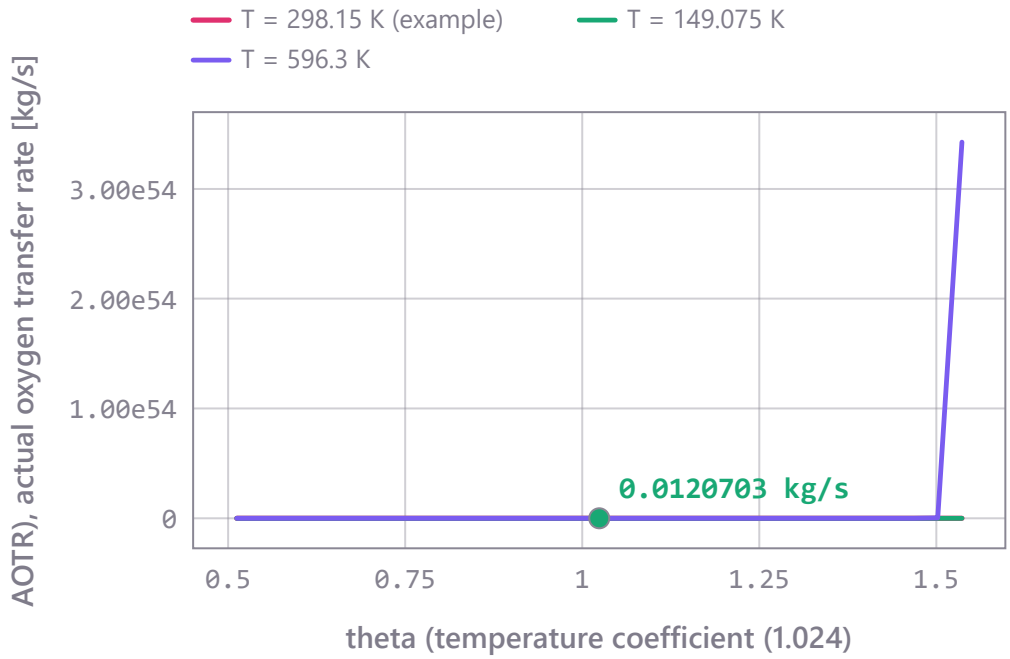
$$AOTR = SOTR\,\alpha\,\theta^{T-20}\,\frac{\beta C_{s,T} F - C_L}{C_{s,20}}$$

input	meaning	condition
`SOTR`	standard oxygen transfer rate	
`alpha`	alpha factor (process/clean water KLa)	
`beta`	beta factor (process/clean water saturation)	
`theta`	temperature coefficient (1.024)	
`CsT`	clean-water saturation DO at the process temperature	
`Cs20`	clean-water saturation DO at 20 degC	
`CL`	operating DO	
`T`	process temperature	
`Fp`	pressure (elevation) correction of saturation	

output	meaning
`AOTR`	actual oxygen transfer rate
`AOTR_kg_per_h`	in kg O_2/h
`ratio`	AOTR/SOTR

- **Method:** AOTR = SOTR alpha θ^{T-20} (beta CsT Fp - CL)/Cs20
- **Assumes:** beta CsT $Fp > CL$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



AOTR (actual oxygen transfer rate) against theta (temperature coefficient (1.024)) from 0.512 to 1.536. One curve for each value of T (process temperature): 298.15 K, 149.075 K, 596.3 K. Other inputs as in the worked example: SOTR = 0.027777777777777776 kg/s, alpha = 0.6, beta = 0.95, CsT = 0.00826 kg/m^3, Cs20 = 0.00909 kg/m^3, CL = 0.002 kg/m^3. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, theta = 1.024, AOTR = 0.0120703 kg/s.

```
environmental_engineering_ai999x.solve('enveng_field_oxygen_transfer', SOTR=0.027777777777777776,
alpha=0.6, beta=0.95, theta=1.024, CsT=0.00826, Cs20=0.00909, CL=0.002, T=298.15)
# -> AOTR = 0.0120703 kg/s, AOTR_kg_per_h = 43.453, ratio = 0.43453   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_field_oxygen_transfer', SOTR=0.06944444444444445,
alpha=0.5, beta=0.98, theta=1.024, CsT=0.01008, Cs20=0.00909, CL=0.0015, T=285.15, Fp=0.9)
# -> AOTR = 0.0233519 kg/s, AOTR_kg_per_h = 84.0667, ratio = 0.336267   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Field oxygen transfer rate: SOTR = 100 kg/h, alpha = 0.6, beta = 0.95, theta = 1.024, CsT = 8.26 mg/L, Cs20 = 9.09 mg/L, CL = 2 mg/L, T = 25 degC	SOLVED	AOTR = 0.0120703 kg/s, AOTR_kg_per_h = 43.453, ratio = 0.43453
Field oxygen transfer rate: SOTR = 250 kg/h, alpha = 0.5, beta = 0.98, theta = 1.024, CsT = 10.08 mg/L, Cs20 = 9.09 mg/L, CL = 1.5 mg/L, T = 12 degC, Fp = 0.9	SOLVED	AOTR = 0.0233519 kg/s, AOTR_kg_per_h = 84.0667, ratio = 0.336267
Field oxygen transfer rate: alpha = 0.6, beta = 0.95, theta = 1.024, CsT = 8.26 mg/L, Cs20 = 9.09 mg/L, CL = 2 mg/L, T = 25 degC	NEEDS_INPUT	enveng_field_oxygen_transfer needs SOTR (standard oxygen transfer rate). Example: solve('enveng_field_oxygen_transfer',
Field oxygen transfer rate: SOTR = 100 kg/h, alpha = 0.6, beta = 0.95, theta =	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
1.024, CsT = 8.26 mg/L, Cs20 = 9.09 mg/L, CL = 2 mg/L, T = 25 degC, zz = 1		environmental_engineering_ai999x.

Overall oxygen transfer coefficient KLa from two DO readings, `enveng_kla_two_point`

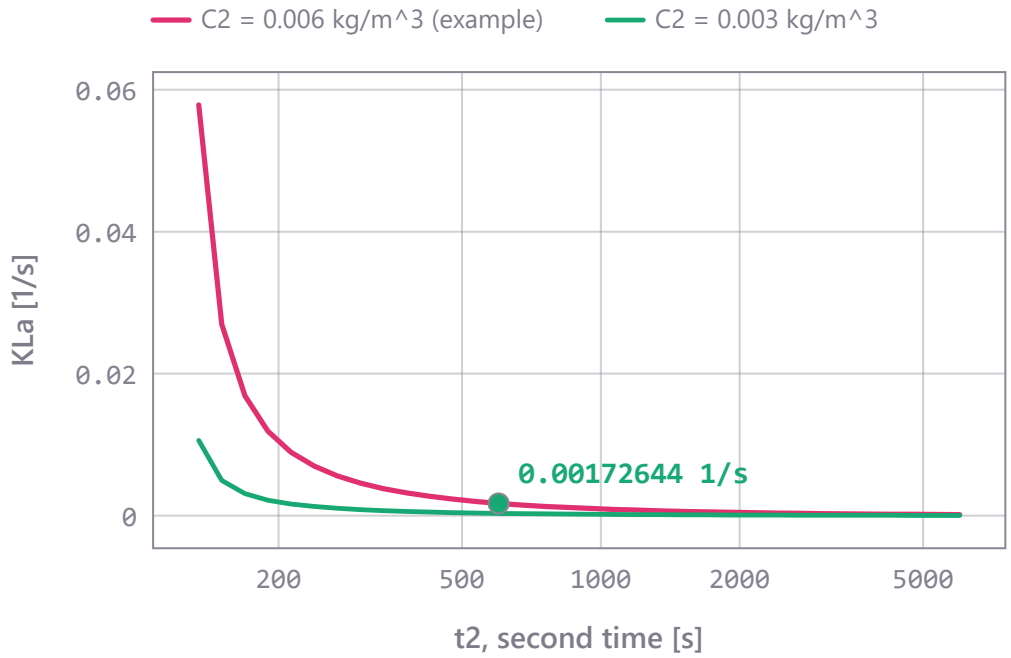
The volumetric mass-transfer coefficient KLa from two dissolved-oxygen readings during reaeration toward a known saturation concentration.

$$K_L a = \frac{\ln \frac{C_s - C_1}{C_s - C_2}}{t_2 - t_1}$$

input	meaning	condition
`Cs`	saturation DO	
`C1`	DO at the first time	
`C2`	DO at the second time	
`t1`	first time	
`t2`	second time	

output	meaning
`KLa`	KLa
`KLa_per_h`	KLa in $1/h$

- **Method:** KLa = ln(($Cs - C_1$)/($Cs - C_2$))/($t_2 - t_1$)
- **Assumes:** $C_1 < C_2 < Cs$; $t_2 > t_1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



KLa (KLa) against t2 (second time) from 60 s to 6000 s on a logarithmic axis. One curve for each value of C2 (DO at the second time): 0.006 kg/m^3, 0.003 kg/m^3. Other inputs as in the worked example: Cs = 0.0091 kg/m^3, C1 = 0.002 kg/m^3, t1 = 120.0 s. 34 points per curve computed by environmental-engineering-ai999x; marked: the worked example, t2 = 600 s, KLa = 0.00172644 1/s.

```
environmental_engineering_ai999x.solve('enveng_kla_two_point', Cs=0.0091, C1=0.002, C2=0.006,
t1=120.0, t2=600.0)
# -> KLa = 0.00172644 1/s, KLa_per_h = 6.2152   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_kla_two_point', Cs=0.0085, C1=0.001, C2=0.0055,
t1=0.0, t2=900.0)
# -> KLa = 0.0010181 1/s, KLa_per_h = 3.66516   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
KLa from two DO readings: Cs = 9.1 mg/L, C1 = 2 mg/L, C2 = 6 mg/L, t1 = 2 min, t2 = 10 min	SOLVED	KLa = 0.00172644 1/s, KLa_per_h = 6.2152
KLa from two DO readings: Cs = 8.5 mg/L, C1 = 1 mg/L, C2 = 5.5 mg/L, t1 = 0 min, t2 = 15 min	SOLVED	KLa = 0.0010181 1/s, KLa_per_h = 3.66516
KLa from two DO readings: C1 = 2 mg/L, C2 = 6 mg/L, t1 = 2 min, t2 = 10 min	NEEDS_INPUT	enveng_kla_two_point needs Cs (saturation DO). Example: solve('enveng_kla_two_point', Cs='0.0091 kg/m^3', C1='0.002 kg/m
KLa from two DO readings: Cs = 9.1 mg/L, C1 = 2 mg/L, C2 = 6 mg/L, t1 = 2 min, t2 = 10 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

KLa fitted to a reaeration test, `enveng_kla_regression`

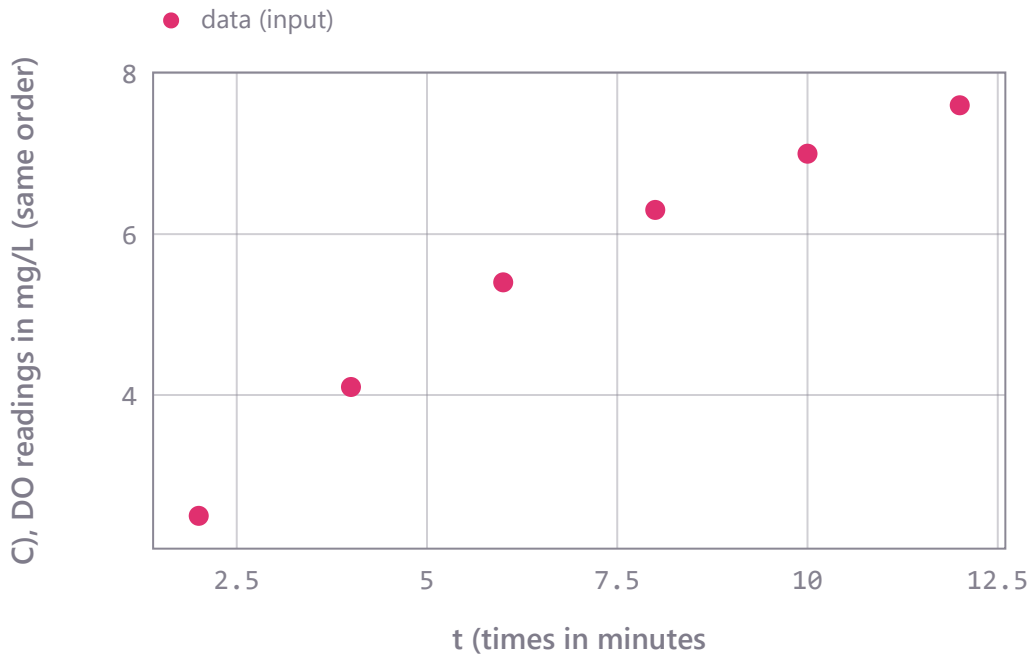
The volumetric oxygen-transfer coefficient KLa from a series of dissolved-oxygen readings during a reaeration test, by linear regression of ln(Cs - C) against time.

$$\ln(C_s - C) = \ln(C_s - C_0) - K_L a\, t$$

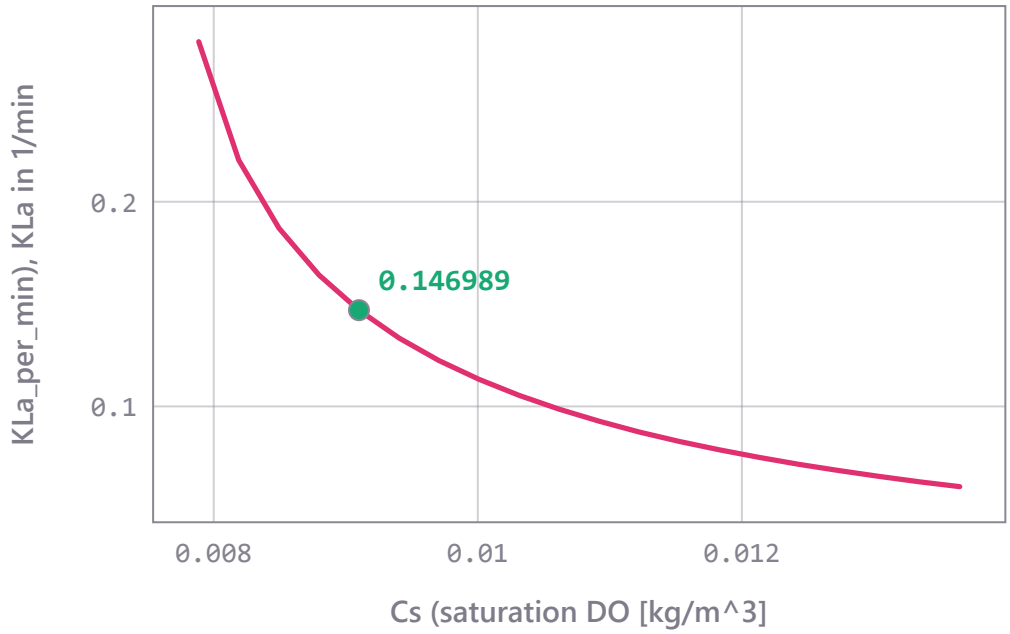
input	meaning	condition
`t`	times in minutes	
`C`	DO readings in mg/L (same order)	
`Cs`	saturation DO	

output	meaning
`KLa_per_min`	KLa in 1/min
`KLa`	KLa
`KLa_per_h`	KLa in $1/h$
`r2`	coefficient of determination

- **Method:** least squares of ln(Cs - C) = ln(Cs - C0) - KLa t
- **Assumes:** every reading below Cs; at least three distinct times
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data C (DO readings in mg/L (same order)) against t (times in minutes) as given in the worked example (6 points). Also returned: KLa_per_min = 0.146989, KLa = 0.00244982 1/s, KLa_per_h = 8.81935, r2 = 0.999181. Other inputs: Cs = 0.0091 kg/m^3.



KLa_per_min (KLa in 1/min) against Cs (saturation DO) from 0.00455 kg/m^3 to 0.01365 kg/m^3. Other inputs as in the worked example: t = [2.0, 4.0, 6.0, 8.0, 10.0, 12.0], C = [2.5, 4.1, 5.4, 6.3, 7.0, 7.6]. 20 points computed by environmental-engineering-ai999x; marked: the worked example, Cs = 0.0091 kg/m^3, KLa_per_min = 0.146989.

```
environmental_engineering_ai999x.solve('enveng_kla_regression', t=[2.0, 4.0, 6.0, 8.0, 10.0, 12.0], C=[2.5, 4.1, 5.4, 6.3, 7.0, 7.6], Cs=0.0091)
# -> KLa_per_min = 0.146989, KLa = 0.00244982 1/s, KLa_per_h = 8.81935, r2 = 0.999181   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_kla_regression', t=[1.0, 3.0, 5.0, 9.0], C=[1.2, 3.0, 4.3, 6.1], Cs=0.0088)
# -> KLa_per_min = 0.128961, KLa = 0.00214935 1/s, KLa_per_h = 7.73766, r2 = 0.999847   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
KLa regression: t = [2, 4, 6, 8, 10, 12], C = [2.5, 4.1, 5.4, 6.3, 7, 7.6], Cs = 9.1 mg/L	SOLVED	KLa_per_min = 0.146989, KLa = 0.00244982 1/s, KLa_per_h = 8.81935, r2 = 0.999181
KLa regression: t = [1, 3, 5, 9], C = [1.2, 3, 4.3, 6.1], Cs = 8.8 mg/L	SOLVED	KLa_per_min = 0.128961, KLa = 0.00214935 1/s, KLa_per_h = 7.73766, r2 = 0.999847
KLa regression: C = [2.5, 4.1, 5.4, 6.3, 7, 7.6], Cs = 9.1 mg/L	NEEDS_INPUT	enveng_kla_regression needs t (times in minutes). Example: solve('enveng_kla_regression', t=[2.0, 4.0, 6.0, 8.0, 10.0, 1
KLa regression: t = [2, 4, 6, 8, 10, 12], C = [2.5, 4.1, 5.4, 6.3, 7, 7.6], Cs = 9.1 mg/L, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Dissolved oxygen during aeration and the transfer rate, `enveng_aeration_do_curve`

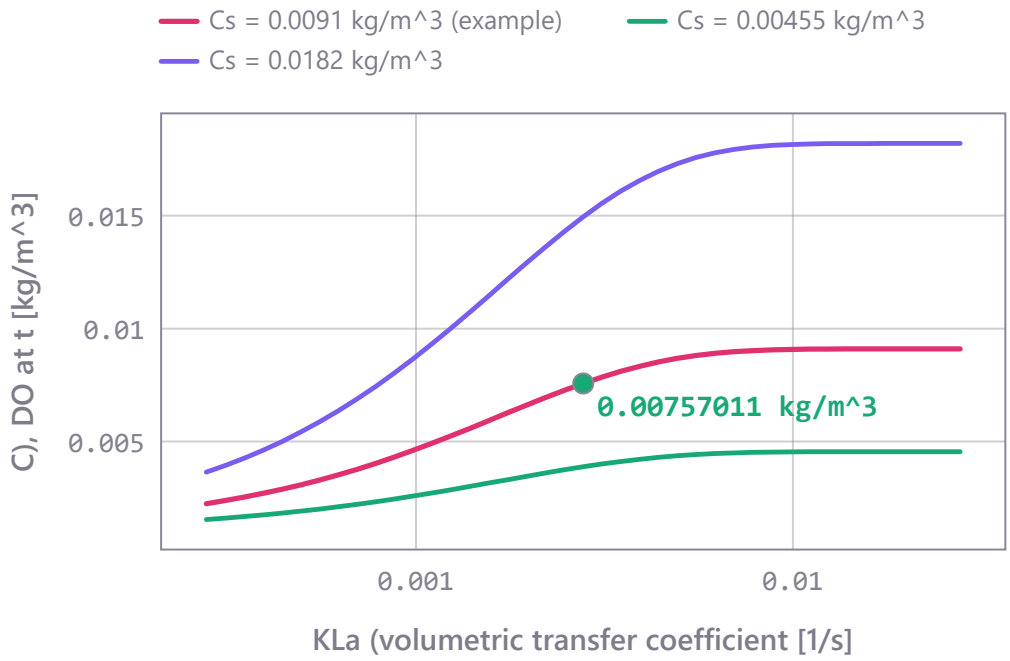
The dissolved oxygen after an aeration time from the starting DO, the saturation DO and KLa, and the oxygen transfer rate KLa (Cs - C) at that moment.

$$C = C_s - (C_s - C_0)e^{-K_L a\, t}$$

input	meaning	condition
`Cs`	saturation DO	
`C0`	starting DO	
`KLa`	volumetric transfer coefficient	
`t`	aeration time	

output	meaning
`C`	DO at t
`C_mgL`	in mg/L
`OTR`	oxygen transfer rate at t
`OTR_mg_per_L_h`	transfer rate in mg/(L h)

- **Method:** dC/dt = KLa (Cs - C), $C = Cs - (Cs - C_0)$ e^(-KLa t)
- **Assumes:** $C_0 \leq Cs$; no oxygen uptake
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (DO at t) against KLa (volumetric transfer coefficient) from 0.000277778 1/s to 0.0277778 1/s on a logarithmic axis. One curve for each value of Cs (saturation DO): 0.0091 kg/m^3, 0.00455 kg/m^3, 0.0182 kg/m^3. Other inputs as in the worked example: C0 = 0.001 kg/m^3, t = 600.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, KLa = 0.00277778 1/s, C = 0.00757011 kg/m^3.

```
environmental_engineering_ai999x.solve('enveng_aeration_do_curve', Cs=0.0091, C0=0.001,
KLa=0.002777777777777778, t=600.0)
# -> C = 0.00757011 kg/m^3, C_mgL = 7.57011, OTR = 4.2497e-06 kg/(m^3*s), OTR_mg_per_L_h = 15.2989
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_aeration_do_curve', Cs=0.0085, C0=0.002,
KLa=0.0011111111111111111, t=1200.0)
# -> C = 0.00678662 kg/m^3, C_mgL = 6.78662, OTR = 1.90376e-06 kg/(m^3*s), OTR_mg_per_L_h = 6.85353
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aeration DO curve: Cs = 9.1 mg/L, C0 = 1 mg/L, KLa = 10 1/h, t = 10 min	SOLVED	C = 0.00757011 kg/m^3, C_mgL = 7.57011, OTR = 4.2497e-06 kg/(m^3*s), OTR_mg_per_L_h = 15.2989
Aeration DO curve: Cs = 8.5 mg/L, C0 = 2 mg/L, KLa = 4 1/h, t = 20 min	SOLVED	C = 0.00678662 kg/m^3, C_mgL = 6.78662, OTR = 1.90376e-06 kg/(m^3*s), OTR_mg_per_L_h = 6.85353
Aeration DO curve: C0 = 1 mg/L, KLa = 10 1/h, t = 10 min	NEEDS_INPUT	enveng_aeration_do_curve needs Cs (saturation DO). Example: solve('enveng_aeration_do_curve', Cs='0.0091 kg/m^3', C0='0.
Aeration DO curve: Cs = 9.1 mg/L, C0 = 1 mg/L, KLa = 10 1/h, t = 10 min, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Henry's law constant conversion between forms, `enveng_henry_constant_conversion`

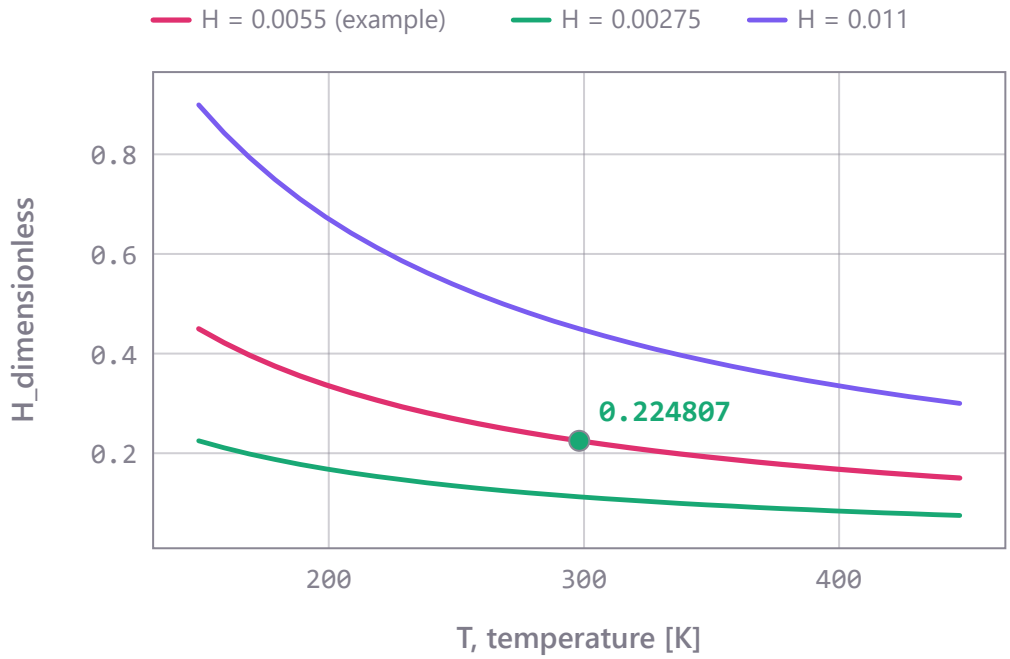
A Henry's law constant converted between its common forms: volatility (Pa m^3/mol, atm m^3/mol), dimensionless gas/liquid ratio at temperature T, mole-fraction form (atm) and solubility forms (mol/(m^3 Pa), mol/(L atm)).

$$H_{cc} = \frac{H_{pc}}{RT}, \quad H_{px} = H_{pc}\, c_w, \quad H_{cp} = \frac{1}{H_{pc}}$$

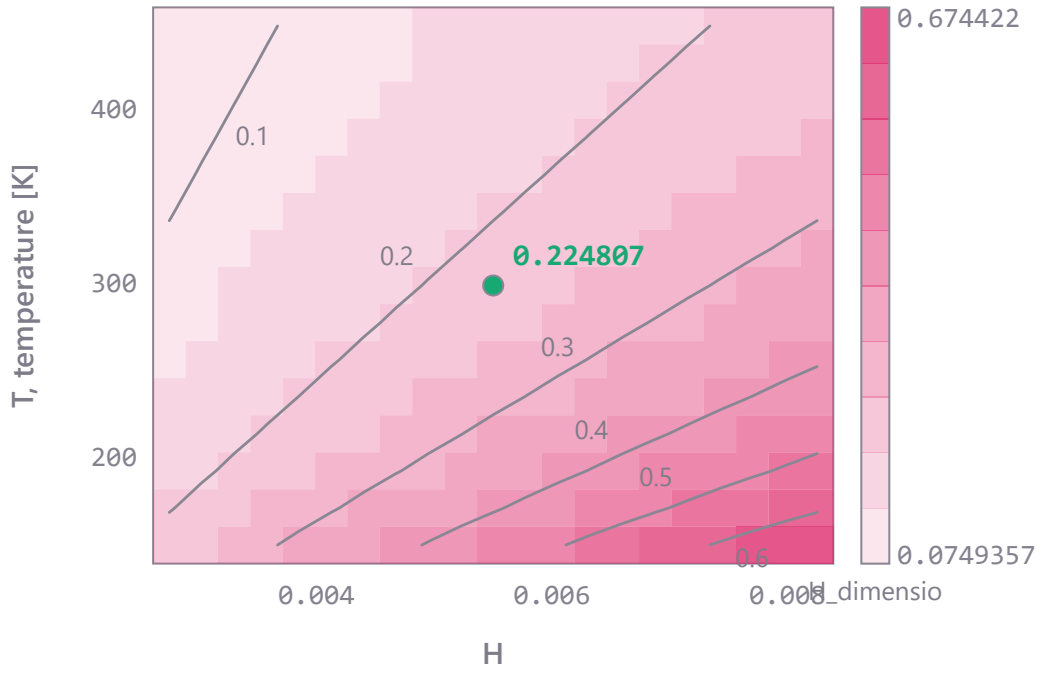
input	meaning	condition
`H`	the Henry's constant value in the stated form	
`form`	form of H: pa_m3_per_mol, atm_m3_per_mol, dimensionless (Cg/Cl), atm (p = H x), mol_per_m3_pa or m_per_atm (mol/(L atm))	
`T`	temperature (for the dimensionless form)	

output	meaning
`H_pa_m3_per_mol`	volatility constant, Pa m^3/mol
`H_atm_m3_per_mol`	volatility constant, atm m^3/mol
`H_dimensionless`	dimensionless gas/liquid ratio at T
`H_atm`	mole-fraction form, atm
`H_mol_per_m3_pa`	solubility constant, mol/(m^3 Pa)
`H_m_per_atm`	solubility constant, mol/(L atm)

- **Method:** all forms through the volatility constant Hpc: Hcc = Hpc/(R T), Hpx = Hpc rho_w, Hcp = 1/Hpc
- **Assumes:** dilute aqueous solution (water 55344.59 mol/m^3)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



H_dimensionless (dimensionless gas/liquid ratio at T) against T (temperature (for the dimensionless form)) from 149.075 K to 447.225 K. One curve for each value of H (the Henry's constant value in the stated form): 0.0055, 0.00275, 0.011. Other inputs as in the worked example: form = atm_m3_per_mol. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, T = 298.15 K, H_dimensionless = 0.224807.



H_dimensionless (dimensionless gas/liquid ratio at T) over H (the Henry's constant value in the stated form) from 0.00275 to 0.00825 and T (temperature (for the dimensionless form)) from 149.075 K to 447.225 K: each of the 21 × 15 cells is one run of environmental-engineering-ai999x, shaded by its value (light 0.0749357 to dark 0.674422); iso-lines at H_dimensionless = 0.1, 0.2, 0.3, 0.4, 0.5, 0.6. Other inputs as in the worked example: form = atm_m3_per_mol. Marked: the worked example, H = 0.0055, T = 298.15 K, H_dimensionless = 0.224807.

```
environmental_engineering_ai999x.solve('enveng_henry_constant_conversion', H=0.0055,
form='atm_m3_per_mol', T=298.15)
# -> H_pa_m3_per_mol = 557.288, H_atm_m3_per_mol = 0.0055, H_dimensionless = 0.224807, H_atm =
304.395, H_mol_per_m3_pa = 0.00179441, H_m_per_atm = 0.181818   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_henry_constant_conversion', H=0.25,
form='dimensionless', T=288.15)
# -> H_pa_m3_per_mol = 598.953, H_atm_m3_per_mol = 0.00591121, H_dimensionless = 0.25, H_atm =
327.153, H_mol_per_m3_pa = 0.00166958, H_m_per_atm = 0.16917   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Henry's constant conversion: H = 0.0055, form = atm_m3_per_mol, T = 298.15 K	SOLVED	H_pa_m3_per_mol = 557.288, H_atm_m3_per_mol = 0.0055, H_dimensionless = 0.224807, H_atm = 304.395, H_mol_per_m3_pa = 0.00179441, H_m_per_atm = 0.181818
Henry's constant conversion: H = 0.25, form = dimensionless, T = 288.15 K	SOLVED	H_pa_m3_per_mol = 598.953, H_atm_m3_per_mol = 0.00591121, H_dimensionless = 0.25, H_atm = 327.153, H_mol_per_m3_pa = 0.00166958, H_m_per_atm = 0.16917
Henry's constant conversion: form = atm_m3_per_mol, T = 298.15 K	NEEDS_INPUT	enveng_henry_constant_conversion needs H (the Henry's constant value in the stated form). Example: solve('enveng_henry_c
Henry's constant conversion: H = 0.0055, form = atm_m3_per_mol, T = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Dissolved concentration in equilibrium with a gas (Henry's law), `enveng_henry_law_equilibrium`

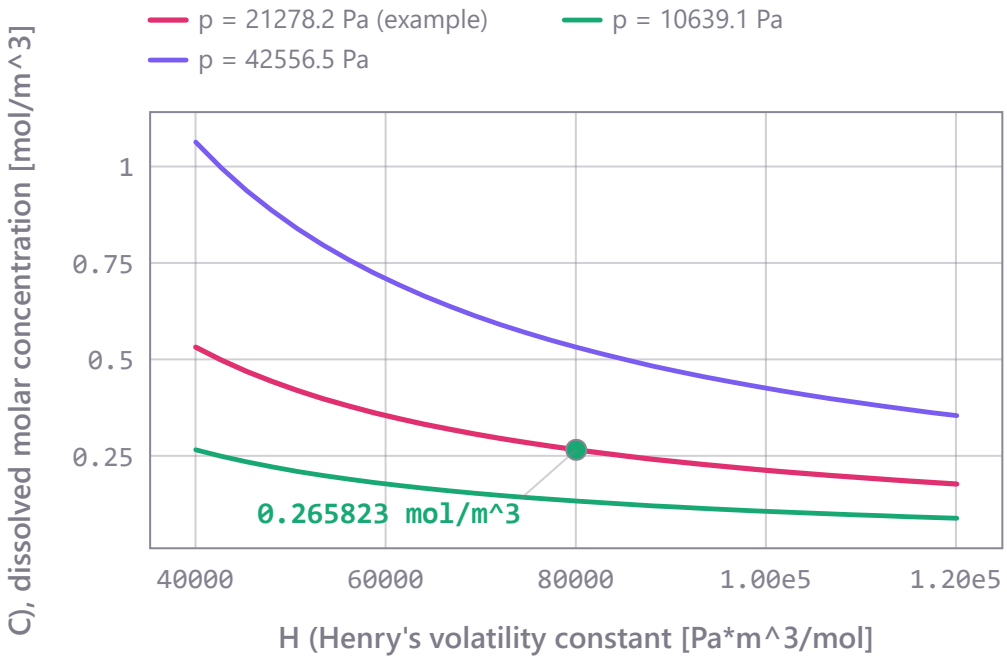
The equilibrium molar (and, with the molar mass, mass) concentration of a gas dissolved in water from its partial pressure and the Henry's volatility constant.

$$C = \frac{p}{H}$$

input	meaning	condition
`p`	partial pressure of the gas	
`H`	Henry's volatility constant	
`M`	molar mass (for the mass concentration)	

output	meaning
`C`	dissolved molar concentration
`C_mass`	dissolved mass concentration
`C_mgL`	in mg/L

- **Method:** $C = p/H$
- **Assumes:** dilute solution, equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C (dissolved molar concentration) against H (Henry's volatility constant) from 40023.4 Pa*m^3/mol to 120070 Pa*m^3/mol. One curve for each value of p (partial pressure of the gas): 21278.2 Pa, 10639.1 Pa, 42556.5 Pa. Other inputs as in the worked example: M = 0.0319988 kg/mol. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, H = 80046.8 Pa*m^3/mol, C = 0.265823 mol/m^3.

```
environmental_engineering_ai999x.solve('enveng_henry_law_equilibrium', p=21278.25, H=80046.75,
M=0.0319988)
# -> C = 0.265823 mol/m^3, C_mass = 0.00850601 kg/m^3, C_mgL = 8.50601   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_henry_law_equilibrium', p=101.325, H=557.2875)
# -> C = 0.181818 mol/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Henry's law equilibrium: p = 0.21 atm, H = 0.79 atm*m^3/mol, M = 31.9988 g/mol	SOLVED	C = 0.265823 mol/m^3, C_mass = 0.00850601 kg/m^3, C_mgL = 8.50601
Henry's law equilibrium: p = 0.001 atm, H = 0.0055 atm*m^3/mol	SOLVED	C = 0.181818 mol/m^3
Henry's law equilibrium: H = 0.79 atm*m^3/mol, M = 31.9988 g/mol	NEEDS_INPUT	enveng_henry_law_equilibrium needs p (partial pressure of the gas). Example: solve('enveng_henry_law_equilibrium', p='21
Henry's law equilibrium: p = 0.21 atm, H = 0.79 atm*m^3/mol, M = 31.9988 g/mol, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Temperature dependence of Henry's constant (van 't Hoff),

`enveng_henry_temperature_dependence`

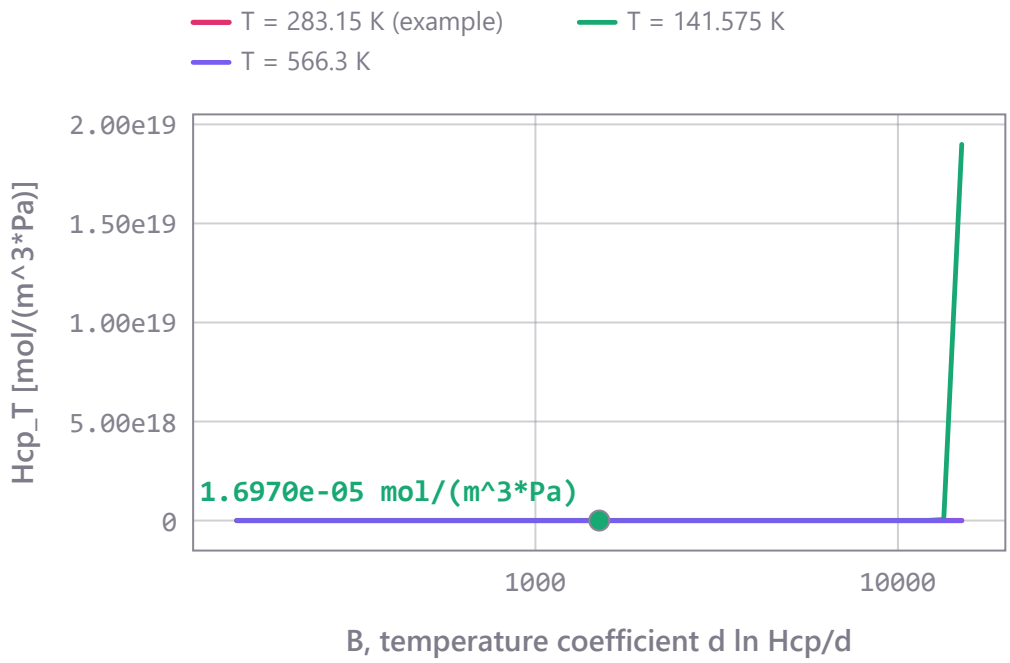
The Henry's solubility constant at a temperature T from its value at a reference temperature and the van 't Hoff coefficient $B = -d$ ln Hcp / d(1/T) (the form tabulated in solubility compilations).

$$H^{cp}(T) = H^{cp}(T_{ref}) \exp\left[B\left(\frac{1}{T} - \frac{1}{T_{ref}}\right)\right]$$

input	meaning	condition
`Hcp`	Henry's solubility constant at Tref	
`B`	temperature coefficient d ln Hcp/d(1/T), in K	
`T`	temperature of interest	
`Tref`	reference temperature (usually 298.15 K)	

output	meaning
`Hcp_T`	solubility constant at T
`Hpc_T`	volatility constant 1/Hcp at T

- **Method:** Hcp(T) = Hcp(Tref) exp(B (1/T - 1/Tref))
- **Assumes:** constant enthalpy of solution over the range
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hcp_T (solubility constant at T) against B (temperature coefficient d ln Hcp/d(1/T), in K) from 150 to 15000 on a logarithmic axis. One curve for each value of T (temperature of interest): 283.15 K, 141.575 K, 566.3 K. Other inputs as in the worked example: Hcp = 1.3e-05 mol/(m^3*Pa), Tref = 298.15 K. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, B = 1500, Hcp_T = 1.6970e-05 mol/(m^3*Pa).

```
environmental_engineering_ai999x.solve('enveng_henry_temperature_dependence', Hcp=1.3e-05,
B=1500.0, T=283.15, Tref=298.15)
# -> Hcp_T = 1.69704e-05 mol/(m^3*Pa), Hpc_T = 58926.2 Pa*m^3/mol   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_henry_temperature_dependence', Hcp=0.0018, B=3200.0,
T=308.15, Tref=298.15)
# -> Hcp_T = 0.0012706 mol/(m^3*Pa), Hpc_T = 787.031 Pa*m^3/mol   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Henry's constant temperature dependence: Hcp = 1.3e-05 mol/(m^3*Pa), B = 1500 K, T = 283.15 K, Tref = 298.15 K	SOLVED	Hcp_T = 1.69704e-05 mol/(m^3*Pa), Hpc_T = 58926.2 Pa*m^3/mol
Henry's constant temperature dependence: Hcp = 0.0018 mol/(m^3*Pa), B = 3200 K, T = 308.15 K, Tref = 298.15 K	SOLVED	Hcp_T = 0.0012706 mol/(m^3*Pa), Hpc_T = 787.031 Pa*m^3/mol
Henry's constant temperature dependence: B = 1500 K, T = 283.15 K, Tref = 298.15 K	NEEDS_INPUT	enveng_henry_temperature_dependence needs Hcp (Henry's solubility constant at Tref). Example: solve('enveng_henry_temper
Henry's constant temperature dependence: Hcp = 1.3e-05 mol/(m^3*Pa), B = 1500 K, T = 283.15 K, Tref = 298.15 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Overall liquid-phase transfer coefficient and flux (two-film theory), `enveng_two_film_transfer`

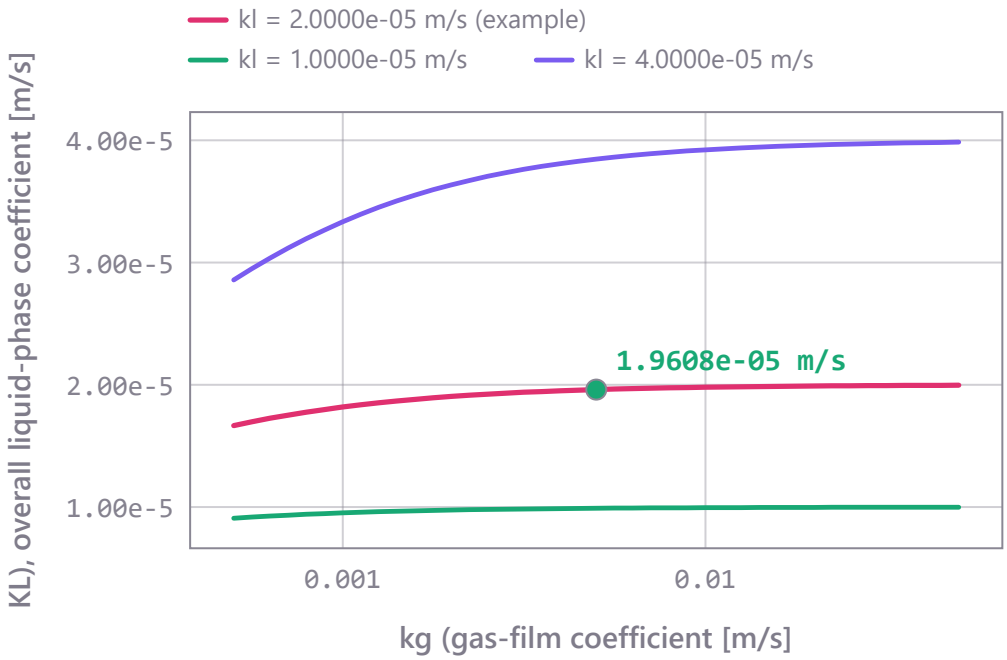
The overall liquid-side mass-transfer coefficient from the liquid- and gas-film coefficients and the dimensionless Henry's constant, the share of resistance in the liquid film, and the flux from water to air (negative when the gas dissolves).

$$\frac{1}{K_L} = \frac{1}{k_l} + \frac{1}{H k_g}, \quad N = K_L \left(C - \frac{C_g}{H} \right)$$

input	meaning	condition
`kl`	liquid-film coefficient	
`kg`	gas-film coefficient	
`H`	dimensionless Henry's constant (Cg/Cl)	
`C`	bulk liquid concentration	
`Cg`	bulk gas concentration	

output	meaning
`KL`	overall liquid-phase coefficient
`flux`	flux from liquid to gas
`liquid_resistance_fraction`	fraction of the resistance in the liquid film

- **Method:** 1/KL = 1/kl + 1/(H kg); flux = KL (C - Cg/H)
- **Assumes:** linear equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



KL (overall liquid-phase coefficient) against kg (gas-film coefficient) from 0.0005 m/s to 0.05 m/s on a logarithmic axis. One curve for each value of kl (liquid-film coefficient): 2.0000e-05 m/s, 1.0000e-05 m/s, 4.0000e-05 m/s. Other inputs as in the worked example: H = 0.2, C = 0.001 kg/m^3, Cg = 0.0 kg/m^3. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, kg = 0.005 m/s, KL = 1.9608e-05 m/s.

```
environmental_engineering_ai999x.solve('enveng_two_film_transfer', kl=2e-05, kg=0.005, H=0.2,
C=0.001, Cg=0.0)
# -> KL = 1.96078e-05 m/s, flux = 1.96078e-08 kg/(m^2*s), liquid_resistance_fraction = 0.980392
(SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_two_film_transfer', kl=5e-05, kg=0.002, H=0.005,
C=0.0005, Cg=2e-08)
# -> KL = 8.33333e-06 m/s, flux = 4.13333e-09 kg/(m^2*s), liquid_resistance_fraction = 0.166667
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-film mass transfer: kl = 2e-05 m/s, kg = 0.005 m/s, H = 0.2, C = 1 mg/L, Cg = 0 ug/m^3	SOLVED	KL = 1.96078e-05 m/s, flux = 1.96078e-08 kg/(m^2*s), liquid_resistance_fraction = 0.980392
Two-film mass transfer: kl = 5e-05 m/s, kg = 0.002 m/s, H = 0.005, C = 0.5 mg/L, Cg = 20 ug/m^3	SOLVED	KL = 8.33333e-06 m/s, flux = 4.13333e-09 kg/(m^2*s), liquid_resistance_fraction = 0.166667
Two-film mass transfer: kg = 0.005 m/s, H = 0.2, C = 1 mg/L, Cg = 0 ug/m^3	NEEDS_INPUT	enveng_two_film_transfer needs kl (liquid-film coefficient). Example: solve('enveng_two_film_transfer', kl='2e-05 m/s',
Two-film mass transfer: kl = 2e-05 m/s, kg = 0.005 m/s, H = 0.2, C = 1 mg/L, Cg = 0 ug/m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Packed-tower air stripping: stripping factor, transfer units and height,

`enveng_air_stripping_tower`

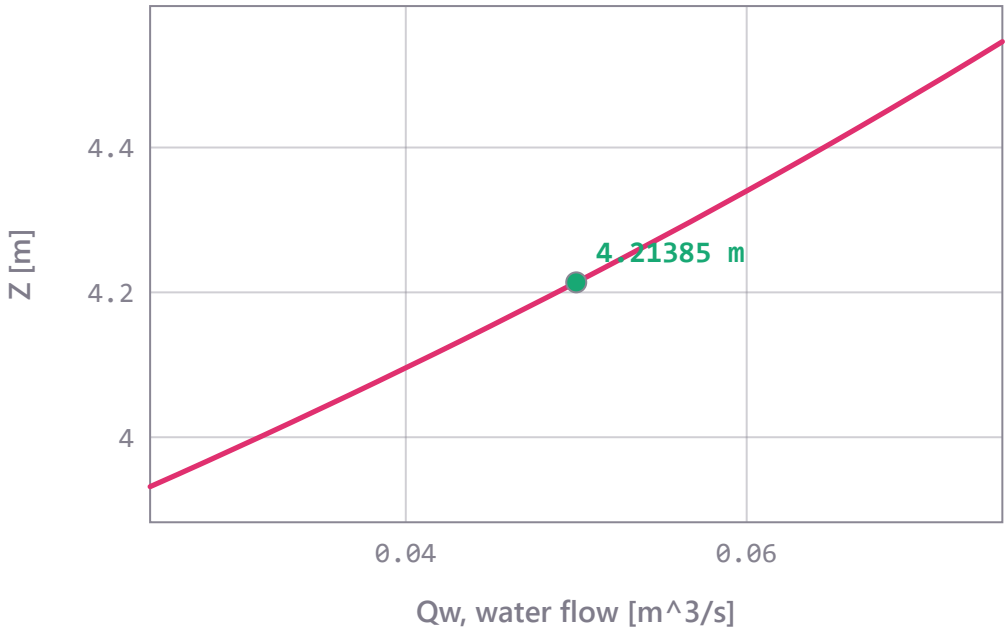
The stripping factor, number of transfer units, packed height and minimum air-to-water ratio of a countercurrent packed tower stripping a volatile compound with clean air.

$$NTU = \frac{R}{R-1}\ln\left[\frac{(C_{in}/C_{out})(R-1)+1}{R}\right], \quad Z = HTU \cdot NTU$$

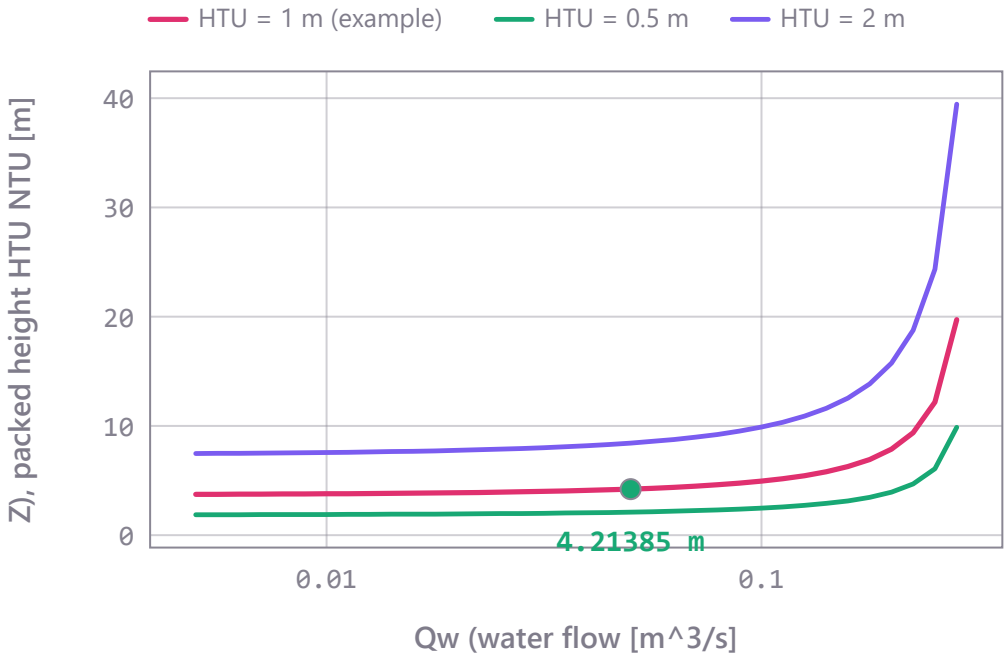
input	meaning	condition
`Qw`	water flow	
`Qa`	air flow	
`H`	dimensionless Henry's constant (Cg/Cl)	
`Cin`	influent concentration	
`Cout`	required effluent concentration	
`HTU`	height of a transfer unit	

output	meaning
`R`	stripping factor H Qa/Qw
`NTU`	number of transfer units
`Z`	packed height HTU NTU
`air_to_water_min`	minimum air-to-water ratio (Cin - Cout)/(H Cin)

- **Method:** $R = H$ Qa/Qw; NTU = R/(R - 1) ln[((Cin/Cout)(R - 1) + 1)/R] (Cin/Cout - 1 when R = 1); Z = HTU NTU
- **Assumes:** Qa/Qw above the minimum; clean inlet air; dilute, linear equilibrium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Z (packed height HTU NTU) against Qw (water flow), with Cin = 2.00e-4 kg/m^3, Cout = 5.00e-6 kg/m^3, H = 0.2, HTU = 1 m, Qa = 1.5 m^3/s. 31 points computed by environmental-engineering-ai999x; marked: Qw = 0.05, Z = 4.21385 m.



Z (packed height HTU NTU) against Qw (water flow) from 0.005 m^3/s to 0.5 m^3/s on a logarithmic axis. One curve for each value of HTU (height of a transfer unit): 1 m, 0.5 m, 2 m. Other inputs as in the worked example: Qa = 1.5 m^3/s, H = 0.2, Cin = 0.0002 kg/m^3, Cout = 5e-06 kg/m^3. 36 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Qw = 0.05 m^3/s, Z = 4.21385 m.

```
environmental_engineering_ai999x.solve('enveng_air_stripping_tower', Qw=0.05, Qa=1.5, H=0.2,
Cin=0.0002, Cout=5e-06, HTU=1.0)
# -> R = 6, NTU = 4.21385, Z = 4.21385 m, air_to_water_min = 4.875   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_air_stripping_tower', Qw=0.1, Qa=2.0, H=0.5,
Cin=0.001, Cout=1e-05, HTU=0.8)
# -> R = 10, NTU = 5.00102, Z = 4.00082 m, air_to_water_min = 1.98   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Air stripping tower: Qw = 0.05 m^3/s, Qa = 1.5 m^3/s, H = 0.2, Cin = 200 ug/L, Cout = 5 ug/L, HTU = 1 m	SOLVED	R = 6, NTU = 4.21385, Z = 4.21385 m, air_to_water_min = 4.875
Air stripping tower: Qw = 0.1 m^3/s, Qa = 2 m^3/s, H = 0.5, Cin = 1000 ug/L, Cout = 10 ug/L, HTU = 0.8 m	SOLVED	R = 10, NTU = 5.00102, Z = 4.00082 m, air_to_water_min = 1.98
Air stripping tower: Qa = 1.5 m^3/s, H = 0.2, Cin = 200 ug/L, Cout = 5 ug/L, HTU = 1 m	NEEDS_INPUT	enveng_air_stripping_tower needs Qw (water flow). Example: solve('enveng_air_stripping_tower', Qw='0.05 m^3/s', Qa='1.5
Air stripping tower: Qw = 0.05 m^3/s, Qa = 1.5 m^3/s, H = 0.2, Cin = 200 ug/L, Cout = 5 ug/L, HTU = 1 m, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Adding sound levels in decibels, `enveng_decibel_addition`

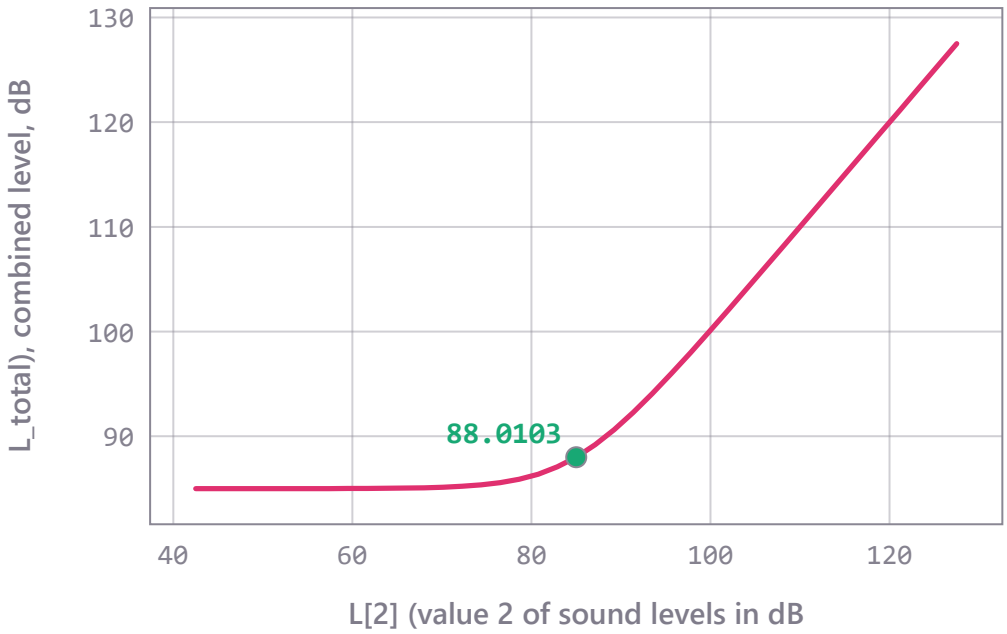
The combined sound pressure level of several incoherent sources, added on an energy basis.

$$L = 10 \log_{10} \sum_i 10^{L_i/10}$$

input	meaning	condition
`L`	sound levels in dB	

output	meaning
`L_total`	combined level, dB

- **Method:** $L = 10$ log10(sum $10^{Li/10}$)
- **Assumes:** incoherent sources
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L_total (combined level, dB) as value 2 of L (sound levels in dB) moves from 42.5 to 127.5, the other values as given ([85.0, 85.0]); 41 runs of environmental-engineering-ai999x. Marked: the worked example, L[2] = 85, L_total = 88.0103.

```
environmental_engineering_ai999x.solve('enveng_decibel_addition', L=[85.0, 85.0])
# -> L_total = 88.0103   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_decibel_addition', L=[90.0, 87.0, 82.0, 75.0])
# -> L_total = 92.2822   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Add sound levels: L = [85, 85]	SOLVED	L_total = 88.0103
Add sound levels: L = [90, 87, 82, 75]	SOLVED	L_total = 92.2822
Add sound levels:	NEEDS_INPUT	enveng_decibel_addition needs L (sound levels in dB). Example: solve('enveng_decibel_addition', L=[85.0, 85.0]).
Add sound levels: L = [85, 85], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Source level after removing background noise, `enveng_background_noise_correction`

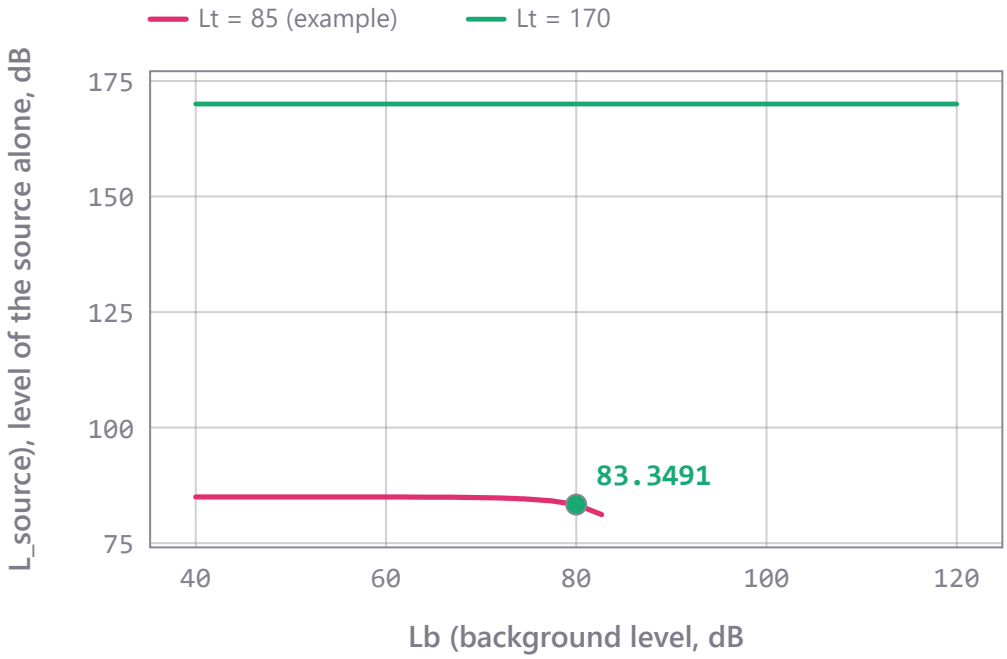
The sound level of a source alone from the total measured level and the background level (energy subtraction).

$$L_s = 10 \log_{10} \left(10^{L_t/10} - 10^{L_b/10} \right)$$

input	meaning	condition
`Lt`	total measured level, dB	
`Lb`	background level, dB	

output	meaning
`L_source`	level of the source alone, dB

- **Method:** $Ls = 10 \log10(10^{Lt/10} - 10^{Lb/10})$
- **Assumes:** $Lt > Lb$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L_source (level of the source alone, dB) against Lb (background level, dB) from 40 to 120. One curve for each value of Lt (total measured level, dB): 85, 170. 17 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Lb = 80, L_source = 83.3491.

```
environmental_engineering_ai999x.solve('enveng_background_noise_correction', Lt=85.0, Lb=80.0)
# -> L_source = 83.3491   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_background_noise_correction', Lt=72.0, Lb=68.0)
# -> L_source = 69.7952   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Background noise correction: Lt = 85 dB, Lb = 80 dB	SOLVED	L_source = 83.3491
Background noise correction: Lt = 72 dB, Lb = 68 dB	SOLVED	L_source = 69.7952
Background noise correction: Lb = 80 dB	NEEDS_INPUT	enveng_background_noise_correction needs Lt (total measured level, dB). Example: solve('enveng_background_noise_correcti
Background noise correction: Lt = 85 dB, Lb = 80 dB, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sound level at another distance (point or line source), `enveng_noise_distance_attenuation`

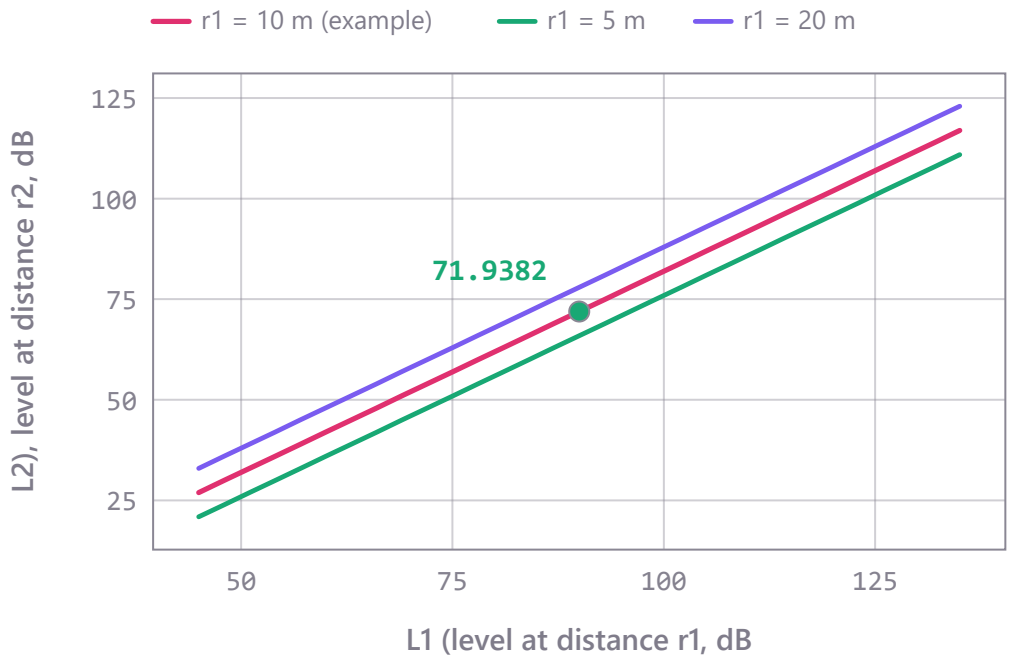
The sound level at a second distance from a point source (6 dB per doubling) or a line source such as a busy road (3 dB per doubling), free field.

$$L_2 = L_1 - 20 \log_{10} \frac{r_2}{r_1} \text{ (point)}$$

input	meaning	condition
`L1`	level at distance r1, dB	
`r1`	first distance	
`r2`	second distance	
`source`	point or line source	

output	meaning
`L2`	level at distance r2, dB

- **Method:** $L_2 = L_1 - 20$ log10(r2/r1) (point), $L_1 - 10$ log10(r2/r1) (line)
- **Assumes:** free field, no absorption
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L2 (level at distance r2, dB) against L1 (level at distance r1, dB) from 45 to 135. One curve for each value of r1 (first distance): 10 m, 5 m, 20 m. Other inputs as in the worked example: r2 = 80.0 m, source = point. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, L1 = 90, L2 = 71.9382.

```
environmental_engineering_ai999x.solve('enveng_noise_distance_attenuation', L1=90.0, r1=10.0,
r2=80.0, source='point')
# -> L2 = 71.9382   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_noise_distance_attenuation', L1=75.0, r1=15.0,
r2=120.0, source='line')
# -> L2 = 65.9691   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Noise distance attenuation: L1 = 90 dB, r1 = 10 m, r2 = 80 m, source = point	SOLVED	L2 = 71.9382
Noise distance attenuation: L1 = 75 dB, r1 = 15 m, r2 = 120 m, source = line	SOLVED	L2 = 65.9691
Noise distance attenuation: r1 = 10 m, r2 = 80 m, source = point	NEEDS_INPUT	enveng_noise_distance_attenuation needs L1 (level at distance r1, dB). Example: solve('enveng_noise_distance_attenuation
Noise distance attenuation: L1 = 90 dB, r1 = 10 m, r2 = 80 m, source = point, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Equivalent continuous sound level Leq, `enveng_equivalent_sound_level`

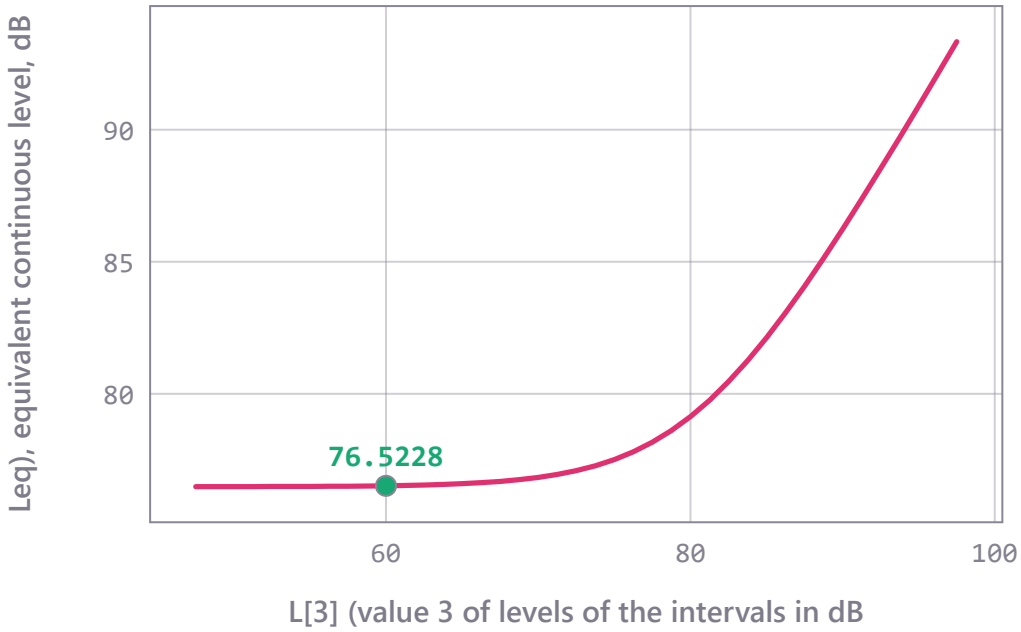
The equivalent continuous sound level of a period made of intervals at different levels (energy average weighted by duration).

$$L_{eq} = 10 \log_{10} \frac{\sum_i t_i 10^{L_i/10}}{\sum_i t_i}$$

input	meaning	condition
`L`	levels of the intervals in dB	
`t`	durations of the intervals (any one unit)	

output	meaning
`Leq`	equivalent continuous level, dB

- **Method:** Leq = 10 log10(sum t_i 10^(L_i/10)/sum t_i)
- **Assumes:** one duration per level
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Leq (equivalent continuous level, dB) as value 3 of L (levels of the intervals in dB) moves from 47.5 to 97.5, the other values as given ([70.0, 85.0, 60.0]); 41 runs of environmental-engineering-ai999x. Other inputs: t = [4.0, 1.0, 3.0]. Marked: the worked example, L[3] = 60, Leq = 76.5228.

```
environmental_engineering_ai999x.solve('enveng_equivalent_sound_level', L=[70.0, 85.0, 60.0], t=
[4.0, 1.0, 3.0])
# -> Leq = 76.5228   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_equivalent_sound_level', L=[92.0, 78.0], t=[2.0,
6.0])
# -> Leq = 86.4694   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equivalent sound level: L = [70, 85, 60], t = [4, 1, 3]	SOLVED	Leq = 76.5228
Equivalent sound level: L = [92, 78], t = [2, 6]	SOLVED	Leq = 86.4694
Equivalent sound level: t = [4, 1, 3]	NEEDS_INPUT	enveng_equivalent_sound_level needs L (levels of the intervals in dB). Example: solve('enveng_equivalent_sound_level', L
Equivalent sound level: L = [70, 85, 60], t = [4, 1, 3], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Day-night (Ldn) and day-evening-night (Lden) sound levels, `enveng_day_night_level`

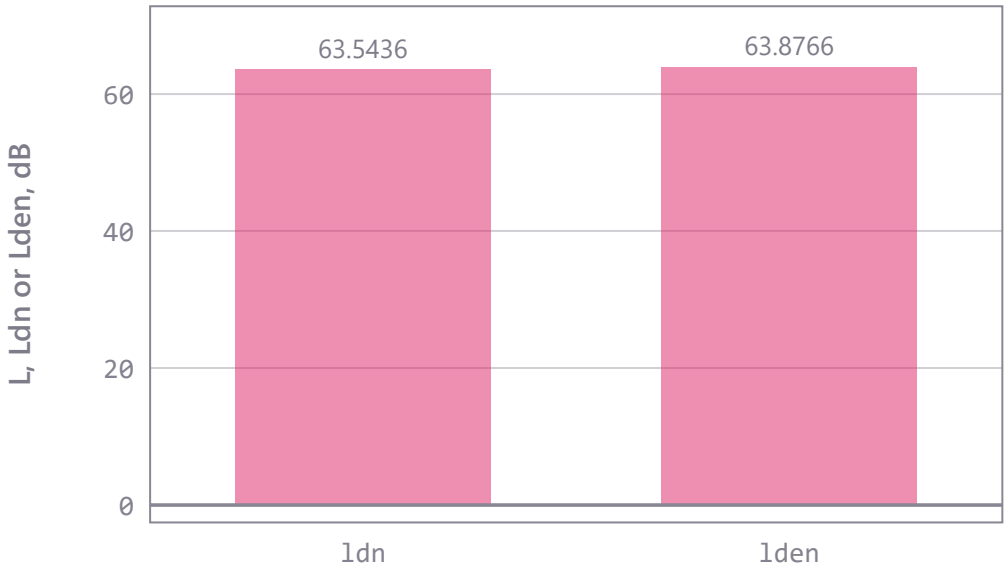
The day-night average level (10 dB night penalty, 22:00 – 07:00) or the day-evening-night level (5 dB evening 19:00 – 23:00, 10 dB night 23:00 – 07:00) from 24 hourly equivalent levels.

$$L_{dn} = 10 \log_{10} \frac{1}{24} \left(\sum_{day} 10^{L_h/10} + \sum_{night} 10^{(L_h+10)/10} \right)$$

input	meaning	condition
`L`	24 hourly Leq values in dB, the first for 00:$00 - 01$:00	
`metric`	ldn or lden	

output	meaning
`L`	Ldn or Lden, dB

- **Method:** $L = 10$ log10($(1/24)$ sum 10^((L_h + penalty_h)/10))
- **Assumes:** exactly 24 hourly levels
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L (Ldn or Lden, dB) for each option of metric (ldn or lden), one run of environmental-engineering-ai999x per option: ldn: 63.5436, lden: 63.8766. The worked example uses ldn. Other inputs as in the worked example: L = [50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 50.0, 65....

```
environmental_engineering_ai999x.solve('enveng_day_night_level', L=[50.0, 50.0, 50.0, 50.0, 50.0,
50.0, 50.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 65.0, 60.0, 60.0,
60.0, 55.0, 55.0], metric='ldn')
# -> L = 63.5436   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_day_night_level', L=[48.0, 48.0, 48.0, 48.0, 48.0,
48.0, 48.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 66.0, 62.0, 62.0,
62.0, 62.0, 52.0], metric='lden')
# -> L = 64.8797   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Day-night sound level: L = [50, 50, 50, 50, 50, 50, 50, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 60, 60, 60, 55, 55], metric = ldn	SOLVED	L = 63.5436

question	outcome	result
Day-night sound level: L = [48, 48, 48, 48, 48, 48, 48, 66, 66, 66, 66, 66, 66, 66, 66, 66, 66, 66, 66, 62, 62, 62, 62, 52], metric = lden	SOLVED	L = 64.8797
Day-night sound level: metric = ldn	NEEDS_INPUT	enveng_day_night_level needs L (24 hourly Leq values in dB, the first for 00:00-01:00). Example: solve('enveng_day_night
Day-night sound level: L = [50, 50, 50, 50, 50, 50, 50, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 65, 60, 60, 60, 55, 55], metric = ldn, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Sound pressure level from sound power level, `enveng_sound_power_to_pressure`

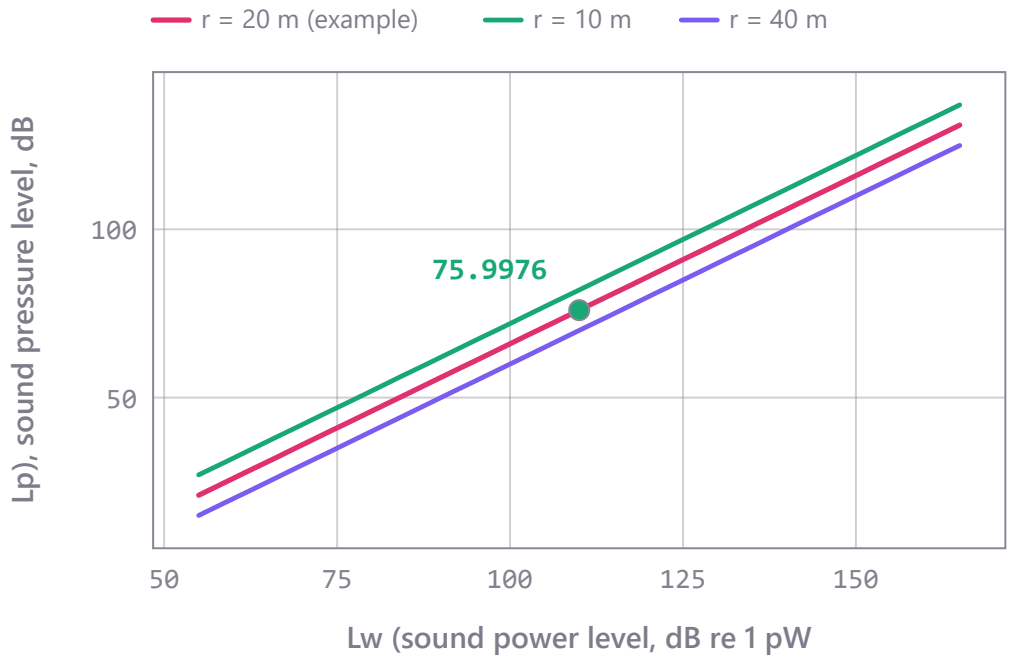
The sound pressure level at a distance from a source of given sound power level and directivity factor (1 free field, 2 on a hard floor, 4 at a wall-floor edge, 8 in a corner).

$$L_p = L_W + 10\log_{10}\frac{Q}{4\pi r^2}$$

input	meaning	condition
`Lw`	sound power level, dB re 1 pW	
`r`	distance from the source	
`Qd`	directivity factor (1, 2, 4 or 8)	

output	meaning
`Lp`	sound pressure level, dB

- **Method:** $Lp = Lw + 10$ log10(Q/(4 pi r^2))
- **Assumes:** far field, no absorption; rho c taken as 400 rayl $(Lp = LI)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lp (sound pressure level, dB) against Lw (sound power level, dB re 1 pW) from 55 to 165. One curve for each value of r (distance from the source): 20 m, 10 m, 40 m. Other inputs as in the worked example: Qd = 2.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, Lw = 110, Lp = 75.9976.

```
environmental_engineering_ai999x.solve('enveng_sound_power_to_pressure', Lw=110.0, r=20.0, Qd=2.0)
# -> Lp = 75.9976   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_sound_power_to_pressure', Lw=95.0, r=5.0, Qd=1.0)
# -> Lp = 70.0285   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sound power to pressure level: Lw = 110 dB, r = 20 m, Qd = 2	SOLVED	Lp = 75.9976
Sound power to pressure level: Lw = 95 dB, r = 5 m, Qd = 1	SOLVED	Lp = 70.0285
Sound power to pressure level: r = 20 m, Qd = 2	NEEDS_INPUT	enveng_sound_power_to_pressure needs Lw (sound power level, dB re 1 pW). Example: solve('enveng_sound_power_to_pressure'
Sound power to pressure level: Lw = 110 dB, r = 20 m, Qd = 2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Occupational noise dose and 8-hour TWA (OSHA, 5 dB exchange),

`enveng_occupational_noise_dose`

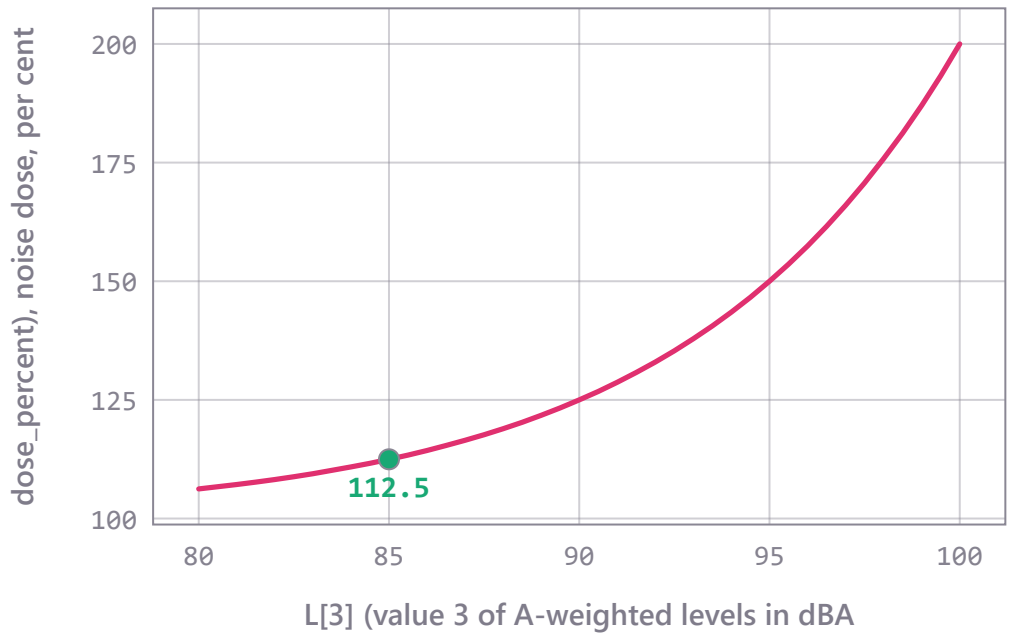
The daily noise dose (per cent of the permissible exposure) and the 8-hour time-weighted average from levels and exposure times, with the 90 dBA criterion, 5 dB exchange rate and 80 dBA threshold.

$$D = 100 \sum_i \frac{C_i}{T_i},\ T_i = \frac{8}{2^{(L_i - 90)/5}},\ TWA = 16.61 \log_{10} \frac{D}{100} + 90$$

input	meaning	condition
`L`	A-weighted levels in dBA	
`hours`	exposure time at each level in hours	

output	meaning
`dose_percent`	noise dose, per cent
`TWA`	8-hour time-weighted average, dBA

- **Method:** $T = 8/2$^((L - 90)/5); $D = 100$ sum C/T; TWA = 16.61 log10(D/100) + 90
- **Assumes:** levels below 80 dBA excluded
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dose_percent (noise dose, per cent) as value 3 of L (A-weighted levels in dBA) moves from 80 to 100, the other values as given ([90.0, 95.0, 85.0]); 41 runs of environmental-engineering-ai999x. Other inputs: hours = [4.0, 2.0, 2.0]. Marked: the worked example, L[3] = 85, dose_percent = 112.5.

```
environmental_engineering_ai999x.solve('enveng_occupational_noise_dose', L=[90.0, 95.0, 85.0],
hours=[4.0, 2.0, 2.0])
# -> dose_percent = 112.5, TWA = 90.8496   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_occupational_noise_dose', L=[100.0, 88.0], hours=
[1.0, 7.0])
# -> dose_percent = 116.313, TWA = 91.0901   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Occupational noise dose: L = [90, 95, 85], hours = [4, 2, 2]	SOLVED	dose_percent = 112.5, TWA = 90.8496
Occupational noise dose: L = [100, 88], hours = [1, 7]	SOLVED	dose_percent = 116.313, TWA = 91.0901
Occupational noise dose: hours = [4, 2, 2]	NEEDS_INPUT	enveng_occupational_noise_dose needs L (A-weighted levels in dBA). Example: solve('enveng_occupational_noise_dose', L=[9
Occupational noise dose: L = [90, 95, 85], hours = [4, 2, 2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Municipal solid waste generation and collection trucks, `enveng_solid_waste_generation`

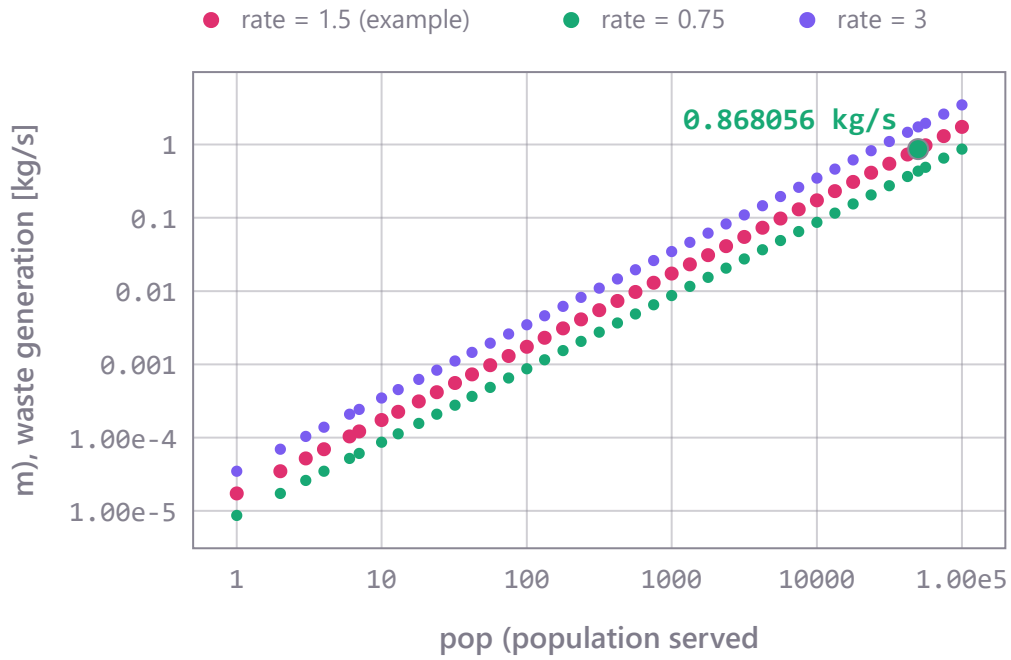
The daily and yearly mass of municipal solid waste of a population, its compacted volume and the number of truck loads per day.

$$\dot{m} = P\,r, \quad n = \left\lceil \frac{\dot{m}}{\rho V_t} \right\rceil$$

input	meaning	condition
`pop`	population served	
`rate`	generation rate in kg per person per day	
`rho`	compacted density in the truck	
`Vt`	truck body volume	

output	meaning
`m`	waste generation
`m_kg_per_day`	in kg/d
`m_t_per_year`	in t per 365-day year
`V_m3_per_day`	compacted volume per day, m^3
`trucks_per_day`	truck loads per day (rounded up)

- **Method:** $m = P$ r; $V = m/\rho$; trucks = ceil(V/Vt)
- **Assumes:** full truck loads at the stated density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



m (waste generation) against pop (population served) from 1 to 100000 on a logarithmic axis, logarithmic output axis. One curve for each value of rate (generation rate in kg per person per day): 1.5, 0.75, 3. Other inputs as in the worked example: rho = 400.0 kg/m^3, Vt = 15.0 m^3. 40 points per curve computed by environmental-engineering-ai999x; marked: the worked example, pop = 50000, m = 0.868056 kg/s.

```
environmental_engineering_ai999x.solve('enveng_solid_waste_generation', pop=50000, rate=1.5,
rho=400.0, Vt=15.0)
# -> m = 0.868056 kg/s, m_kg_per_day = 75000, m_t_per_year = 27375, V_m3_per_day = 187.5,
trucks_per_day = 13   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_solid_waste_generation', pop=120000, rate=2.0,
rho=350.0, Vt=20.0)
# -> m = 2.77778 kg/s, m_kg_per_day = 240000, m_t_per_year = 87600, V_m3_per_day = 685.714,
trucks_per_day = 35   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solid waste generation: pop = 50000, rate = 1.5 kg/cap/d, rho = 400 kg/m^3, Vt = 15 m^3	SOLVED	m = 0.868056 kg/s, m_kg_per_day = 75000, m_t_per_year = 27375, V_m3_per_day = 187.5, trucks_per_day = 13
Solid waste generation: pop = 120000, rate = 2 kg/cap/d, rho = 350 kg/m^3, Vt = 20 m^3	SOLVED	m = 2.77778 kg/s, m_kg_per_day = 240000, m_t_per_year = 87600, V_m3_per_day = 685.714, trucks_per_day = 35
Solid waste generation: rate = 1.5 kg/cap/d, rho = 400 kg/m^3, Vt = 15 m^3	NEEDS_INPUT	enveng_solid_waste_generation needs pop (population served). Example: solve('enveng_solid_waste_generation', pop=50000,
Solid waste generation: pop = 50000, rate = 1.5 kg/cap/d, rho = 400 kg/m^3, Vt = 15 m^3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Landfill life with daily cover and waste growth, `enveng_landfill_life`

The landfill volume used in the first year (waste at its in-place density plus cover soil) and the life of a landfill of given capacity when the waste grows at a constant yearly rate.

$$n = \frac{\ln\left(1 + \frac{Cg}{V_0}\right)}{\ln(1+g)}, \quad V_0 = \frac{M(1+c)}{\rho}$$

input	meaning	condition
`capacity`	available landfill volume	
`M`	waste delivered in the first year	
`rho`	in-place (compacted) waste density	
`cover`	cover soil volume per volume of waste (e.g. 0.25)	
`g`	yearly growth of the waste quantity (fraction or %; 0 for none)	

output	meaning
`V_first_year`	volume used in the first year, m^3
`life_years`	landfill life, years

- **Method:** $V_0 = M$ (1 + cover)/rho per year; capacity = V0 ((1 + g)^n - 1)/g, so $n = ln$(1 + capacity g/V0)/ln(1 + g)
- **Assumes:** growth compounded yearly, life read continuously
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



life_years (landfill life, years) against M (waste delivered in the first year) from 0.792202 kg/s to 2.37661 kg/s. One curve for each value of capacity (available landfill volume): 2e+06 m^3, 1e+06 m^3, 4e+06 m^3. Other inputs as in the worked example: rho = 600.0 kg/m^3, cover = 0.25, g = 0.0. 31 points per curve computed by environmental-engineering-ai999x; marked: the worked example, M = 1.5844 kg/s, life_years = 19.2.

```
environmental_engineering_ai999x.solve('enveng_landfill_life', capacity=2000000.0,
M=1.5844043907014476, rho=600.0, cover=0.25, g=0.0)
# -> V_first_year = 104167, life_years = 19.2   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_landfill_life', capacity=5000000.0,
M=3.802570537683474, rho=700.0, cover=0.2, g=0.03)
# -> V_first_year = 205714, life_years = 18.5271   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Landfill life: capacity = 2000000 m^3, M = 50000 t/yr, rho = 600 kg/m^3, cover = 0.25, g = 0	SOLVED	V_first_year = 104167, life_years = 19.2
Landfill life: capacity = 5000000 m^3, M = 120000 t/yr, rho = 700 kg/m^3, cover = 0.2, g = 0.03	SOLVED	V_first_year = 205714, life_years = 18.5271
Landfill life: M = 50000 t/yr, rho = 600 kg/m^3, cover = 0.25, g = 0	NEEDS_INPUT	enveng_landfill_life needs capacity (available landfill volume). Example: solve('enveng_landfill_life', capacity='2e+06
Landfill life: capacity = 2000000 m^3, M = 50000 t/yr, rho = 600 kg/m^3, cover = 0.25, g = 0, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.

Landfill methane generation (first-order decay, constant acceptance), `enveng_landfill_gas_generation`

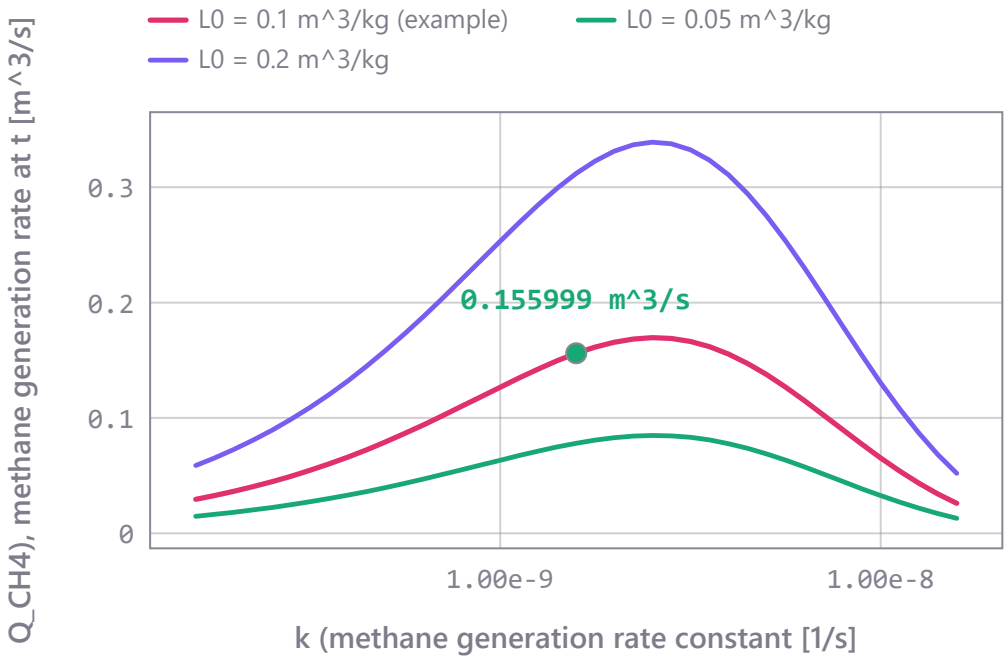
The methane generation rate and the cumulative methane of a landfill receiving waste at a constant rate for its operating life, each deposit decaying at first order (LandGEM-type model in continuous form), and the landfill gas flow at 50 % methane.

$$Q(t) = \int_0^{\min(t,t_c)} kL_0 M\, e^{-k(t-\tau)}\, d\tau = ML_0 \left(e^{-k(t-\min(t,t_c))} - e^{-kt} \right)$$

input	meaning	condition
`M`	waste acceptance rate	
`L0`	methane generation potential	
`k`	methane generation rate constant	
`tc`	operating (filling) period	
`t`	time since opening	

output	meaning
`Q_CH4`	methane generation rate at t
`Q_CH4_m3_per_year`	in m^3 per year
`V_CH4`	cumulative methane generated up to t
`Q_LFG`	landfill gas rate at 50 % methane

- **Method:** Q(t) = M L0 (e^(-k (t - min(t, tc))) - e^(-k t)); cumulative volume by integrating Q
- **Assumes:** constant acceptance from opening to tc; first-order decay of every deposit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q_CH4 (methane generation rate at t) against k (methane generation rate constant) from 1.5844e-10 1/s to 1.5844e-08 1/s on a logarithmic axis. One curve for each value of L0 (methane generation potential): 0.1 m^3/kg, 0.05 m^3/kg, 0.2 m^3/kg. Other inputs as in the worked example: M = 3.168808781402895 kg/s, tc = 631152000.0 s, t = 788940000.0 s. 41 points per curve computed by environmental-engineering-ai999x; marked: the worked example, k = 1.5844e-09 1/s, Q_CH4 = 0.155999 m^3/s.

```
environmental_engineering_ai999x.solve('enveng_landfill_gas_generation', M=3.168808781402895,
L0=0.1, k=1.5844043907014477e-09, tc=631152000.0, t=788940000.0)
# -> Q_CH4 = 0.155999 m^3/s, Q_CH4_m3_per_year = 4.92296e+06, V_CH4 = 1.01541e+08 m^3, Q_LFG =
0.311998 m^3/s   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_landfill_gas_generation', M=1.5844043907014476,
L0=0.17, k=1.267523512561158e-09, tc=473364000.0, t=315576000.0)
# -> Q_CH4 = 0.0887989 m^3/s, Q_CH4_m3_per_year = 2.80228e+06, V_CH4 = 1.4943e+07 m^3, Q_LFG =
0.177598 m^3/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Landfill gas generation: M = 100000 t/yr, L0 = 100 m^3/t, k = 0.05 1/yr, tc = 20 yr, t = 25 yr	SOLVED	Q_CH4 = 0.155999 m^3/s, Q_CH4_m3_per_year = 4.92296e+06, V_CH4 = 1.01541e+08 m^3, Q_LFG = 0.311998 m^3/s
Landfill gas generation: M = 50000 t/yr, L0 = 170 m^3/t, k = 0.04 1/yr, tc = 15 yr, t = 10 yr	SOLVED	Q_CH4 = 0.0887989 m^3/s, Q_CH4_m3_per_year = 2.80228e+06, V_CH4 = 1.4943e+07 m^3, Q_LFG = 0.177598 m^3/s
Landfill gas generation: L0 = 100 m^3/t, k = 0.05 1/yr, tc = 20 yr, t = 25 yr	NEEDS_INPUT	enveng_landfill_gas_generation needs M (waste acceptance rate). Example: solve('enveng_landfill_gas_generation', M='3.16
Landfill gas generation: M = 100000 t/yr, L0 = 100 m^3/t, k =	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
0.05 1/yr, tc = 20 yr, t = 25 yr, zz = 1		environmental_engineering_ai999x.

Heating value of waste from its elemental composition (modified Dulong),

`enveng_dulong_heating_value`

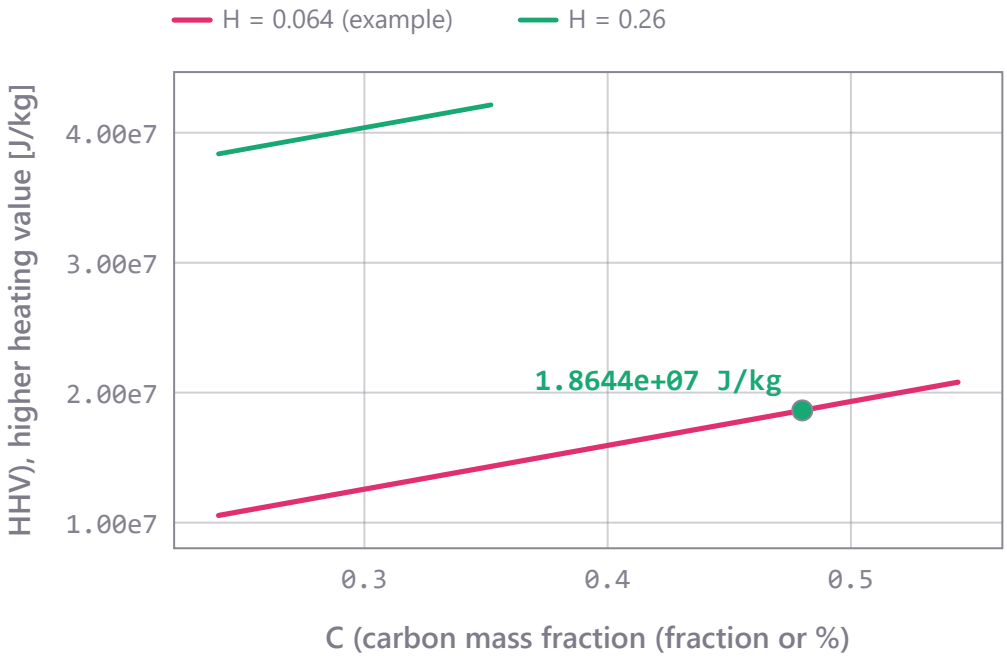
The higher heating value of a waste or fuel from its dry mass fractions of carbon, hydrogen, oxygen, sulfur and nitrogen by the modified Dulong formula.

$$HHV = 337C + 1419\left(H - \frac{O}{8}\right) + 93S + 23N \;\; [\mathrm{kJ/kg},\ \%]$$

input	meaning	condition
`C`	carbon mass fraction (fraction or %)	
`H`	hydrogen mass fraction	
`O`	oxygen mass fraction	
`S`	sulfur mass fraction	
`N`	nitrogen mass fraction	

output	meaning
`HHV`	higher heating value
`HHV_kJ_per_kg`	in kJ/kg
`HHV_MJ_per_kg`	in MJ/kg

- **Method:** HHV $(kJ/kg) = 337\,C + 1419\,(H - O/8) + 93\,S + 23$ N, mass per cent
- **Assumes:** fractions on the same (dry) basis, sum at most 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



HHV (higher heating value) against C (carbon mass fraction (fraction or %)) from 0.24 to 0.72. One curve for each value of H (hydrogen mass fraction): 0.064, 0.26. Other inputs as in the worked example: O = 0.376, S = 0.004, N = 0.008. 20 points per curve computed by environmental-engineering-ai999x; marked: the worked example, C = 0.48, HHV = 1.8644e+07 J/kg.

```
environmental_engineering_ai999x.solve('enveng_dulong_heating_value', C=0.48, H=0.064, O=0.376,
S=0.004, N=0.008)
# -> HHV = 1.86439e+07 J/kg, HHV_kJ_per_kg = 18643.9, HHV_MJ_per_kg = 18.6439   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_dulong_heating_value', C=0.6, H=0.072, O=0.228,
S=0.002, N=0.012)
# -> HHV = 2.64388e+07 J/kg, HHV_kJ_per_kg = 26438.8, HHV_MJ_per_kg = 26.4388   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dulong heating value: C = 0.48, H = 0.064, O = 0.376, S = 0.004, N = 0.008	SOLVED	HHV = 1.86439e+07 J/kg, HHV_kJ_per_kg = 18643.9, HHV_MJ_per_kg = 18.6439
Dulong heating value: C = 0.6, H = 0.072, O = 0.228, S = 0.002, N = 0.012	SOLVED	HHV = 2.64388e+07 J/kg, HHV_kJ_per_kg = 26438.8, HHV_MJ_per_kg = 26.4388
Dulong heating value: H = 0.064, O = 0.376, S = 0.004, N = 0.008	NEEDS_INPUT	enveng_dulong_heating_value needs C (carbon mass fraction (fraction or %)). Example: solve('enveng_dulong_heating_value'
Dulong heating value: C = 0.48, H = 0.064, O = 0.376, S = 0.004, N = 0.008, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.
Dulong heating value: C = 48%, H = 6.4%, O = 37.6%, S = 0.4%, N = 0.8%	SOLVED	HHV = 1.86439e+07 J/kg, HHV_kJ_per_kg = 18643.9, HHV_MJ_per_kg = 18.6439

Moisture content of mixed solid waste, `enveng_waste_moisture_content`

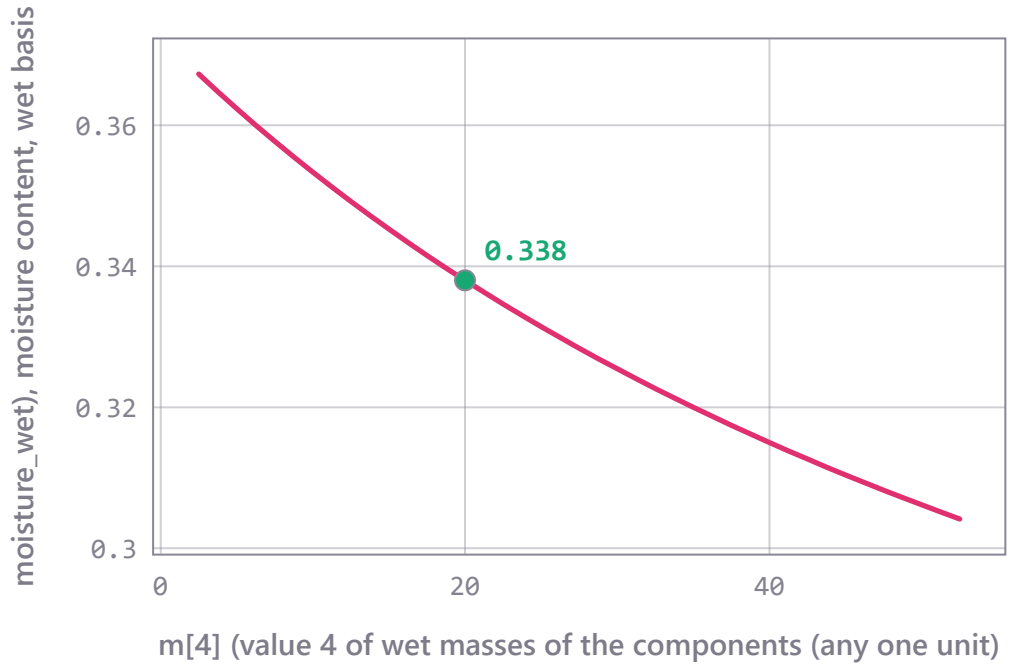
The moisture content (wet and dry basis) and the dry mass of a waste mixture from the masses of its components and their moisture fractions.

$$M_{wet} = \frac{\sum m_i w_i}{\sum m_i}$$

input	meaning	condition
`m`	wet masses of the components (any one unit)	
`w`	moisture fractions of the components, wet basis	

output	meaning
`moisture_wet`	moisture content, wet basis
`dry_mass`	dry mass, in the unit of m
`moisture_dry`	moisture content, dry basis

- **Method:** water = sum m_i w_i; M_wet = water/sum m; M_dry = water/(sum m - water)
- **Assumes:** moisture fractions below 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



moisture_wet (moisture content, wet basis) as value 4 of m (wet masses of the components (any one unit)) moves from 2.5 to 52.5, the other values as given ([40.0, 25.0, 15.0, 20.0]); 41 runs of environmental-engineering-ai999x. Other inputs: w = [0.7, 0.06, 0.02, 0.2]. Marked: the worked example, m[4] = 20, moisture_wet = 0.338.

```
environmental_engineering_ai999x.solve('enveng_waste_moisture_content', m=[40.0, 25.0, 15.0, 20.0],
w=[0.7, 0.06, 0.02, 0.2])
# -> moisture_wet = 0.338, dry_mass = 66.2, moisture_dry = 0.510574   (SOLVED)
```

```
environmental_engineering_ai999x.solve('enveng_waste_moisture_content', m=[10.0, 30.0], w=[0.6,
0.1])
# -> moisture_wet = 0.225, dry_mass = 31, moisture_dry = 0.290323   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Waste moisture content: m = [40, 25, 15, 20], w = [0.7, 0.06, 0.02, 0.2]	SOLVED	moisture_wet = 0.338, dry_mass = 66.2, moisture_dry = 0.510574
Waste moisture content: m = [10, 30], w = [0.6, 0.1]	SOLVED	moisture_wet = 0.225, dry_mass = 31, moisture_dry = 0.290323
Waste moisture content: w = [0.7, 0.06, 0.02, 0.2]	NEEDS_INPUT	enveng_waste_moisture_content needs m (wet masses of the components (any one unit)). Example: solve('enveng_waste_moistu
Waste moisture content: m = [40, 25, 15, 20], w = [0.7, 0.06, 0.02, 0.2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: environmental_engineering_ai999x.